



ORACLE
NetSuite

SuiteCloud Development Framework

2025.2

January 7, 2026



Copyright © 2005, 2025, Oracle and/or its affiliates.

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software, software documentation, data (as defined in the Federal Acquisition Regulation), or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software," "commercial computer software documentation," or "limited rights data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle®, Java, and MySQL are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

If this document is in public or private pre-General Availability status:

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

If this document is in private pre-General Availability status:

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, timing, and pricing of any features or functionality described in this document may change and remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Documentation Accessibility

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at <https://www.oracle.com/corporate/accessibility>.

Access to Oracle Support

Oracle customers that have purchased support have access to electronic support through My Oracle Support. For information, visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=info> or visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=trs> if you are hearing impaired.

Sample Code

Oracle may provide sample code in SuiteAnswers, the Help Center, User Guides, or elsewhere through help links. All such sample code is provided "as is" and "as available", for use only with an authorized NetSuite Service account, and is made available as a SuiteCloud Technology subject to the SuiteCloud Terms of Service at www.netsuite.com/tos.

Oracle may modify or remove sample code at any time without notice.

No Excessive Use of the Service

As the Service is a multi-tenant service offering on shared databases, Customer may not use the Service in excess of limits or thresholds that Oracle considers commercially reasonable for the Service. If Oracle reasonably concludes that a Customer's use is excessive and/or will cause immediate or ongoing performance issues for one or more of Oracle's other customers, Oracle may slow down or throttle Customer's excess use until such time that Customer's use stays within reasonable limits. If Customer's particular usage pattern requires a higher limit or threshold, then the Customer should procure a subscription to the Service that accommodates a higher limit and/or threshold that more effectively aligns with the Customer's actual usage pattern.

Beta Features

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software" or "commercial computer software documentation" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, and timing of any features or functionality described in this document remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Table of Contents

SuiteCloud Development Framework	1
Getting Started with SuiteCloud Development Framework	3
Customizations Supported by SuiteCloud Development Framework	3
SuiteCloud Development Framework Architecture	7
SuiteCloud Development Framework Setup	8
Enabling SuiteCloud Development Framework in the Target NetSuite Account (Administrator Only)	8
Assigning the Developer Role (Administrator Only)	9
SuiteCloud Development Framework Account Preferences (SDF Developers Only)	10
SuiteCloud Project Types	11
Account Customization Projects	11
SuiteApp Projects	12
Differences Between SuiteCloud Projects	14
SuiteCloud Project Structure and File Components	19
SuiteCloud Development Framework Tutorial	20
Set Up a NetSuite Account (Administrator Only)	21
Set Up Your IDE Environment	21
Create an Account Customization Project	22
Create a Custom Record Definition	23
Create a User Event Script	25
Create a Script Record and Script Deployment Definition	27
Validate and Deploy the SuiteCloud Project to a Target NetSuite Account	29
Test the SuiteCloud Project in the Target NetSuite Account	30
Developing SuiteCloud Projects for SuiteCloud Development Framework	32
Account Component Imports to SuiteCloud Projects	32
SDF Custom Object Dependencies in SuiteApps	33
Manually Defining SDF Custom Object Dependencies in a SuiteApp Using the SuiteCloud Project Manifest	33
SuiteApp Bundle Object Dependencies for SuiteCloud Projects	34
Account Component Dependencies in SuiteCloud Projects	37
Manually Defining Account Component Dependencies in the Account Customization Project Manifest	38
Feature Dependencies for SDF Custom Objects in SuiteCloud Projects	39
Enhanced Feature Dependencies for SDF Custom Objects	40
Enhanced Feature Dependencies for SDF Custom Objects Containing OneWorld Components	41
Manually Defining Feature Dependencies in the SuiteCloud Project Manifest	41
Account Configuration from an Account Customization Project	43
SDF Custom Object and File Development in SuiteCloud Projects	45
SuiteScript Files in SuiteCloud Projects	45
Modifying SuiteCloud Project File and Folder Attributes in an XML Definition	46
Installation Preferences in the SuiteApp of SuiteCloud Project	49
SuiteCloud Project Validation	62
SuiteCloud Project Deployment Preparation	62
Customizing SuiteApp Project Deployment using SDF Installation Scripts	64
Deployment Logs for SuiteCloud Projects	69
Viewing and Uninstalling SDF SuiteApps in a NetSuite Account	70
Publishing an SDF SuiteApp	70
Developing SDF Custom Objects	71
Syntax Guidelines and Reference Formats for SDF Custom Objects	72
Translatable Fields on SDF Custom Objects	75
Downloading the XML Definition of an Object from a NetSuite Account	79
SDF Custom Object File Structure	80

Account-Specific Values Validation in SuiteCloud Projects	86
Date and Time Formats in SDF Custom Objects	87
Lists, Records, and Fields	94
Custom Lists as XML Definitions	94
Custom Other Record Fields as XML Definitions	95
Custom Item Fields as XML Definitions	101
Custom CRM Fields as XML Definitions	106
Custom Entity Fields as XML Definitions	112
Custom Item Number Fields as XML Definitions	117
Custom Transaction Body Fields as XML Definitions	124
Custom Transaction Line Fields as XML Definitions	130
Custom Transaction Item Option Fields as XML Definitions	136
Custom Record Types as XML Definitions	142
Custom Segments as XML Definitions	151
Custom Transaction Record Types as XML Definitions	165
Reports	171
Financial Reports as XML Definitions	171
Forms	172
Custom Address Forms as XML Definitions	172
Custom Entry Forms as XML Definitions	178
Custom Code Tab and Actions Tab in XML Definitions	186
Transaction Forms as XML Definitions	188
Sublists as XML Definitions	201
Subtabs as XML Definitions	205
Import and Export	206
Saved CSV Import Maps as XML Definitions	206
Saved Searches as XML Definitions	214
Published Dashboards and Portlets	215
Published Dashboards as XML Definitions	215
Supported Published Dashboard Object Portlets	218
Scripting	234
User Event Scripts as XML Definitions	234
Scheduled Scripts as XML Definitions	237
SDF Installation Scripts as XML Definitions	241
Workflows as XML Definitions	250
Translation Collections as XML Definitions	269
Map/Reduce Script Objects as XML Definitions	273
Portlet Scripts as XML Definitions	278
Client Scripts as XML Definitions	283
Integration Records as XML Definitions	287
Mass Update Scripts as XML Definitions	287
Suitelet Scripts as XML Definitions	290
RESTlet Scripts as XML Definitions	294
Secrets as XML Definitions	298
Single Page Applications as XML Definitions	298
Custom Tool Scripts as XML Definitions	300
Plug-ins	301
Custom Plug-ins as XML Definitions	301
Custom GL Lines Plug-ins as XML Definitions	306
Email Capture Plug-ins as XML Definitions	309
Financial Institution Connectivity Plug-ins as XML Definitions	311
Financial Institution Parser Plug-ins as XML Definitions	313
Bank Statement Parser Plug-ins as XML Definitions	316
Dataset Builder Plug-ins as XML Definitions	317
Workbook Builder Plug-ins as XML Definitions	319

Centers and Tabs	321
Centers as XML Definitions	321
Center Categories as XML Definitions	321
Center Tabs as XML Definitions	322
KPI Scorecards as XML Definitions	323
Users and Roles in SDF	330
Roles and Permissions as an XML Definition	331
Templates	336
Email Templates as XML Definitions	336
Advanced HTML/PDF Templates as XML Definitions	338
SuiteCommerce Web Site Management	340
CMS Content Types as XML Definitions	340
SSP Applications as XML Definitions	340
SuiteAnalytics Workbooks	345
Workbooks as XML Definitions	345
Artificial Intelligence (AI)	347
Prompts as XML Definitions	348
Text Enhance Actions as XML Definitions	348
Conversion of a Bundle to an Account Customization Project for SDF	350
Converting a Bundle into an Account Customization Project from a NetSuite Account	351
SuiteCloud Development Framework Error Handling	352
entryForm and transactionForm Errors	352
The object value is either invalid or does not exist in the account	352
The field under the group is invalid	353
The property value of the object cannot be modified	353
The sameRowAsPrevious field must be set to false because it is the first entry	353
The object field must not be NOT_YET_SUPPORTED	354
Account Configuration Errors	354
Resolve the following conflicts to disable the feature	354
Disable the dependent feature before disabling the feature	354
The feature must be enabled to deploy this project	355
The feature is duplicated	355
Enable the dependent feature before enabling the feature	355
The feature is missing its field	356
The feature's field requires a value	356
The feature is not available on this account	356
The configuration field from the feature is missing	356
Feature is not modifiable	357
Feature(s) definition must start with tag but found tag	357
An invalid feature ID	357
Is an invalid feature state	357
You cannot use SuiteCloud Development Framework to enable the feature because it requires a Terms of Service user agreement	358
Circular Dependency Warnings	358

SuiteCloud Development Framework

SuiteCloud Development Framework (**SDF**) is a development framework that you can use to create SDF SuiteApps, or to customize NetSuite accounts, using an integrated development environment (IDE) on your local computer. It provides a foundation for a structured development process and supports familiar development concepts and patterns. Information about the new release is included in the [Release Notes](#).

Both SDF SuiteApps and account customizations can be created using **SuiteCloud project** components, sometimes referred to as customizations or SDF custom objects, for commercial distribution or internal use within your organization. The two types of SuiteCloud projects are account customization and SuiteApp. The projects are file-based and use XML definitions of SDF custom objects representing NetSuite objects and record types such as custom records, custom forms, and workflows. SuiteCloud projects also support SuiteScript files. Information about SuiteCloud projects is available in [SuiteCloud Project Types](#).

For information about the directory structure and files in SuiteCloud projects, see [SuiteCloud Project Structure and File Components](#) and [SDF Custom Object File Structure](#).

For an overview of development and deployment using SDF, see the following information:

- [Development Overview](#)
- [Deployment Overview](#)
- [Versioning and Revision Control for SDF](#)
- [SuiteCloud Development Framework Architecture](#)

To get started using SuiteCloud Development Framework, see [Getting Started with SuiteCloud Development Framework](#).

Additional information about SDF is included in the following topics:

- [Customizations Supported by SuiteCloud Development Framework](#)
- The reference content is also available in PDF format in the  [SuiteCloud Development Framework XML Reference](#).
- [Syntax Guidelines and Reference Formats for SDF Custom Objects](#)
- [SuiteCloud Development Framework Architecture](#)
- [SuiteCloud Development Framework Setup](#)
- [SuiteCloud Project Types](#)
- [SuiteCloud Development Framework Tutorial](#)

Development Overview

SDF provides rich code completion for the SDF custom object XML definitions used in SuiteCloud projects.

You do not need to be connected to a NetSuite account when developing SuiteCloud projects.

SDF enables you to build, validate, and deploy SuiteCloud projects from your local computer without tying its components to a NetSuite account. You can also create and modify custom objects in the NetSuite UI, and then use SDF to import those objects into SuiteCloud projects.

By decoupling the development process from NetSuite accounts, SDF improves code portability and provides a streamlined deployment process across various NetSuite account types. For information about account types, see the help topic [Understanding NetSuite Account Types](#).

✓ **Tip:** Development accounts can be used with SDF. For information about who can benefit from using development accounts and how to get one, see the help topic [Uses for Development Accounts](#).

You can use SuiteCloud Software Development Kit (SuiteCloud SDK) to develop and deploy SuiteCloud projects to NetSuite accounts. The following tools are useful for getting started developing SDF custom objects:

- **SuiteCloud extension** provides a user interface for NetSuite platform development using SuiteCloud Development Framework (SDF).
 - [SuiteCloud Extension for Visual Studio Code Overview](#)
 -  [SuiteCloud Extension for Visual Studio Code basic commands](#)
- **SuiteCloud IDE plug-in** is an integrated development environment (IDE) that is packaged for NetSuite platform development. IDE plug-in interfaces with SDF and enables you to create SuiteCloud projects. For more information about SuiteCloud IDE plug-in, see the help topic [SuiteCloud IDE Plug-in for WebStorm Overview](#).
- SuiteCloud Development Framework Command Line Interface (**CLI**) is a tool that can be used with your IDE to develop SuiteCloud projects. You can also create batch and shell scripts that use CLI commands to automate SuiteCloud project validation and deployment processes. For more information about SuiteCloud CLI, see the following:
 - [SuiteCloud CLI for Node.js Guide](#)
 - [SuiteCloud CLI for Java Guide](#)

For more information about using SuiteCloud SDK, see the help topic [SuiteCloud SDK Overview](#).

Deployment Overview

SuiteCloud projects can be deployed directly to development, sandbox, or production accounts where you have access to create NetSuite objects. SDF provides on-demand validation of your SuiteCloud projects during the development process against NetSuite accounts without requiring deployment. You can review rich deployment logs and audit trails that include detailed information about the components being deployed and the deployment status.

You can also use SuiteApp Control Center to distribute SDF SuiteApp projects as SDF SuiteApps. For information about distributing SDF SuiteApps, see the help topic [SuiteApp Control Center Overview](#).

To get started using SuiteCloud Extension for Visual Studio Code or SuiteCloud IDE Plug-in for WebStorm to create a SuiteCloud project and deploy the project to a NetSuite account, see [SuiteCloud Development Framework Tutorial](#).

Versioning and Revision Control for SDF

SuiteCloud Development Framework (SDF) is integrated with SuiteCloud SDK so that, in addition to your code, you can manage other components of SuiteCloud projects. With SDF, you can maintain custom objects as XML files and code outside of a NetSuite account, so you can use your own revision control system to provide your development team with versioning, change management, and team collaboration.

Getting Started with SuiteCloud Development Framework

You can get started with SuiteCloud Development Framework (SDF) by learning from the following topics:

- [SuiteCloud Development Framework Setup](#)
 - [Enabling SuiteCloud Development Framework in the Target NetSuite Account \(Administrator Only\)](#)
 - [Assigning the Developer Role \(Administrator Only\)](#)
 - [SuiteCloud Development Framework Account Preferences \(SDF Developers Only\)](#)
- [SuiteCloud Project Types](#)
 - [Account Customization Projects](#)
 - [SuiteApp Projects](#)
 - [Differences Between SuiteCloud Projects](#)
 - [SuiteCloud Project Structure and File Components](#)

To get started using SDF by working on a customization in a SuiteCloud project, begin the [SuiteCloud Development Framework Tutorial](#).



Tip: Development accounts can be used with SDF. For information about who can benefit from using development accounts and how to get one, see the help topic [Uses for Development Accounts](#).

To use SDF with an extension or plug-in, see:

- [Installing and Setting Up SuiteCloud Extension for Visual Studio Code](#)
- [Installing and Setting Up SuiteCloud IDE Plug-in for WebStorm](#)

For a demonstration of setting up SDF and creating a project, watch the following videos:

-  [SuiteCloud Development Framework: Setting up NetSuite to Use SDF](#)
-  [Creating a SuiteApp Project in SuiteCloud CLI for Node.js](#)

Customizations Supported by SuiteCloud Development Framework

SuiteCloud Development Framework (SDF) provides you with a way to customize NetSuite using SuiteCloud project components instead of using the NetSuite interface. These components can consist of files, scripts, and SDF custom objects such as custom records, custom forms, and workflows. Such components are sometimes referred to as customizations, SDF custom objects, and XML definitions. For more information, see .



Tip: You can see the root XML tag and script ID prefix for each SDF custom object in [SDF Custom Object File Structure](#).

The following customizations are supported as SDF custom objects:

Lists, Records, & Fields

- List, see [Custom Lists as XML Definitions](#) and [customlist](#)

- Record type, see [Custom Record Types as XML Definitions](#) and [customrecordtype](#)
- Custom segment, see [Custom Segments as XML Definitions](#) and [customsegment](#)
- Transaction type, see [Custom Transaction Record Types as XML Definitions](#) and [customtransactiontype](#)
- Entity field, see [Custom Entity Fields as XML Definitions](#) and [entitycustomfield](#)
- Item field, see [Custom Item Fields as XML Definitions](#) and [itemcustomfield](#)
- CRM field, see [Custom CRM Fields as XML Definitions](#) and [crmcustomfield](#)
- Transaction body field, see [Custom Transaction Body Fields as XML Definitions](#) and [transactionbodycustomfield](#)
- Transaction line field (formerly transaction column field), see [Custom Transaction Line Fields as XML Definitions](#) and [transactioncolumncustomfield](#)
- Transaction Item option field, see [Custom Transaction Item Option Fields as XML Definitions](#) and [itemoptioncustomfield](#)
- Item number field, see [Custom Item Number Fields as XML Definitions](#) and [itemnumbercustomfield](#)
- Other record field (formerly other custom field), see [Custom Other Record Fields as XML Definitions](#) and [othercustomfield](#)

Forms

Note: You cannot create new address forms, entry forms and transaction forms using SDF. To work with those forms in SDF, you must first import them from a NetSuite account into a SuiteCloud project. See [Account Component Imports to SuiteCloud Projects](#).

- Address form, see [Custom Address Forms as XML Definitions](#) and [addressForm](#)
- Entry form, see [Custom Entry Forms as XML Definitions](#) and [entryForm](#)
- Transaction form, see [Transaction Forms as XML Definitions](#) and [transactionForm](#)
- Sublist, see [Sublists as XML Definitions](#) and [sublist](#)
- Subtab, see [Subtabs as XML Definitions](#) and [subtab](#)

Import/Export

- Saved CSV import, see [Saved CSV Import Maps as XML Definitions](#) and [savedcsvimport](#)
- Saved search, see [Saved Searches as XML Definitions](#) and [savedsearch](#)

Note: Do not manually edit the savedsearch XML in SDF. Modifications to the system-generated XML may result in validation and deployment failures. You should customize saved searches with NetSuite rather than SDF.

Published Dashboards & Portlets

- Published Dashboard, see [Published Dashboards as XML Definitions](#) and [publisheddashboard](#)
The following portlets are supported:
 - Analytics Portlet
 - Calendar Portlet
 - Custom Portlet

- Custom Search Portlet
- Event Scheduler Portlet
- Key Performance Indicator (KPI) Portlet
- Key Performance Indicator (KPI) Scorecard Portlet
- KPI Meter Portlet
- Links Portlet
- List Portlet
- Quick Search Portlet
- Recent Records Portlet
- Recent Reports Portlet
- Reminders Portlet
- Search Form Portlet
- Settings Portlet
- Tasks Portlet
- Trend Graph Portlet

 **Note:** Each portlet type is supported by SDF as part of its support for published dashboards. For more information about defining portlets in SDF, see [Supported Published Dashboard Object Portlets](#).

Scripting

- Bundle installation script, see the help topics [SuiteScript 2.x Bundle Installation Script Type](#) and [bundleinstallationscript](#)
- Client script, see [Client Scripts as XML Definitions](#) and [clientscript](#)
- Map/Reduce script, see [Map/Reduce Script Objects as XML Definitions](#) and [mapreducescript](#)
- Mass update script, see [Mass Update Scripts as XML Definitions](#) and [massupdatescript](#)
- Portlet script, see [Portlet Scripts as XML Definitions](#) and [portlet](#)
- RESTlet script, see [RESTlet Scripts as XML Definitions](#) and [restlet](#)
- Scheduled script, see [Scheduled Scripts as XML Definitions](#) and [scheduledscript](#)
- Suitelet script, see [Suitelet Scripts as XML Definitions](#) and [suitelet](#)
- User event script, see [User Event Scripts as XML Definitions](#) and [usereventscript](#)
- Workflow action script, see the help topics [SuiteScript 2.x Workflow Action Script Type](#) and [workflowactionscript](#)
- SDF Installation script, see [SDF Installation Scripts as XML Definitions](#) and [sdfinstallationscript](#)
- Translation Collections, see [Translation Collections as XML Definitions](#) and [translationcollection](#)

 **Note:** When importing translationcollection objects into a SuiteCloud project, the corresponding XLIFF files are not created. For more information, see [Translation Collections as XML Definitions](#).

- Workflow, see [Workflows as XML Definitions](#) and [workflow](#)
- Integration record, see [Integration Records as XML Definitions](#) and [integration](#)
- API secret, see [Secrets as XML Definitions](#) and [secret](#)

- Single page application (SPA), see [Single Page Applications as XML Definitions](#) and [singlepageapp](#)

Note: SPA objects cannot be copied using Copy to Account.

- Custom tool script, see [Custom Tool Scripts as XML Definitions](#) and [tool](#)

Note: Custom tool objects cannot be copied using Copy to Account.

Plug-ins

- Plug-in – Financial Institution Connectivity, see [Financial Institution Connectivity Plug-ins as XML Definitions](#) and [ficonnectivityplugin](#)
- Plug-in – Financial Institution Parser, see [Financial Institution Parser Plug-ins as XML Definitions](#) and [fiparserplugin](#)
- Plug-in – Bank Statement Parser, see [Bank Statement Parser Plug-ins as XML Definitions](#) and [bankstatementparserplugin](#)
- Plug-in – Custom GL Lines, see [Custom GL Lines Plug-ins as XML Definitions](#) and [customglplugin](#)
- Plug-in – Email Capture, see [Email Capture Plug-ins as XML Definitions](#) and [emailcaptureplugin](#)
- Plug-in – Dataset Builder, see [Dataset Builder Plug-ins as XML Definitions](#) and [datasetbuilderplugin](#)
- Plug-in – Workbook Builder, see [Workbook Builder Plug-ins as XML Definitions](#) and [workbookbuilderplugin](#)
- Plug-in – Custom Plug-in Type, see [Creating a Custom Plug-in Type in an XML Definition](#) and [plugintype](#)
- Plug-in – Custom Plug-in Implementation, see [Creating a Custom Plug-in Implementation in an XML Definition](#) and [pluginimplementation](#)

Centers and Tabs

- Center, see [Centers as XML Definitions](#) and [center](#)
- Center category, see [Center Categories as XML Definitions](#) and [centercategory](#)
- Center tab, see [Center Tabs as XML Definitions](#) and [centertab](#)
- KPI Scorecard, see [KPI Scorecards as XML Definitions](#) and [kpiscorecard](#)

Users/Roles

- Role, see [Custom Roles as XML Definitions](#) and [role](#)

Templates

- Email template, see [Email Templates as XML Definitions](#) and [emailtemplate](#)
- Advanced HTML/PDF template, see [Advanced HTML/PDF Templates as XML Definitions](#) and [advancedpdftemplate](#)

Reports

- Financial reports, see [Financial Reports as XML Definitions](#), [reportdefinition](#) and [financiallayout](#)

SuiteCommerce Web Site Management

- CMS Content Type (also known as Custom Content Type), see [CMS Content Types as XML Definitions](#) see the help topic [cmscontenttype](#)
- SSP Application, see [SSP Applications as XML Definitions](#) and [sspapplication](#)

SuiteAnalytics Workbooks

- Dataset, see [Custom Dataset Objects](#) and [dataset](#)
- Workbook, see [Workbooks as XML Definitions](#) and [workbook](#)

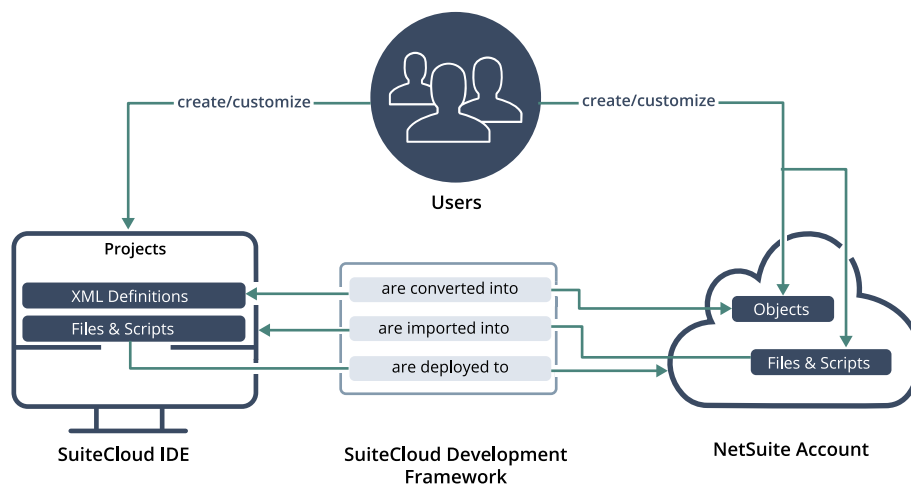


Important: Do not modify the contents of the definition structure or its character data. Modifications may damage the workbook data when you deploy the modified object to a target NetSuite account.

You should use NetSuite to modify custom workbooks rather than using SDF to modify workbook objects. After you save your changes in NetSuite and re-import the workbook into your SuiteCloud project, the workbook object gets updated safely and automatically.

SuiteCloud Development Framework Architecture

SuiteCloud Development Framework (SDF) acts as a deployment mechanism that operates between the SuiteCloud SDK and a NetSuite account. With this mechanism, you can work with account components locally in the SuiteCloud SDK tools. **NetSuite components**, sometimes referred to as customizations, consist of files, scripts, and custom objects such as custom records, custom forms, and workflows.



When you use SuiteCloud SDK to create SuiteCloud projects, you can create, import, and customize account components. NetSuite objects and record types are represented as XML definitions in SuiteCloud projects.

When you deploy SuiteCloud projects to a NetSuite account, SDF converts the XML definitions to NetSuite objects and record types. The SDF custom objects overwrite earlier versions of objects that have the same script IDs in the target NetSuite account. The updated files and scripts overwrite earlier versions of files and scripts with the same names in the file cabinet.

You can customize account components either in a SuiteCloud project or in a NetSuite account. SDF includes a validation process that informs you of any differences or conflicts between SuiteCloud components and account components.

SuiteCloud Development Framework Setup

To enable SuiteCloud Development Framework (SDF), administrative tasks must first be performed on the target NetSuite account. For information about how an account administrator can set up the requirements for SDF, see the following topics:

- [Enabling SuiteCloud Development Framework in the Target NetSuite Account \(Administrator Only\)](#)
- [Assigning the Developer Role \(Administrator Only\)](#)
- [SuiteCloud Development Framework Account Preferences \(SDF Developers Only\)](#)

Next, download a SuiteCloud IDE plug-in or extension and add the target NetSuite account. For experienced users, SuiteCloud CLI is available to use with your own IDE. For more information, see the following topics:

- [Installing and Setting Up SuiteCloud Extension for Visual Studio Code](#)
- [Installing and Setting Up SuiteCloud IDE Plug-in for WebStorm](#)
- [SuiteCloud CLI for Java Guide](#)
- [SuiteCloud CLI for Node.js Guide](#)

Enabling SuiteCloud Development Framework in the Target NetSuite Account (Administrator Only)

To use SuiteCloud Development Framework (SDF) with a target NetSuite account, certain features must be enabled by an account administrator. For information about enabling features, see:

- [Enabling Features](#)
- [SuiteCloud Features](#)

To set up a NetSuite account for SDF

1. Log in to NetSuite with an administrator role.
2. Go to Setup > Company > Enable Features.
3. On the **SuiteCloud** tab, enable the following features:
 - Custom Records
 - SuiteCloud Development Framework
 - To use the OAuth 2.0 protocol with SuiteCloud SDK, enable the OAuth 2.0 feature in the account. For more information, see the help topic [OAuth 2.0 Authentication for SuiteCloud SDK](#).
 - To use custom code and server SuiteScript, enable **Client SuiteScript** and **Server SuiteScript**.

- To use Transaction fields, enable **Item Options**.
- To use Custom Transactions, enable **Custom Transactions**.
- To use advanced PDF/HTML templates, enable **Advanced PDF/HTML Templates**.
- To use SSP Applications, enable **SuiteScript Server Pages**.

 **Note:** If you use SSP applications, the SuiteCommerce module must be provisioned to the NetSuite account.

- To use Workflows, enable **SuiteFlow**.
- To use Custom Segments, enable **Custom Segments**.
- To use the Workbook API, including Dataset Builder Plug-ins and Workbook Builder Plug-ins, enable **Workbook API**.

On the **Company** tab, to use Translation Collections, enable **Multi-Language**.

4. Depending on the types of records you work with, enable the following features:

- To use KPI scorecards, on the **Analytics** tab, enable **KPI Scorecards**.
- To use SuiteAnalytics Workbook or the Workbook API, on the **Analytics** tab, enable **SuiteAnalytics Workbook**.
- To use address forms, go to the **Company** tab. On the **Classifications** Subtab, enable **Locations**.
- To use SSP Applications, on the **Web Presence** tab, enable **Web Site** and **Host HTML Files**.
- To use CMS Content Types, on the **Web Presence** tab, enable **Site Management Tools**.

 **Note:** Site management tools are only available in SuiteCommerce Advanced accounts. For more information, see the help topic [Upgrade SuiteCommerce to SuiteCommerce Advanced](#).

Next, you should create and assign a customized Developer role to use with the account for SDF development. For more information, see [Assigning the Developer Role \(Administrator Only\)](#).

Assigning the Developer Role (Administrator Only)

The account administrator should assign SuiteCloud Development Framework (SDF) users a Developer role that provides full or partial access to account customization, depending on business needs and security policy. The standard Developer role includes a set of permissions that provide the ability to create, modify, and view the scripts, files, and SDF custom objects in SuiteCloud projects. For the full list of permissions given to the standard Developer role, see the help topic [Standard Roles Permissions Table](#).

Permissions cannot be changed or added to the standard role. To edit permissions, you can customize the Developer role.

 **Important:** To use or customize the Developer role, the SuiteCloud Development Framework (SDF) feature must be enabled in the target NetSuite account. For more information, see [Enabling SuiteCloud Development Framework in the Target NetSuite Account \(Administrator Only\)](#).

The following topics provide information for assigning and customizing the Developer role:

- [Assigning the Standard Developer Role \(Administrator Only\)](#)
- [Customizing the Developer Role \(Administrator Only\)](#)

Assigning the Standard Developer Role (Administrator Only)

Administrators can assign the standard Developer role to employees. If you use the standard Developer role, permissions are dynamically adjusted when you enable or disable features in the target account.

To assign a standard Developer role to a target NetSuite account:

1. Log in to NetSuite with an administrator role.
2. Go to Lists > Employees > Employees.
3. Click **Edit** next to the name of the employee you want to assign the Developer role to.
4. Click the **Access** subtab.
5. In the **Role** field, select the **Developer** role for this employee.
6. Click **Add**.
7. Click **Save**.

For more information, see the help topic [Assigning Roles to an Employee](#).

Customizing the Developer Role (Administrator Only)

To change permissions from those provided by the standard Developer role, you can create and assign a custom Developer role. The new customized role inherits the standard Developer role's permissions.

Note: On custom Developer roles, permissions are not updated dynamically when features are enabled in the target account. Permissions are set manually on a custom Developer role. For convenience, you should enable all the features you need for working in SDF before customizing the Developer role. See [Enabling SuiteCloud Development Framework in the Target NetSuite Account \(Administrator Only\)](#).

For more information about customizing roles, see:

- [Customizing or Creating NetSuite Roles](#)
- [NetSuite Permissions Overview](#)

To create a custom Developer role from a target NetSuite account:

1. Log in to NetSuite with an administrator role.
2. Go to Setup > Users/Roles > Manage Roles.
3. Click the **Customize** link next to the Developer role.
4. Under **Permissions**, add and set the level of access for other permissions you want for the role.
5. Click **Save**.

After creating a custom Developer role, you can assign the role to employees. For more information, see the help topic [Assigning Roles to an Employee](#).

To use the OAuth 2.0 protocol with SuiteCloud SDK, enable the OAuth 2.0 feature in the account. For more information, see the help topic [OAuth 2.0 Authentication for SuiteCloud SDK](#).

SuiteCloud Development Framework Account Preferences (SDF Developers Only)

SuiteCloud Development Framework (SDF) preferences for the target NetSuite account are specified on the General subtab of the Set Preferences page.

To access the Set Preferences page, go to Home > Set Preferences. The General subtab is displayed by default. On this subtab, you can set SDF preferences in the SuiteCloud Development Framework section.

In this section, you can enable the following preferences:

- **Show ID Field on Sublists** — This preference displays script ID field in sublists that are supported by SDF.
- **Show App ID Field** — This preference displays the App ID in NetSuite customizations that belong to an SDF SuiteApp that was installed on the account.

In addition to enabling SDF preferences, you should show internal IDs in the target account. For more information, see the following:

- [Showing Record and Field IDs in Your Account](#)
- [General Personal Preferences](#)

For information about setting preferences in SuiteCloud IDE Plug-in for WebStorm, such as file backups, code templates, and validation, see the following:

- [Setting Up SuiteCloud IDE Plug-in for WebStorm](#)
- [Setting Installation Preferences in SuiteCloud IDE Plug-in for WebStorm](#)

For information about enabling permissions, see [Assigning the Developer Role \(Administrator Only\)](#).

SuiteCloud Project Types

You can use SuiteCloud Development Framework (SDF) to create **SuiteCloud project** components, sometimes referred to as customizations, for internal use within your organization or for commercial distribution. These file-based projects use XML definitions of NetSuite objects and record types, such as custom records, custom forms, and workflows. SuiteCloud projects also support SuiteScript files. You can use SDF to deploy SuiteCloud projects to a NetSuite account.

 **Note:** SuiteCloud projects were formerly referred to as SDF projects.

You can create the following SuiteCloud project types:

- [Account Customization Projects](#)
- [SuiteApp Projects](#)

Account customization projects are intended for customizations on accounts you own, such as your production, development, and sandbox account types.

SuiteApp Projects are for SuiteCloud Developer Network (SDN) members to develop and deploy SDF SuiteApps to their NetSuite accounts.

For information about the directory structure and files in SuiteCloud projects, see [SuiteCloud Project Structure and File Components](#).

For information that compares the SuiteCloud project types, see [Differences Between SuiteCloud Projects](#).

Account Customization Projects

An account customization project is a type of SuiteCloud project for SuiteCloud Development Framework (SDF). Account customization projects are intended for customizations on accounts you own, such as your production, development, and sandbox account types.

Note: SuiteCloud projects were formerly referred to as SDF projects.

To create an account customization project using SuiteCloud SDK tools, see the following topics:

- [Properties of an Account Customization Project](#)
- [Creating a SuiteCloud Project in SuiteCloud Extension for Visual Studio Code](#)
- [Creating an Account Customization Project in SuiteCloud IDE Plug-in for WebStorm](#)
- For CLI for Java, see the help topic [createproject](#)
- For CLI for Node.js, see the help topic [project:create](#)

Note: The maximum file size of your SuiteCloud project (compressed as a zip file during deployment) is 50 MB. To maintain a small project size, you should use File Cabinet to upload large files and images to NetSuite rather than SDF.

For information about creating and deploying a SuiteCloud project to a target NetSuite account, see [SuiteCloud Development Framework Tutorial](#).

Important: Account customization projects cannot be uninstalled from the target account after they have been deployed. Unwanted account components need to be manually deleted.

Properties of an Account Customization Project

The following table lists the properties which must be specified when you create an account customization project.

Property	Description
Project Name	The name of your project.

SuiteApp Projects

A SuiteApp project is a type of SuiteCloud project for SuiteCloud Development Framework (SDF). As a SuiteCloud Developer Network (SDN) member, you can use SuiteApp projects to create SDF SuiteApps. SuiteApp projects are self-contained, standalone projects you can use to develop and deploy SDF SuiteApps to target NetSuite accounts.

Note: SuiteCloud projects were formerly referred to as SDF projects.

SuiteApp projects can include SDF installation scripts that are automatically executed when the project is deployed. For more information, see the help topic [SuiteScript 2.x SDF Installation Script Type](#).

Note: To create an SDF SuiteApp, you must provide the publisher ID that SDN has assigned to you. Contact SDN if you are a partner and you do not know your publisher ID.

SuiteApp Project Creation

For SuiteApp project requirements, see the following topics:

- [Properties of a SuiteApp Project](#)

- SuiteApp Application ID
- Publisher ID

To publish a SuiteApp using SDF, you need to use the SuiteApp project type. The following topics cover how to create a SuiteApp project using your preferred NetSuite development tool:

- [Creating a SuiteCloud Project in SuiteCloud Extension for Visual Studio Code](#)
- [Creating a SuiteApp Project in SuiteCloud IDE Plug-in for WebStorm](#)
- For CLI for Java, see the help topic [createproject](#)
- For CLI for Node.js, see the help topic [project:create](#)

Properties of a SuiteApp Project

The following table lists the properties which must be specified when you create a SuiteApp project.

After you create a SuiteApp project, its application ID includes the Project ID and Publisher ID. For more information, see [SuiteApp Application ID](#).

Note: The maximum file size of your SuiteCloud project (compressed as a zip file during deployment) is 50 MB. To maintain a small project size, you should use File Cabinet to upload large files and images to NetSuite rather than SDF.

Property	Description
Publisher ID	The publisher ID of your SuiteApp project.  Important: To create an SDF SuiteApp, you must provide the publisher ID that SDN assigned to you.
Project ID	A unique identifier to assign to the project.
Project Name	A name for your project.
Project Version	The version to assign to your project. This value must use semantic versioning. For example, specify the following project version if your SDF SuiteApp is version 1.02, minor revision #01242017: 1 1.02.01242017 Modifying the project version value updates the SuiteApp version in the target account the next time the SuiteApp is deployed. The SuiteApp version can be viewed on the Installed SuiteApps page. For more information, see Viewing and Uninstalling SDF SuiteApps in a NetSuite Account.

SuiteApp Application ID

The SuiteApp workspace name is the application ID, consisting of the following components:



Application ID

A SuiteApp application ID is based on the publisher ID and project ID. SuiteApp project names are automatically set to their application IDs. The application ID is a fully qualified name with the following notation: <publisherid>.<projectid>. For example:

- com.example.mysuiteapp
- com.mycompany.helloworld



Important: Application IDs must be no longer than 80 characters. SuiteApps created, imported, or converted as SuiteCloud projects are always represented by an application ID.

Publisher ID

A SuiteApp publisher ID is a unique ID that is assigned to each SDN partner. The publisher ID consists of a domain name, such as com or org, and a publisher name, such as the name of your company or organization. This has the notation: <domain>.<company>. For example:

- com.example
- com.mycompany



Important: Publisher ID is a requirement for creating SuiteApp projects. You do not need a publisher ID to create account customization projects.

Publisher IDs are issued and registered by SDN. For more information about SDN, see <http://www.netsuite.com/portal/developers/sdn.shtml>.

Project ID

A SuiteApp project ID refers to the specific SuiteApp project that you want to deploy or publish. This ID is specified when the project is created. The following are project ID examples:

- mysuiteapp
- helloworld



Note: The project ID must be a single word that consists of lowercase alphanumeric characters. It must be less than 40 characters.

Differences Between SuiteCloud Projects

SuiteCloud Development Framework (SDF) supports two types of SuiteCloud projects: account customization projects and SuiteApp projects. These project types are optimized for their intended use. If you have never worked with a SuiteCloud project before, start with [SuiteCloud Project Types](#).



Note: SuiteCloud projects were formerly referred to as SDF projects.

The following table covers supported SDF actions for account customization projects and SuiteApp projects. Review these project type comparisons if:

- You use both types of SuiteCloud projects.

- You want more familiarity with expected SDF behavior.

For information about removing content after it is deployed, depending on project type, see [Deleting Components in a SuiteCloud Project that were Deployed to a Target Account](#).

Action	Account Customization Project	SuiteApp Project
Adding Dependencies to the Manifest	Supported You can add feature, file, and object dependencies to the Manifest file. For more information, see Account Component Dependencies in SuiteCloud Projects , SDF Custom Object Dependencies in SuiteApps , and Feature Dependencies for SDF Custom Objects in SuiteCloud Projects .	Partially Supported You can add feature dependencies and external SDF object in a SuiteApp dependencies to the Manifest file. For more information, see SDF Custom Object Dependencies in SuiteApps and Feature Dependencies for SDF Custom Objects in SuiteCloud Projects . Dependencies on files or objects that are native to the target account are not supported.
Configuring Account Features	Supported For more information, see Account Configuration from an Account Customization Project	Not Supported
Editing Custom Transaction Status from the NetSuite Account UI	Allowed	Not allowed A custom transaction status sublist is not editable in the UI if it is part of a custom transaction originating from a SuiteApp.
Hiding Files	Not Supported	Supported For more information, see Installation Preferences in the SuiteApp of SuiteCloud Project .
Importing Bundle Components	Supported For more information, see Conversion of a Bundle to an Account Customization Project for SDF	Not Supported
Importing Files	Supported You can import files into the account customization project from a NetSuite account.  Note: The maximum file size of your SuiteCloud project (compressed as a zip file during deployment) is 50 MB. To maintain a small project size, you should use File Cabinet to upload large files and images to NetSuite rather than SDF.	Not Supported
Locking Objects	Not Supported	Partially Supported For more information, see Lockable SDF Custom Objects and Installation Preferences in the SuiteApp of SuiteCloud Project .

Action	Account Customization Project	SuiteApp Project
Importing Objects	Partially Allowed You can import objects that are native to the account and not installed by a SuiteApp.	Allowed You can import objects that are native to the account or installed by a SuiteApp.
Moving and Copying Files	Allowed When you move files in an account customization project and deploy the change, SDF creates copies in the new location and original files remain in the original location in the NetSuite account. Note: Moving files in the project results in duplicated files in the target NetSuite account. You can manually remove the duplicated files from the File Cabinet.	Allowed When you move files in a SuiteApp project and deploy the change, SDF creates copies in the new location, and removes the original files from the original location. Note: Moving files in the project results in deletion of the original files.
Referencing Files	Allowed You can reference files in the project or in the account.	Partially Allowed You can reference files that exist within the same SuiteApp project.
Referencing Objects	Allowed An account customization project can reference objects in the same project, or objects in the account. The object must be declared as a dependency in the manifest file when from an account.	Partially Allowed A SuiteApp project can reference objects in the project or in a SuiteApp installed to the account.
Using Code Completion for File References	When you specify a file reference from the IDE plug-in, SDF lists files that are present in the project and files that are defined as dependencies in the Manifest file.	When you specify a file reference from the IDE plug-in, SDF can suggest only files that are present in the project.
Using Code Completion for Script ID Attributes	When you specify a script ID attribute from the IDE plug-in, SDF can suggest script IDs from objects present in the project and objects defined as dependencies in the Manifest file.	When you specify a script ID attribute from the IDE plug-in, SDF can suggest script IDs from objects present in the project and objects defined as dependencies in the Manifest file.
Validating Files	Existing files in the project and files that are defined as dependencies in the manifest file are validated.	Existing files in the project are validated.
Validating Script IDs	Script IDs from existing objects in the project and objects defined as dependencies in the manifest file are validated.	Script IDs from existing objects in the project are validated.
Re-deploying SDF Custom Objects	SDF custom objects in an account customization project always replace the custom objects in the NetSuite account when you deploy, regardless of any previous changes made to those objects in the account.	When you update a previously deployed an SDF custom object in the NetSuite account and re-deploy the same SDF custom object from a SuiteCloud SDK tool, the SDF custom object in the NetSuite account does not get updated.

Deleting Components in a SuiteCloud Project that were Deployed to a Target Account

The content and SuiteCloud project type affects the modifications you can perform using SuiteCloud Development Framework (SDF).

Components that are created in an account customization project cannot be uninstalled after they are deployed to a target account. You must manually delete the components from the target NetSuite account.

When a SuiteApp project is deployed to a target account, any object with the same application ID that already exists in the account and is not part of the deployment is deleted.

See the following table for details and exceptions when deleting components from a SuiteCloud project.

Action	Account Customization Project	SuiteApp Project
Deleting Files	Not Allowed Files are not deleted in the account.	Partially Allowed Only .js files are removed from the File Cabinet in the account when you deploy a SuiteApp project that does not contain those .js files. Other file types are not removed from the File Cabinet in the account when you deploy a SuiteApp project that does not contain them. For example, if you rename a file in the project and redeploy the project, the File Cabinet will now have two versions of the file.  Note: If the SuiteApp is uninstalled from the account, files under the SuiteApps folder for that SuiteApp are deleted.
Deleting Instances	Not Allowed Custom record type object instances are not deleted in the account.	Partially Allowed Regular custom record type object instances are deleted when a new version of the SuiteApp project is deployed and it does not either contain the object or reference the object in the deploy.xml file. Custom record type objects that are associated with custom segments behave differently. The instances are not deleted because this behavior would impact the GL.
Deleting Objects	Not Allowed To help protect data in the NetSuite account, when you delete an object from an account customization project and deploy the project, the object is not deleted from the account. You must manually delete objects directly from the NetSuite account.	Allowed Objects are deleted when a new version of the SuiteApp project is deployed and it does not either contain the object or reference the object in the deploy.xml file.

Action	Account Customization Project	SuiteApp Project
	 Important: When an object is deployed to an account that contains a different type of object with the same ID, the object in the account is deleted.	
Deleting Sublist Items	Allowed When you delete a sublist item from an account customization project and deploy the project, only the sublist items in the project can exist in the account. Any sublist items that already exist in the account are deleted. The permissions in the target account will not be changed or removed for role permissions that exist in the target account but not in the permission field group of the following SDF custom objects in the account customization project (ACP): customrecordtype, customsegment, customtransactiontype. The permissions in the target account will be changed to the XML values for the permissions in the account customization project (ACP) for role permissions that exist in both the permission field group of the following SDF custom objects in the ACP and the target account: customrecordtype, customsegment, customtransactiontype.	Partially Allowed Sublists or sublist line items are deleted when a new version of the SuiteApp project is deployed and it does not either contain the sublist or line item or reference the sublist or line item in the deploy.xml file. If there are existing line items defined in the account, they are retained. If a SuiteApp is updated and it contains sublists without script IDs, only those sublist line items still included in the SuiteApp XML will remain. Existing line items defined in the account only are deleted. If the entire SuiteApp is removed or uninstalled, the sublist and its line items are deleted from the XML and account. The permissions in the target account will not be changed or removed for role permissions that exist in the target account but not in the permission field group of the following SDF custom objects in the SDF SuiteApp (or SuiteApp project): customrecordtype, customsegment, customtransactiontype. The permissions in the target account will be changed to the XML values for the permissions in the SDF SuiteApp or SuiteApp project for role permissions that exist in both the permission field group of the following SDF custom objects in the SDF SuiteApp (or SuiteApp project) and the target account: customrecordtype, customsegment, customtransactiontype.
Deleting Subrecords	Not Allowed Subrecords are not deleted in the account.  Important: Exceptions can occur for the system to enforce uniqueness of an ID. When a subrecord is deployed to an account that contains a different type of subrecord with the same ID, the subrecord in the account is deleted. For example: When an existing workflow action has the same script ID as a new workflow action, and the workflow actions are under the same workflow, but the two workflow actions have different action types, the existing workflow action in the account	Allowed Subrecords in the SuiteApp are deleted when a new version of the SuiteApp project does not contain either the subrecord or reference to the subrecord in the deploy.xml file.

Action	Account Customization Project	SuiteApp Project
	is deleted prior to creation of the new workflow action. When an existing script deployment has the same record type as a new script deployment, and both deployments are under the same script record, the existing script deployment in the account is deleted prior to creation of the new script deployment.	

SuiteCloud Project Structure and File Components

In SuiteCloud Development Framework (SDF), all SuiteCloud components are stored in a file system SuiteCloud project instead of the target NetSuite account. This file storage facilitates the use of standard software development processes during project development, including team-based development, standard quality assurance and integration testing processes, and the use of third-party revision control systems.

SuiteCloud projects consist of the following hierarchy of files used to represent schema, logic, project properties, and deployment directives:

- **Manifest File** – contains the project metadata. This metadata includes the project name, framework version, and any dependencies on features, records, or files. The manifest filename is `manifest.xml` and located at the root of the SuiteCloud project folder. For additional information about using the manifest file, see the following topics:
 - [Manually Defining Feature Dependencies in the SuiteCloud Project Manifest](#)
 - [Manually Defining SDF Custom Object Dependencies in a SuiteApp Using the SuiteCloud Project Manifest](#)
- **Deploy File** – references the file paths used in the project. This file specifies the order in which files are uploaded to the File Cabinet, and in which objects are created in the account. The deploy filename is `deploy.xml` and located at the root of the SuiteCloud project folder. For more information, see [SuiteCloud Project Deployment Preparation](#).
- **Object Files** – are the XML definitions of SDF custom objects, such as custom records, custom forms, and workflows. These files contain all of the properties that are needed to create or modify objects in the target NetSuite account. The object files are stored in the `Objects` folder, located at the root of the SuiteCloud project folder. For more information about defining supported SDF custom objects in your SuiteCloud project, see the following topics:
 - [SDF Custom Object File Structure](#)
 - [Customizations Supported by SuiteCloud Development Framework](#)
- **File Cabinet Files** – are the files and SuiteScripts to be deployed to the File Cabinet in the target NetSuite account. File links are not supported by SDF. For information about the attributes that are available for file cabinet files in your SuiteCloud project, see [Modifying SuiteCloud Project File and Folder Attributes in an XML Definition](#).

For account customization projects, all files must be physically stored in one or more of the following folders of your SuiteCloud project workspace:

- `FileCabinet/SuiteScripts/`
- `FileCabinet/Templates/E-mail Templates/`
- `FileCabinet/Templates/Marketing Templates/`
- `FileCabinet/Templates/Web Site Hosting Files/`

- FileCabinet/Web Site Hosting Files/*/SSP Applications/<app id>/



Note: You must enable the Customer Relationship Management feature to access the E-mail Templates record. To enable the feature, go to Setup > Company > Enable Feature. Click the **CRM** subtab and check **Customer Relationship Management**.

For SuiteApp projects, all files must be physically stored in the FileCabinet/SuiteApps/<appid> folder of your SuiteCloud project workspace:

- FileCabinet/SuiteApps/<app id>/
- FileCabinet/Web Site Hosting Files/*/SSP Applications/<app id>/
- **Translations Files** — are the XLIFF files referenced in translationcollection objects. These files contain the translated strings for a target language. For more information, see [Translation Collections as XML Definitions](#).
- **Account Configuration Files (Account customization projects only)** – are XML definitions of account configuration settings that can be modified in an account customization project. Account features can be enabled or disabled in the AccountConfiguration/features.xml file. For more information, see [Account Configuration from an Account Customization Project](#).
- **Installation Preferences Files (SuiteApp projects only)** – are XML definitions of installation preference settings that can be modified in a SuiteApp project. Installation preferences are specified in the InstallationPreferences folder. They are enabled or disabled using the Apply Installation Preferences box that appears when you deploy, validate, or preview a SuiteApp project. For more information, see [Installation Preferences in the SuiteApp of SuiteCloud Project](#).

SuiteCloud Development Framework Tutorial

This tutorial is intended as a starting point to familiarize yourself with the development process for SuiteCloud Development Framework (SDF).

In this tutorial, you create a sample account customization project using the SuiteCloud Extension for Visual Studio Code or the SuiteCloud IDE Plug-in for WebStorm that includes the following components:

- Custom record
- SuiteScript 2.x user event script
- Script record
- Script deployment record

The custom record is created from an XML template that includes one custom text field. In this project, you create a user event script object that sets a default value for the text field each time the custom record loads in the UI. You also set up a script record and a script deployment record for this script.

The following table describes the tutorial steps. Each step builds on the previous step, so you must complete them in order.

Step	Description
Step One: Set Up a NetSuite Account (Administrator Only)	Enable the SuiteCloud Development Framework feature on the target NetSuite account.
Step Two: Set Up Your IDE Environment	Set up your IDE and SuiteCloud IDE for use with SuiteCloud projects.
Step Three: Create an Account Customization Project	Create and set up a new SuiteCloud project in SuiteCloud IDE.

Step	Description
Step Four: Create a Custom Record Definition	Set up an XML definition of a custom record object.
Step Five: Create a User Event Script	Use SuiteScript 2.x to create a user event script.
Step Six: Create a Script Record and Script Deployment Definition	Set up an XML definition of a script deployment object that deploys the user event script.
Step Seven: Validate and Deploy the SuiteCloud Project to a Target NetSuite Account	Validate SuiteCloud project to ensure that there are no errors, and then deploy it to a target account.
Step Eight: Test the SuiteCloud Project in the Target NetSuite Account	Verify that the SuiteCloud project was deployed to the target account and functions correctly.

This tutorial assumes that you have Administrator access to the target account, and familiarity with SuiteScript 2.x and JavaScript concepts. For more information about SuiteScript 2.x, see the help topics [SuiteScript 2.x](#) and [SuiteScript Versioning Guidelines](#).

Set Up a NetSuite Account (Administrator Only)

The following steps show you how to enable the SuiteCloud Development Framework (SDF) feature in a target NetSuite account.



Tip: Development accounts can be used with SDF. For information about who can benefit from using development accounts and how to get one, see the help topic [Uses for Development Accounts](#).

To set up a NetSuite account:

1. Enable SuiteCloud Development Framework and feature dependencies in the target NetSuite account.
For more information, see [Enabling SuiteCloud Development Framework in the Target NetSuite Account \(Administrator Only\)](#).
2. Assign a customized Developer role to employees that should use SDF. All remaining steps in this tutorial should be performed using the Developer role instead of the Administrator role.
For more information, see [Assigning the Developer Role \(Administrator Only\)](#) and [Assigning Roles to an Employee](#).

Continue to Step Two: [Set Up Your IDE Environment](#).

Set Up Your IDE Environment

This topic is step two of the [SuiteCloud Development Framework Tutorial](#). If you have not already done so, complete step one: [Set Up a NetSuite Account \(Administrator Only\)](#).

Depending on your IDE, refer to one of the following procedures:

- [To set up your Visual Studio Code environment](#)
- [To set up your WebStorm environment](#)

To set up your Visual Studio Code environment

1. Verify that all software prerequisites are met.
For more information, see the help topic [Installation Prerequisites for SuiteCloud Extension for Visual Studio Code](#).
2. Download and install Visual Studio Code.
For more information, see the help topic [Downloading Visual Studio Code](#).
3. Download and install SuiteCloud Extension for Visual Studio Code.
For more information, see the help topic [Installing SuiteCloud Extension for Visual Studio Code](#).
4. To use the OAuth 2.0 protocol with SuiteCloud Extension for Visual Studio Code, enable the OAuth 2.0 feature in the account. For more information, see the help topic [OAuth 2.0 Authentication for SuiteCloud SDK](#).

To set up your WebStorm environment

1. Verify that all software prerequisites are met.
For more information, see the help topics [Installation Requirements for SuiteCloud IDE Plug-in for WebStorm](#) and [Supported Operating Systems for SuiteCloud IDE Plug-in for WebStorm](#).
2. Download and install WebStorm.
For more information, see the help topic [Downloading WebStorm](#).
3. Download and install SuiteCloud IDE plug-in.
For more information, see the help topic [Adding SuiteCloud IDE Plug-in to WebStorm](#).
4. Set up the IDE plug-in. For more information, see the help topic [Setting Up SuiteCloud IDE Plug-in for WebStorm](#).
5. To use the OAuth 2.0 protocol with SuiteCloud IDE, enable the OAuth 2.0 feature in the account.
For more information, see the help topic [OAuth 2.0 Authentication for SuiteCloud SDK](#).

Continue to Step Three: [Create an Account Customization Project](#).

Create an Account Customization Project

This topic is step three of the [SuiteCloud Development Framework Tutorial](#). If you have not already done so, complete step two: [Set Up Your IDE Environment](#).

The following steps show you how to create an account customization project in your IDE. For more information about account customization projects, see [SuiteCloud Project Types](#).

Depending on your IDE, refer to one of the following procedures:

- [To create an account customization project in Visual Studio Code](#)
- [To create an account customization project in WebStorm](#)
- For CLI for Java, see the help topic [createproject](#)
- For CLI for Node.js, see the help topic [project:create](#)

To create an account customization project in Visual Studio Code

1. Press Ctrl+Shift+P to open the Command Palette and enter suitecloud.
From the dropdown list, select **SuiteCloud: Create Project**.
2. Select the parent folder for your new SuiteCloud project, and then click **Select Folder**.

Note: You can select the same parent folder for different SuiteCloud projects, because each project is automatically created in a subfolder of the selected parent folder. Do not create a SuiteCloud project in another SuiteCloud project subfolder.

3. For the type of project you want to create, select **Account Customization**.
4. For the project name, enter SDFTutorial1 and press Enter.
5. Do not specify to use Jest testing framework.

The SuiteCloud project is created for you, with the operation status displayed in the **Output** tab. When the extension is finished creating the SuiteCloud project, you can choose how to open the project folder by selecting either **This Window** or **New Window**. If there is already a folder open in the current window, selecting **This Window** closes that folder and opens the one for the project you created.

To create an account customization project in WebStorm

1. Select **File > New > Project...**
2. From the **New Project** dialog, in the left-hand panel, select **Account Customization**.
3. In the **Project Name** field, enter SDFTutorial1, and click **Create**.

SuiteCloud IDE creates a new SuiteCloud project.

Continue to Step Four: [Create a Custom Record Definition](#).

Create a Custom Record Definition

This topic is step four of the [SuiteCloud Development Framework Tutorial](#). If you have not already done so, complete step three: [Create an Account Customization Project](#).

SuiteCloud IDE plug-in includes templates for each supported object type and includes code completion for all supported object definitions. You can use these templates as a base to define SDF custom objects for a SuiteCloud project. In this manner, you can add or edit SDF custom objects in the SuiteCloud IDE plug-in.

The following steps show you how to create a custom record object that contains a single text field. For more information about custom records, see the help topic [Custom Records](#).

Depending on your IDE, refer to one of the following procedures:

- [To create a custom record definition in Visual Studio Code](#)
- [To create a custom record definition in WebStorm](#)

To create a custom record definition in Visual Studio Code

1. With the SDFTutorial1 folder open in Visual Studio Code, select **File > New File**.
2. In the Command Palette, select **Text File**.

Note: This procedure manually creates the custom record definition. Although you cannot create a custom record definition from a template using SuiteCloud Extension for Visual Studio Code, you can import a custom record definition from your NetSuite account. For more information, see the help topic [Importing SDF Custom Objects from a NetSuite Account with SuiteCloud Extension for Visual Studio Code](#).

3. Enter the following code into the new file:

```

1 <customrecordtype scriptid="customrecord_tutorial">
2   <description></description>
3   <recordname>SDF Tutorial</recordname>
4   <customrecordcustomfields>
5     <customrecordcustomfield scriptid="custrecord_tut_memo">
6       <description></description>
7       <displaytype>NORMAL</displaytype>
8       <fieldtype>TEXT</fieldtype>
9       <label>Memo</label>
10      <ismandatory>F</ismandatory>
11    </customrecordcustomfield>
12  </customrecordcustomfields>
13 </customrecordtype>

```

4. Save the file as `customrecord_tutorial.xml` in the subfolder `SDFTutorial/src/Objects`.
5. Press `Ctrl+Shift+P` to open the Command Palette, and enter `suitecloud`. From the dropdown list, select **SuiteCloud: Add Dependency References to the Manifest**. For more information, see the help topic [Adding Dependencies to the Manifest in SuiteCloud Extension for Visual Studio Code](#).



Tip: You can also use the following keyboard shortcuts to add dependencies:

- Linux: Press `Ctrl+Shift+Alt+A`
- Mac OS: Press `Ctrl+Alt+Cmd+A`
- Windows: Press `Ctrl+Shift+Alt+A`

The command adds the following code to the manifest:

```

1 <dependencies>
2   <features>
3     <feature required="true">CUSTOMRECORDS</feature>
4   </features>
5 </dependencies>

```

To create a custom record definition in WebStorm

1. Right-click the **Objects** folder within the `SDFTutorial` project, and select **New > Custom Object**.
2. Set the following properties:
 - From the **Object Type** list, select **Record Type**.
 - In the **Script ID** field, enter `customrecord_tutorial`.
3. Click **OK**.

The `customrecord_tutorial` object is created based on a template that partially defines a single custom record field. The new file contains errors that are resolved when you specify a record name and a label for the custom record field.

4. Specify the values of the `recordname` and `label` tags, and update the `script ID` attribute of the `customrecordcustomfield` tag.

Your `customrecord_tutorial.xml` file should look like the following code sample:

```

1 <customrecordtype scriptid="customrecord_tutorial">
2   <allowattachments>T</allowattachments>
3   <allowinlinedetaching>T</allowinlinedetaching>
4   <allowquickadd>T</allowquickadd>
5   <enableidle>T</enableidle>
6   <enablekeywords>T</enablekeywords>

```

```

7      <enableoptimisticlocking>T</enableoptimisticlocking>
8      <enablesystemnotes>T</enablesystemnotes>
9      <iconbuiltin>T</iconbuiltin>
10     <includeinsearchmenu>T</includeinsearchmenu>
11     <includename>T</includename>
12     <shownotes>T</shownotes>
13     <recordname>SDF Tutorial</recordname>
14     <customrecordcustomfields>
15         <customrecordcustomfieldscriptid="custrecord_tut_memo">
16             <accesslevel>2</accesslevel>
17             <description></description>
18             <displaytype>NORMAL</displaytype>
19             <fieldtype>TEXT</fieldtype>
20             <label>MEMO</label>
21             <searchlevel>2</searchlevel>
22             <storevalue>T</storevalue>
23         </customrecordcustomfield>
24     </customrecordcustomfields>
25 </customrecordtype>

```

The preceding code sample creates a custom record type object for an SDF Tutorial record with the customrecord_tutorial1 script ID. The record contains a text field with the custrecord_tut_memo script ID.

5. Save the file.

Continue to Step Five: [Create a User Event Script](#).

Create a User Event Script

This topic is step five of the [SuiteCloud Development Framework Tutorial](#). If you have not already done so, complete step four: [Create a Custom Record Definition](#).

User event scripts are executed on the NetSuite server when users perform certain actions, such as create, load, update, copy, or delete. In this step, you create a user event script that sets the value of the custom memo field each time a user loads the SuiteCloud Development Framework (SDF) Tutorial record in the UI. For more information about user event scripts, see the help topics [SuiteScript 2.x User Event Script Type](#) and [SuiteScript 2.x User Event Script Entry Points and API](#).

The following steps show you how to create a user event script file. Depending on your IDE, refer to one of the following procedures:

- [To create a user event script in Visual Studio Code](#)
- [To create a user event script in WebStorm](#)

To create a user event script in Visual Studio Code

1. With the SDFTutorial1 open in Visual Studio Code, press Ctrl+Shift+P to open the Command Palette and enter suitecloud. Then from the dropdown list, select SuiteCloud: Create SuiteScript File.
2. For the type of SuiteScript file to create, select User Event Script.
3. Without any modules selected, click **OK** to go to the next step.
4. Save the file in the following folder location: /SuiteScripts.
5. Enter the following name for the SuiteScript file: UserEventScript.js.
The SuiteScript file is created in the specified folder location and opened for editing in Visual Studio Code.
6. In the beforeLoad function, enter the following code sample:

```

1 | function beforeLoad(scriptContext) {

```



```

2   if (scriptContext.type === scriptContext.UserEventType.CREATE) {
3       var customRecord = scriptContext.newRecord;
4       customRecord.setValue({
5           fieldId: 'custrecord_tut_memo',
6           value: 'My first account customization project'
7       });
8   }
9 }

```

The preceding code sample sets a value for the `custrecord_tut_memo` text field before the SDF Tutorial record type loads. For more information about setting record values, see the help topics [record.Record](#) and [Record.setValue\(options\)](#).

7. Save the file.

To create a user event script in WebStorm

1. Select **FileCabinet** and right-click the **SuiteScripts** folder within the SDFTutorial project. Next, select **New > SuiteScript File**.
2. In the **New SuiteScript File** dialog, do the following:
 - From the **Type** dropdown list, select **User Event Script**.
 - In the **Name** field, enter `UserEventScript`.

3. Click **OK**.

The user event script file is created based on a template that contains user event script entry points.

4. In the `beforeLoad` function, enter the following code:

```

1  function beforeLoad(scriptContext) {
2      if (scriptContext.type === scriptContext.UserEventType.CREATE) {
3          var customRecord = scriptContext.newRecord;
4          customRecord.setValue({
5              fieldId: 'custrecord_tut_memo',
6              value: 'My first account customization project'
7          });
8      }
9  }

```

5. Your code should look like the following code sample:

```

1  /**
2   * @ApiVersion 2.x
3   * @NScriptType UserEventScript
4   * @NModuleScope SameAccount
5   */
6  define([],
7
8      function() {
9
10         /**
11          * Function definition to be triggered before record is loaded.
12          *
13          * @param {Object} scriptContext
14          * @param {Record} scriptContext.newRecord - New record
15          * @param {string} scriptContext.type - Trigger type
16          * @param {Form} scriptContext.form - Current form
17          * @Since 2015.2
18          */
19         function beforeLoad(scriptContext) {
20             if (scriptContext.type === scriptContext.UserEventType.CREATE) {
21                 var customRecord = scriptContext.newRecord;
22                 customRecord.setValue({
23                     fieldId: 'custrecord_tut_memo',
24                     value: 'My first account customization project'

```

```

25     });
26     }
27     }
28
29     /**
30     * Function definition to be triggered before record is loaded.
31     *
32     * @param {Object} scriptContext
33     * @param {Record} scriptContext.newRecord - New record
34     * @param {Record} scriptContext.oldRecord - Old record
35     * @param {string} scriptContext.type - Trigger type
36     * @Since 2015.2
37     */
38     function beforeSubmit(scriptContext) {
39
40     }
41
42     /**
43     * Function definition to be triggered before record is loaded.
44     *
45     * @param {Object} scriptContext
46     * @param {Record} scriptContext.newRecord - New record
47     * @param {Record} scriptContext.oldRecord - Old record
48     * @param {string} scriptContext.type - Trigger type
49     * @Since 2015.2
50     */
51     function afterSubmit(scriptContext) {
52
53     }
54
55     return {
56         beforeLoad: beforeLoad,
57         beforeSubmit: beforeSubmit,
58         afterSubmit: afterSubmit
59     };
60
61 });

```

The preceding code sample sets a value for the `custrecord_tut_memo` text field before the SDF Tutorial record type loads. For more information about setting record values, see the help topics [record.Record](#) and [Record.setValue\(options\)](#).

6. Save the file.

Continue to Step Six: [Create a Script Record and Script Deployment Definition](#).

Create a Script Record and Script Deployment Definition

This is step six of the [SuiteCloud Development Framework Tutorial](#). If you have not already done so, complete step five: [Create a User Event Script](#).

To apply the user event script to the SuiteCloud Development Framework (SDF) Tutorial record type, create a script record for the user event and a script deployment definition. For more information, see the help topics [Script Record Creation](#) and [Script Deployment](#).

The following steps show you how to create a user event script record and script deployment definition. Depending on your IDE, refer to one of the following procedures:

- [To create a script record and script deployment definition in Visual Studio Code](#).
- [To create a script record and script deployment definition in WebStorm](#).

To create a script record and script deployment definition in Visual Studio Code

1. With the SDFTutorial folder open in Visual Studio Code, select **File > New File**.
2. In the Command Palette, select **Text File**.

Note: This procedure manually creates the script record and script deployment record definitions. Although you cannot create those definitions from a template using SuiteCloud Extension for Visual Studio Code, you can import the definitions from your NetSuite account. For more information, see the help topic [Importing SDF Custom Objects from a NetSuite Account with SuiteCloud Extension for Visual Studio Code](#).

3. Enter the following code into the new file:

```

1 <usereventscript scriptid="customscript_userevent">
2   <description></description>
3   <isinactive></isinactive>
4   <name>SDF Tutorial User Event</name>
5   <notifyowner>T</notifyowner>
6   <scriptfile>[/SuiteScripts/UserEventScript.js]</scriptfile>
7   <scriptdeployments>
8     <scriptdeployment scriptid="customdeploy_ue_tut">
9       <isdeployed>T</isdeployed>
10      <loglevel>DEBUG</loglevel>
11      <recordtype>[customrecord_tutorial]</recordtype>
12      <status>TESTING</status>
13    </scriptdeployment>
14  </scriptdeployments>
15 </usereventscript>

```

4. Save the file as customscript_userevent.xml in the subfolder SDFTutorial/src/Objects.
5. Press Ctrl+Shift+P to open the Command Palette, and enter suitecloud. From the dropdown list, select **SuiteCloud: Add Dependency References to the Manifest**. For more information, see the help topic [Adding Dependencies to the Manifest in SuiteCloud Extension for Visual Studio Code](#).

Tip: You can also use the following keyboard shortcuts to add dependencies:

- Linux: Press Ctrl+Shift+Alt+A
- Mac OS: Press Ctrl+Alt+Cmd+A
- Windows: Press Ctrl+Shift+Alt+A

The command adds the following code to the manifest:

```

1 <dependencies>
2   <features>
3     <feature required="true">CUSTOMRECORDS</feature>
4     <feature required="true">SERVERSIDESCRIPING</feature>
5   </features>
6 </dependencies>

```

To create a script record and script deployment definition in WebStorm

1. Select the **Objects** folder within the SDFTutorial project, and right-click **New > Custom Object**.
2. Set the following properties:
 - From the **Object Type** list, select **Script – User Event**.
 - In the **Script ID** field, enter customscript_userevent.
3. Click **OK**.

The customscript_userevent object is based on a template that partially defines a script record. The new file contains errors that get resolved when certain tags are specified.

- Specify the values of the name, scriptfile and recordtype tags, and update the script ID attribute of the scriptdeployment tag.

Your customscript_userevent.xml file should look like the following:

```

1 <usereventscript scriptid="customscript_userevent">
2   <name>SDF Tutorial User Event</name>
3   <notifyowner>T</notifyowner>
4   <scriptfile>[/SuiteScripts/UserEventScript.js]</scriptfile>
5   <scriptdeployments>
6     <scriptdeploymentscriptid="customdeploy_ue_tut">
7       <isdeployed>T</isdeployed>
8       <loglevel>DEBUG</loglevel>
9       <recordtype>[customrecord_tutorial]</recordtype>
10      <status>TESTING</status>
11    </scriptdeployment>
12  </scriptdeployments>
13 </usereventscript>

```

- Save the file.
- Right-click the **manifest.xml** file within the SDFTutorial project, and select **NetSuite > Add Dependency References to Manifest**.

Continue to Step Seven: [Validate and Deploy the SuiteCloud Project to a Target NetSuite Account](#).

Validate and Deploy the SuiteCloud Project to a Target NetSuite Account

This is step seven of the [SuiteCloud Development Framework Tutorial](#). If you have not already done so, complete step six: [Create a Script Record and Script Deployment Definition](#).

The following steps show you how to validate and deploy your project to a target NetSuite account.

Note: If you are using SuiteCloud Extension for Visual Studio Code, validate and deploy your project using the command **SuiteCloud: Deploy Project**. For more information, see the help topic [Deploying a SuiteCloud Project to Your NetSuite Account with SuiteCloud Extension for Visual Studio Code](#).

To validate and deploy the project to a target NetSuite account in WebStorm:

- Right-click the project and go to **NetSuite > Validate Project Against Account**.
- Verify that your project gets validated against the correct account and role, and then click **Validate**.

A window displays the progress of the validation and automatically closes when the validation is complete. The following text should appear in the console window to indicate that validation succeeded:

```

1 Validating using account (ACCOUNTNAME).
2 Validate manifest -- Success
3 Validate deploy file -- Success
4 Validate configuration -- Success
5 Validate objects -- Success
6 Validate files -- Success
7 Validate folders -- Success
8 Validate account settings -- Success
9 Validation COMPLETE

```

- Review the console window in the SuiteCloud IDE to verify that the project is valid. Error messages appear in red.

4. Right-click the project and go to **NetSuite > Deploy**.
5. Verify that your project gets deployed to the correct account and role, and then click **Deploy**.

If you chose to deploy to a NetSuite production account, a notification window appears. Confirm that you want to deploy to the production account to continue the deployment process.

A window displays the progress of the deployment and automatically closes when the deployment is complete.

The console window shows the deployment progress on the server. Use this log to review specific actions and to troubleshoot issues with deployment. The text that appears in the console window should be similar to the following text:

```

1 | 2020-08-09 06:05:10 (PST) Installation started
2 | Info -- Account [(PRODUCTION) ACCOUNTNAME]
3 | Info -- Account Customization Project [SDFtutorial]
4 | Info -- Framework Version [1.0]
5 | Validate manifest -- Success
6 | Validate deploy file -- Success
7 | Validate configuration -- Success
8 | Validate objects -- Success
9 | Validate files -- Success
10 | Validate folders -- Success
11 | Validate account settings -- Success
12 | Begin deployment
13 | Upload file -- ~/FileCabinet/SuiteScripts/UserEventScript.js
14 | Create object -- customrecord_tutorial (customrecordtype)
15 | Create object -- customscript_userevent (usereventscript)
16 | Create object -- customrecord_tutorial.custrecord_tut_memo (customrecordcustomfield)
17 | Create object -- customscript_userevent.customdeploy_ue_tut (scriptdeployment)
18 | 2020-08-09 06:05:37 (PST) Installation COMPLETE (0 minutes 27 seconds)

```

Continue to Step Eight: [Test the SuiteCloud Project in the Target NetSuite Account](#).

Test the SuiteCloud Project in the Target NetSuite Account

This is step eight of the [SuiteCloud Development Framework Tutorial](#). If you have not already done so, complete step seven: [Validate and Deploy the SuiteCloud Project to a Target NetSuite Account](#).

The following steps show you how to verify that all objects have been created in the target NetSuite account and that the user event script has been uploaded.

To test the project in the Target NetSuite account:

1. Log in to the target NetSuite account as an administrator.
2. Verify that UserEventScript.js was uploaded to the account.
Go to Documents > Files > File Cabinet, and then click the **SuiteScripts** folder.
UserEventScript.js should appear in the SuiteScripts folder.
3. Verify that the script record was created.
Go to Customization > Scripting, and then click **Scripts**.
The SuiteCloud Development Framework (SDF) Tutorial User Event script, customscript_userevent, should be listed.
4. Verify that the script deployment record was created.
Go to Customization > Scripting, and then click **Script Deployments**.
The customdeploy_ue_tut script should be listed.
5. Verify that the custom record was created.

Go to Customization > Lists, Records, & Fields, and then click **Record Types**.

The SDF Tutorial script record, `customrecord_tutorial`, should be listed.

6. Verify that the user event script works on the SDF Tutorial script record.

In the SDF Tutorial row, click **New Record**.

The new record template loads, and the Memo field should automatically state **My first account customization project**.

Developing SuiteCloud Projects for SuiteCloud Development Framework

The following topics provide information about SuiteCloud Development Framework (SDF) tasks:

- [Account Component Imports to SuiteCloud Projects](#)
- [SDF Custom Object Dependencies in SuiteApps](#)
- [Account Component Dependencies in SuiteCloud Projects](#)
- [Feature Dependencies for SDF Custom Objects in SuiteCloud Projects](#)
- [Account Configuration from an Account Customization Project](#)
- [SDF Custom Object and File Development in SuiteCloud Projects](#)
- [SuiteCloud Project Validation](#)
- [SuiteCloud Project Deployment Preparation](#)
- [Deployment Logs for SuiteCloud Projects](#)
- [Viewing and Uninstalling SDF SuiteApps in a NetSuite Account](#)
- [Publishing an SDF SuiteApp](#)

Account Component Imports to SuiteCloud Projects

Watch the following video to learn how to import custom objects into your SuiteCloud project.

 [Importing SDF Custom Objects into Your SuiteCloud Project](#)

You can use SuiteCloud SDK tools to import account components from your NetSuite account into your SuiteCloud project for use with SuiteCloud Development Framework (SDF).

For account customization projects, you can import custom record types, customized record types, objects, files, and scripts from within your NetSuite account.

For the SuiteApp projects, you can only import custom record types, customized record types, and objects from your NetSuite account.

If a SuiteCloud component has the same script ID as an account component you import, it gets overwritten. For more information about account components supported by SDF in SuiteCloud projects, see [Customizations Supported by SuiteCloud Development Framework](#).

If you're working with account components that don't need direct updates through the SuiteCloud project, you should reference the account component instead of importing them into your project. For more information, see [Account Component Dependencies in SuiteCloud Projects](#) and [SDF Custom Object Dependencies in SuiteApps](#).



Important: You cannot import locked record types into SuiteCloud projects. You must either unlock the record types before you import them, or reference them outside of the SuiteCloud project. For more information about locked record types, see [Locking SDF Custom Objects in the SuiteApp Type of SuiteCloud Project](#) and [Locking Objects in Customization Bundles](#).

When importing an object from a NetSuite account to an SDF SuiteApp project:

- The object's application ID must match the app ID for the destination SDF SuiteApp project.

- If the application ID values do not match, an error is returned when that project is deployed to a NetSuite account.

To import account components into a SuiteCloud project:

- Using SuiteCloud Extension for Visual Studio Code, see the help topic [Importing SDF Custom Objects from a NetSuite Account with SuiteCloud Extension for Visual Studio Code](#).
- Using SuiteCloud IDE plug-in for WebStorm, see the help topic [Importing Account Components to your SuiteCloud Project with SuiteCloud IDE Plug-in for WebStorm](#).
- Using CLI for java, see the help topics [listfiles](#), [listobjects](#), [importfiles](#), and [importobjects](#).
- Using CLI for Node.js, see the help topic [object:import](#)

SDF Custom Object Dependencies in SuiteApps

Note: You can reference only the objects in a SuiteCloud Development Framework (SDF) SuiteApp. You cannot reference the entire SuiteApp, its files, or its scripts.

For the SuiteApp projects, you can define your SuiteApp to contain dependencies on objects in another SuiteApp, even though those objects are external to your SuiteCloud project. Those object dependencies must be objects supported by SuiteCloud Development Framework (SDF).

To set up dependencies, you create object dependencies in the project manifest. These dependencies require fully qualified script IDs and an application ID. When you add these dependencies to the project manifest, you can refer to your external objects without adding them to your project. For valid references, those objects must already be installed in the target account.

See the following help topics related to SDF custom objects:

- For information about NetSuite components supported as SDF custom objects, see [Customizations Supported by SuiteCloud Development Framework](#).
- For information about the XML format guidelines, see [Specifying an SDF Custom Object Reference](#).

To define an external SDF custom object in a SuiteApp dependency:

- Using SuiteCloud Extension for Visual Studio Code, see the help topic [Adding Dependencies to the Manifest in SuiteCloud Extension for Visual Studio Code](#).
- Using SuiteCloud IDE plug-in for WebStorm, see the help topic [Defining Dependencies on SDF Custom Objects in a SuiteApp and Bundle Objects with SuiteCloud IDE Plug-in for WebStorm](#).
- Manually, see [Manually Defining SDF Custom Object Dependencies in a SuiteApp Using the SuiteCloud Project Manifest](#).

Note: If you want to use SDF to make changes to an SDF custom object, you must import the object into your project instead. For more information, see the help topic [Importing Account Components to your SuiteCloud Project with SuiteCloud IDE Plug-in for WebStorm](#). For information about referencing objects using SDF CLI, see the help topics [listmissingdependencies](#) and [adddependencies](#).

Manually Defining SDF Custom Object Dependencies in a SuiteApp Using the SuiteCloud Project Manifest

In SuiteCloud Development Framework (SDF), you can define dependencies on SDF custom objects that are external to your SuiteCloud project, such as objects in another SuiteApp.

To manually define an external SuiteApp dependency:

1. Open the `manifest.xml` file.
2. Add the following elements if they aren't already there:
Add a `dependencies` element.
Inside the `dependencies` element, add an `applications` element.
3. Inside the `applications` element, add an `application` element. Specify the application id, for example: `<application id="publisherid.projectid">`.
4. Inside the `application` element, add an `objects` element. Use the object XML tag to reference an SDF custom object in a NetSuite account by the script id.

For example, your `dependencies` element should look similar to the following XML if you reference an SDF custom object named `customrecord_sample` that is part of the `com.samples.mysuiteapp` project:

```

1 <dependencies>
2
3   <applications>
4
5     <application id="com.example.mysuiteapp">
6
7       <objects>
8
9         <object>customrecord_sample</object>
10
11       </objects>
12
13     </application>
14
15   </applications>
16
17 </dependencies>

```

Note: Your `dependencies` element may also include a `features` element if your project uses any feature dependencies. For more information, see the help topic [Defining Feature Dependencies in SuiteCloud IDE Plug-in for WebStorm](#).

5. Save the file.
You can now reference an object that is external to your SuiteCloud project. For more information, see [Specifying a Reference to an SDF Custom Object in Another SuiteApp](#).
6. Validate the project against the target NetSuite account to make sure that the dependencies are correct.
For more information about validation, see the help topic [Deploying a SuiteCloud Project to Your NetSuite Account with SuiteCloud Extension for Visual Studio Code](#) or [Validating a SuiteCloud Project with SuiteCloud IDE Plug-in for WebStorm](#).

SuiteApp Bundle Object Dependencies for SuiteCloud Projects

 [Referencing a SuiteBundle from Your SuiteApp Project](#)

Note: SuiteBundler is still supported, but it will not be updated with any new features.

To take advantage of new features for packaging and distributing customizations, you can use the Copy to Account and SuiteCloud Development (SDF) features instead of SuiteBundler.

Copy to Account is an administrator tool that you can use to copy custom objects between your accounts. The tool can copy one custom object at a time, including dependencies and data. For more information, see the help topic [Copy to Account](#).

SuiteCloud Development Framework is a development framework that you can use to create SuiteApps from an integrated development environment (IDE) on your local computer. For more information, see .

You can define dependencies on bundle objects that are external to your SuiteCloud project in SuiteCloud Development Framework (SDF). To set up dependencies, you add object dependencies in the project manifest. These dependencies require a script ID and bundle ID. When you add these dependencies to the project manifest, you can refer to your external bundle objects from object XML definitions in your project. For a valid reference, the object and either the referenced bundle or a version of that bundle must already be installed to the target account.

You can reference a bundle or objects in the bundle that are from a SuiteApp. The object must be of a type supported by the SuiteCloud Development Framework (SDF) supported types. The bundle must be installed or copied to the account where the SuiteApp will be deployed, and the target account can't be the source account where the bundle was originally created. You can't reference files in the bundle

Important: Renamed bundle script IDs are not supported in SDF.

For information about NetSuite components supported as SDF custom objects, see [Customizations Supported by SuiteCloud Development Framework](#).

For information about the XML format guidelines, see [Specifying an SDF Custom Object Reference](#).

To define a bundle object dependency:

- Using SuiteCloud IDE plug-in for WebStorm, see the help topic [Defining Dependencies on SDF Custom Objects in a SuiteApp and Bundle Objects with SuiteCloud IDE Plug-in for WebStorm](#).
- Manually, see [Manually Defining Bundle Object Dependencies from the SuiteCloud Project Manifest](#).

Note: For information about referencing objects using SDF CLI, see the help topics [listmissingdependencies](#) and [adddependencies](#).

Manually Defining Bundle Object Dependencies from the SuiteCloud Project Manifest

You can define dependencies on bundle objects that are outside your SuiteCloud project if they are NetSuite components supported by SuiteCloud Development Framework (SDF). You need to specify the object(s) your bundle depends on, and the bundle that the object is part of.

To define the bundles your SuiteApp project depends on:

1. Open the `manifest.xml` file.
2. Add the following elements if they do not already exist:
Add a `dependencies` element.
Inside the `dependencies` element, add a `bundles` element.
3. Inside the `bundles` element, add a `bundle` element. Specify the bundle id, for example: `<bundle id="123">`

Note: Bundle IDs are always digits only.

4. Inside the `bundle` element, add an `objects` element. Use the object XML tag to reference an object by script id.

For example, your `dependencies` element should look similar to the following XML if you reference bundles 100, 200, and 300, which are copies of one another:

```
1 <bundles>
2
3   <bundle id="100|200|300"/>
4
5 </bundles>
```

Note: Your `dependencies` element may also include a `features` element if your project uses any feature dependencies. For more information, see the help topic [Defining Dependencies on SDF Custom Objects in a SuiteApp and Bundle Objects with SuiteCloud IDE Plug-in for WebStorm](#).

5. Save the file.
You can now reference an object that is external to your SuiteCloud project. For more information, see [Specifying a Bundle Object Reference](#).
6. Validate the project against the target NetSuite account to make sure that the dependencies are correct.
For more information about validation, see the help topic [Deploying a SuiteCloud Project to Your NetSuite Account with SuiteCloud Extension for Visual Studio Code](#) or [Validating a SuiteCloud Project with SuiteCloud IDE Plug-in for WebStorm](#).

To define objects in the bundle your SuiteApp project depends on:

1. Open the `manifest.xml` file.
2. Add the following elements if they do not already exist:
Add a `dependencies` element.
Inside the `dependencies` element, add a `bundles` element.
3. Inside the `bundles` element, add a `bundle` element. Specify the bundle id, for example: `<bundle id="123">`

Note: Bundle IDs are always digits only.

4. Inside the `bundle` element, add an `objects` element. Use the object XML tag to reference an object by script id.

For example, your `dependencies` element should look similar to the following XML if you reference the SDF custom objects named `custrecord_wheel` and `customrecord_train` that are part of the bundle 351:

```

1 <dependencies>
2
3   <bundles>
4
5     <bundle id="351">
6
7       <objects>
8
9         <object>customrecord_car.custrecord_wheel</object>
10
11        <object>customrecord_train</object>
12
13      </objects>
14
15    </bundle>
16
17  </bundles>
18
19 </dependencies>

```

Note: Your dependencies element may also include a features element if your project uses any feature dependencies. For more information, see the help topic [Defining Dependencies on SDF Custom Objects in a SuiteApp and Bundle Objects with SuiteCloud IDE Plug-in for WebStorm](#).

5. Save the file.

You can now reference an object that is external to your SuiteCloud project. For more information, see [Specifying a Bundle Object Reference](#).

6. Validate the project against the target NetSuite account to make sure that the dependencies are correct.

For more information, see the help topic [Deploying a SuiteCloud Project to Your NetSuite Account with SuiteCloud Extension for Visual Studio Code](#) and [Validating a SuiteCloud Project with SuiteCloud IDE Plug-in for WebStorm](#).

Account Component Dependencies in SuiteCloud Projects

In account customization projects, you can set up dependencies on existing SDF custom objects, files, and scripts from a NetSuite account. You can set up dependencies by adding object or file dependencies in the project manifest. These dependencies need fully qualified IDs for objects and path references for files and scripts. When you add these dependencies to the project manifest, you can refer to the target account components without adding them to your project or overwriting them during project deployment.

Your project can depend on SDF custom objects in the same project, or in the target account using an attribute=value pair format that specifies the script ID. For information about supported SDF objects and XML guidelines, see [Customizations Supported by SuiteCloud Development Framework](#) and [Specifying an SDF Custom Object Reference](#).

To define account component dependencies:

- Using SuiteCloud Extension for Visual Studio Code, see the help topic [Adding Dependencies to the Manifest in SuiteCloud Extension for Visual Studio Code](#).
- Using SuiteCloud IDE plug-in for WebStorm, see the help topic [Defining Account Component Dependencies from an Account Customization Project in SuiteCloud IDE Plug-in for WebStorm](#).

- Manually, see [Manually Defining Account Component Dependencies in the Account Customization Project Manifest](#).

For more information about account component dependencies, see the following help topics:

- For information about referencing an existing SDF custom object that is already deployed to the account, see [SDF Custom Object Dependencies in SuiteApps](#).
- For information about using SDF to make changes to an account component, see [Account Component Imports to SuiteCloud Projects](#).
- For information about referencing account components in an account customization project using CLI for Java, see the help topics [listmissingdependencies](#) and [adddependencies](#).
- For information about referencing account components in an account customization project using CLI for Node.js, see the help topic [project:adddependencies](#).

Manually Defining Account Component Dependencies in the Account Customization Project Manifest

In account customization projects, you can set up dependencies on existing SDF custom objects, files, and scripts from a NetSuite account.

To manually define dependencies on account components:

1. Open the `manifest.xml` file.
2. Add a `dependencies` element if one does not already exist.
3. Inside the `dependencies` element, add an `objects` element to reference your SDF custom objects that exist in the target account or a `files` element to reference your files and scripts that exist in the target account.
4. To reference an object, inside the `objects` element, use the `object` XML tag to reference an SDF custom object that exists in the target NetSuite account.
5. To reference a file, inside the `files` element, use the `file` XML tag to reference the path to a file or script that exists in the target NetSuite account.

Always use a forward slash (/) prefix in front of the path. The leading forward slash represents the root directory of the File Cabinet.

For example, your `dependencies` element should look similar to the following XML if you reference an SDF custom object named `customrecord_sample` located in the target account, and a script named `ext_UserScript.js` located in the `SuiteScript/reference` folder of the File Cabinet:

```

1  <dependencies>
2
3    <objects>
4
5      <object>customrecord</object>
6
7    </objects>
8
9    <files>
10
11      <file>/SuiteScript/reference/ext_UserScript.js</file>
12
13    </files>
14
15  </dependencies>
```

Note: Your dependencies element may also include a features element if your project uses any feature dependencies. For more information, see the help topic [Defining Dependencies on SDF Custom Objects in a SuiteApp and Bundle Objects with SuiteCloud IDE Plug-in for WebStorm](#).

6. Save the file.

You can now reference an objects, files, and scripts that are external to your SuiteCloud project. For more information, see [Specifying an SDF Custom Object Reference](#) or [Specifying a Custom File or Script Reference](#).

7. Validate the project against the target NetSuite account to make sure that the dependency references are correct.

For more information, see the help topic [Deploying a SuiteCloud Project to Your NetSuite Account with SuiteCloud Extension for Visual Studio Code](#) or [Validating a SuiteCloud Project with SuiteCloud IDE Plug-in for WebStorm](#).

Feature Dependencies for SDF Custom Objects in SuiteCloud Projects

When you use SuiteCloud Development Framework (SDF) to build SuiteApps or account customizations in SuiteCloud projects, you should consider which NetSuite features your SDF custom objects require and which ones are optional:

- For some SDF custom objects, you have to add NetSuite features to the project manifest so your SuiteApp successfully installs or your project deploys to the target account. For more information, see [Standard Feature Dependencies for SDF Custom Objects](#) and [Enhanced Feature Dependencies for SDF Custom Objects Containing OneWorld Components](#).
- Some SDF custom objects have optional NetSuite features, even if a record type or field depends on a feature. For those SDF custom objects, the SuiteApp installation (or SuiteCloud project deployment) succeeds, but those objects or fields that need those features won't show up for users in the target account. For more information, see [Enhanced Feature Dependencies for SDF Custom Objects](#).

Standard Feature Dependencies for SDF Custom Objects

For SDF custom objects that are not listed in [Enhanced Feature Dependencies for SDF Custom Objects](#) and [Enhanced Feature Dependencies for SDF Custom Objects Containing OneWorld Components](#), you need to reference the feature dependencies in the SuiteCloud project so that validation and deployment succeed. This makes sure that the required features are enabled in the target account so you can use those SDF custom objects.

You reference dependencies using the feature element in the SuiteCloud project manifest. Each element has a `required` attribute that accepts a string value of `true` or `false`. You can update the project manifest manually or let SDF evaluate the project and update the manifest for you.

When you set a `required` element to `true`, the referenced feature has to be enabled in the target NetSuite account. Any SDF custom object in the SuiteCloud project that is feature-dependent should work as intended.

When you set a `required` element to `false`, the referenced feature doesn't need to be enabled in the target NetSuite account. Any SDF custom object in the SuiteCloud project that is feature-dependent may

cause errors or warnings during validation. The element contents are ignored if they depend on features that are not required. These elements cause warnings when the SuiteCloud project is validated.

To add and define feature dependencies:

- Using SuiteCloud Extension for Visual Studio Code, see the help topic [Adding Dependencies to the Manifest in SuiteCloud Extension for Visual Studio Code](#).
- Using SuiteCloud IDE plug-in for WebStorm, see the help topic [Defining Feature Dependencies in SuiteCloud IDE Plug-in for WebStorm](#).
- Using CLI for Node.js, see the help topic [project:adddependencies](#).
- Using CLI for Java, see the help topics [listmissingdependencies](#) and [adddependencies](#).
- Manually, see [Manually Defining Feature Dependencies in the SuiteCloud Project Manifest](#).

Enhanced Feature Dependencies for SDF Custom Objects

Your users can fully install an SDF SuiteApp even if some NetSuite features aren't enabled in the target account. If the account administrator later enables those features, the relevant SDF custom objects become available to your SuiteApp's users.

This feature works for a limited set of SDF custom objects for SuiteCloud Development Framework (SDF), which you can include in your SDF SuiteApp. Your users can install these SDF SuiteApps without getting warnings or errors about missing features.

The following SDF custom objects support this feature dependency enhancement:

- clientscript
- customrecordtype
- transactionForm
- dataset
- entryForm
- itemcustomfield
- othercustomfield
- role
- savedsearch
- usereventscript
- workbook



Note: For information about enhanced feature dependencies in OneWorld accounts, see [Enhanced Feature Dependencies for SDF Custom Objects Containing OneWorld Components](#).

Note the following information when including SDF custom objects in your SDF SuiteApps that support this enhancement:

- SDF SuiteApp installation succeeds without notifying the user.
- SDF SuiteApp deployment succeeds without any log entries outputted to the console of SuiteCloud IDE or the SuiteCloud CLI.
- Validating a SuiteCloud project in SuiteCloud IDE or the SDF CLI doesn't notify you about features when they aren't enabled in the target account.

- During SDF SuiteApp installation, SDF custom objects that are restricted by features are created in the target account, but they'll remain hidden until the feature is enabled.

Uninstalling an SDF SuiteApp also removes those SDF custom objects from the account, even when those objects were hidden.

- You can still make the feature required for the target account by setting the `required` attribute to `true` in the manifest, even if you already used SuiteCloud IDE to add the dependency references for you. That attribute value means that the installation will fail if the feature isn't enabled in the target account. To manually define the feature dependency, see [Manually Defining Feature Dependencies in the SuiteCloud Project Manifest](#).

Not specifying a feature in the manifest is functionally the same as setting the feature's `required` attribute to `false`.

The same functionality is also available for account customization projects (ACP) that contain supported SDF custom objects. You can deploy these projects to an account without the required features, and the custom objects will become available when the features are enabled.

For all other SDF custom objects, the standard feature dependency behavior remains true. For information about standard feature dependencies, see [Standard Feature Dependencies for SDF Custom Objects](#).

Enhanced Feature Dependencies for SDF Custom Objects Containing OneWorld Components

Your users can successfully install an SDF SuiteApp with OneWorld components in accounts that don't have OneWorld functionality. They won't get any warnings or errors about missing features when installing these SuiteApps. This functionality is available for the following subset of SDF custom objects in SuiteCloud Development Framework (SDF):

- `customrecordcustomfield`
- `otherrecordfield`



Note: For standard feature dependencies for other SDF custom objects, see [Standard Feature Dependencies for SDF Custom Objects](#).

Note the following behaviors when you install an SDF SuiteApp containing the `customrecordcustomfield` object with a `Subsidiary` field:

- In OneWorld accounts, the installation will succeed.
- In accounts that are not OneWorld, the installation will only succeed if the `Subsidiary` field isn't required. You'll get a warning message during deployment saying the `Subsidiary` field has been removed, but the rest of the SDF SuiteApp will deploy successfully. If the `Subsidiary` field is required, the installation will fail.

For information about enhanced feature dependencies in accounts that are not OneWorld, see [Enhanced Feature Dependencies for SDF Custom Objects](#).

Manually Defining Feature Dependencies in the SuiteCloud Project Manifest

You can define feature dependencies by updating the manifest file for your SuiteCloud project in SuiteCloud Development Framework (SDF).

To manually define feature dependencies:

1. Open the `manifest.xml` file.
2. After the `frameworkversion` element, add a `dependencies` element if one does not already exist.
Then inside the `dependencies` element, add a `features` element if one does not already exist.
3. Type the following element inside the `features` element:

```
1 | <feature required=""></feature>
```

Your `dependencies` element should look similar to the following XML:

```
1 | <dependencies>
2 |
3 |   <features>
4 |
5 |     <feature required=""></feature>
6 |
7 |   </features>
8 |
9 | </dependencies>
```

4. With the cursor placed inside the `feature` element, enter the feature value. For the list of features, see the help topic [configurable_features](#).
5. Set the `required` attribute to a string to one of the following values:

- **true** – Specifies that the feature is required for the SDF SuiteApp to successfully install (or SDF account customization project to successfully deploy) to the target account. If the target account doesn't have the feature enabled, the installation (or deployment) will fail with an error.

The following XML sample is an example of a feature dependency that is required for the SuiteApp to be successfully installed:

```
1 | <dependencies>
2 |
3 |   <features>
4 |
5 |     <feature required="true">EXAMPLEFEATURE</feature>
6 |
7 |   </features>
8 |
9 | </dependencies>
```

- **false** – Specifies that the feature is not required for the SDF SuiteApp to successfully install (or SDF account customization project to successfully deploy) to the target account. SuiteApps (and account customization projects) containing SDF custom objects that support enhanced feature dependency can still successfully deploy without errors when this value is set. For more information, see [Enhanced Feature Dependencies for SDF Custom Objects](#).

The following XML sample is an example of a feature dependency that is not required for the SuiteApp to be successfully installed:

```
1 | <dependencies>
2 |
3 |   <features>
4 |
5 |     <feature required="false">EXAMPLEFEATURE</feature>
6 |
7 |   </features>
8 |
9 | </dependencies>
```

6. Save the file.

Account Configuration from an Account Customization Project

In SuiteCloud Development Framework (SDF), you can enable or disable account features from an SDF account customization project. To find an account's feature configuration in NetSuite, go to **Setup> Company > Enable Features**.

You can either import the most recent feature configuration from a NetSuite account or create a new feature configuration in the account customization project. By controlling account features in your project, you ensure that more project components operate as intended without requiring manual adjustments to the target account features. Additionally, you can use this feature to compare the current configuration of an account to its previous configuration when a project was deployed to it.

Features that require a Terms of Service user agreement cannot be enabled or disabled using SuiteCloud Development Framework (SDF). The following features **cannot** be controlled using SDF:

- MARKETING (Marketing Automation)
- MAILMERGE (Mail Merge)
- DIRECTDEPOSIT (Direct Deposit)
- WEBSERVICEEXTERNAL (Web Services)
- SUITESIGNON (SuiteSignOn)
- OPENIDSSO (OpenID Single Sign-on)
- SAMLSO (SAML Single Sign-on)
- TBA (Token-based Authentication)
- ITEMOPTIONS (Item Options)
- CUSTOMRECORDS (Custom Records)
- ADVANCEDPRINTING (Advanced PDF/HTML Templates)
- AUTOLOCATIONASSIGNMENT (Automatic Location Assignment)
- WORKFLOW (SuiteFlow)
- CREATESUITEBUNDLES (Create bundles with SuiteBundler)
- SUITEAPPDEVELOPMENTFRAMEWORK (SuiteCloud Development Framework)
- INSTALLSUITEBUNDLES (Install bundles with SuiteBundler)
- SERVERSIDESCRIPING (Server SuiteScript)
- WEBAPPLICATIONVERSIONING (SuiteScript Server Pages Versioning)
- WEBAPPLICATIONS (SuiteScript Server Pages)
- CUSTOMGLLINES (Custom GL Lines)
- CUSTOMTRANSACTIONS (Custom Transactions)
- CUSTOMSEGMENTS (Custom Segments)
- CUSTOMCODE (Client SuiteScript)
- FXRATEUPDATES (Currency Exchange Rate Integration)
- MOBILEPUSHNTF (Mobile Push Notification)
- WEEKLYTIMESHEETSNEWUI (Weekly Timesheets Interface)
- WEBDUPLICATEEMAILMANAGEMENT (Web Site Duplicate Email Management)
- PAYROLL (Payroll)

Account customization projects contain an AccountConfiguration folder. By default, this folder is empty. By creating or importing a features.xml file in this folder, you can enable or disable features. Account configuration is stored in the features.xml file. In addition to creating the new file, the deploy.xml file in the project must reference the features.xml file path.

For information about referencing dependencies using CLI for Java, see the help topics [listconfiguration](#) and [importconfiguration](#).

Account Configuration Validation

Due to the complexity of account feature dependencies on other account features, you may encounter several validation errors when enabling or disabling features. You should validate your project often while adjusting the account configuration in a SuiteCloud project. Use the validation error messages and logs while working with an account configuration to get it in a working state. For more information, see [SuiteCloud Project Validation](#) and [Account Configuration Errors](#).

The following example shows some possible validation issues that may occur after modifying an account configuration:

```

1 2020-08-29 07:22:33 (PST) started
2
3 Info -- Account [(PRODUCTION) MyNetSuiteAccount]
4
5 Info -- Account Customization Project [AccountCustomizationProjectSample]
6
7 Info -- Framework Version [1.0]
8
9 Validate manifest -- Success
10
11 Validate deploy file -- Success
12
13 Validate configuration -- Success
14
15 Validate script file -- Success
16
17 Validate objects -- Success
18
19 Validate files -- Success
20
21 Validate folders -- Success
22
23 Validate account settings -- Success
24
25 Validate Custom Objects against the Account -- Success
26
27
28
29 WARNING -- One or more potential issues were found during configuration validation.
30
31 Details: You cannot use SDF to enable the SERVERSIDESCRIPITING(Server SuiteScript) feature because it requires a Terms of Service
32 user agreement.
33
34 Details: You cannot use SDF to enable the WORKFLOW(SuiteFlow) feature because it requires a Terms of Service user agreement.
35
36 Details: Configuration of MULTILOCINVT(Multi-Location Inventory) feature reported the following warning(s): Once you have distrib
37 uted items, you will not be able to disable the Multi-Location Inventory feature. Are you sure you want to enable this feature?
38
39 File: ~/AccountConfiguration/features.xml
40
41 Begin deployment
42
43 Configure feature -- The MULTILANGUAGE(Multi-Language) feature has been left UNCHANGED with status ENABLED
44
45 Configure feature -- The MULTISHIPTO(Multiple Shipping Routes) feature has been left UNCHANGED with status DISABLED
46
47 Configure feature -- The LOCATIONS(Locations) feature has been ENABLED

```

```

47 Configure feature -- The MULTILOCINVT(Multi-Location Inventory) feature has been ENABLED
48
49 Configure feature -- The ACCOUNTING(Accounting) feature has been left UNCHANGED with status ENABLED
50
51 Configure feature -- Disabling of the SERVERSIDESCRIPING(Server SuiteScript) feature has FAILED
52
53 Configure feature -- Disabling of the WORKFLOW(SuiteFlow) feature has FAILED
54
55 Create object -- customrecord_ (customrecordtype)
56
57
58
59 WARNING -- One or more potential issues were found during custom object creation. (customrecord_)
60
61 Details: Once you have distributed items, you will not be able to disable the Multi-Location Inventory feature. Are you sure you
    want to enable this feature?
62
63 Details: To complete multi-location inventory setup, please visit 'Setup > Company > Locations' to define your company's locations
    and then 'Transactions > Inventory > Distribute Inventory' to distribute your inventory before entering transactions or running
    reports.
64
65 File: ~/Objects/customrecord_.xml
66
67 Create object -- customrecord_.custrecord_ (customrecordcustomfield)
68
69 2020-08-29 07:22:48 (PST) Installation COMPLETE (0 minutes 16 seconds)

```

SDF Custom Object and File Development in SuiteCloud Projects

The following topics explain various aspects of SuiteCloud project development for SDF custom objects and files in SuiteCloud Development Framework (SDF):

- [SuiteScript Files in SuiteCloud Projects](#)
- [Modifying SuiteCloud Project File and Folder Attributes in an XML Definition](#)
- [Installation Preferences in the SuiteApp of SuiteCloud Project](#)
- [Developing SDF Custom Objects](#)
- [Managing SuiteCloud Projects as Compressed Files in SuiteCloud IDE Plug-in for WebStorm](#)



Important: You cannot deploy SDF custom objects from account customization projects if the objects are locked in the target NetSuite account. You also cannot use account customization records to add subrecords to locked record types. For more information about locked record types, see [Locking SDF Custom Objects in the SuiteApp Type of SuiteCloud Project](#) and [Locking Objects in Customization Bundles](#).

SuiteScript Files in SuiteCloud Projects

Working with SuiteScript files in a SuiteCloud project for SuiteCloud Development Framework (SDF) is similar to working with those in a SuiteScript project.

For information about:

- Creating SuiteScript files in SuiteCloud projects, see one of the following topics:
 - [Creating a SuiteScript File in SuiteCloud Extension for Visual Studio Code](#)
 - [Creating a SuiteScript File in SuiteCloud IDE Plug-in for WebStorm](#)
 - [file:create](#)

- [createfile](#)
- Importing SuiteScript files into a SuiteCloud project, see [Account Component Imports to SuiteCloud Projects](#).
- Deploying SuiteScript files to a target NetSuite account based on the structure of the deploy file, see [SuiteCloud Project Deployment Preparation](#).

Modifying SuiteCloud Project File and Folder Attributes in an XML Definition

In SuiteCloud Development Framework (SDF), when you use a SuiteCloud SDK tool to create a file or folder in the File Cabinet of your SuiteCloud project, an XML file is automatically created in a `.attributes` subfolder. The XML file contains attributes for the file or folder.

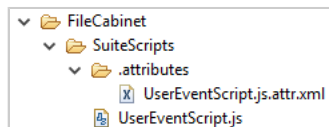
When you create a file or folder in the File Cabinet of your SuiteCloud project using a tool other than SuiteCloud IDE, you must also create the attribute file and folder manually.

File Attributes

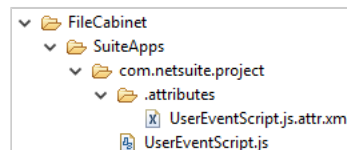
File attributes are stored in an XML file located inside a `.attributes` folder. That `.attributes` folder is located in the same folder as the File Cabinet file. For example, if the File Cabinet file is located in the SuiteScripts folder, the `.attributes` folder is also located in the SuiteScripts folder.

The filename for the attributes file is of the form `filename.attr.xml`, where *filename* is the filename for a File Cabinet file. For example, the file `UserEventScript.js` has a corresponding attribute file `UserEventScript.js.attr.xml` in the `.attributes` folder.

The following graphic is an example of file attributes in an account customization type of SuiteCloud project:



The following graphic is an example of file attributes in a SuiteApp type of SuiteCloud project:



Consider the following when working with files attributes in SDF:

- Attribute files for File Cabinet files and their parent folder are located in the same `.attributes` folder.
- In account customization projects, attributes for files are located in subdirectories of the FileCabinet/SuiteScripts folder.
- In SuiteApp projects, attributes for files are located in subdirectories of the FileCabinet/SuiteApps folder.
- Only store file attributes in the `.attributes` folder. Other files that are stored in the `.attributes` folder are not deployed.
- For information about uploading files to a NetSuite account, see the help topic [Uploading Files to the File Cabinet](#).
- When adding files to the Marketing Templates folder, you must specify the Marketing Automation feature as a dependency in the manifest

File attributes are represented by following fields for the **file** element:

Field	Type	Description
availablewithoutlogin	boolean	Sets the file to be accessible externally. The file can be accessed with a URL that points to the file record. The value F sets the file to be only accessible to those who are logged in to NetSuite with permission to view File Cabinet documents. The default value is F.  Note: When the Web Site Hosting Files Always Available general company preference is enabled and the SuiteApps folder contains subfolders with two periods in the folder name, files in those subfolders are accessible externally, even when the availablewithoutlogin attribute is set to F for those files. For example, / SuiteApps/folder1.1.2/file.txt.
bundleable	boolean	Includes the file in a bundle. If you include this file in a bundle, setting the value to F does not remove the file from the bundle. To remove the file from a bundle, update the bundle and remove the file. The default value is F.
description	string	Provides a description for the file. The description only appears on the file record. By default, the value is empty.
generateurltimestamp	boolean	Adds a time stamp parameter to the URL for the file. This is useful when you are building a web site, and you want to view the most recent updates to the files on your site. The URL time stamp is ignored by NetSuite, but indicates to the browser cache and network that content has changed. The default value is F.
hideinbundle	boolean	Hides the file, preventing anyone who installs the bundle from accessing the file in the target account. This attribute corresponds to the Hide in SuiteBundle box in the NetSuite File Cabinet. The hideinbundle attribute cannot be used to hide client script SuiteScript files. The default value is F.
isinactive	boolean	Indicates that the file is inactive. The file only appears to users on lists if Show Inactives is enabled in the NetSuite UI. The default value is F.

File Attributes

The following example is an XML definition of a file attributes:

```

1 <file>
2
3   <availablewithoutlogin>T</availablewithoutlogin>

```

```

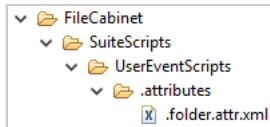
4  <bundleable>T</bundleable>
5
6
7  <description>User event script.</description>
8
9  <generateurltimestamp>T</generateurltimestamp>
10
11 <hideinbundle>F</hideinbundle>
12
13 <isinactive>F</isinactive>
14
15 </file>

```

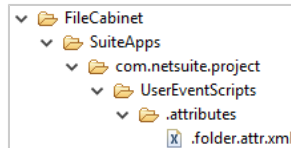
Folder Attributes

Folder attributes are stored in a `folder.attr.xml` file in a `.attributes` folder. There is a `.attributes` folder for each File Cabinet subfolder. For example, if you create a `UserEventScripts` folder in the File Cabinet of your SuiteCloud project, it must contain a `.attributes` folder that contains a `.folder.attr.xml` file.

The following graphic is an example of folder attributes in an account customization type of SuiteCloud project:



The following graphic is an example of folder attributes in a SuiteApp type of SuiteCloud project:



Consider the following when working with folder attributes in SDF:

- In account customization projects, attributes for folders are located in subdirectories of the FileCabinet/SuiteScripts folder.
- In SuiteApp projects, attributes for folders are located in subdirectories of the FileCabinet/SuiteApps folder.
- System folder attributes cannot be modified. You can only modify attributes for subfolders of the FileCabinet/SuiteScripts and FileCabinet/SuiteApps folders.
- To be deployed, subfolders must be located either in FileCabinet/SuiteScripts or FileCabinet/SuiteApps.
- Attribute files for File Cabinet files and their parent folder are located in the same `.attributes` folder.
- System folders such as SuiteScripts and SuiteApps do not have their own folder attributes.
- For information about creating folders in a NetSuite account, see the help topic [Creating File Cabinet Folders](#).

Folder attributes are represented by following fields for the **folder** element:

Attribute	Type	Value
bundleable	boolean	Includes the folder in a bundle. If you include this folder in a bundle, setting to F does not remove the folder from the bundle. To remove the folder from a bundle, update the bundle and remove the folder. The default value is F.

Attribute	Type	Value
description	string	Provides a description for the folder. The description only appears on the folder record. By default, the value is empty.
isinactive	boolean	Indicates that the folder is inactive. The folder only appears to users on lists if Show Inactives is enabled in the NetSuite UI. The default value is F.
isprivate	boolean	Specifies that the contents of this folder visible to the folder creator and the account administrator. The default value is F.

Folder Attributes

The following example is an XML definition of folder attributes.

```

1 <folder>
2
3   <bundleable>F</bundleable>
4
5   <description>This folder contains all of the user event scripts.</description>
6
7   <isinactive>F</isinactive>
8
9   <isprivate>F</isprivate>
10
11 </folder>

```

Installation Preferences in the SuiteApp of SuiteCloud Project

Hiding and Locking Preferences

In SuiteCloud Development Framework (SDF), installation preferences apply to SuiteCloud projects of type SuiteApp, and provide you with a way to configure general or detailed preferences for an element, group of elements, or the project. You can apply the preferences per deployment or validation.

You can use SDF installation preferences as a way to determine the SuiteApp's customizations that your SuiteApp users can see, modify, or both see and modify. You can also use SDF installation preferences to determine what customizations are preserved or overwritten during SuiteApp upgrade. For information about which customizations are supported by installation preferences, Consult the documentation for each installation preference file.

The files that are used for installation preferences are included in the InstallationPreferences folder when you create a SuiteCloud project of type SuiteApp. There are three types of installation preferences files:

- Locking preferences are stored in the locking.xml file. When an SDF custom object or file is locked using SuiteCloud Development Framework (SDF), the file's fields cannot be modified from the account. For more information, see [Locking SDF Custom Objects in the SuiteApp Type of SuiteCloud Project](#) and [Locking Files in the SuiteApp Type of SuiteCloud Project](#).
- Hiding preferences are stored in the hiding.xml. When a file is hidden using SDF, it cannot be viewed, downloaded, or overwritten. For more information, see [Hiding Files in the SuiteApp Type of SuiteCloud Project](#).

- Overwriting preferences are stored in the `overwriting.xml` file. For more information, see [Overwrite Settings for SuiteApp Projects](#).

Note: Even when using only locking preferences without hiding preferences or hiding preferences without locking preferences, your SuiteCloud project must include both the `locking.xml` and `hiding.xml` files for the project to successfully deploy to a target account.

To test or apply installation preferences against a target account, you can use the **Apply Installation Preferences** option. Enabling this option triggers SDF to apply installation preferences to the validation or deployment instance. For more information about using the **Apply Installation Preferences** option, see the following topics:

- [Creating an Installation Preference File with SuiteCloud IDE Plug-in for WebStorm](#)
- [Default Installation Preferences](#)
- [Applying Installation Preferences During SuiteCloud Project Validation and Deployment](#)
- [Installation Preferences Validation in SuiteCloud Projects](#)

Note: For information about applying installation preferences using SuiteCloud CLI for Java, see the help topics [deploy](#), [preview](#), and [validate](#).

Default Installation Preferences

You can use the default action to define a setting for all applicable elements in the project except those exceptions that you define in the apply action list.

The default action lets you set a general preference for elements in your SuiteApp project, allowing you to apply and change installation preferences in fewer steps. From the preferences file, you can define the action you want to set for most elements in the project. The apply action list can contain the exceptions. For more information, see [Locking SDF Custom Objects in the SuiteApp Type of SuiteCloud Project](#) or [Hiding Files in the SuiteApp Type of SuiteCloud Project](#).

Note that the default action applies only to elements that support the preference, and only when deployed with Apply Installation Preferences enabled. For example, if a default action of "LOCK" is set, lockable objects in the Objects folder of the SuiteApp project are locked, unless specified differently in the apply action list for locking preferences. The default action applies to custom objects in your project that are not in the apply action list of your installation preferences file.

Locking SDF Custom Objects in the SuiteApp Type of SuiteCloud Project

In a SuiteCloud Development Framework (SDF), you can consider an installation preference to lock supported SDF custom object types in your SuiteCloud project when the project is of type SuiteApp. Locking SDF custom objects helps protect an object's fields from being edited by your SuiteApp users.

Consider the following information when locking SDF custom objects:

- When an object is successfully locked, the object's fields cannot be modified in target accounts where the SuiteApp is installed.
- Locking an object prevents target account users from editing the object.
- Locking an object does not prevent target account users from seeing the object.
- The NetSuite pages for locked objects do not show actions that allow changes to the object, such as **Edit**, **Delete**, and **Change ID**. Also, all list and view pages display a lock icon to provide complete visibility of locked objects in installed SuiteApps.

- For a list of SDF custom object types that can be locked, see [Lockable SDF Custom Objects](#). See the Notes column for information about locking exceptions. Certain objects have different behavior. For example, a small subset of fields can be modified on a locked workflow object.

Note: Even when using only locking preferences without hiding preferences, your SuiteCloud project must include both the `locking.xml` and `hiding.xml` files for the project to successfully deploy to a target account.

To lock SDF custom objects in a SuiteCloud project of SuiteApp type

- Create a `locking.xml` file in the `InstallationPreferences` folder of your SuiteCloud project of SuiteApp type. To create the file using a SuiteCloud IDE, see the help topic [Creating an Installation Preference File with SuiteCloud IDE Plug-in for WebStorm](#). If you already have a `locking.xml` file in your SuiteCloud project, proceed directly to the next step.
- In the `locking.xml` file in your SuiteCloud project, enter values for the following attributes and fields:

- For the `type` attribute of the `<preference>` element, specify the installation preference type to be Locking. For more information about installation preferences, see [Installation Preferences in the SuiteApp of SuiteCloud Project](#).
- For the `defaultAction` attribute of the `<preference>` element, specify a default locking action for all SDF custom objects in your SuiteCloud project. Supported values for an action are `LOCK` and `UNLOCK`.

Setting a value for the default action can reduce the steps when applying a preference to many objects at a time. The default action applies to SDF custom objects in your project that are not in the `apply` action list of your `locking.xml` file. In the included example, the installation locking preference for lockable SDF custom objects in the SuiteApp project is `UNLOCK`, so all SDF custom objects in the SuiteApp other than `customlist_example1` are unlocked.

- For the `action` attribute of the `<apply>` element, specify a value that will apply to the SDF custom objects. Valid values are `LOCK` and `UNLOCK`. For example, `<apply action="LOCK">` locks the specified SDF custom object types.
- For the `<object>` element, specify the script ID of the SDF custom object that is being locked. For example, `<object>customlist_example1</object>`. For additional SDF custom objects, you can add `<object></object>` elements as needed. For the list of lockable SDF custom object types, see [Lockable SDF Custom Objects](#).

- Save your changes in the `locking.xml` file.

When you later deploy the SuiteCloud project, to enable the values specified in the `locking.xml` file, you must select the **Apply Installation Preferences** box. For more information about selecting that preference during deployment, see [Applying Installation Preferences During SuiteCloud Project Validation and Deployment](#).

Locking a Custom Object using SDF Example

The following XML sample is an example of locking preference in a `locking.xml` file in a SuiteCloud project of SuiteApp type. The example locks the `customlist_example1` SDF custom object, sets the preference type to `LOCKING`, sets the default action to `UNLOCK`, sets the `apply` action to `LOCK`, and adds the object:

```

1 <preference type="LOCKING" defaultAction="UNLOCK">
2
3   <apply action="LOCK">
4
5     <object>customlist_example1</object>
6
7   </apply></preference>
```

Lockable SDF Custom Objects

The following table lists the SDF custom object types that you can lock in the SuiteApp type of SuiteCloud project. The effects of locking vary according to object type.

Object Type	Notes
customlist	Values cannot be added, edited, or deleted in locked custom lists in target accounts.
customrecordtype	- ID of a locked custom record cannot be changed or deleted. - Original custom field IDs cannot be changed or deleted. - Custom fields cannot be added. - Custom record instances can be added and the ID can be changed or deleted. - ID of custom record instances that came with the SuiteApp cannot be changed or deleted. - Changing the ID of any custom record Instance is not allowed.
customsegment	To lock a custom segment object, you must apply the lock to the custom segment object along with its associated custom record type object.
Fields: - crmcustomfield - entitycustomfield - itemcustomfield - itemoptioncustomfield - itemnumbercustomfield - transactionbodycustomfield - transactioncolumncustomfield - workflow_othercustomfield	Target account administrators can control access to locked custom fields through form customization as before.
Forms: - addressform - entryform - transactionform	New instances can be created and customized
Plug-ins: - advancedrevrecplugin - advancedrevrecplugin - bankstatementparserplugin - consolidatedrateadjustorplugin - custommglplugin - datasetbuilderplugin - emailcaptureplugin - ficonnectivityplugin - fiparserplugin - paymentgatewayplugin - shippingpartnersplugin	Locked plug-ins can be configured

Object Type	Notes
- taxcalculationplugin - workbookbuilderplugin	
plugintype	—
plugintypeimpl	—
prompt	- Values cannot be added, edited, or deleted in locked prompts. - When viewing the prompt list in Prompt Studio, a locked prompt has a Customize link (instead of an Edit link) and a positive ID. - When viewing the prompt details in Prompt Studio, a locked prompt has a Locked indicator in the upper-right corner.
role	New instances can be created and customized
savedsearch	Users who have access to the saved search can create a new copy of the saved search and add or remove filters and results columns.
scripts: - bundleinstallationscript - clientscript - mapreducescript - massupdatescript - restlet - scheduledscript - suitelet - usereventscript - workflowactionscrip - singlepageapp - tool	- IDs for original custom fields on the script cannot be deleted or changed. - New custom fields can be added and then changed or deleted. - Script deployments are not lockable and you can create new deployments or edit existing ones - For single page applications (SPAs): - You can apply installation preferences to SPAs. - SPAs support object locking, hiding, and locking of files.
SSP applications	The SSP application record is lockable.
textenhanceaction	- Values cannot be added, edited, or deleted in locked Text Enhance actions. - When viewing the Text Enhance action list in Prompt Studio, a locked Text Enhance action has a Customize link (instead of an Edit link) and a positive ID. - When viewing the Text Enhance action details in Prompt Studio, a locked Text Enhance action has a Locked indicator in the upper-right corner.
Workflows	Workflow objects are partially editable. - The following basic Information fields from a locked workflow can be edited: Release Status, Keep Instance and History, Enable Logging, Inactive. - You may not edit the definition, instance fields, states, transitions, or actions of a locked workflow. Also, you may not add transitions or states.
translationcollection	When an SDF SuiteApp deployment locks a Translation Collection in the target account, subsequent deployments can still update that Translation Collection including its strings.

Object Type	Notes
customtransactiontype	Values cannot be added, edited, or deleted in locked custom transaction

Hiding Files in the SuiteApp Type of SuiteCloud Project

In SuiteCloud Development Framework (SDF), you can hide files in your SuiteCloud project when that project is of type SuiteApp. Hiding a file prevents anyone who installs the SuiteApp from accessing the file in the target account.

When hiding files, consider the following important information:

- SuiteCloud Development Framework (SDF) validates that hiding preferences match with any existing `hideinbundle` file attributes. You cannot use a conflicting hiding preference and `hideinbundle` attribute on the same file. For information about changing a file attribute, see [Modifying SuiteCloud Project File and Folder Attributes in an XML Definition](#).
- Even when using only hiding preferences without locking preferences, your SuiteCloud project must include both the `hiding.xml` and `locking.xml` files for the project to successfully deploy to a target account.
- It is not possible to hide client script SuiteScript files in a SuiteCloud project of SuiteApp type. For client script files, do not set the HIDING preference action value to HIDE.

To hide files in a SuiteCloud project of SuiteApp type

1. Create a `hiding.xml` file in the `InstallationPreferences` folder of your SuiteCloud project of SuiteApp type. To create the file using a SuiteCloud IDE, see the help topic [Creating an Installation Preference File with SuiteCloud IDE Plug-in for WebStorm](#). If you already have a `hiding.xml` file in your SuiteCloud project, proceed directly to the next step.
2. In the `hiding.xml` of your SuiteCloud project, enter values for the following attributes and fields:
 - For the type attribute of the `<preference>` element, specify the installation preference type to be HIDING. For more information about installation preferences, see [Installation Preferences in the SuiteApp of SuiteCloud Project](#).
 - For the `defaultAction` attribute of the `<preference>` element, specify a default hiding action for all files in your SuiteCloud project. Valid values are HIDE and UNHIDE.

The default action applies to content in your project that is not in the apply list of your `hiding.xml` file. In the following example, the default preference for files in the SuiteCloud project is UNHIDE, so all files except the `myScript.js` file are not hidden:

```

1 <preference type="HIDING" defaultAction="UNHIDE">
2
3   <apply action="HIDE">
4
5     <path>~/FileCabinet/SuiteApps/com.example.myFile/myScript.js</path>
6
7   </apply>
8
9 </preference>
```

- For the action attribute of the `<apply>` element, specify a value that will apply to the file or files. Valid values are HIDE and UNHIDE. For example, `<apply action="HIDE">` hides the specified file or files.
- For the `<path>` element, specify a file path for the files to hide. The following are the possible ways to specify paths:

- You can specify a path to a single file. For example, `<path>~/FileCabinet/SuiteApps/com.example.myFile/myScript.js</path>`.
- You can specify a path to all files in a target folder (multiple files) using the wildcard (*) and the `appliesTo` attribute for the `<path>` element. For example, `<path appliesTo="FILES">~/FileCabinet/SuiteApps/com.example.myFiles/*</path>`.



Note: A `<path>` element with a value that contains the wildcard (*) requires an `appliesTo` attribute to be set.

3. Save your changes in the `hiding.xml` file.

When you later deploy the SuiteCloud project, to enable the values specified in the `hiding.xml` file, you must select the **Apply Installation Preferences** box. For more information about selecting that preference during deployment, see [Applying Installation Preferences During SuiteCloud Project Validation and Deployment](#).

Hiding a File using SDF Example

The following `hiding.xml` file example hides the `myScript.js` file by setting the preference type to `HIDING`, the apply action to `HIDE`, and the path to the file.

```

1 <preference type="HIDING" defaultAction="UNHIDE">
2
3   <apply action="HIDE">
4
5     <path>~/FileCabinet/SuiteApps/com.example.myFile/myScript.js</path>
6
7   </apply>
8
9 </preference>
```

Hiding Multiple Files using SDF Example

The following `hiding.xml` file example hides all of the files in the folder `com.example.myFiles` by using the wildcard (*) and the `appliesTo="FILES"` attribute:

```

1 <preference type="HIDING" defaultAction="UNHIDE">
2
3   <apply action="HIDE">
4
5     <path appliesTo="FILES">~/FileCabinet/SuiteApps/com.example.myFiles/*</path>
6
7   </apply>
8
9 </preference>
```

Locking Files in the SuiteApp Type of SuiteCloud Project

In SuiteCloud Development Framework (SDF), you can lock files and folders in your SuiteCloud project when that project is of type `SuiteApp`. After project deployment (or SuiteApp installation), a locked file or folder will appear to your SuiteApp users as read-only.



Note: Even when using only locking preferences without hiding preferences, your SuiteCloud project must include both the `locking.xml` and `hiding.xml` files for the project to successfully deploy to a target account.

To lock files in a SuiteCloud project of SuiteApp type

1. Create a `locking.xml` file in the `InstallationPreferences` folder of your SuiteCloud project of SuiteApp type. To create the file using a SuiteCloud IDE, see the help topic [Creating an Installation Preference File with SuiteCloud IDE Plug-in for WebStorm](#). If you already have a `locking.xml` file in your SuiteCloud project, proceed directly to the next step.
2. In the `locking.xml` file in your SuiteCloud project, enter values for the following attributes and fields:
 - For the `type` attribute of the `<preference>` element, specify the installation preference type to be Locking. For more information about installation preferences, see [Installation Preferences in the SuiteApp of SuiteCloud Project](#).
 - For the `defaultAction` attribute of the `<preference>` element, specify a default locking action for all files, folders, or files and folders in your SuiteCloud project. Supported values for an action are LOCK and UNLOCK.

Setting a value for the default action can reduce the steps when applying a preference to many files at a time. The default action applies to files in your project that are not in the apply action list of your `locking.xml` file. In the following example, the installation locking preference for files in the SuiteApp project is UNLOCK, so all SuiteApp files other than `myScript.js` are unlocked:

```

1 <preference type="LOCKING" defaultAction="UNLOCK">
2
3   <apply action="LOCK">
4
5     <path>~/FileCabinet/SuiteApps/com.example.helloWorld/myScript.js</path>
6
7   </apply></preference>

```

- For the `action` attribute of the `<apply>` element, specify a value that will apply to the files, folders, or both files and folders. Valid values are LOCK and UNLOCK. For example, `<apply action="LOCK">` locks the specified files or folders.
- For the `<path>` element, specify a file path for the files or folders (or both files and folders) to lock. The following are the possible ways to specify paths:
 - You can specify a path to a single file. For example, `<path>~/FileCabinet/SuiteApps/com.example.helloWorld/myScript.js</path>`.
 - You can specify a path to all files in a target folder (multiple files) using the wildcard (*) and the `appliesTo` attribute for the `<path>` element. For example, `<path appliesTo="FILES">~/FileCabinet/SuiteApps/com.example.helloWorld/*</path>`.
 - You can specify a path to all files and folders in a target folder (multiple files and folders) using the wildcard (*) and the `appliesTo` attribute for the `<path>` element. For example, `<path appliesTo="FILES_AND_FOLDERS" includeRoot="F">~/FileCabinet/SuiteApps/com.example.helloWorld/myFolder/*</path>`.

To lock the root folder as well, set the attribute value `includeRoot="T"` on the `<path>` element. For example: `<path appliesTo="FILES_AND_FOLDERS" includeRoot="T">~/FileCabinet/SuiteApps/com.example.helloWorld/myFolder/*</path>`.

3. Save your changes in the `locking.xml` file.

When you later deploy the SuiteCloud project, to enable the values specified in the `locking.xml` file, you must select the **Apply Installation Preferences** box. For more information about selecting that preference during deployment, see [Applying Installation Preferences During SuiteCloud Project Validation and Deployment](#).

Locking a File Using SDF Example

The following `locking.xml` file example locks the `myScript.js` file by setting the preference type to LOCKING, the default action to UNLOCK, and the apply action to LOCK, and includes a path to the file:

```

1 <preference type="LOCKING" defaultAction="UNLOCK">
2
3   <apply action="LOCK">
4
5     <path>~/FileCabinet/SuiteApps/com.example.helloWorld/myScript.js</path>
6
7   </apply></preference>

```

Locking Multiple Files Using SDF Example

The following locking.xml file example locks all files in a target folder using the wildcard (*) in the path with the appliesTo="FILES" attribute value set for the <path> element:

```

1 <preference type="LOCKING" defaultAction="UNLOCK">
2
3   <apply action="LOCK">
4
5     <path appliesTo="FILES">~/FileCabinet/SuiteApps/com.example.helloWorld/*</path>
6
7   </apply></preference>

```

Locking Files and Folders Using SDF Example

The following locking.xml file example locks all files and folders in myFolder by using the wildcard (*) and the appliesTo="FILES_AND_FOLDERS" attribute for the <path> element. The myFolder folder itself is not locked because includeRoot="F".

```

1 <preference type="LOCKING" defaultAction="UNLOCK">
2
3   <apply action="LOCK">
4
5     <path appliesTo="FILES_AND_FOLDERS" includeRoot="F">~/FileCabinet/SuiteApps/com.example.helloWorld/myFolder/*</path>
6
7   </apply></preference>

```

The following locking.xml file example locks all files and folders in myFolder, including myFolder, by using the wildcard (*) and the appliesTo="FILES_AND_FOLDERS" attribute for the <path> element with includeRoot="T".

```

1 <preference type="LOCKING" defaultAction="UNLOCK">
2
3   <apply action="LOCK">
4
5     <path appliesTo="FILES_AND_FOLDERS" includeRoot="T">~/FileCabinet/SuiteApps/com.example.helloWorld/myFolder/*</path>
6
7   </apply></preference>

```

Overwrite Settings for SuiteApp Projects

When an SDF SuiteApp is updated (or a SuiteApp project deployed), your installation preferences configuration specifies how to preserve some or all of the existing data in the target account for script deployments, custom record types, and custom lists. Preserving data can be useful when you do not want to overwrite things your SuiteApp users have modified in the target account, such as script deployments, custom record instances, or custom list values.

You can configure SDF SuiteApp installation preferences in SuiteCloud Development Framework (SDF) by editing the code block <preference type="OVERWRITING"> in the overwriting.xml file, which is located in the InstallationPreferences folder of your SuiteApp project.

The `overwriting.xml` file is not required in the SuiteApp project, so you can still validate and deploy your SuiteApp project when there are no overwriting installation preferences configured for it.



Important: When you deploy the project, enable the **Apply Installation Preferences** option to apply the settings in the `overwriting.xml` file.

The following information describes the fields you can set for SDF SuiteApp overwriting installation preferences:

- [Overwriting Script Deployments](#)
- [Overwriting Custom Record Types](#)
- [Overwriting Custom Lists](#)

Overwriting Script Deployments

For script deployments, you can set script deployment default preferences for the `scriptdeployments` field, and individual script deployment preferences for the `scriptdeployment` field.

For the `scriptdeployments` field, you can set the `defaultAction` attribute to `OVERWRITE` or `PRESERVE`. This setting is the default applied to all script deployments that are not explicitly set by the `scriptdeployment` field's `action` attribute.

For the `scriptdeployment` field, you can set the `action` attribute to `OVERWRITE` or `PRESERVE`. This setting applies to an individually specified script deployment.

When a script deployment already exists in the target account, specifying `OVERWRITE` overwrites it with the one from your SuiteApp, and specifying `PRESERVE` retains the account version of the script deployment instead of the one from your SuiteApp.

The `scriptid` attribute must follow the pattern as `customscript_{parent_scriptid}.customdeploy_{scriptdeployment_scriptid}`, where `{parent_scriptid}` is the script ID for the script SDF custom object that contains the script deployment, and `{scriptdeployment_scriptid}` is the script ID for the script deployment itself.

The following code sample configures an SDF SuiteApp's installation preferences to overwrite all script deployments of the SuiteApp in the target account by default, with the exception to preserve the script deployment `customscript_user_event_example.customdeploy_ue_example`:

```
1 <scriptdeployments defaultAction="OVERWRITE">
2
3   <scriptdeployment scriptid="customscript_user_event_example.customdeploy_ue_example action="PRESERVE"/>
4
5 </scriptdeployments>
```



Important: In the preceding code sample, all of the script deployments that belong to other objects in the SuiteApp are also overwritten in the target account, not only the script deployments that belong to `customscript_user_event_example`.

For more information about script deployments, see the help topic [Script Deployment](#).

Overwriting Custom Record Types

For custom record types, you can set instance default preferences for the `instances` field, and individual instance preferences for the `instance` field.

For the instances field, you can set the defaultAction attribute to OVERWRITE or PRESERVE. This setting is the default applied to all custom record instances that are not explicitly set by the instance field's action attribute.

For the instance field, you can set the action attribute to OVERWRITE or PRESERVE. This setting applies to an individually specified custom record type instance.

The following code sample configures an SDF SuiteApp's installation preferences to preserve all of the instances for custom record type customrecord_type_example in the target account by default, except for the instance customrecord_instance_example which will be overwritten:

```

1 <customrecordtypes>
2
3   <customrecordtype scriptid="customrecord_type_example">
4
5     <instances defaultAction="PRESERVE">
6
7       <instance scriptid="customrecord_instance_example" action="OVERWRITE"/>
8
9     </instances>
10
11   </customrecordtype>
12
13 </customrecordtypes>

```

For more information about custom record types, see the following topics:

- [Custom Records](#)
- [Custom Record Types as XML Definitions](#)

Overwriting Custom Lists

For custom lists, you can set custom value default preferences for the customvalues field, and individual custom value preferences for the customvalue field.

For the customvalues field, you can set the defaultAction attribute to OVERWRITE or PRESERVE. This setting is the default applied to all custom list values that are not explicitly set by the customvalue field's action attribute.

For the customvalue field, you can set the action attribute to OVERWRITE or PRESERVE. This setting applies to an individually specified list value.

The following code sample configures an SDF SuiteApp's installation preferences to overwrite all of customlist_example custom list's values in the target account by default, except for the customlist_value_example value which will be preserved:

```

1 <customlists>
2
3   <customlist scriptid="customlist_example">
4
5     <customvalues defaultAction="OVERWRITE">
6
7       <customvalue scriptid="customlist_value_example" action="PRESERVE"/>
8
9     </customvalues>
10
11   </customlist>
12
13 </customlists>

```

For more information about custom lists, see the following topics:

- [Custom Lists](#)
- [Custom Lists as XML Definitions](#)

Applying Installation Preferences During SuiteCloud Project Validation and Deployment

The Apply Installation Preferences option appears each time you validate or deploy your SuiteApp project using SuiteCloud SDK tools for SuiteCloud Development Framework (SDF). You must enable this option to apply installation preferences that were specified in the InstallationPreferences folder of the project. For example, to lock specific objects, or hide files by default when deployed. If you do not want to apply installation preferences to a deployment instance, clear the Apply Installation Preferences box when you deploy the project.

If an installation preference for an element changed from the previous deployment, or is applied for the first time, the log will capture the change.

```

1 2018-02-02 06:35:43 (PST) Installation started
2
3 Info -- Account [(PRODUCTION) MyNetSuiteAccount]
4
5 Info -- Publisher ID [com.samples]
6
7 Info -- SuiteApp [com.samples.example2 (1.0.1)]
8
9 Info -- Framework Version [1.0]
10
11 Validate manifest -- Success
12
13 Validate deploy file -- Success
14
15 Validate script file -- Success
16
17 Validate objects -- Success
18
19 Validate files -- Success
20
21 Validate folders -- Success
22
23 Validate preferences -- Success
24
25 Validate flags -- Success
26
27 Validate account settings -- Success
28
29 Validate Custom Objects against the Account -- Success
30
31 Info -- Current installed version [1.0.1]
32
33 Begin deployment -- com.samples.example2 (1.0.1)
34
35 The LOCKING preference type action for the customrecord_sample object has changed to LOCK.
36
37 2018-02-02 06:35:45 (PST) Installation COMPLETE (0 minutes 2 seconds)

```

For more information about installation preferences, see [Installation Preferences in the SuiteApp of SuiteCloud Project](#).

Installation Preferences Validation in SuiteCloud Projects

When you modify installation preferences for the SuiteApp project type of SuiteCloud project for SuiteCloud Development Framework (SDF), ensure you validate your project against the target account for each modification. For more information, see [SuiteCloud Project Validation](#).

The following example shows some possible validation issues that may occur after modifying installation preferences:

Installation Preference	Supported Value	Notes
Preference Type	■ HIDING ■ LOCKING ■ OVERWRITING	Preference and action values must be compatible.
Action defaultAction	■ HIDE ■ UNHIDE ■ LOCK ■ UNLOCK ■ OVERWRITE ■ PRESERVE	Hiding is supported for files only. Locking is supported for certain objects types. For a list, see a Lockable SDF Custom Objects.

Installation Preference File Structure in SuiteCloud Projects

In SuiteCloud projects for SuiteCloud Development Framework, there are three types of preference files: locking preferences, hiding preferences and overwriting. To use these preferences you need to create locking, hiding, and overwriting XML files in the InstallationPreferences folder of your SuiteApp project.

For more information, see the following topics:

- [Installation Preferences in the SuiteApp of SuiteCloud Project](#)
- If you are using the WebStorm plug-in, see the help topic [Creating an Installation Preference File with SuiteCloud IDE Plug-in for WebStorm](#).

To create a locking preference file, the file name must be **locking.xml** and the XML structure must follow the following template:

```

1 <preference type="LOCKING" defaultAction="">
2
3   <apply action="">
4
5   </apply>
6
7 </preference>
```

To create a hiding preference file, the file name must be **hiding.xml** and the XML structure must follow the following template:

```

1 <preference type="HIDING" defaultAction="">
2
3   <apply action="">
4
5   </apply>
6
7 </preference>
```

To create an overwriting preference file, the file name must be **overwriting.xml** and the XML structure must follow the following template:

```

1 <preference type="OVERWRITING">
2
3   <scriptdeployments defaultAction="">
4
```

```

5
6
7     </scriptdeployments>
8
9 </preference>

```

SuiteCloud Project Validation

Validation, Dependency Checking, and Deployment with SuiteCloud Extension for Visual Studio Code

SuiteCloud Development Framework (SDF) validates SuiteCloud projects against the target NetSuite account. This server-side validation catches errors that may occur during the project deployment process.

The following tasks occur during validation:

- The feature and file dependencies in the account get validated.
- The project structure, object definitions, and object dependencies get validated independently of the NetSuite account.
- Field values are validated for missing required fields. Field values from lists are validated against the possible values.
- Validation attempts to collate all errors and warnings unless a fatal error occurs. Fatal errors stop the validation process.
- A validation log is created at the end of execution.

You can use the SuiteCloud IDE plug-in to validate SuiteCloud projects against any NetSuite account to which you have access to. Server-side validation also occurs when you deploy a SuiteCloud project to a target NetSuite account.

For information about validating a SuiteCloud project using:

- SuiteCloud Extension for Visual Studio Code, see the help topic [Deploying a SuiteCloud Project to Your NetSuite Account with SuiteCloud Extension for Visual Studio Code](#),
- SuiteCloud IDE plug-in for Webstorm, see the help topic [Validating a SuiteCloud Project with SuiteCloud IDE Plug-in for WebStorm](#).
- CLI for Java, see the help topic [validate](#).
- CLI for Node.js, see the help topic [project:validate](#).

SuiteCloud Project Deployment Preparation

Validation, Dependency Checking, and Deployment with SuiteCloud Extension for Visual Studio Code

In SuiteCloud Development Framework (SDF), to prepare a SuiteCloud project for deployment, reference the NetSuite components that you want to deploy in the `deploy.xml` file. SDF uses this file to determine the order in which files and objects are deployed to a target NetSuite account.

When you create a SuiteCloud project, a deploy file is automatically generated. By default, the file is configured to deploy all supported files in the `FileCabinet/SuiteScripts` folder and all SDF custom objects in the `Objects` folder. In an account customization project, the file is configured to deploy the account configuration set in the `AccountConfiguration` folder. You can customize which NetSuite components to deploy by adjusting the file paths specified in the **path** elements that are nested in the **configuration**, **files** and **objects** elements. SDF processes each **path** element sequentially when the project is deployed. You can create multiple **files** and **objects** elements to control the order in which files and objects are deployed.



Important: When customizing the deploy file, ensure that the NetSuite components that are dependent on other NetSuite components are deployed afterwards. This practice helps prevent order validation errors from occurring.

Each path must start with a tilde (~), which denotes the root path of your project.

You can use an asterisk (*) in a path to represent all files in a particular folder. During deployment, SDF ensures that all the files in that folder are deployed to a target account in an appropriate order that does not cause order validation errors.

In a SuiteApp project, you can only reference NetSuite components that exist in the project, or objects from an SDF SuiteApp installed to the account. SuiteApp projects can include SDF installation scripts that are automatically executed when the project is deployed. For more information, see [Customizing SuiteApp Project Deployment using SDF Installation Scripts](#).

In an account customization project, you can reference components that exist in the target NetSuite account by declaring them as dependencies in the project manifest. Path names are case-sensitive.

When deploying a project from SuiteCloud IDE plug-in, all files within the project folder are compressed and submitted to the server. For example, omitting a custom employee record from deploy.xml does not prevent the object from being included in the compressed file that is sent to the server. However, when the compressed file is on the server, only files and objects defined in deploy.xml are deployed to the target NetSuite account.

There can be a negative performance impact on deployment when there are a large number of files in the project that are not specified for deployment. Deployment performance can be improved by only including project files that are intended to be deployed.

The maximum file size of your SuiteCloud project (compressed as a zip file during deployment) is 50 MB. To maintain a small project size, you should use File Cabinet to upload large files and images to NetSuite rather than SDF.



Note: Custom objects in an account customization project always replace the custom objects in the NetSuite account when you deploy, regardless of any previous changes made to those objects in the account.

For more information about deploying SuiteCloud projects to a target NetSuite account, see:

- [Deploying a SuiteCloud Project to Your NetSuite Account with SuiteCloud Extension for Visual Studio Code](#)
- [Deploying a SuiteCloud Project to Your NetSuite Account with SuiteCloud IDE Plug-in for WebStorm](#)
- [deploy](#) in Java CLI
- [project:deploy](#) in Node CLI

Example

The following example shows a deploy file for an account customization project. In this example, the project is set to deploy the account configuration that is specified in the AccountConfiguration folder. All files in the FileCabinet/SuiteScripts folder are deployed, and two SDF custom objects, customrecord_employee.xml and customrecord_company.xml, are deployed.

```

1 <deploy>
2
3   <configuration>
4
```

```

5      <path>~/AccountConfiguration/*</path>
6
7    </configuration>
8
9    <files>
10
11      <path>~/FileCabinet/SuiteScripts/*</path>
12
13    </files>
14
15    <objects>
16
17      <path>~/Objects/customrecord_employee.xml</path>
18
19      <path>~/Objects/customrecord_company.xml</path>
20
21    </objects>
22
23  </deploy>

```

Deploy File Templates

In case you need to restore your deploy file, copy the following XML content in a local deploy.xml file and save it in the project folder:

- For an Account customization project, download [this file](#).
- For a SuiteApp, download [this file](#). Change the <path> value to the File Cabinet location for your SuiteApp.

Customizing SuiteApp Project Deployment using SDF Installation Scripts

You can incorporate SuiteCloud Development Framework (SDF) installation scripts into your SuiteApp project deployment. An SDF installation script is a type of SuiteScript that can be triggered and run on the target account during SuiteCloud project deployment. This provides capability to perform customization, set up the account, migrate data, and run other tasks during installation of the SuiteApp. For more information, see the help topics [SuiteScript 2.x SDF Installation Script Type](#) and [SDF Installation Scripts as XML Definitions](#).

To use this feature, you must specify the SDF installation script deployment and when you want it to run from the deploy file of your SuiteApp project. You must include the references you need using script and run elements in the deploy file. SDF deploys referenced components to the target NetSuite account sequentially, first to last in the order they appear.

Depending on the placement of the run element, you can emulate the following script endpoints that were originally included in bundle installation scripts:

- before install
- after install
- before update
- after update

 **Note:** SDF installation scripts cannot be run before uninstall of an SDF SuiteApp.

The following sections provide more information about customizing SuiteApp project deployment using SDF installation scripts:

- [Defining When an SDF Installation Script is Run](#)
- [Handling Exceptions and Monitoring SDF Installation Scripts](#)
- [Deployment that Successfully Runs an SDF Installation Script](#)
- [Deployment Containing a User-defined Script Exception](#)
- [Deployment with JavaScript Exception](#)
- [Deployment with SuiteScript Error \(JSON format\)](#)

Defining When an SDF Installation Script is Run

You can perform this task in a SuiteApp project that contains a SDF installation script file and an `sdfinstallationscript` object. The deployment log includes a run scripts line if a valid `<run>` tag is specified in the `deploy.xml`. See [Handling Exceptions and Monitoring SDF Installation Scripts](#).

For information about the `sdfinstallationscript` object, see [SDF Installation Scripts as XML Definitions](#).

For an example, see [Syntax for Triggering an SDF Installation Script Example](#).

Note: Installation scripts are executed as the role you used to deploy the SuiteApp project.

1. From your SuiteApp project, open the `deploy.xml`.
2. Insert the `run` element where you want the script to be triggered.
3. Add a `script` element inside the `run` element.
4. Within the `script` element, reference the XML file path for the `sdfinstallationscript` object and deployment record script ID for the script that you want to execute.

```

1 <deploy>
2
3   <run>
4
5     <script>
6
7       <path>~/Objects/YOURSCRIPTRECORD.xml</path>
8
9       <deployment>your_deployment_script_id</deployment>
10
11     </script>
12
13   </run>
14
15 </deploy>

```

5. (Optional) To run multiple SDF installation scripts, you can individually add more script references inside the same `run` element, or in a separate `run` element. See [Triggering Multiple SDF Installation Scripts during SuiteApp Project Deployment Example](#).

Note: You cannot use a wildcard (*) in the path to run multiple installation scripts in a folder.

Syntax for Triggering an SDF Installation Script Example

The following `deploy.xml` sample contains a `run` element that uses the file path and script ID of the deployment record to reference the XML definition of the SDF installation script (`customscript_install_script.xml`) and its deployment record (`customdeploy1`).


```

1 <deploy>
2
3   <run>
4
5     <script>
6
7       <path>~/Objects/customscript_install_script.xml</path>
8
9       <deployment>customdeploy1</deployment>
10
11     </script>
12
13   </run>
14
15 </deploy>

```

Triggering Multiple SDF Installation Scripts during SuiteApp Project Deployment Example

In the following example, multiple SDF installation scripts are triggered at different points during deployment through placement of the run tags.

The following actions will occur for each of the three referenced scripts when they are executed:

1. The script file is uploaded.
2. The script record and deployment is created from the XML definition.
3. The script is executed.

```

1 <deploy>
2
3   <files>
4
5     <path>~/FileCabinet/SuiteApps/com.example.scriptdemo/*</path>
6
7   </files>
8
9   <run>
10
11     <script>
12
13       <path>~/Objects/customscript_execute.xml</path>
14
15       <deployment>customdeploy_before_objects</deployment>
16
17     </script>
18
19   </run>
20
21   <objects>
22
23     <path>~/Objects/*</path>
24
25   </objects>
26
27   <run>
28
29     <script>
30
31       <path>~/Objects/customscript_execute2.xml</path>
32
33       <deployment>customdeploy_1</deployment>
34
35     </script>
36
37     <script>
38

```

```

39      <path>~/Objects/customscript_execute.xml</path>
40
41      <deployment>customdeploy_after_objects</deployment>
42
43    </script>
44
45  </run>
46
47</deploy>

```

Handling Exceptions and Monitoring SDF Installation Scripts

To monitor the progress of your installation scripts and the deployment process of your SuiteApp project, you can do the following:

- From SuiteCloud IDE, view the deployment log. The deployment log includes a run scripts line if a valid run element is specified in the deploy.xml. For more information about deployment logs, see [Deployment Logs for SuiteCloud Projects](#).
- In the target NetSuite account, script execution details are logged on the **Execution Log** subtab that appears on the Script page and Script Deployment page. For more information, see the help topic [SuiteScript Monitoring, Auditing, and Logging](#).

SDF installation script failures terminate SuiteApp deployments and can include their own error handling in addition to errors thrown by SDF and SuiteScript. An error thrown by an SDF installation script returns an error followed by the text defined by the script author.

The following deployment logs are sample outputs when a valid run element referencing an SDF installation script is included in the deploy file:

- [Deployment that Successfully Runs an SDF Installation Script](#)
- [Deployment Containing a User-defined Script Exception](#)
- [Deployment with JavaScript Exception](#)
- [Deployment with SuiteScript Error \(JSON format\)](#)

Deployment that Successfully Runs an SDF Installation Script

The following text output is representative of a successful SuiteApp project deployment that runs an SDF installation script:

```

1 2018-08-07 09:37:04 (PST) Installation started
2
3 Info -- Account [(PRODUCTION) DB93_Regress_OW_1326288]
4
5 Info -- Publisher ID [com.netsuite]
6
7 Info -- SuiteApp [com.netsuite.example(1.0.0)]
8
9 Info -- Framework Version [1.0]
10
11 Validate manifest -- Success
12
13 Validate deploy file -- Success
14
15 Validate script file -- Success
16
17 Validate objects -- Success
18
19 Validate files -- Success
20
21 Validate folders -- Success

```

```

22
23 Validate preferences -- Success
24
25 Validate flags -- Success
26
27 Validate account settings -- Success
28
29 Validate Custom Objects against the Account -- Success
30
31 Info -- Current installed version [1.0.0]
32
33 Begin deployment -- com.netsuite.example(1.0.0)
34
35 Run installation script -- customscript_execute (sdfinstallationscript)
36
37 2028-08-07 09:47:18 (PST) Installation COMPLETE (0 minutes 55 seconds)

```

Deployment Containing a User-defined Script Exception

In an SDF installation script, you can write custom code to throw a friendlier error to users that install the SuiteApp and to keep in the SuiteApp project deployment log.

```

1 ...
2
3 Validate account settings -- Success
4
5 Validate Custom Objects against the Account -- Success
6
7 Info -- Current installed version [1.0.0]
8
9 Begin deployment -- com.netsuite.example(1.0.0)
10
11 Run installation script -- customscript_execute (sdfinstallationscript)
12
13 *** ERROR ***
14
15
16
17 An installation script execution error occurred.
18
19 Details: I am a user-defined thrown exception.
20
21 File: ~/FileCabinet/SuiteApps/com.netsuite.example/execute.js
22
23 2028-08-07 09:38:05 (PST) Installation FAILED (1 minute 2 seconds)

```

Deployment with JavaScript Exception

The following log captures a JavaScript error pointing to a problem in the SDF installation script, in this case a type error.

```

1 ...
2
3 Validate account settings -- Success
4
5 Validate Custom Objects against the Account -- Success
6
7 Info -- Current installed version [1.0.0]
8
9 Begin deployment -- com.netsuite.example(1.0.0)
10
11 Run installation script -- customscript_execute (sdfinstallationscript)
12
13 *** ERROR ***
14
15

```

```

16
17 An installation script execution error occurred.
18
19 Details: org.mozilla.javascript.EcmaError: TypeError: Cannot find function isFeatsureInEffect in object runtime. (/SuiteApps/
com.netsuite.example/execute.js#7)
20
21 File: ~/FileCabinet/SuiteApps/com.netsuite.example/execute.js
22
23 2028-08-07 09:38:05 (PST) Installation FAILED (1 minute 2 seconds)

```

Deployment with SuiteScript Error (JSON format)

The following log displays a sample SuiteScript error log in JSON format. The deployment was stopped when a problem occurred during execution of the SDF installation script. The error is generated by SuiteScript as a full stacktrace in JSON format.

In an SDF installation script, you can include a try/catch block with logic to throw your own user-facing error message in place of the SuiteScript error.

```

1 ....
2
3 Validate account settings -- Success
4
5 Validate Custom Objects against the Account -- Success
6
7 Info -- Current installed version [1.0.0]
8
9 Begin deployment -- com.netsuite.example(1.0.0)
10
11 Run installation script -- customscript_execute (sdfinstallationscript)
12
13 *** ERROR ***
14
15
16
17 An installation script execution error occurred.
18
19 Details: {"type":"error.SuiteScriptError","name":"INSUFFICIENT_PERMISSION","message":"Permission Violation: You need the
'Transactions -> Sales Order' permission to access this page. Please contact your account administrator.,"stack":["createEr
ror(N/error.js:469)"," <anonymous>(SYSTEM_LIBS:3704)"," <anonymous>(SYSTEM_LIBS:2702)"," <anonymous>(SYSTEM_LIBS:2819)","in
vokeOn(SYSTEM_LIBS:3703)","createRecord_impl(N/record/recordImpl.js:24)","createRecord(N/record/recordImpl.js:16)","createRe
cord(N/record.js:15)","run(/SuiteApps/com.netsuite.example/execute.js:16)","<anonymous>(INVOCATION_WRAPPER:1)"],"notifyOf
f":false,"id":"","notifyOff":false,"userFacing":false}
20
21 File: ~/FileCabinet/SuiteApps/com.netsuite.example/execute.js
22
23 2028-08-07 09:38:05 (PST) Installation FAILED (1 minute 2 seconds)

```

Deployment Logs for SuiteCloud Projects

When you deploy a SuiteCloud project for SuiteCloud Development Framework (SDF) to the target NetSuite account, an installation log is created and uploaded to that account.

You can review and download installation logs from the **Deployment Audit Trail** page of the NetSuite account. This page is located under Customization > SuiteCloud Development.

The installation log is a step-by-step progress report that includes the following information:

- The objects and files that were installed.
- The date and time when objects and files were installed.
- Any warnings or errors that occurred during deployment.

Note: Server-side validation is always performed prior to a deployment. For more information about validation, see [SuiteCloud Project Validation](#).

Important: You cannot deploy SDF custom objects from account customization projects if the objects are locked in the target NetSuite account. You also cannot use account customization projects to add subrecords to locked record types. For more information about locked record types, see [Locking SDF Custom Objects in the SuiteApp Type of SuiteCloud Project](#) and [Locking Objects in Customization Bundles](#).

For information about deploying a SuiteCloud project using:

- [Deploying a SuiteCloud Project to Your NetSuite Account with SuiteCloud Extension for Visual Studio Code](#).
- SuiteCloud IDE plug-in for WebStorm, see the help topic [Deploying a SuiteCloud Project to Your NetSuite Account with SuiteCloud IDE Plug-in for WebStorm](#).
- Java CLI, see the help topic [deploy](#).
- Node CLI, see the help topic [project:deploy](#).

Viewing and Uninstalling SDF SuiteApps in a NetSuite Account

You can view and uninstall a SuiteCloud Development Framework (SDF) SuiteApp that was previously installed in a NetSuite account by going to Customization > SuiteCloud Development > Installed SuiteApp List.

Note: If you do not see the SuiteCloud Development menu option, you may need to enable the SuiteCloud Development Framework feature. For more information, see [Enabling SuiteCloud Development Framework in the Target NetSuite Account \(Administrator Only\)](#).

To uninstall an SDF SuiteApp from the list of installed SuiteApps, click Uninstall under the Action column.

Important: Account customization projects cannot be uninstalled from a target NetSuite account after they have been deployed. Unwanted account components need to be manually deleted.

For information about how objects are deleted when an SDF SuiteApp is updated or uninstalled, see [Deleting Components in a SuiteCloud Project that were Deployed to a Target Account](#)

Publishing an SDF SuiteApp

For information about publishing a SuiteCloud Development Framework SuiteApp (also known as an SDF SuiteApp) to the SuiteApp Marketplace, see the following topics:

- [Requirements for SuiteApp Marketplace Availability](#)
- [Publisher ID for SDN Partners](#)
- [Getting Started with SuiteApp Control Center](#)

Developing SDF Custom Objects

The following topics provide information about developing XML definitions for SDF custom objects in a SuiteCloud project for SuiteCloud Development Framework (SDF), and validation of SDF custom objects when deploying those projects to a target NetSuite account:

SDF Object Format and Rules

- [Downloading the XML Definition of an Object from a NetSuite Account](#)
- [Translatable Fields on SDF Custom Objects](#)
- [Downloading the XML Definition of an Object from a NetSuite Account](#)
- [SDF Custom Object File Structure](#)
- [Account-Specific Values Validation in SuiteCloud Projects](#)
- [Date and Time Formats in SDF Custom Objects](#)

Custom NetSuite Records, Subrecords, Scripts, Fields, and Lists that are Supported as SDF Custom Objects

- [Custom Address Forms as XML Definitions](#)
- [Custom Entry Forms as XML Definitions](#)
- [Custom Code Tab and Actions Tab in XML Definitions](#)
- [Custom Lists as XML Definitions](#)
- [Financial Reports as XML Definitions](#)
- [Custom Other Record Fields as XML Definitions](#)
- [Custom Item Fields as XML Definitions](#)
- [Custom CRM Fields as XML Definitions](#)
- [Custom Entity Fields as XML Definitions](#)
- [Custom Item Number Fields as XML Definitions](#)
- [Custom Transaction Body Fields as XML Definitions](#)
- [Custom Transaction Line Fields as XML Definitions](#)
- [Custom Transaction Item Option Fields as XML Definitions](#)
- [Custom Record Types as XML Definitions](#)
- [Custom Segments as XML Definitions](#)
- [Custom Transaction Record Types as XML Definitions](#)
- [Transaction Forms as XML Definitions](#)
- [Sublists as XML Definitions](#)
- [Saved CSV Import Maps as XML Definitions](#)
- [Published Dashboards as XML Definitions](#)
- [Scheduled Scripts as XML Definitions](#)
- [SDF Installation Scripts as XML Definitions](#)

- [SSP Applications as XML Definitions](#)
- [Workflows as XML Definitions](#)
- [Custom Plug-ins as XML Definitions](#)
- [KPI Scorecards as XML Definitions](#)
- [Roles and Permissions as an XML Definition](#)
- [Email Templates as XML Definitions](#)
- [Advanced HTML/PDF Templates as XML Definitions](#)
- [CMS Content Types as XML Definitions](#)
- [Translation Collections as XML Definitions](#)
- [Translatable Fields on SDF Custom Objects](#)
- [User Event Scripts as XML Definitions](#)
- [Workbooks as XML Definitions](#)
- [Dataset Builder Plug-ins as XML Definitions](#)
- [Workbook Builder Plug-ins as XML Definitions](#)
- [Integration Records as XML Definitions](#)
- [Secrets as XML Definitions](#)
- [Single Page Applications as XML Definitions](#)
- [Prompts as XML Definitions](#)
- [Text Enhance Actions as XML Definitions](#)
- [Custom Tool Scripts as XML Definitions](#)

Adding SDF Custom Objects to Your SuiteCloud Project

The following topics provide information that you can use to add SDF custom objects to your SuiteCloud project:

- [Creating a SuiteScript File in SuiteCloud Extension for Visual Studio Code](#)
- [Downloading the XML Definition of an Object from a NetSuite Account](#)

Syntax Guidelines and Reference Formats for SDF Custom Objects

Depending on the type of reference or value you want to define in your SDF custom object for SuiteCloud Development Framework (SDF), see the following topics for syntax guidelines and examples:

- [Specifying a Boolean Value](#)
- [Specifying a System Enum](#)
- [Working with Password Fields](#)
- [Specifying a Standard Object Reference to NetSuite Components](#)
- [Specifying an SDF Custom Object Reference](#)
- [Specifying a Custom File or Script Reference](#)
- [Specifying a Custom Segment Reference](#)

To reference an object that is external to your SuiteCloud project, see the following topics:

- [Specifying a Reference to an SDF Custom Object in Another SuiteApp](#)
- [Specifying a Bundle Object Reference](#)

Specifying a Boolean Value

To specify a Boolean value, set the content to either **T** or **F**. For example:

```
1 | <inactive>F</inactive>
```

Specifying a System Enum

To specify a system enum, ensure that the content is in upper case. For example:

```
1 | <permkey>ADMIN_CUSTRECORD</permkey>
```

Working with Password Fields

When working with password fields, note that any content that you specify is ignored when you deploy the SuiteCloud project. When you import a custom object from a NetSuite account, the password fields are masked as **(Encrypted)**. For example:

```
1 | <custrecord_password>(Encrypted)</custrecord_password>
```

Specifying a Standard Object Reference to NetSuite Components

To specify a standard object reference to NetSuite components, ensure that the content is in upper case. For example:

```
1 | <recordtype>CUSTOMER</recordtype>
```

Specifying an SDF Custom Object Reference

To specify an SDF custom object reference, ensure that you use a `scriptid` attribute=value pair encased in square brackets. For example, the following is an SDF custom object reference to an object with a script ID of `custobject_tutorial`:

```
1 | <recordtype>[scriptid=customrecord_sample]</recordtype>
```

Script ID descriptors can be set up according to a parent/child relationship, such as a custom field on a record. To specify an object in a hierarchical context, from the object's XML file, include the reference within square brackets using the following format: `[parentscriptid=<scriptid>. childscriptid=<scriptid>]`. In the following example, the script ID of the object is prefixed with the script ID of the parent, where parent script ID is `customrecordscriptid` and child script ID is `customrecordcustomfieldscriptid` delimited by a period(.):

```
1 | [scriptid=customrecordscriptid.customrecordcustomfieldscriptid]
```


Specifying a Reference to an SDF Custom Object in Another SuiteApp

In addition to the `scriptid` descriptor, SDF custom object references support an `appid` descriptor.

To specify an external SDF custom object reference, include the reference within square brackets in the object's XML file, using the following format: `[appid=<appid>, scriptid=<scriptid>]`. For example, the following is an SDF custom object reference to an object with an application ID of `com.samples.mysuiteapp` and a script ID of `custobject_tutorial`:

```
1 | [appid=com.example.mysuiteapp, scriptid=customrecord_example_mysuiteapp_objectname]
```

When creating script IDs for SDF custom objects in your SuiteApp using SDF or NetSuite, you should always include your company and application names to mitigate Script ID collisions.

Note: In an account customization project, any custom fields or objects that you deploy to a target account cannot be deleted from that account using SuiteCloud Development Framework (SDF). You can only create or modify objects. To delete an SDF custom object, you must remove all object references, and manually delete it from the account.

Specifying a Bundle Object Reference

To specify an external bundle object reference, you must include a bundle ID descriptor.

In the object's XML file where you want to add the reference, use the following format: `[bundleid=<number>, scriptid=<scriptid>]`. For example, the following is a reference to an SDF custom object with a bundle ID of 351 and a script ID of `custobject_tutorial`:

```
1 | [bundleid=351, scriptid=customrecord]
```

Note: You can only reference one bundle ID at the object level.

Specifying a Custom Segment Reference

When working with custom segment references, a type descriptor can be used to reference unified IDs. For example, the following is an SDF custom object reference to a `customrecordcustomfield` object with a script ID of `customrecordscriptid.customrecordcustomfieldscriptid` that exists in an SDF SuiteApp with the application ID of `com.samples.mysuiteapp`:

```
1 | [appid=com.samples.mysuiteapp, type=customrecordcustomfield, scriptid=customrecordscriptid.customrecordcustomfieldscriptid]
```

Important: The type descriptor is only supported in custom field references that are generated by the application of a custom segment to a record. These references are typically custom segment unified field IDs. You should always enable unified IDs on custom segments when using them with SDF. By using unified IDs, the script IDs in your custom segment objects do not contain account-specific values, making them more portable and accessible to other NetSuite accounts. For more information about creating a custom segment in NetSuite with unified IDs, see the help topic [Creating a Custom Segment](#).

Specifying a Custom File or Script Reference

To specify a custom file or script reference, ensure that the content is the file path enclosed in square brackets. For example:

```
1 | <scriptfile>[/SuiteScripts/UserEventScript.js]</scriptfile>
```

Translatable Fields on SDF Custom Objects

When creating SDF custom objects in SuiteCloud projects for SuiteCloud Development Framework (SDF) that have translatable fields, you can reference translation strings from translation collections. After the custom objects and the translation collections are deployed to an account, the NetSuite UI displays the translation for each translatable field in the account language. If a field has no translation in the account language, the default translation is displayed in the NetSuite UI.

For information about translatable fields function in a NetSuite account, see the help topic [String Hierarchy and Translation String Linking](#).

For more information about translatable fields in SDF custom objects, see the following:

- [Using Translatable Fields in SDF Custom Objects](#)
- [Translatable Fields Supported in SDF Custom Objects](#)
- [Considerations For Translatable Fields](#)
- [Translation Reference Format](#)
- [Translatable Field XML Examples](#)

Using Translatable Fields in SDF Custom Objects

The following steps contain the high-level process for using translations in XML definitions for SDF custom objects:

1. Identify the SDF custom object that contains the translatable fields that you intend to translate. For information about that, see [Translatable Fields Supported in SDF Custom Objects](#).
To create an SDF custom object from a template, see the following topics:
 - [Creating a SuiteScript File in SuiteCloud Extension for Visual Studio Code](#)
 - [Creating an XML Definition for an SDF Custom Object in SuiteCloud IDE Plug-in for WebStorm](#)
 To create or import or an existing SDF custom object from a NetSuite account, see the following topics:
 - [Importing SDF Custom Objects from a NetSuite Account with SuiteCloud Extension for Visual Studio Code](#)
 - [Importing SDF custom objects into a SuiteCloud Project with SuiteCloud IDE Plug-in for WebStorm](#)
 - [Downloading the XML Definition of an Object from a NetSuite Account](#)
2. Create or import a translation collection, and then add translation strings (terms) for each SDF custom object field value that you would like to translate. For more information, see [Translation Collections as XML Definitions](#).
3. For the translatable fields in the SDF custom object from step 1, set the attribute `translate="T"`, and then reference the term from the translation collection from step 2.

The following is an example of the `displaylabel` field referencing the hotel term in the `custcollection_accommodations` translation collection:

```
1 | <displaylabel translate="T">[scriptId=custcollection_accommodations.hotel]</displaylabel>
```

For information about the translation reference format and code samples, see [Translation Reference Format](#) and [Translatable Field XML Examples](#).

4. Export the XLIFF file for the translation collection from step 2, and send it to your translators. For information about exporting XLIFF files, see the help topic [Exporting a Translation Collection](#).
5. Add XLIFF files for various languages received from the translators to the translations folder in the SuiteCloud project. For more information about SuiteCloud project folders, see [SuiteCloud Project Structure and File Components](#).

Translatable Fields Supported in SDF Custom Objects

You can identify translatable fields that are supported in SDF custom objects by searching in the for the following text: **This field accepts references to the string custom type**.

The string custom type refers to the [string](#) topic that has information about the translation collection fields. For more information about those fields, see [Translation Collections as XML Definitions](#)

Considerations For Translatable Fields

Consider the following information when using translatable fields in XML definitions:

- A translatable field cannot reference translation strings that are not in the same SuiteCloud project.
- Translation strings can be reused in multiple translatable fields within your SDF SuiteApp. Consider the following information when reusing translation strings for translatable fields:
 - Multiple SDF custom objects within the same SuiteApp can reference the same translation string. SDF custom objects in other SuiteApps cannot reference that same translation string.
 - Multiple fields within the same SDF custom object can reference the same translation string.
- Setting the translation=F attribute for a translatable field signifies that the value is a literal string. For information about the translate attribute and reference format, see [Translation Reference Format](#).

Translation Reference Format

Note: The translation reference format requires a translation collection object. For information about using translation collections in SDF, see [Translation Collections as XML Definitions](#).

You can translate fields by using the following format:

- Set the field translate attribute to true, by setting translate=T.
- For the field value, specify a reference to the translation using the following format: *translationCollection.string*, where: *translationCollection* is the name of the translation collection and *string* is the script id for the translation string in the specified translation collection. For example, *scriptid=custcollection_accomodations.hotel* is a script ID reference to the string hotel in the translation collection *custcollection_accomodations*.

The following example combines the two previous examples and presents them in XML format:

```
1 | <displaylabel translate="T">[scriptid=custcollection_accomodations.hotel]</displaylabel>
```

You can provide a literal string for translatable fields by specifying translate attribute as false and providing a literal string value. For example:

```
1 | <displaylabel translate="F">Hotel</displaylabel>
```

If the translate attribute is omitted, it is the same as specifying translate="F".

Translatable Field XML Examples

The following example is a workflow SDF custom object with a label field that contains a reference to the term approve in the translation collection custcollection_approvals:

```

1 <workflow scriptid="customworkflow1">
2
3   <name>My custom workflow</name>
4
5   <recordtypes>LEAD</recordtypes>
6
7   ...
8
9   <workflowstates>
10
11    ...
12
13    <workflowstate scriptid="workflowstate_approvalpending">
14
15      <name>State 2: Pending Approval</name>
16
17      ...
18
19      <workflowactions triggertype="BEFORELOAD">
20
21        <addbuttonaction scriptid="workflowaction_approvebutton">
22
23          ...
24
25          <label translate="T">[scriptid=custcollection_approvals.approve]</label>
26
27          ...
28
29        </addbuttonaction>
30
31      </workflowactions>
32
33    </workflowstate>
34
35    ...
36
37  </workflowstates>
38
39  ...
40
41 </workflow>

```

The following example is a custom workflow object where:

- The workflow name references the string opportunityname in the translation collection custcollection_opportunity.
- The workflow state name references the string state1 in the translation collection custcollection_opportunity.

```

1 <workflow scriptid="customworkflow_opportunity">
2
3   ...
4
5   <name translate="T">[scriptid=custcollection_opportunity.opportunityname]</name>
6
7   ...
8
9   <workflowstates>
10
11    <workflowstate scriptid="workflowstate1">
12
13      <description></description>
14
15      <donotexitworkflow>F</donotexitworkflow>

```

```

16
17     <name translate=T>[scriptid=custcollection_opportunity.state1]</name>
18
19     <positionx>243</positionx>
20
21     <positiony>133</positiony>
22
23     <workflowactions triggerType="ONENTRY">
24
25         <setfieldmandatoryaction scriptid="workflowaction1">
26
27             <clienttriggerfields></clienttriggerfields>
28
29             <conditionsavedsearch></conditionsavedsearch>
30
31             <contexttypes></contexttypes>
32
33             <eventtypes></eventtypes>
34
35             <field>STDBODYTITLE</field>
36
37             <isinactive>F</isinactive>
38
39             <ismandatory>T</ismandatory>
40
41             <initcondition>
42
43                 <formula></formula>
44
45                 <type>VISUAL_BUILDER</type>
46
47             </initcondition>
48
49         </setfieldmandatoryaction>
50
51     </workflowactions>
52
53 </workflowstate>
54
55 </workflowstates>
56
57 </workflow>

```

The following example is a translation collection object where its own name and description fields are translated:

```

1 <translationcollection scriptid='custcollection_accommodation'>
2
3     <name translate="T">[scriptid=custcollection_accommodation'.accommodation]</name>
4
5     <description translate="T">[scriptid=custcollection_accommodation.accommodation_description]</description>
6
7     <defaultlanguage>en-US</defaultlanguage>
8
9     <description>This Translation Collection contains terms for the Hotel customization.</description>
10
11     <strings>
12
13         <string scriptid='accommodation'>
14
15             <defaulttranslation>Accommodation Translation Collection</defaulttranslation>
16
17         </string>
18
19     </strings>
20
21     <strings>
22
23         <string scriptid='acommodation_description'>
24
25             <defaulttranslation>This translation collection is translated into multiple languages.</defaulttranslation>
26

```

```

27     </string>
28
29   </strings>
30
31 </translationcollection>

```

In the preceding example, the name of the translation collection references the string accommodation and the description for the translation collection references the string accommodation_description. Both the name and description fields reference strings in the custcollection_accommodation translation collection, which is the same translation collection for those name and description fields.

Downloading the XML Definition of an Object from a NetSuite Account

For some NetSuite record types supported as SDF custom objects by SuiteCloud Development Framework (SDF), you can download XML definitions directly from your NetSuite account. You can preview the downloaded file to learn how an SDF custom object is structured, or manually incorporate the definition in a SuiteCloud project.

You can also learn about the following:

- [Developing SDF Custom Objects](#)
- [Creating an XML Definition for an SDF Custom Object in SuiteCloud IDE Plug-in for WebStorm](#)
- [Comparing an SDF Custom Object with an Object from the Account in SuiteCloud IDE Plug-in for WebStorm](#)
- [Overwriting SDF Custom Objects with their Matching Objects Imported from the Account in SuiteCloud IDE Plug-in for WebStorm](#)

To download an XML definition from a NetSuite account:

1. Log in to a NetSuite account.
2. Open a record type that is supported by SuiteCloud Development Framework as an SDF custom object.
 For example, go to Customization > Lists, Records, & Fields > Record Types, and select any of the available record types.
 For information about NetSuite components supported as SDF custom objects, see [Customizations Supported by SuiteCloud Development Framework](#).
3. In the **Actions** menu, click **Download XML** or **Download XML & Include Record Instances**.
 Your browser prompts you to download and save the XML definition of the SDF custom object for the record type that you opened.
 If you downloaded the XML definition with record instances, the references to the record instances are nested in an instances XML element.

Alternatively, you can use SuiteCloud IDE plug-in to import XML definitions from a NetSuite account to a SuiteCloud project. For more information, see:

- Using the extension for Visual Studio Code, see the help topics [Importing Files from a NetSuite Account with SuiteCloud Extension for Visual Studio Code](#) and [Importing SDF Custom Objects from a NetSuite Account with SuiteCloud Extension for Visual Studio Code](#).
- Using IDE plug-in for WebStorm, see the help topic [Importing Account Components to your SuiteCloud Project with SuiteCloud IDE Plug-in for WebStorm](#).
- Using CLI for Node.js, see the help topics [file:import](#) and [object:import](#).
- Using CLI for Java, see the help topics [importfiles](#) and [importobjects](#).

Important: You cannot import locked record types into SuiteCloud projects. You must either unlock the record types before you import them, or reference them outside of the SuiteCloud project. For more information about locked record types, see [Locking SDF Custom Objects in the SuiteApp Type of SuiteCloud Project](#) and [Locking Objects in Customization Bundles](#).

SDF Custom Object File Structure

This topic provides the script ID prefix and root XML tag for each supported SDF custom object in SuiteCloud Development Framework (SDF). This information can be useful when you are creating a new XML definition of an SDF custom object from your SuiteCloud project.

Important: Objects must be stored in the Objects folder of the SuiteCloud project. The correct prefix must be used for its object type. The script ID for an SDF custom object must match that object's XML filename.

Tip: For additional information about supported SDF custom objects, see [Customizations Supported by SuiteCloud Development Framework](#) and .

SDF Custom Object Type	Script ID Prefix	Root XML Tag	Topics
addressform	custaddr essform_	<pre> 1 <addressForm scriptid="custform_"> 2 3 ... 4 5 </addressForm> </pre>	Custom Address Forms as XML Definitions and addressForm
advancedpdftemplate	custtm pl_	<pre> 1 <advancedpdftemplate scriptid="custtmpl_"> 2 3 ... 4 5 </advancedpdftemplate> </pre>	Advanced HTML/ PDF Templates as XML Definitions and advancedpdf template
bankstatementparserplugin	customs cript	<pre> 1 <bankstatementparserplugin scriptid="customscript_"> 2 3 . 4 5 </bankstatementparserplugin> </pre>	Bank Statement Parser Plug-ins as XML Definitions and bankstatemen tparserplugin
bundleinstallationscript	customsc ript_	<pre> 1 <bundleinstallationscript scriptid="customscript_"> 2 3 ... 4 5 </bundleinstallationscript> </pre>	SuiteScript 2.x Bundle Installation Script Type and b undleinstallations cript
center	custcen ter_	<pre> 1 <center scriptid="custcenter_"> 2 3 ... 4 5 </center> </pre>	Centers as XML Definitions and center
centercategory	custcen tercate gory_	<pre> 1 <centercategory scriptid="custcentercategory_"> 2 3 ... 4 5 </centercategory> </pre>	Center Categories as XML Definitions and centercategory
centerlink	custli nk_	<pre> 1 <centerlink scriptid="custlink_"> 2 3 ... 4 5 </centerlink> </pre>	Center Categories as XML Definitions and centerlink

SDF Custom Object Type	Script ID Prefix	Root XML Tag	Topics
centertab	custcentertab_	<pre> 1 <centercentertab scriptid="custcentertab_"> 2 3 ... 4 5 </centercentertab> </pre>	Center Tabs as XML Definitions and centertab
clientscript	customscript_	<pre> 1 <clientscript scriptid="customscript_"> 2 3 ... 4 5 </clientscript> </pre>	Client Scripts as XML Definitions and clientscript
cmscontenttype	custcontenttype_	<pre> 1 <cmscontenttype scriptid="custcontenttype_"> 2 3 ... 4 5 </cmscontenttype> </pre>	CMS Content Types as XML Definitions see the help topic cmscontenttype
crmcustomfield	crmcustomfield_	<pre> 1 <crmcustomfield scriptid="crmcustomfield_"> 2 3 ... 4 5 </crmcustomfield> </pre>	Custom CRM Fields as XML Definitions and crmcustomfield
customglplugin	customscript_	<pre> 1 <customglplugin scriptid="customscript_"> 2 3 ... 4 5 </customglplugin> </pre>	Custom GL Lines Plug-ins as XML Definitions and customglplugin
customlist	customlist_	<pre> 1 <customlist scriptid="customlist_"> 2 3 ... 4 5 </customlist> </pre>	Custom Lists as XML Definitions and customlist
customrecordtype	customrecord_	<pre> 1 <customrecordtype scriptid="customrecord_"> 2 3 ... 4 5 </customrecordtype> </pre>	Custom Record Types as XML Definitions and customrecordtype
customsegment	customsegment_	<pre> 1 <customsegment scriptid="cseg_"> 2 3 ... 4 5 </customsegment> </pre>	Custom Segments as XML Definitions and customsegment
customtransactiontype	customtransaction_	<pre> 1 <customtransactiontype scriptid="customtransaction_"> 2 3 ... 4 5 </customtransactiontype> </pre>	Custom Transaction Record Types as XML Definitions and customtransactiontype
dataset	custdataset_	<pre> 1 <dataset scriptid="custdataset_"> 2 3 ... 4 5 </dataset> </pre>	Custom Dataset Objects and dataset



Note: Custom segment objects must be associated with a custom record object type.

SDF Custom Object Type	Script ID Prefix	Root XML Tag	Topics
		 Important: Do not modify the contents of the definition structure or its character data. Modifications may damage the workbook data when you deploy the modified object to a target NetSuite account. You should use NetSuite to modify custom workbooks rather than using SDF to modify workbook objects. After you save your changes in NetSuite and re-import the workbook into your SuiteCloud project, the workbook object gets updated safely and automatically.	
datasetbuilderplugin	customscript_	<pre> 1 <datasetbuilderplugin scriptid="customscript_"> 2 3 ... 4 5 </datasetbuilderplugin> </pre>	Dataset Builder Plug-ins as XML Definitions and datasetbuilderplugin
emailcaptureplugin	customscript_	<pre> 1 <emailcaptureplugin scriptid="customscript_"> 2 3 ... 4 5 </emailcaptureplugin> </pre>	Email Capture Plug-ins as XML Definitions and emailcaptureplugin
emailtemplate	customemailtpl_	<pre> 1 <emailtemplate scriptid="customemailtpl_"> 2 3 ... 4 5 </emailtemplate> </pre>	Email Templates as XML Definitions and emailtemplate
entitycustomfield	customentity_	<pre> 1 <entitycustomfield scriptid="customentity_"> 2 3 ... 4 5 </entitycustomfield> </pre>	Custom Entity Fields as XML Definitions and entitycustomfield
entryForm	customform_	<pre> 1 <entryform scriptid="customform_"> 2 3 ... 4 5 </entryform> </pre>  Important: Due to the complexity of this object, you should import an existing instance of this object type from a NetSuite account when you want to work with this object in the SuiteCloud project.	Custom Entry Forms as XML Definitions and entryForm
ficonnectivityplugin	customscript	<pre> 1 <ficonnectivityplugin scriptid="customscript_"> 2 3 ... 4 5 </ficonnectivityplugin> </pre>	Financial Institution Connectivity Plug-ins as XML Definitions and ficonnectivityplugin
financiallayout	customlayout_	<pre> 1 <financiallayout scriptid="customlayout_"> 2 3 ... 4 5 </financiallayout> </pre>	Financial Reports as XML Definitions, reportdefinition and financiallayout
fiparserplugin	customscript	<pre> 1 <fiparserplugin scriptid="customscript_"> 2 3 ... 4 5 </fiparserplugin> </pre>	Financial Institution Parser Plug-ins as XML Definitions and fiparserplugin
integration	custominteg	<pre> 1 <integration scriptid="custominteg_"> 2 3 ... 4 5 </integration> </pre>	Integration Records as XML Definitions and integration

SDF Custom Object Type	Script ID Prefix	Root XML Tag	Topics
itemcustomfield	custitem_	<pre> 1 <itemcustomfield scriptid="custitem_"> 2 ... 3 ... 4 ... 5 </itemcustomfield> </pre>	Custom Item Fields as XML Definitions and itemcustomfield
itemnumbercustomfield	custitemnumber_	<pre> 1 <itemnumbercustomfield scriptid="custitemnumber_"> 2 ... 3 ... 4 ... 5 </itemnumbercustomfield> </pre>	Custom Item Number Fields as XML Definitions and itemnumbercustomfield
itemoptioncustomfield	custcol_	<pre> 1 <itemoptioncustomfield scriptid="custcol_"> 2 ... 3 ... 4 ... 5 </itemoptioncustomfield> </pre>	Custom Transaction Item Option Fields as XML Definitions and itemoptioncustomfield
kpiscorecard	custkpi scorecard_	<pre> 1 <kpiscorecard scriptid="custkpi scorecard_"> 2 ... 3 ... 4 ... 5 </kpiscorecard> </pre>	KPI Scorecards as XML Definitions and kpiscorecard
mapreducescript	customscript_	<pre> 1 <mapreducescript scriptid="customscript_"> 2 ... 3 ... 4 ... 5 </mapreducescript> </pre>	Map/Reduce Script Objects as XML Definitions and mapreducescript
massupdatescript	customscript_	<pre> 1 <massupdatescript scriptid="customscript_"> 2 ... 3 ... 4 ... 5 </massupdatescript> </pre>	Mass Update Scripts as XML Definitions and massupdatescript
othercustomfield	customrecord_	<pre> 1 <othercustomfield scriptid="customrecord_"> 2 ... 3 ... 4 ... 5 </othercustomfield> </pre>	Custom Other Record Fields as XML Definitions and othercustomfield
pluginimplementation	customscript_	<pre> 1 <pluginimplementation scriptid="customscript_"> 2 ... 3 ... 4 ... 5 </pluginimplementation> </pre>	Creating a Custom Plug-in Implementation in an XML Definition and pluginimplementation
plugintype	customscript_	<pre> 1 <plugintype scriptid="customscript_"> 2 ... 3 ... 4 ... 5 </plugintype> </pre>	Creating a Custom Plug-in Type in an XML Definition and plugintype
portlet	customscript_	<pre> 1 <portlet scriptid="customscript_"> 2 ... 3 ... 4 ... 5 </portlet> </pre>	Portlet Scripts as XML Definitions and portlet
publisheddashboard	custpub dashboard_	<pre> 1 <publisheddashboard scriptid="custpub dashboard_"> 2 ... 3 ... 4 ... 5 </publisheddashboard> </pre>	Published Dashboards as XML Definitions and publisheddashboard
reportdefinition	customreport_	<pre> 1 <reportdefinition scriptid="customreport_"> 2 ... 3 ... 4 ... 5 </reportdefinition> </pre>	Financial Reports as XML Definitions, reportdefinition and financiallayout

SDF Custom Object Type	Script ID Prefix	Root XML Tag	Topics
restlet	customscript_	<pre> 1 <restlet scriptid="customscript_"> 2 3 ... 4 5 </restlet> </pre>	RESTlet Scripts as XML Definitions and restlet
role	customrole_	<pre> 1 <role scriptid="customrole_"> 2 3 ... 4 5 </role> </pre>	Custom Roles as XML Definitions and role
savedcsvimport	custimport_	<pre> 1 <savedcsvimport scriptid="custimport_"> 2 3 ... 4 5 </savedcsvimport> </pre>	Saved CSV Import Maps as XML Definitions savedcsvimport
savedsearch	customsearch_	<pre> 1 <savedsearch scriptid="customsearch_"> 2 3 4 ... 5 6 7 </savedsearch> </pre>  Warning: Do not manually edit the savedsearch XML in SDF. Modifications to the system-generated XML may result in validation and deployment failures. You should customize saved searches with NetSuite rather than SDF.	Saved Searches as XML Definitions and savedsearch
scheduledscript	customscript_	<pre> 1 <scheduledscript scriptid="customscript_"> 2 3 ... 4 5 </scheduledscript> </pre>	Scheduled Scripts as XML Definitions and scheduledscript
sdfinstallationscript	customscript_	<pre> 1 <sdfinstallationscript scriptid="customscript_"> 2 3 ... 4 5 </sdfinstallationscript> </pre>	SDF Installation Scripts as XML Definitions sdfinstallationscript
secret	custsecret_	<pre> 1 <secret scriptid="custsecret_"> 2 3 </secret> </pre>	Secrets as XML Definitions and secret
singlepageapp	custspa_	<pre> 1 <singlepageapp scriptid="custspa_"> 2 3 ... 4 5 </singlepageapp> </pre>	Single Page Applications as XML Definitions and singlepageapp
sspapplication	webapp_	<pre> 1 <sspapplication scriptid="webapp_"> 2 3 ... 4 5 </sspapplication> </pre>	SSP Applications as XML Definitions and sspapplication
sublist	custsublist_	<pre> 1 <sublist scriptid="custsublist_"> 2 3 ... 4 5 </sublist> </pre>	Sublists as XML Definitions and sublist
subtab	custtab_	<pre> 1 <subtab scriptid="custtab_"> 2 3 ... 4 5 </subtab> </pre>	Subtabs as XML Definitions and subtab

SDF Custom Object Type	Script ID Prefix	Root XML Tag	Topics
suitelet	customscript_	<pre> 1 <suitelet scriptid="customscript_"> 2 3 ... 4 5 </suitelet> </pre>	Suitelet Scripts as XML Definitions and suitelet
tool	customtool_	<pre> 1 <tool scriptid="customtool_"> 2 3 ... 4 5 </tool> </pre>	Custom Tool Scripts as XML Definitions and tool
transactionForm	custform_	<pre> 1 <transactionForm scriptid="custform_"> 2 3 ... 4 5 </transactionForm> </pre>  Important: Due to the complexity of this object, you should import an existing instance of this object type from a NetSuite account when you want to work with this object in the SuiteCloud project.	Transaction Forms as XML Definitions and transaction Form
transactionbodycustomfield	custbody_	<pre> 1 <transactionbodycustomfield scriptid="custbody_"> 2 3 ... 4 5 </transactionbodycustomfield> </pre>	Custom Transaction Body Fields as XML Definitions and transactionbodycustomfield
transactioncolumncustomfield	custcol_	<pre> 1 <transactioncolumncustomfield scriptid="custcol_"> 2 3 ... 4 5 </transactioncolumncustomfield> </pre>	Custom Transaction Line Fields as XML Definitions and transactioncolumncustomfield
translationcollection	custcollection_	<pre> 1 <translationcollection scriptid="custcollection_"> 2 3 4 5 </translationcollection> </pre>	Translation Collections as XML Definitions and translationcollection
usereventscript	customscript_	<pre> 1 <usereventscript scriptid="customscript_"> 2 3 ... 4 5 </usereventscript> </pre>	User Event Scripts as XML Definitions and usereventscript
workbook	custworkbook_	<pre> 1 <workbook scriptid="custworkbook_"> 2 3 ... 4 5 </workbook> </pre>  Important: Do not modify the contents of the definition structure or its character data. Modifications may damage the workbook data when you deploy the modified object to a target NetSuite account. You should use NetSuite to modify custom workbooks rather than using SDF to modify workbook objects. After you save your changes in NetSuite and re-import the workbook into your SuiteCloud project, the workbook object gets updated safely and automatically.	Workbooks as XML Definitions and workbook
workbookbuilderplugin	customscript_	<pre> 1 <workbookbuilderplugin scriptid="customscript_"> 2 3 ... 4 5 </workbookbuilderplugin> </pre>	Workbook Builder Plug-ins as XML Definitions and workbookbuilderplugin

SDF Custom Object Type	Script ID Prefix	Root XML Tag	Topics
workflow	customworkflow_	<pre> 1 <workflow scriptid="customworkflow_"> 2 3 ... 4 5 </workflow> </pre>	Workflows as XML Definitions and workflow
workflowactionscript	customscript	<pre> 1 <workflowactionscript scriptid="customscript_"> 2 3 ... 4 5 </workflowactionscript> </pre>	workflowactionscript

Account-Specific Values Validation in SuiteCloud Projects

Account-specific values refer to information defined in a NetSuite account, such as internal IDs, record instances, sensitive data, and personal data entry. Dependencies on this type of value can be problematic in applications or customizations that are developed outside a NetSuite account. By default, SuiteCloud Development Framework (SDF) prevents deployment of any SuiteCloud project that contains account-specific values. Deployment is stopped and an error is thrown when an account-specific value is detected.

When working with account customization projects, you can override the default behavior by updating the project settings, or changing the Action on Account-Specific Values option for a validation or deployment instance.

To change how account-specific values are validated, set the Action on Account-Specific Values option to one of the following values:

- ERROR
- WARNING

For more information about setting this option using SuiteCloud Extension for Visual Studio Code, see the following topic:

- [Deploying a SuiteCloud Project to Your NetSuite Account with SuiteCloud Extension for Visual Studio Code.](#)

For more information about setting this option using SuiteCloud IDE plug-in for Webstorm, see the following topics:

- [Changing Project Settings in SuiteCloud IDE Plug-in for WebStorm](#)
- [Validating a SuiteCloud Project with SuiteCloud IDE Plug-in for WebStorm](#)
- [Deploying a SuiteCloud Project to Your NetSuite Account with SuiteCloud IDE Plug-in for WebStorm](#)

For information about setting this option using CLI for Java, see the following topics:

- [deploy](#)
- [validate](#)

For information about setting this option using CLI for Node.js, see the following topics:

- [project:deploy](#)
- [project:validate](#)

When set to WARNING, SDF captures warnings about the account-specific values. Deployment can proceed and you can change the values after deployment. You can choose this setting when you are an advanced user with a particular development or testing scenario. For example, this setting can help you when

you recently synchronized your production and sandbox databases and you want to deploy an account customization containing references to equivalent internal IDs that exist in both accounts.

 **Note:** The Action on Account-Specific Values setting only applies to instances of account-specific values in optional fields. SDF returns an error if an account-specific value exists in a required field.

When set to ERROR, SDF throws error messages for account-specific values and stops deployment of the project. To successfully deploy the project, you must remove instances of [ACCOUNT_SPECIFIC_VALUE]. The default setting is error.

You should remove all references to account-specific values to make your project compatible with any account type. Unexpected behavior and data changes can occur if you rely on account-specific values in a customization. The value may not exist in the target account, or a different object may use that value. For example, if a project uses an internal ID to find an employee and send a payroll notification, and you deploy it to a different account, some possibilities are:

- Notifications are not sent because the id was not found, or the object type with that ID was not compatible.
- Notifications are sent to the wrong person because the internal ID matches the reference to that person.

 **Note:** The Action on Account-Specific Values setting is not supported in SuiteCloud IDE plug-in for WebStorm.

Date and Time Formats in SDF Custom Objects

SuiteCloud Development Framework (SDF) uses a standard date and time format across all supported SDF custom objects. The date and time format is based on the UTC standard, and is ISO compliant to ensure data consistency and compatibility in SuiteCloud projects.

If the incorrect date and time format is specified, the validation log returns an error with the correct date and time format to use. For information about the custom fields that are supported in NetSuite, see the help topic [Field Type Descriptions for Custom Fields](#).

For information about the UTC standard, see:

- <https://www.timeanddate.com/time/aboututc.html>
- https://en.wikipedia.org/wiki/Coordinated_Universal_Time
- https://en.wikipedia.org/wiki/ISO_8601

 **Important:** SDF custom objects do not support seconds. If you deploy an SDF custom object that contains a field with a Time or Date And Time value that contains seconds, the seconds are not retained in the target NetSuite account.

Date and time can be specified in the following ways:

- **Date** is expressed in yyyy-MM-dd format. For example, 2020-07-24 is a valid Date value.
- **Time** can be expressed in following ways:
 - **Time With Time Zone** is expressed in HH:mm:00Z format, where Z represents the UTC standard and HH is a 24 hour value. For example, 09:30:00Z is a valid Time With Time Zone value. Some fields, such as custom record custom fields, do not have time zones, so they require Time values that do not specify a time zone.

When a Time With Time Zone value is deployed to an account, the time is converted from the UTC standard to the time zone preference of the account. For example, if you deploy a Time With Time

Zone value of 09:30:00Z to an account with the time zone set to Pacific Daylight Time (PDT or UTC–7), the date remains the same but the time is set to 2:30 PM.

- **Time Without Time Zone** is expressed in HH:mm:00 format, and represents a time of day that occurs regardless of the time zone. For example, 09:30:00 is a valid Time Without Time Zone value.
- **Date And Time** is expressed in yyyy-MM-ddTHH:mm:00Z format, where Z represents the UTC standard and HH is a 24 hour value. For example, 2020-07-24T09:30:00Z is a valid Date And Time value.

When a Date And Time value is deployed to an account, the time is converted from the UTC standard to the time zone preference of the account. For example, if you deploy a Date And Time value of 2020-06-23T09:30:00Z to an account with the time zone set to Pacific Daylight Time (PDT or UTC–7), the date remains the same but the time is set to 2:30 PM.

Example

The following example shows date and time used in a custom record and an instance of that custom record:

```

1 <customrecordtype scriptid="customrecord24">
2
3 <accesstype>CUSTRECORDENTRYPERM</accesstype>
4
5 <allowattachments>F</allowattachments>
6
7 <allowinlinedeleting>F</allowinlinedeleting>
8
9 <allowinlinedetaching>F</allowinlinedetaching>
10
11 <allowinlineediting>F</allowinlineediting>
12
13 <allowmobileaccess>F</allowmobileaccess>
14
15 <allownumberingoverride>F</allownumberingoverride>
16
17 <allowquickadd>F</allowquickadd>
18
19 <allowquicksearch>F</allowquicksearch>
20
21 <allowuiaccess>T</allowuiaccess>
22
23 <description></description>
24
25 <enabledle>F</enabledle>
26
27 <enablekeywords>F</enablekeywords>
28
29 <enablemailmerge>F</enablemailmerge>
30
31 <enablenumbering>F</enablenumbering>
32
33 <enableoptimisticlocking>F</enableoptimisticlocking>
34
35 <enablesystemnotes>F</enablesystemnotes>
36
37 <hierarchical>F</hierarchical>
38
39 <icon></icon>
40
41 <iconbuiltin>T</iconbuiltin>
42
43 <iconindex></iconindex>
44
45 <includeinsearchmenu>F</includeinsearchmenu>
46
47 <includename>T</includename>
48
49 <isinactive>F</isinactive>

```

```

50
51 <isordered>F</isordered>
52
53 <numberinginit></numberinginit>
54
55 <numberingmindigits></numberingmindigits>
56
57 <numberingprefix></numberingprefix>
58
59 <numberingsuffix></numberingsuffix>
60
61 <recordname>Customer Contact</recordname>
62
63 <showcreationdate>F</showcreationdate>
64
65 <showcreationdateonlist>F</showcreationdateonlist>
66
67 <showid>F</showid>
68
69 <showlastmodified>F</showlastmodified>
70
71 <showlastmodifiedonlist>F</showlastmodifiedonlist>
72
73 <shownotes>F</shownotes>
74
75 <showowner>F</showowner>
76
77 <showownerallowchange>F</showownerallowchange>
78
79 <showowneronlist>F</showowneronlist>
80
81 <customrecordcustomfields>
82
83   <customrecordcustomfield scriptid="custrecord1">
84
85     <accesslevel>2</accesslevel>
86
87     <allowquickadd>F</allowquickadd>
88
89     <applyformatting>F</applyformatting>
90
91     <checkspelling>F</checkspelling>
92
93     <defaultchecked>F</defaultchecked>
94
95     <defaultselection></defaultselection>
96
97     <defaultvalue></defaultvalue>
98
99     <description></description>
100
101     <displayheight></displayheight>
102
103     <displaytype>NORMAL</displaytype>
104
105     <displaywidth></displaywidth>
106
107     <dynamicdefault></dynamicdefault>
108
109     <encryptatrest>F</encryptatrest>
110
111     <fieldtype>TIMEOFDAY</fieldtype>
112
113     <globalsearch>F</globalsearch>
114
115     <help></help>
116
117     <isformula>F</isformula>
118
119     <ismandatory>F</ismandatory>
120
121     <isparent>F</isparent>
122

```



```

123 <label>Time Contacted</label>
124
125 <linktext></linktext>
126
127 <maxlength></maxlength>
128
129 <maxvalue></maxvalue>
130
131 <minvalue></minvalue>
132
133 <onparentdelete></onparentdelete>
134
135 <parentsubtab></parentsubtab>
136
137 <rolerestrict>F</rolerestrict>
138
139 <searchcomparefield></searchcomparefield>
140
141 <searchdefault></searchdefault>
142
143 <searchlevel>2</searchlevel>
144
145 <selectrecordtype></selectrecordtype>
146
147 <showinlist>F</showinlist>
148
149 <sourcefilterby></sourcefilterby>
150
151 <sourcefrom></sourcefrom>
152
153 <sourcelist></sourcelist>
154
155 <storevalue>T</storevalue>
156
157 <subtab></subtab>
158
159 </customrecordcustomfield>
160
161 <customrecordcustomfield scriptid="custrecord2">
162
163 <accesslevel>2</accesslevel>
164
165 <allowquickadd>F</allowquickadd>
166
167 <applyformatting>F</applyformatting>
168
169 <checkspelling>F</checkspelling>
170
171 <defaultchecked>F</defaultchecked>
172
173 <defaultselection></defaultselection>
174
175 <defaultvalue></defaultvalue>
176
177 <description></description>
178
179 <displayheight></displayheight>
180
181 <displaytype>NORMAL</displaytype>
182
183 <displaywidth></displaywidth>
184
185 <dynamicdefault></dynamicdefault>
186
187 <encryptatrest>F</encryptatrest>
188
189 <fieldtype>DATE</fieldtype>
190
191 <globalsearch>F</globalsearch>
192
193 <help></help>
194
195 <isformula>F</isformula>

```

```

196     <ismandatory>F</ismandatory>
197
198     <isparent>F</isparent>
199
200     <label>Date of Follow-up</label>
201
202     <linktext></linktext>
203
204     <maxlength></maxlength>
205
206     <maxvalue></maxvalue>
207
208     <minvalue></minvalue>
209
210
211     <onparentdelete></onparentdelete>
212
213     <parentsubtab></parentsubtab>
214
215     <rolerestrict>F</rolerestrict>
216
217     <searchcomparefield></searchcomparefield>
218
219     <searchdefault></searchdefault>
220
221     <searchlevel>2</searchlevel>
222
223     <selectrecordtype></selectrecordtype>
224
225     <showinlist>F</showinlist>
226
227     <sourcefilterby></sourcefilterby>
228
229     <sourcefrom></sourcefrom>
230
231     <sourcelist></sourcelist>
232
233     <storevalue>T</storevalue>
234
235     <subtab></subtab>
236
237 </customrecordcustomfield>
238
239 <customrecordcustomfield scriptid="custrecord3">
240
241     <accesslevel>2</accesslevel>
242
243     <allowquickadd>F</allowquickadd>
244
245     <applyformatting>F</applyformatting>
246
247     <checkspelling>F</checkspelling>
248
249     <defaultchecked>F</defaultchecked>
250
251     <defaultselection></defaultselection>
252
253     <defaultvalue></defaultvalue>
254
255     <description></description>
256
257     <displayheight></displayheight>
258
259     <displaytype>NORMAL</displaytype>
260
261     <displaywidth></displaywidth>
262
263     <dynamicdefault></dynamicdefault>
264
265     <encryptatrest>F</encryptatrest>
266
267     <fieldtype>DATETIME</fieldtype>
268

```

```

269 <globalsearch>F</globalsearch>
270
271 <help></help>
272
273 <isformula>F</isformula>
274
275 <ismandatory>F</ismandatory>
276
277 <isparent>F</isparent>
278
279 <label>Date and Time of Response</label>
280
281 <linktext></linktext>
282
283 <maxlength></maxlength>
284
285 <maxvalue></maxvalue>
286
287 <minvalue></minvalue>
288
289 <onparentdelete></onparentdelete>
290
291 <parentsubtab></parentsubtab>
292
293 <rolerestrict>F</rolerestrict>
294
295 <searchcomparefield></searchcomparefield>
296
297 <searchdefault></searchdefault>
298
299 <searchlevel>2</searchlevel>
300
301 <selectrecordtype></selectrecordtype>
302
303 <showinlist>F</showinlist>
304
305 <sourcefilterby></sourcefilterby>
306
307 <sourcefrom></sourcefrom>
308
309 <sourcelist></sourcelist>
310
311 <storevalue>T</storevalue>
312
313 <subtab></subtab>
314
315 </customrecordcustomfield>
316
317 <customrecordcustomfield scriptid="custrecord4">
318
319 <accesslevel>2</accesslevel>
320
321 <allowquickadd>F</allowquickadd>
322
323 <applyformatting>F</applyformatting>
324
325 <checkspelling>F</checkspelling>
326
327 <defaultchecked>F</defaultchecked>
328
329 <defaultselection></defaultselection>
330
331 <defaultvalue></defaultvalue>
332
333 <description></description>
334
335 <displayheight></displayheight>
336
337 <displaytype>NORMAL</displaytype>
338
339 <displaywidth></displaywidth>
340
341 <dynamicdefault></dynamicdefault>

```

```

342     <encryptatrest>F</encryptatrest>
343
344     <fieldtype>TIMEOFDAY</fieldtype>
345
346     <globalsearch>F</globalsearch>
347
348     <help></help>
349
350     <isformula>F</isformula>
351
352     <ismandatory>F</ismandatory>
353
354     <isparent>F</isparent>
355
356     <label>Regularly Unavailable Time</label>
357
358     <linktext></linktext>
359
360     <maxlength></maxlength>
361
362     <maxvalue></maxvalue>
363
364     <minvalue></minvalue>
365
366     <onparentdelete></onparentdelete>
367
368     <parentsubtab></parentsubtab>
369
370     <rolerestrict>F</rolerestrict>
371
372     <searchcomparefield></searchcomparefield>
373
374     <searchdefault></searchdefault>
375
376     <searchlevel>2</searchlevel>
377
378     <selectrecordtype></selectrecordtype>
379
380     <showinlist>F</showinlist>
381
382     <sourcefilterby></sourcefilterby>
383
384     <sourcefrom></sourcefrom>
385
386     <sourcelist></sourcelist>
387
388     <storevalue>T</storevalue>
389
390     <subtab></subtab>
391
392 </customrecordcustomfield>
393
394 </customrecordcustomfields>
395
396 <instances>
397
398     <instance scriptid="val_51121_mstrwlf_384">
399
400         <custrecord1>22:30:00</custrecord1>
401
402         <custrecord2>2020-08-03</custrecord2>
403
404         <custrecord3>2020-02-02T03:30:00Z</custrecord3>
405
406         <custrecord4>18:30:00:00Z</custrecord4>
407
408         <isinactive>F</isinactive>
409
410         <name>Susie Smith</name>
411
412     </instance>
413
414

```

```

415 | </instances>
416 |
417 | </customrecordtype>

```

Lists, Records, and Fields

For SuiteCloud projects in SuiteCloud Development Framework (SDF), the following topics provide information about the SDF custom objects that pertain to lists, records and fields:

- [Custom Lists as XML Definitions](#)
- [Custom Other Record Fields as XML Definitions](#)
- [Custom Item Fields as XML Definitions](#)
- [Custom CRM Fields as XML Definitions](#)
- [Custom Entity Fields as XML Definitions](#)
- [Custom Item Number Fields as XML Definitions](#)
- [Custom Transaction Body Fields as XML Definitions](#)
- [Custom Transaction Line Fields as XML Definitions](#)
- [Custom Transaction Item Option Fields as XML Definitions](#)
- [Custom Record Types as XML Definitions](#)
- [Custom Segments as XML Definitions](#)
- [Custom Transaction Record Types as XML Definitions](#)

Custom Lists as XML Definitions

A custom list is a list of values that you can use in custom fields on your forms and records. Custom lists enable you to set up predefined choices for your employees and customers to select when entering transactions and records. You can create an unlimited number of custom lists and an unlimited number of values for each list.

Create and manage custom lists using SuiteCloud Development Framework (SDF). You can model a custom list in a SuiteCloud project and deploy the project to a target NetSuite account. You can either create new custom lists in your project or import existing lists from a NetSuite account. For more information about working with custom lists in a NetSuite account, see the help topic [Creating a Custom List](#).

A custom list object is comprised of the following:

- **Description** – You can enter a description of the custom list in the Description field.
- **Inactive** – You can make the list inactive by setting the `isinactive` value to T, or make it active by setting the `isinactive` value to F. An inactive list does not appear on any forms or in global searches, like a deleted list. You also cannot select this list from any lists on entities, items, or transactions.
- **Matrix option** – You can make the list available on matrix items by setting the `ismatrixoption` value to T, or make it unavailable by setting the `ismatrixoption` value to F. If you designate the list as a matrix option list, you must create a unique abbreviation for each list value.



Warning: If this field appears in the project, you must reference the `MATRIXITEMS` feature in the manifest file to avoid project warnings. In the manifest file, you can specify whether this feature is required in the target account. `MATRIXITEMS` must be enabled for this field to appear in the target account.

- **Ordered** – You can sort your list by the order in which you enter them by setting the `isordered` value to T, or sort it by alphabetical order by setting the `isordered` value to F.

- **Name** – You must have a name for the custom list.

Your custom list appears on custom fields when deployed to an account. The following is an XML definition example of a custom list. When deployed to an account, go to your custom field to select **List/Record** under **Type**, and select your custom list under **List/Record**. Under the **Validation & Defaulting** subtab, the **Default Selection** displays the values from your custom list that you can select:

```

1 <customlist scriptid="customlist_accounttype">
2   <description>Account type is used to distinguish between primary and secondary accounts.</description>
3   <inactive>F</inactive>
4   <ismatrixoption>F</ismatrixoption>
5   <isordered>T</isordered>
6   <name>Account Type</name>
7   <customvalues>
8     <customvalue scriptid="val_53391_399334_433">
9       <abbreviation>P</abbreviation>
10      <inactive>F</inactive>
11      <value>Primary</value>
12    </customvalue>
13    <customvalue scriptid="val_53392_399334_459">
14      <abbreviation>S</abbreviation>
15      <inactive>F</inactive>
16      <value>Secondary</value>
17    </customvalue>
18  </customvalues>
19 </customlist>

```

You can create custom values in a custom list. In a custom list object, custom values are grouped within the customvalues structure. In this structure, you can create as many custom values needed by specifying a customvalue structure for each one. For more information, see the help topic [customvalue](#).

Your custom list can be used in custom fields on your forms and records. You can create custom fields in both SDF and NetSuite. For more information, see the help topic [Creating a Custom Field](#).

For more information about custom lists, see the following topics:

- [customlist](#)
- [Custom Lists](#)
- [Adding Translations for Custom Lists](#)
- [Managing Large Custom Lists](#)

Custom Other Record Fields as XML Definitions

In SuiteCloud Development Framework (SDF), custom other record fields are represented by the othercustomfield SDF custom object. To gather information specific to your business needs, you can create different types of custom fields and apply them to your custom forms or records. Use the custom other record field in the following situations:

- To create a more specialized custom field when the existing custom fields available in NetSuite do not fit your needs
- To use on records without custom forms associated with them

For more information about othercustomfield SDF custom object, see the following topics:

- [Setting Values on the othercustomfield SDF Custom Object](#)
- [Custom Other Record Field XML Definition Examples](#)

Note: In the NetSuite UI, custom other field is referred to as Other Record field.

Here is an example of an othercustomfield SDF custom object shown as **Other Record Field** in the NetSuite UI. To view this page from the NetSuite UI, go to Customization > Lists, Records, & Fields > Other Record Fields > New.

When the othercustomfield SDF custom object is included in a form, it can look like the following example in the NetSuite UI:

See the following topics in the Customization help for more information about working with the other record field:

- [Creating Other Record Fields](#)

- [Creating a Custom Field](#)
- [Custom Field Types](#)

Applying a Custom Other Record Field to a Record

The following table maps record names to the values accepted by the `rectype` and `selectrecordtype` elements of the `othercustomfield` SDF custom object. Values are negative numbers. For example: `<rectype>-290</rectype>`.

Record Name	Value
Account	-112
Address	-289
Bin	-242
Campaign Event	-107
Charge	-290
Charge Rule	-276
Class	-101
Company	-117
Competitor	-108
Department	-102
Expense Category	-126
Inventory Cost Template	-298
Inventory Demand Plan	-246
Inventory Supply Plan	-247
Location	-103
Item Demand Plan	-246
Manufacturing Cost Template	-294
Manufacturing Routing	-288
Note	-303
Payroll Item	-265
Planned Standard Cost	-249
Product	-115
Product Version	-113
Standard Cost Version	-248
Role	-118
Tax Code	-128

Workflow	-196
Workplace	-196
Domain	-438
Format Profile	-553
Subsidiary	-117
Subsidiary Settings	-551
Vendor-Subsidiary Relationship	-549

Setting Values on the othercustomfield SDF Custom Object

The following sections provide information about setting values for these othercustomfield SDF custom object XML elements:

- [Script ID](#)
- [Access Level](#)
- [Allow Delete of List/Record Values](#)
- [Field Filter](#)
- [Translation subtab](#)

To view the reference documentation that contains the full list of supported fields and values for the othercustomfield SDF custom object, see the help topic [othercustomfield](#). The reference also lists the type, defaults, and notes about an element and whether it is mandatory, conditionally mandatory, or optional.

Script ID

The prefix of the custom field object's script ID should be custrecord_. The remainder of the ID is your choice. To enhance readability, you should create an ID that includes an underscore (_) after the custom field's prefix and adding a meaningful name after the underscore.

Access Level

The following table lists the values you can use to set access level.

Access Level	Value
None	0
View	1
Run	
Edit	2

In SDF, role access to custom other field is supported as a structured field with its own embedded structure. For more information about its supported fields and values, see the help topic [roleaccesses](#).

This structured field is equivalent to the Role section in the following NetSuite UI:

Other Record Field List

Save & New Cancel Reset

RECORD TYPE * DESCRIPTION

LABEL * TYPE

ID LIST/RECORD

OWNER

Display Validation & Defaulting Sourcing & Filtering **Access** Translation

DEFAULT ACCESS LEVEL Edit DEFAULT LEVEL FOR SEARCH/REPORTING Edit

Role	Department	Subsidiary
ROLE *	ACCESS LEVEL *	LEVEL FOR SEARCH/REPORTING *
Administrator	View	Run

Add Cancel Insert Remove

Allow Delete of List/Record Values

You can use the Allow Delete of List/Record Values field when the fieldtype value on the custom field is set to List/Record.

The Allow Delete of List/Record Values field is represented by the onparentdelete element and takes the following values:

- NO_ACTION — Prevent and Return Error
- SET_NULL — Allow and Set Independent Values to Null

The following graphic shows the field and its options on the **Validation & Defaulting** subtab of the record as it appears in the NetSuite UI:

Other Record Field

Save Cancel Reset

RECORD TYPE * DESCRIPTION

LABEL * TYPE

ID LIST/RECORD

OWNER A Wolfe

Display **Validation & Defaulting** Sourcing & Filtering Access

☐ MANDATORY

☐ CHECK SPELLING

DEFAULT SELECTION <Enter first few letters then tab>

DYNAMIC DEFAULT

ALLOW DELETE OF LIST/RECORD VALUES?

Prevent and Return Error

Prevent and Return Error

Allow and Set Dependent Field Values to Null

Save Cancel Reset

Field Filter

In SDF, a field filter on the `othercustomfield` SDF custom object is supported as a structured field with its own embedded structure. For more information about its supported fields and values, see the help topic [customfieldfilters](#).

This structured field sets fields that appear on the Sourcing and Filtering subtab on other record field in the NetSuite UI.

If you change the field type or the list/record for a field that is used for sourcing or filtering on other custom fields, all sourcing and filtering based on the field is removed.

Translation subtab

The **Translation** subtab is not currently supported by SDF. To work with translations using SDF, use the `translationcollection` SDF custom object. For more information, see [Translation Collection Object Fields in XML Definitions](#).

Custom Other Record Field XML Definition Examples

The following code samples are SuiteCloud Development Framework (SDF) examples of `othercustomfield` SDF custom objects:

```
1 <othercustomfield scriptid="custrecord_specialtextfield"
2   <accesslevel>2</accesslevel>
3   <displaytype>NORMAL</displaytype>
4   <fieldtype>TEXT</fieldtype>
5   <label>Important Special Field</label>
6   <rectype>-289</rectype>
7   <searchlevel>2</searchlevel>
8   <storevalue>T</storevalue>
9 </othercustomfield>
```

```
1 <othercustomfield scriptid="custrecord_system_role_checkbox">
2   <accesslevel>2</accesslevel>
3   <applyformatting>F</applyformatting>
4   <checkspelling>F</checkspelling>
5   <defaultchecked>F</defaultchecked>
6   <defaultselection/>
7   <defaultvalue/>
8   <description/>
9   <displayheight/>
10  <displaytype>NORMAL</displaytype>
11  <displaywidth/>
12  <dynamicdefault/>
13  <encryptatrest>F</encryptatrest>
14  <fieldtype>CHECKBOX</fieldtype>
15  <help/>
16  <isformula>F</isformula>
17  <ismandatory>F</ismandatory>
18  <label>System Role</label>
19  <linktext/>
20  <maxlength/>
21  <maxvalue/>
22  <minvalue/>
23  <onparentdelete/>
24  <rectype>-118</rectype>
25  <searchcomparefield/>
26  <searchdefault/>
27  <searchlevel>2</searchlevel>
28  <selectrecordtype/>
29  <showhierarchy>F</showhierarchy>
30  <showinlist>F</showinlist>
```

```

31 <sourcefilterby/>
32 <sourcefrom/>
33 <sourcelist/>
34 <storevalue>T</storevalue>
35 </othercustomfield>

```

Custom Item Fields as XML Definitions

You can create XML definitions for custom item fields as SDF custom objects using SuiteCloud Development Framework (SDF). You can either create an `itemcustomfield` object in a SuiteCloud project or import a custom item field from a NetSuite account into a SuiteCloud project. For information about creating a custom item field in a NetSuite account, see the following:

- [Creating Custom Item Fields](#)
- [Field Type Descriptions for Custom Fields](#)

Add custom item fields to your item records to gather information specific to your business needs.

After an `itemcustomfield` object is deployed to an account, a custom item field is created. The field appears on the item records that you applied the object to. You can create custom item records in a NetSuite account or in your SuiteCloud IDE. For more information about creating custom item records, see the help topic [Creating Item Records](#).

Setting Values on the `itemcustomfield` SDF Custom Object

Provide a readable script ID attribute by using the underscore followed by a meaningful name for the SDF custom object. NetSuite automatically adds a `custitem` prefix that indicates the type of field, for example, `custitem_mynewfield`.

The following fields are required for `itemcustomfield` objects:

- `label` - Specifies the label for the custom field. The label is the text that your users see as the name of the custom field when it is displayed on the record.
- `fieldtype` - Specifies the type of custom field. For more information about the possible values of `fieldtype`, see the help topic [generic_customfield_fieldtype](#).
- `selectrecordtype` (conditional) - Specifies the list or record that contains the items for the custom field. This field is required only when the `fieldtype` value is equal to `SELECT` or `MULTISELECT`.

The following XML definition example is a basic `itemcustomfield` object with the required fields:

```

1 <itemcustomfield scriptid="custitem_mynewfield">
2 ...
3 <fieldtype>SELECT</fieldtype>
4 <label>SAMPLE ITEMCUSTOMFIELD</label>
5 <selectrecordtype>-112</selectrecordtype>
6 ...
7 </itemcustomfield>

```

For more information about possible fields for `itemcustomfield` objects, see the help topic [itemcustomfield](#).

If you include any SDF custom objects, files, or scripts in your fields, you must define the dependencies for them in your SuiteCloud project's manifest. For more information, see [SDF Custom Object Dependencies in SuiteApps](#).

To index a custom field to be included in global search, set the `globalsearch` field value to T. The `globalsearch` field can equal to T only if the following field conditions are met:

- `storevalue` equals to T
- `fieldtype` equals to one of the following:
 - CURRENCY
 - FLOAT
 - EMAIL
 - TEXT
 - HELP
 - URL
 - INLINEHTML
 - INTEGER
 - PERCENT
 - PHONE
 - TEXTAREA



Warning: You cannot index a custom field for global search if any of your `searchlevel` field values equal to 0.

Embedding Structured Fields Within an `itemcustomfield` SDF Custom Object

You can create custom filters for your custom field by embedding `customfieldfilter` structured fields within the `itemcustomfield` SDF custom object. You can also customize the default access that a role has to the custom field by embedding `roleaccess` structured fields within the object.

For more information about the possible fields to define for these structured fields, see the following:

- [Custom Field Filters](#)
- [Role Accesses](#)

To view a full XML definition example for an `itemcustomfield` object, see [Example of an `itemcustomfield` SDF Custom Object](#).

Custom Field Filters

You can add custom field filters to a list/record or multiple select custom field to filter the choices available in that custom field on records and transactions. In SDF, you can use custom field filters by embedding `customfieldfilter` structured fields within a custom field object.



Note: Filtering can apply only when the `fieldtype` value of your custom field object is equal to `SELECT` or `MULTISELECT`. It does not apply to custom lists, meaning if you include `customfieldfilter` structured fields, your `selectrecordtype` value cannot equal a custom list scriptid.

`customfieldfilters` is the parent element and `customfieldfilter` is the child element. You can include multiple `customfieldfilter` child elements within the parent.

The `fldfilter` subfield specifies a standard field on the list/record that you entered for the `selectrecordtype` field. The `fldfilter` subfield is required. For more information about the supported fields and values for `customfieldfilter` structured fields, see the help topic [customfieldfilter](#).

When an object with a `customfieldfilter` structured field is deployed to a target account, the **Sourcing & Filtering** subtab of the custom field record is populated with the field values specified by the structured field. The choices available to users in a record or transaction for that custom field are tailored by the filter settings from the structured field.

Here is an example of a custom item field configuration page in NetSuite with filters set for the field:



Important: If you change the field type or the list/record for a field that is used for sourcing or filtering on other custom fields, all sourcing and filtering based on the field is removed.

Additionally, you can define custom field filters in NetSuite. For more information, see the help topic [Setting Filtering Criteria](#).

Example of a customfieldfilter Structured Field

The following XML definition example is a `customfieldfilter` structured field embedded within an `itemcustomfield` object whose `fieldtype` equals `SELECT`. After this object is deployed to the target account, a custom item field labeled `SAMPLE_ITEMCUSTOMFIELD` appears on the **Main** tab of all Inventory item records. The `fldfiltercomparetype` equals `LIKE` and `fldfilterval` equals `paid`. Therefore, the field's dropdown list is filtered to display only accounts that contain "paid" in the account name.

```

1 <itemcustomfield scriptid="custitem_sample_itemcust">
2   <appliedtoinventory>T</appliedtoinventory>
3   <fieldtype>SELECT</fieldtype>
4   <label>SAMPLE_ITEMCUSTOMFIELD</label>
5   <selectrecordtype>-112</selectrecordtype>
6   ...
7   <customfieldfilters>
8     <customfieldfilter>
9       <fldcomparefield></fldcomparefield>
10      <fldfilter>STORECORDACCOUNTACCTNAME</fldfilter>

```

```

11      <fldfilterchecked></fldfilterchecked>
12      <fldfiltercomparetype>LIKE</fldfiltercomparetype>
13      <fldfilternotnull>F</fldfilternotnull>
14      <fldfilternull>F</fldfilternull>
15      <fldfiltersel></fldfiltersel>
16      <fldfilterval>paid>
17    </customfieldfilter>
18  </customfieldfilters>
19  ...
20 </itemcustomfield>

```

Role Accesses

In SDF, you can control how roles access the information in custom fields by embedding `roleaccess` structured fields within a custom field object. The accesses that you specify determines how the custom field can be accessed on the record or through search results and reports. This helps you to maintain the security of your business information.

`roleaccesses` is the parent element and `roleaccess` is the child element. You can include multiple `roleaccess` child elements within the parent.

After an object with a `roleaccess` structured field is deployed to a NetSuite account, the **Role** section under the **Access** subtab of the custom field record is populated with the access level values of the fields specified by the structured field.

Here is an example of a custom item field configuration page in NetSuite with access levels set for a role:

The screenshot shows the 'Custom Item Field' configuration page in NetSuite. The 'Access' subtab is selected, and a red box highlights the 'Role' section. This section contains a table with columns for 'Role', 'Access Level', and 'Level for Search/Reporting'. The 'Role' column has a dropdown menu with 'Accountant' selected. The 'Access Level' column has a dropdown menu with 'View' selected. The 'Level for Search/Reporting' column has a dropdown menu with 'Run' selected. Below the table are buttons for 'Add', 'Cancel', 'Insert', and 'Remove'.

The following subfields are required for `roleaccess` structured fields:

- `role` - Specifies the role.
- `accesslevel` - Specifies the access level granted to the role for the custom field object.

Possible access level values:

- 0 - None
- 1 - View
- 2 - Edit

- **searchlevel** - Specifies the access level granted to the role for the custom field object through search results and reports.

Possible access level values:

- 0 - None
- 1 - Run
- 2 - Edit

For more information about the supported fields and values for **roleaccess**, see the help topic [roleaccess](#).

Additionally, you can define role accesses to custom fields from a NetSuite account. For more information, see the help topic [Restricting Access to Custom Fields](#).

Example of a roleaccess Structured Field

The following XML definition example is a **roleaccess** structured field embedded within a **itemcustomfield** object. In this example, the **accesslevel** for the Accountant role equals 1 and the **searchlevel** equals 1. Therefore, the role's access to the field is set to **View** and the access level through searching and reporting is set to **Run**.

```

1 <itemcustomfield scriptid="custitem_sample_itemcust">
2   <appliedtoinventory>T</appliedtoinventory>
3   <fieldtype>SELECT</fieldtype>
4   <label>SAMPLE ITEMCUSTOMFIELD</label>
5   <selectrecordtype>-112</selectrecordtype>
6   ...
7   <roleaccesses>
8     <roleaccess>
9       <accesslevel>1</accesslevel>
10      <role>ACCOUNTANT</role>
11      <searchlevel>1</searchlevel>
12    </roleaccess>
13  </roleaccesses>
14  ...
15 </itemcustomfield>

```

Example of an itemcustomfield SDF Custom Object

The following XML definition example is for an **itemcustomfield** SDF custom object. After this SDF custom object is deployed to the target account, a custom item field labeled **SAMPLE ITEMCUSTOMFIELD** appears on the **Main** tab of all item records that the object is applied to. In this example, the object applies to inventory item records. The **fieldtype** equals **SELECT**. Therefore, a value for **selectrecordtype** is required.

```

1 <itemcustomfield scriptid="custitem_sample_itemcust">
2   <accesslevel>2</accesslevel>
3   <appliedtogroup>F</appliedtogroup>
4   <appliedtoinventory>T</appliedtoinventory>
5   <appliedtokit>F</appliedtokit>
6   <appliedtononinventory>F</appliedtononinventory>
7   <appliedtoothercharge>F</appliedtoothercharge>
8   <appliedtopricelist>F</appliedtopricelist>
9   <appliedtoservice>F</appliedtoservice>
10  <appliedtospecificitems>F</appliedtospecificitems>
11  <appliedtosubplan>F</appliedtosubplan>
12  <applyformatting>F</applyformatting>
13  <checkspelling>F</checkspelling>
14  <defaultchecked>F</defaultchecked>
15  <defaultselection></defaultselection>
16  <defaultvalue></defaultvalue>
17  <description>Sample description for field.</description>

```



```

18 <displayheight></displayheight>
19 <displaytype>NORMAL</displaytype>
20 <displaywidth></displaywidth>
21 <dynamicdefault></dynamicdefault>
22 <encryptatrest>F</encryptatrest>
23 <fieldtype>SELECT</fieldtype>
24 <globalsearch>F</globalsearch>
25 <help>Enter a brief explanation of the kind of information you want entered in this field.</help>
26 <includechilditems>F</includechilditems>
27 <isformula>F</isformula>
28 <ismandatory>F</ismandatory>
29 <isparent>F</isparent>
30 <itemmatrix>F</itemmatrix>
31 <itemssubtype>BOTH</itemssubtype>
32 <label>SAMPLE_ITEMCUSTOMFIELD</label>
33 <linktext></linktext>
34 <maxlength></maxlength>
35 <maxvalue></maxvalue>
36 <minvalue></minvalue>
37 <onparentdelete>NO_ACTION</onparentdelete>
38 <parentsubtab></parentsubtab>
39 <searchcomparefield></searchcomparefield>
40 <searchdefault></searchdefault>
41 <searchlevel>2</searchlevel>
42 <selectrecordtype>-112</selectrecordtype>
43 <showhierarchy>T</showhierarchy>
44 <showinlist>F</showinlist>
45 <sourcefilterby></sourcefilterby>
46 <sourcefrom></sourcefrom>
47 <sourcelist></sourcelist>
48 <storevalue>T</storevalue>
49 <subtab>ITEMMAIN</subtab>
50 <customfieldfilters>
51   <customfieldfilter>
52     <fldcomparefield></fldcomparefield>
53     <fldfilter>STDRECORDACCOUNTACCTNAME</fldfilter>
54     <fldfilterchecked></fldfilterchecked>
55     <fldfiltercomparetype>LIKE</fldfiltercomparetype>
56     <fldfilternotnull>F</fldfilternotnull>
57     <fldfilternull>F</fldfilternull>
58     <fldfiltersel></fldfiltersel>
59     <fldfilterval>paid</fldfilterval>
60   </customfieldfilter>
61 </customfieldfilters>
62 <roleaccesses>
63   <roleaccess>
64     <accesslevel>1</accesslevel>
65     <role>ACCOUNTANT</role>
66     <searchlevel>1</searchlevel>
67   </roleaccess>
68 </roleaccesses>
69 </itemcustomfield>

```

Custom CRM Fields as XML Definitions

You can create XML definitions for custom CRM fields as SDF custom objects using SuiteCloud Development Framework (SDF). You can either create a `crmcustomfield` object in a SuiteCloud project or import a custom CRM field from a NetSuite account into a SuiteCloud project. For information about creating a custom CRM field in a NetSuite account, see the following:

- [Creating Custom CRM Fields](#)
- [Field Type Descriptions for Custom Fields](#)

Add custom CRM fields to your CRM records to gather information specific to your business needs.

These CRM records include:

- task records
- phone call records
- campaign records
- case records
- solution records
- event records

After a `crmcustomfield` object is deployed to an account, a custom CRM field is created. The field appears on the CRM records that you applied the object to.

Setting Values on the `crmcustomfield` SDF Custom Object

Provide a readable script ID attribute by using the underscore followed by a meaningful name for the SDF custom object. NetSuite automatically adds a `custevent` prefix that indicates the type of field, for example, `custevent_mynewfield`.

The following fields are required for `crmcustomfield` objects:

- `label` - Specifies the label for the custom field. The label is the text that your users see as the name of the custom field when it is displayed on the record.
- `fieldtype` - Specifies the type of custom field. For more information about the possible values of `fieldtype`, see the help topic [generic_customfield_fieldtype](#).
- `selectrecordtype` (conditional) - Specifies the list or record that contains the items for the custom field. This field is required only when the `fieldtype` value is equal to `SELECT` or `MULTISELECT`.

The following XML definition example is a basic `crmcustomfield` object with the required fields.

```

1 <crmcustomfield scriptid="custevent_mynewfield">
2 ...
3   <fieldtype>SELECT</fieldtype>
4   <label>SAMPLE CRMFIELD</label>
5   <selectrecordtype>-103</selectrecordtype>
6 ...
7 </crmcustomfield>

```

For more information about possible fields for `crmcustomfield` objects, see the help topic [crmcustomfield](#).

If you include any SDF custom objects, files, or scripts in your fields, you must define the dependencies for them in your SuiteCloud project's manifest. For more information, see [SDF Custom Object Dependencies in SuiteApps](#).

To index a custom field to be included in global search, set the `globalsearch` field value to `T`. The `globalsearch` field can equal to `T` only if the following field conditions are met:

- `storevalue` equals to `T`
- `fieldtype` equals to one of the following:
 - `CURRENCY`
 - `FLOAT`
 - `EMAIL`
 - `TEXT`

- HELP
- URL
- INLINEHTML
- INTEGER
- PERCENT
- PHONE
- TEXTAREA



Warning: You cannot index a custom field for global search if any of your searchlevel field values equal to 0.

Embedding Structured Fields Within a crmcustomfield SDF Custom Object

You can create custom filters for your custom field by embedding customfieldfilter structured fields within the crmcustomfield SDF custom object. You can also customize the default access that a role has to the custom field by embedding roleaccess structured fields within the object.

For more information about the possible fields to define for these structured fields, see the following:

- [Custom Field Filters](#)
- [Role Accesses](#)

To view a full XML definition example for a crmcustomfield object, see [Example of a crmcustomfield SDF Custom Object](#).

Custom Field Filters

You can add custom field filters to a list/record or multiple select custom field to filter the choices available in that custom field on records and transactions. In SDF, you can use custom field filters by embedding customfieldfilter structured fields within a custom field object.



Note: Filtering can apply only when the fieldtype value of your custom field object is equal to SELECT or MULTISELECT. It does not apply to custom lists, meaning if you include customfieldfilter structured fields, your selectrecordtype value cannot equal a custom list scriptid.

customfieldfilters is the parent element and customfieldfilter is the child element. You can include multiple customfieldfilter child elements within the parent.

The fldfilter subfield specifies a standard field on the list/record that you entered for the selectrecordtype field. The fldfilter subfield is required. For more information about the supported fields and values for customfieldfilter structured fields, see the help topic [customfieldfilter](#).

When an object with a customfieldfilter structured field is deployed to a target account, the **Sourcing & Filtering** subtab of the custom field record is populated with the field values specified by the structured field. The choices available to users in a record or transaction for that custom field are tailored by the filter settings from the structured field.

Here is an example of a custom item field configuration page in NetSuite with filters set for the field:

Custom Item Field ← → List Copy to Account (Beta)

Save Cancel Reset Change ID Apply to Forms Actions

LABEL *
SAMPLE ITEMCUSTOMFIELD

PUBLISHER ID

APP ID

ID
custitem_sample_itemcust

INTERNAL ID
543

OWNER
Larry Nelson

DESCRIPTION

MATRIX OPTION

TYPE
List/Record

LIST/RECORD
Account

☒ STORE VALUE
 ☐ USE ENCRYPTED FORMAT
☐ SHOW IN LIST
☐ GLOBAL SEARCH
☐ RECORD IS PARENT
☐ INACTIVE

Applies To Display Validation & Defaulting **Sourcing & Filtering** Access Translation History

SOURCE LIST

SOURCE FROM

SOURCE FILTER BY

FILTER USING*	IS CHECKED	COMPARE TYPE	COMPARE VALUE TO	VALUE IS	IS NOT EMPTY	IS EMPTY	COMPARE TO FIELD
Name		contains	paid				
		equal		<Type & tab for single value>			

Add Cancel Insert Remove



Important: If you change the field type or the list/record for a field that is used for sourcing or filtering on other custom fields, all sourcing and filtering based on the field is removed.

Additionally, you can define custom field filters in NetSuite. For more information, see the help topic [Setting Filtering Criteria](#).

Example of a customfieldfilter Structured Field

The following XML definition example is a customfieldfilter structured field embedded within an itemcustomfield object whose fieldtype equals SELECT. After this object is deployed to the target account, a custom item field labeled SAMPLE ITEMCUSTOMFIELD appears on the **Main** tab of all Inventory item records. The fldfiltercomparetype equals LIKE and fldfilterval equals paid. Therefore, the field's dropdown list is filtered to display only accounts that contain "paid" in the account name.

```

1 <itemcustomfield scriptid="custitem_sample_itemcust">
2   <appliestoinventory>T</appliestoinventory>
3   <fieldtype>SELECT</fieldtype>
4   <label>SAMPLE ITEMCUSTOMFIELD</label>
5   <selectrecordtype>-112</selectrecordtype>
6   ...
7   <customfieldfilters>
8     <customfieldfilter>
9       <fldcomparefield></fldcomparefield>
10      <fldfilter>STDRECORDACCOUNTACCTNAME</fldfilter>
11      <fldfilterchecked></fldfilterchecked>
12      <fldfiltercomparetype>LIKE</fldfiltercomparetype>
13      <fldfilternotnull>F</fldfilternotnull>
14      <fldfilternull>F</fldfilternull>
15      <fldfiltersel></fldfiltersel>
16      <fldfilterval>paid</fldfilterval>
17    </customfieldfilter>
18  </customfieldfilters>
19  ...
20 </itemcustomfield>

```

Role Accesses

In SDF, you can control how roles access the information in custom fields by embedding roleaccess structured fields within a custom field object. The accesses that you specify determines how the custom

field can be accessed on the record or through search results and reports. This helps you to maintain the security of your business information.

roleaccesses is the parent element and roleaccess is the child element. You can include multiple roleaccess child elements within the parent.

After an object with a roleaccess structured field is deployed to a NetSuite account, the **Role** section under the **Access** subtab of the custom field record is populated with the access level values of the fields specified by the structured field.

Here is an example of a custom item field configuration page in NetSuite with access levels set for a role:

Custom Item Field

Save Cancel Reset Change ID Apply to Forms Actions

LABEL *
 SAMPLE ITEMCUSTOMFIELD
 PUBLISHER ID
 APP ID
 ID
 custitem_sample_itemcust
 INTERNAL ID
 543
 OWNER
 Larry Nelson

DESCRIPTION
 TYPE
 List/Record
 LIST/RECORD
 Account

☒ STORE VALUE ☐ USE ENCRYPTED FORMAT
☐ SHOW IN LIST
☐ GLOBAL SEARCH
☐ RECORD IS PARENT
☐ INACTIVE

Applies To Display Validation & Defaulting Sourcing & Filtering **Access** Translation History

DEFAULT ACCESS LEVEL
 Edit
 DEFAULT LEVEL FOR SEARCH/REPORTING
 Edit

Role	Department	Subsidiary	History
ROLE *			
Accountant			

Add Cancel Insert Remove

The following subfields are required for roleaccess structured fields:

- **role** - Specifies the role.
- **accesslevel** - Specifies the access level granted to the role for the custom field object.
Possible access level values:
 - 0 - None
 - 1 - View
 - 2 - Edit
- **searchlevel** - Specifies the access level granted to the role for the custom field object through search results and reports.
Possible access level values:
 - 0 - None
 - 1 - Run
 - 2 - Edit

For more information about the supported fields and values for roleaccess, see the help topic [roleaccess](#).

Additionally, you can define role accesses to custom fields from a NetSuite account. For more information, see the help topic [Restricting Access to Custom Fields](#).

Example of a roleaccess Structured Field

The following XML definition example is a roleaccess structured field embedded within a itemcustomfield object. In this example, the accesslevel for the Accountant role equals 1 and the searchlevel equals 1. Therefore, the role's access to the field is set to **View** and the access level through searching and reporting is set to **Run**.

```

1 <itemcustomfield scriptid="custitem_sample_itemcust">
2   <appliestoinventory>T</appliestoinventory>
3   <fieldtype>SELECT</fieldtype>
4   <label>SAMPLE ITEMCUSTOMFIELD</label>
5   <selectrecordtype>-112</selectrecordtype>
6   ...
7   <roleaccesses>
8     <roleaccess>
9       <accesslevel>1</accesslevel>
10      <role>ACCOUNTANT</role>
11      <searchlevel>1</searchlevel>
12    </roleaccess>
13  </roleaccesses>
14  ...
15 </itemcustomfield>

```

Example of a crmcustomfield SDF Custom Object

The following XML definition example is for a crmcustomfield object. After this SDF custom object is deployed to the target account, a custom item field labeled **SAMPLE CRMFIELD** appears on the **System Information** subtab of all CRM records that the object is applied to. In this example, the object applies to phone call records. The fieldtype equals SELECT. Therefore, a value for selectrecordtype is required.

```

1 <crmcustomfield scriptid="custevent_sample_crm">
2   <accesslevel>2</accesslevel>
3   <appliesperkeyword>F</appliesperkeyword>
4   <appliestocampaign>F</appliestocampaign>
5   <appliestocase>F</appliestocase>
6   <appliestoevent>F</appliestoevent>
7   <appliestophonecall>T</appliestophonecall>
8   <appliestoprojecttask>F</appliestoprojecttask>
9   <appliestoresourceallocation>F</appliestoresourceallocation>
10  <appliestotask>F</appliestotask>
11  <applyformatting>F</applyformatting>
12  <availableexternally>F</availableexternally>
13  <checkspelling>F</checkspelling>
14  <defaultchecked>F</defaultchecked>
15  <defaultselection></defaultselection>
16  <defaultvalue></defaultvalue>
17  <description>Enter a description of this field.</description>
18  <displayheight></displayheight>
19  <displaytype>NORMAL</displaytype>
20  <displaywidth></displaywidth>
21  <dynamicdefault></dynamicdefault>
22  <encryptatrest>F</encryptatrest>
23  <fieldtype>SELECT</fieldtype>
24  <globalsearch>F</globalsearch>
25  <help>Enter a brief explanation of the kind of information you want entered in this field.</help>
26  <isformula>F</isformula>
27  <ismandatory>F</ismandatory>
28  <isparent>F</isparent>
29  <label>SAMPLE CRMFIELD</label>
30  <linktext></linktext>
31  <maxlength></maxlength>
32  <maxvalue></maxvalue>
33  <minvalue></minvalue>
34  <onparentdelete>NO_ACTION</onparentdelete>
35  <parentsubtab></parentsubtab>
36  <searchcomparefield></searchcomparefield>
37  <searchdefault></searchdefault>

```

```

38 <searchlevel>2</searchlevel>
39 <selectrecordtype>-103</selectrecordtype>
40 <showhierarchy>T</showhierarchy>
41 <showinlist>F</showinlist>
42 <sourcefilterby></sourcefilterby>
43 <sourcefrom></sourcefrom>
44 <sourcelist></sourcelist>
45 <storevalue>T</storevalue>
46 <subtab>CRMSYSTEMINFORMATION</subtab>
47 <customfieldfilters>
48   <customfieldfilter>
49     <fldcomparefield></fldcomparefield>
50     <fldfilter>STDRECORDLOCATIONNAME</fldfilter>
51     <fldfilterchecked></fldfilterchecked>
52     <fldfiltercomparetype>LIKE</fldfiltercomparetype>
53     <fldfilternotnull>F</fldfilternotnull>
54     <fldfilternull>F</fldfilternull>
55     <fldfiltersel></fldfiltersel>
56     <fldfilterval>warehouse</fldfilterval>
57   </customfieldfilter>
58 </customfieldfilters>
59 <roleaccesses>
60   <roleaccess>
61     <accesslevel>0</accesslevel>
62     <role>ACCOUNTANT</role>
63     <searchlevel>0</searchlevel>
64   </roleaccess>
65 </roleaccesses>
66 </crmcustomfield>

```

Custom Entity Fields as XML Definitions

You can create XML definitions for custom entity fields as SDF custom objects using SuiteCloud Development Framework (SDF). You can either create an `entitycustomfield` object in a SuiteCloud project or import a custom entity field from a NetSuite account into a SuiteCloud project. For information about creating a custom entity field in a NetSuite account, see the following:

- [Creating Custom Entity Fields](#)
- [Field Type Descriptions for Custom Fields](#)

You can add custom entity fields to your entity records to gather information specific to your business needs. Entity custom fields can be added to existing and custom subtabs on the entry forms you use to enter entity records in a NetSuite account.

After an `entitycustomfield` object is deployed to an account, a custom entity field is created. The field appears on the entity records that you applied the object to.

Setting Values on the `entitycustomfield` SDF Custom Object

Provide a readable script ID attribute by using the underscore followed by a meaningful name for the SDF custom object. NetSuite automatically adds a `custentity` prefix that indicates the type of field, for example, `custentity_mynewfield`.

The following fields are required for `entitycustomfield` objects:

- `label` - Specifies the label for the custom field. The label is the text that your users see as the name of the custom field when it is displayed on the record.
- `fieldtype` - Specifies the type of custom field. For more information about the possible values of `fieldtype`, see the help topic [generic_customfield_fieldtype](#).
- `selectrecordtype` (conditional) - Specifies the list or record that contains the items for the custom field. This field is required only when the `fieldtype` value is equal to `SELECT` or `MULTISELECT`.

The following XML definition example is a basic `entitycustomfield` object with the required fields:

```

1 <entitycustomfield scriptid="custentity_mynewfield">
2   ...
3   <fieldtype>SELECT</fieldtype>
4   <label>SAMPLE ENTITYFIELD</label>
5   <selectrecordtype>-112</selectrecordtype>
6   ...
7 </entitycustomfield>

```

For more information about possible fields for `entitycustomfield` objects, see the help topic [entitycustomfield](#).

If you include any SDF custom objects, files, or scripts in your fields, you must define the dependencies for them in your SuiteCloud project's manifest. For more information, see [SDF Custom Object Dependencies in SuiteApps](#).

To index a custom field to be included in global search, set the `globalsearch` field value to T. The `globalsearch` field can equal to T only if the following field conditions are met:

- `storevalue` equals to T
- `fieldtype` equals to one of the following:
 - CURRENCY
 - FLOAT
 - EMAIL
 - TEXT
 - HELP
 - URL
 - INLINEHTML
 - INTEGER
 - PERCENT
 - PHONE
 - TEXTAREA



Warning: You cannot index a custom field for global search if any of your `searchlevel` field values equal to 0.

Embedding Structured Fields Within an `entitycustomfield` SDF Custom Object

You can create custom filters for your custom field by embedding `customfieldfilter` structured fields within the `entitycustomfield` SDF custom object. You can also customize the default access that a role has to the custom field by embedding `roleaccess` structured fields within the object.

For more information about the possible fields to define for these structured fields, see the following:

- [Custom Field Filters](#)
- [Role Accesses](#)

To view a full XML definition example for an `entitycustomfield` object, see [Example of an entitycustomfield SDF Custom Object](#).

Custom Field Filters

You can add custom field filters to a list/record or multiple select custom field to filter the choices available in that custom field on records and transactions. In SDF, you can use custom field filters by embedding customfieldfilter structured fields within a custom field object.

Note: Filtering can apply only when the fieldtype value of your custom field object is equal to SELECT or MULTISELECT. It does not apply to custom lists, meaning if you include customfieldfilter structured fields, your selectrecordtype value cannot equal a custom list scriptid.

customfieldfilters is the parent element and customfieldfilter is the child element. You can include multiple customfieldfilter child elements within the parent.

The fldfilter subfield specifies a standard field on the list/record that you entered for the selectrecordtype field. The fldfilter subfield is required. For more information about the supported fields and values for customfieldfilter structured fields, see the help topic [customfieldfilter](#).

When an object with a customfieldfilter structured field is deployed to a target account, the **Sourcing & Filtering** subtab of the custom field record is populated with the field values specified by the structured field. The choices available to users in a record or transaction for that custom field are tailored by the filter settings from the structured field.

Here is an example of a custom item field configuration page in NetSuite with filters set for the field:

Custom Item Field

Save Cancel Reset Change ID Apply to Forms Actions

LABEL *
SAMPLE ITEMCUSTOMFIELD

PUBLISHER ID

APP ID

ID
custitem_sample_itemcust

INTERNAL ID
543

OWNER
Larry Nelson

DESCRIPTION

MATRIX OPTION

TYPE
List/Record

LIST/RECORD
Account

STORE VALUE ☒ USE ENCRYPTED FORMAT ☐

SHOW IN LIST ☐

GLOBAL SEARCH ☐

RECORD IS PARENT ☐

INACTIVE ☐

Applies To Display Validation & Defaulting **Sourcing & Filtering** Access Translation History

SOURCE LIST

SOURCE FROM

SOURCE FILTER BY

FILTER USING *	IS CHECKED	COMPARE TYPE	COMPARE VALUE TO	VALUE IS	IS NOT EMPTY	IS EMPTY	COMPARE TO FIELD
Name		contains	paid				
		equal		<Type & tab for single value>			

Add Cancel Insert Remove

Important: If you change the field type or the list/record for a field that is used for sourcing or filtering on other custom fields, all sourcing and filtering based on the field is removed.

Additionally, you can define custom field filters in NetSuite. For more information, see the help topic [Setting Filtering Criteria](#).

Example of a customfieldfilter Structured Field

The following XML definition example is a customfieldfilter structured field embedded within an itemcustomfield object whose fieldtype equals SELECT. After this object is deployed to the target account,

a custom item field labeled `SAMPLE_ITEMCUSTOMFIELD` appears on the **Main** tab of all Inventory item records. The `fldfiltercomparetype` equals `LIKE` and `fldfilternull` equals `paid`. Therefore, the field's dropdown list is filtered to display only accounts that contain "paid" in the account name.

```

1 <itemcustomfield scriptid="custitem_sample_itemcust">
2   <appliedtoinventory>T</appliedtoinventory>
3   <fieldtype>SELECT</fieldtype>
4   <label>SAMPLE_ITEMCUSTOMFIELD</label>
5   <selectrecordtype>-112</selectrecordtype>
6   ...
7   <customfieldfilters>
8     <customfieldfilter>
9       <fldcomparefield></fldcomparefield>
10      <fldfilter>STDRECORDACCOUNTACCTNAME</fldfilter>
11      <fldfilterchecked></fldfilterchecked>
12      <fldfiltercomparetype>LIKE</fldfiltercomparetype>
13      <fldfilternotnull>F</fldfilternotnull>
14      <fldfilternull>F</fldfilternull>
15      <fldfiltersel></fldfiltersel>
16      <fldfilterinterval>paid</fldfilterinterval>
17    </customfieldfilter>
18  </customfieldfilters>
19  ...
20 </itemcustomfield>

```

Role Accesses

In SDF, you can control how roles access the information in custom fields by embedding `roleaccess` structured fields within a custom field object. The accesses that you specify determines how the custom field can be accessed on the record or through search results and reports. This helps you to maintain the security of your business information.

`roleaccesses` is the parent element and `roleaccess` is the child element. You can include multiple `roleaccess` child elements within the parent.

After an object with a `roleaccess` structured field is deployed to a NetSuite account, the **Role** section under the **Access** subtab of the custom field record is populated with the access level values of the fields specified by the structured field.

Here is an example of a custom item field configuration page in NetSuite with access levels set for a role:

Custom Item Field | List | Copy to Account (Beta)

Save | Cancel | Reset | Change ID | Apply to Forms | Actions

LABEL *
SAMPLE_ITEMCUSTOMFIELD

PUBLISHER ID

APP ID

ID
custitem_sample_itemcust

INTERNAL ID
543

OWNER
Larry Nelson

DESCRIPTION

MATRIX OPTION

TYPE
List/Record

LIST/RECORD
Account

STORE VALUE ☒ USE ENCRYPTED FORMAT ☐

SHOW IN LIST ☐

GLOBAL SEARCH ☐

RECORD IS PARENT ☐

INACTIVE ☐

Applies To | Display | Validation & Defaulting | Sourcing & Filtering | **Access** | Translation | History

DEFAULT ACCESS LEVEL
Edit

DEFAULT LEVEL FOR SEARCH/REPORTING
Edit

ROLE *	ACCESS LEVEL *	LEVEL FOR SEARCH/REPORTING *
Accountant	View	Run

✓ Add | ✗ Cancel | + Insert | - Remove

The following subfields are required for `roleaccess` structured fields:

- `role` - Specifies the role.
- `accesslevel` - Specifies the access level granted to the role for the custom field object.
Possible access level values:
 - 0 - None
 - 1 - View
 - 2 - Edit
- `searchlevel` - Specifies the access level granted to the role for the custom field object through search results and reports.
Possible access level values:
 - 0 - None
 - 1 - Run
 - 2 - Edit

For more information about the supported fields and values for `roleaccess`, see the help topic [roleaccess](#).

Additionally, you can define role accesses to custom fields from a NetSuite account. For more information, see the help topic [Restricting Access to Custom Fields](#).

Example of a `roleaccess` Structured Field

The following XML definition example is a `roleaccess` structured field embedded within a `itemcustomfield` object. In this example, the `accesslevel` for the Accountant role equals 1 and the `searchlevel` equals 1. Therefore, the role's access to the field is set to **View** and the access level through searching and reporting is set to **Run**.

```

1 <itemcustomfield scriptid="custitem_sample_itemcust">
2   <appliedtoinventory>T</appliedtoinventory>
3   <fieldtype>SELECT</fieldtype>
4   <label>SAMPLE_ITEMCUSTOMFIELD</label>
5   <selectrecordtype>-112</selectrecordtype>
6   ...
7   <roleaccesses>
8     <roleaccess>
9       <accesslevel>1</accesslevel>
10      <role>ACCOUNTANT</role>
11      <searchlevel>1</searchlevel>
12    </roleaccess>
13  </roleaccesses>
14  ...
15 </itemcustomfield>

```

Example of an `entitycustomfield` SDF Custom Object

The following XML definition example is for an `entitycustomfield` SDF custom object. After this object is deployed to the target account, a custom entity field labeled **SAMPLE ENTITYFIELD** appears on the Company Profile subtab of all entity records that the object applies to. In this example, the object applies to contact and customer records. The `fieldtype` equals `SELECT`. Therefore, a value for `selectrecordtype` is required.

```

1 <entitycustomfield scriptid="custentity_sample_test">
2   <accesslevel>2</accesslevel>
3   <appliedtocontact>T</appliedtocontact>
4   <appliedtocustomer>T</appliedtocustomer>
5   <appliedtoemployee>F</appliedtoemployee>
6   <appliedtogenericsrc>F</appliedtogenericsrc>

```

```

7      <appliestogroup>F</appliestogroup>
8      <appliestoothername>F</appliestoothername>
9      <appliestopartner>F</appliestopartner>
10     <appliestopricelist>F</appliestopricelist>
11     <appliestoproject>F</appliestoproject>
12     <appliestoprojecttemplate>F</appliestoprojecttemplate>
13     <appliestostatement>F</appliestostatement>
14     <appliestovendor>F</appliestovendor>
15     <appliestowebiste>F</appliestowebiste>
16     <applyformatting>F</applyformatting>
17     <availableexternally>F</availableexternally>
18     <checkspelling>F</checkspelling>
19     <defaultchecked>F</defaultchecked>
20     <defaultselection></defaultselection>
21     <defaultvalue></defaultvalue>
22     <description>Sample description for field.</description>
23     <displayheight></displayheight>
24     <displaytype>NORMAL</displaytype>
25     <displaywidth></displaywidth>
26     <dynamicdefault></dynamicdefault>
27     <encryptatrest>F</encryptatrest>
28     <fieldtype>SELECT</fieldtype>
29     <globalsearch>F</globalsearch>
30     <help>Enter a brief explanation of the kind of information you want entered in this field.</help>
31     <isformula>F</isformula>
32     <ismandatory>F</ismandatory>
33     <isparent>F</isparent>
34     <label>SAMPLE_ENTITYFIELD</label>
35     <linktext></linktext>
36     <maxlength></maxlength>
37     <maxvalue></maxvalue>
38     <minvalue></minvalue>
39     <onparentdelete>NO_ACTION</onparentdelete>
40     <parentsubtab></parentsubtab>
41     <searchcomparefield></searchcomparefield>
42     <searchdefault></searchdefault>
43     <searchlevel>2</searchlevel>
44     <selectrecordtype>112</selectrecordtype>
45     <showhierarchy>T</showhierarchy>
46     <showinlist>T</showinlist>
47     <sourcefilterby></sourcefilterby>
48     <sourcefrom></sourcefrom>
49     <sourcelist></sourcelist>
50     <storevalue>T</storevalue>
51     <subtab>ENTITYRELATIONSHIPS</subtab>
52     <customfieldfilters>
53       <customfieldfilter>
54         <fldcomparefield></fldcomparefield>
55         <fldfilter>STDRECORDACCOUNTACCTNAME</fldfilter>
56         <fldfilterchecked></fldfilterchecked>
57         <fldfiltercomparetype>LIKE</fldfiltercomparetype>
58         <fldfilternotnull>F</fldfilternotnull>
59         <fldfilternull>F</fldfilternull>
60         <fldfiltersel></fldfiltersel>
61         <fldfilterval>GST</fldfilterval>
62       </customfieldfilter>
63     </customfieldfilters>
64     <roleaccesses>
65       <roleaccess>
66         <accesslevel>1</accesslevel>
67         <role>ACCOUNTANT</role>
68         <searchlevel>1</searchlevel>
69       </roleaccess>
70     </roleaccesses>
71 </entitycustomfield>

```

Custom Item Number Fields as XML Definitions

You can create XML definitions of custom item number fields as SDF custom objects using SuiteCloud Development Framework (SDF). You can either create an `itemnumbercustomfield` object in a SuiteCloud

project or import custom item number fields from a NetSuite account into a SuiteCloud project. For information about creating a custom item number field in a NetSuite account, see the following:

- [Creating Custom Item Number Fields](#)
- [Field Type Descriptions for Custom Fields](#)

Add custom item number fields to serialized or lot numbered inventory item records to track information specific to each item or workflow unique to your business. For example, you can track quality control procedures or recall information.

Prerequisites for Using Custom Item Number Fields

Before creating a custom item number field, complete the following tasks to ensure the field can be used in a target account. These tasks include:

1. [Enabling Serial Numbered Inventory](#) and [Lot Numbered Items](#).

To create serialized or lot numbered inventory items, enable both of the inventory features.

2. [Inventory Adjustments](#).

To enter new serial or lot numbers for serialized or lot numbered inventory items, enter an Inventory Adjustment. The quantity of serial or lot numbers entered must equal the quantity of the adjustment.

Adjust the inventory if the inventory item has a quantity on hand amount of zero.

3. [Advanced Bin / Numbered Inventory Management](#).

To edit the value of your custom item number field, you must enable the Advanced Bin / Numbered Inventory Management feature. The Advanced Bin / Numbered Inventory Management feature lets you to view an item's Inventory Detail subrecord, which contains the serial/lot number record.

Setting Values on the itemnumbercustomfield SDF Custom Object

Provide a readable script ID attribute by using the underscore followed by a meaningful name for the SDF custom object. NetSuite automatically adds a customitemnumber prefix that indicates the type of field, for example, customitemnumber_mynewfield

The following fields are required for itemnumbercustomfield objects:

- `label` - Specifies the label for the custom field. The label is the text that your users see as the name of the custom field when it is displayed on the record.
- `fieldtype` - Specifies the type of custom field. For more information about the possible values of `fieldtype`, see the help topic [generic_customfield_fieldtype](#).
- `selectrecordtype` (conditional) - Specifies the list or record that contains the items for the custom field. This field is required only when the `fieldtype` value is equal to `SELECT` or `MULTISELECT`.

The following XML definition example is a basic itemnumbercustomfield object with the required fields:

```
1 <itemnumbercustomfield scriptid="custitemnumber_mynewfield">
2 ...
3   <fieldtype>SELECT</fieldtype>
```

```

4      <label>SAMPLE ITEMNUMBERFIELD</label>
5      <selectrecordtype>-127</selectrecordtype>
6      ...
7  </itemnumbercustomfield>

```

For more information about possible fields for `itemnumbercustomfield` objects, see the help topic [itemnumbercustomfield](#).

Important: Your `appliestoall` field can equal T if either `appliestolots`, `appliestoserialized`, or both are set to T as well. Otherwise, the `itemnumbercustomfield` object does not appear on the serial or lot number records.

If you include any SDF custom objects, files, or scripts in your fields, you must define the dependencies for them in your SuiteCloud project's manifest. For more information, see [SDF Custom Object Dependencies in SuiteApps](#).

After an `itemnumbercustomfield` object is deployed to an account, an item number field is created.

To apply the custom field to particular items, go to the object's item number field configuration page. Choose **Specific Items**. In the **Applies To** subtab, choose **Select Items to Apply Field** field, hold down the Ctrl key and click all the items you want this field to apply to, and click **Save**.

Here is an example of a custom item number field configuration page that applies to specific inventory items:

Item Number Field ← → List Copy to Account

Save **Cancel** **Reset** **Change ID** **Actions**

LABEL *
SAMPLE ITEMNUMBERFIELD

PUBLISHER ID

APP ID

ID
custitemnumber_record_nbr

INTERNAL ID
544

OWNER
Larry Nelson

DESCRIPTION
Sample description for field.

TYPE
List/Record

LIST/RECORD
Tax Period

☒ STORE VALUE ☐ USE ENCRYPTED FORMAT

☐ SHOW IN LIST

☐ INACTIVE

Applies To **Display** **Validation & Defaulting** **Sourcing & Filtering** **Access** **Translation** **History**

☒ **SPECIFIC ITEMS**

SELECT ITEMS TO APPLY FIELD

CARPETS/FLOORCOVERINGS : SAMPLE ITEMNAME 2

SAMPLE ITEMNAME

SAMPLE LOTITEM

Serialized-Test

z\$100 Gift Certificate

☐ **ALL ITEMS**

☐ SERIALIZED

☐ LOTS

☐ GIFT CERTIFICATE

Save **Cancel** **Reset** **Change ID** **Actions**

To view and edit the custom item number field, go to an inventory item record that has the custom field applied to it. On the **Inventory Detail** subtab, select a number under the **Serial/Lot Number** column. When you click a number, the serial or lot number record displays in a popup window. The field you created is included here.

Here is an example of a custom item number field included on a serial number record:

Serial Number Record

Edit

Back

Search Transactions

ITEM NAME Serialized-Test	MEMO
SERIAL/LOT NUMBER 1	SAMPLE ITEMNUMBERFIELD FY 2013 : Q4 2013 : Dec 2013
STATUS Available	TEST 1

Note: In the NetSuite UI, the inventory detail subrecord also displays as a popup window on records and transaction forms when you click the **Inventory Detail** icon ( or ). On the popup window, click a number under the **Serial/Lot Number** column to display the serial or lot number record.

Embedding Structured Fields Within an itemnumbercustomfield SDF Custom Object

You can create custom filters for your custom field by embedding customfieldfilter structured fields within the itemnumbercustomfield SDF custom object. You can also customize the default access that a role has to the custom field by embedding roleaccess structured fields within the itemnumbercustomfield object.

For more information about the possible fields to define for these structured fields, see the following:

- [Custom Field Filters](#)
- [Role Accesses](#)

To view a full XML definition example for an itemnumbercustomfield object, see [Example of an itemnumbercustomfield SDF Custom Object](#).

Custom Field Filters

You can add custom field filters to a list/record or multiple select custom field to filter the choices available in that custom field on records and transactions. In SDF, you can use custom field filters by embedding customfieldfilter structured fields within a custom field object.

Note: Filtering can apply only when the fieldtype value of your custom field object is equal to SELECT or MULTISELECT. It does not apply to custom lists, meaning if you include customfieldfilter structured fields, your selectrecordtype value cannot equal a custom list scriptid.

customfieldfilters is the parent element and customfieldfilter is the child element. You can include multiple customfieldfilter child elements within the parent.

The fldfilter subfield specifies a standard field on the list/record that you entered for the selectrecordtype field. The fldfilter subfield is required. For more information about the supported fields and values for customfieldfilter structured fields, see the help topic [customfieldfilter](#).

When an object with a customfieldfilter structured field is deployed to a target account, the **Sourcing & Filtering** subtab of the custom field record is populated with the field values specified by the structured field. The choices available to users in a record or transaction for that custom field are tailored by the filter settings from the structured field.

Here is an example of a custom item field configuration page in NetSuite with filters set for the field:



Important: If you change the field type or the list/record for a field that is used for sourcing or filtering on other custom fields, all sourcing and filtering based on the field is removed.

Additionally, you can define custom field filters in NetSuite. For more information, see the help topic [Setting Filtering Criteria](#).

Example of a customfieldfilter Structured Field

The following XML definition example is a customfieldfilter structured field embedded within an itemcustomfield object whose fieldtype equals SELECT. After this object is deployed to the target account, a custom item field labeled SAMPLE_ITEMCUSTOMFIELD appears on the **Main** tab of all Inventory item records. The fldfiltercomparetype equals LIKE and fldfilterinterval equals paid. Therefore, the field's dropdown list is filtered to display only accounts that contain "paid" in the account name.

```

1 <itemcustomfield scriptid="custitem_sample_itemcust">
2   <appliedtoinventory>T</appliedtoinventory>
3   <fieldtype>SELECT</fieldtype>
4   <label>SAMPLE_ITEMCUSTOMFIELD</label>
5   <selectrecordtype>-112</selectrecordtype>
6   ...
7   <customfieldfilters>
8     <customfieldfilter>
9       <fldcomparefield></fldcomparefield>
10      <fldfilter>STDCORDACCOUNTACCTNAME</fldfilter>
11      <fldfilterchecked></fldfilterchecked>
12      <fldfiltercomparetype>LIKE</fldfiltercomparetype>
13      <fldfilternotnull>F</fldfilternotnull>
14      <fldfilternull>F</fldfilternull>
15      <fldfiltersel></fldfiltersel>

```



```

16         <fldfilterval>paid>
17     </customfieldfilter>
18 </customfieldfilters>
19 ...
20 </itemcustomfield>

```

Role Accesses

In SDF, you can control how roles access the information in custom fields by embedding `roleaccess` structured fields within a custom field object. The accesses that you specify determines how the custom field can be accessed on the record or through search results and reports. This helps you to maintain the security of your business information.

`roleaccesses` is the parent element and `roleaccess` is the child element. You can include multiple `roleaccess` child elements within the parent.

After an object with a `roleaccess` structured field is deployed to a NetSuite account, the **Role** section under the **Access** subtab of the custom field record is populated with the access level values of the fields specified by the structured field.

Here is an example of a custom item field configuration page in NetSuite with access levels set for a role:

Custom Item Field

Save Cancel Reset Change ID Apply to Forms Actions

LABEL *
SAMPLE ITEMCUSTOMFIELD

PUBLISHER ID

APP ID

ID
custitem_sample_itemcust

INTERNAL ID
543

OWNER
Larry Nelson

DESCRIPTION

MATRIX OPTION

TYPE
List/Record

LIST/RECORD
Account

STORE VALUE ☒ USE ENCRYPTED FORMAT ☐

SHOW IN LIST ☐

GLOBAL SEARCH ☐

RECORD IS PARENT ☐

INACTIVE ☐

Applies To Display Validation & Defaulting Sourcing & Filtering **Access** Translation History

DEFAULT ACCESS LEVEL
Edit

DEFAULT LEVEL FOR SEARCH/REPORTING
Edit

Role *	ACCESS LEVEL *	LEVEL FOR SEARCH/REPORTING *
Accountant	View	Run

Add Cancel Insert Remove

The following subfields are required for `roleaccess` structured fields:

- `role` - Specifies the role.
- `accesslevel` - Specifies the access level granted to the role for the custom field object.
Possible access level values:
 - 0 - None
 - 1 - View
 - 2 - Edit
- `searchlevel` - Specifies the access level granted to the role for the custom field object through search results and reports.
Possible access level values:

- 0 - None
- 1 - Run
- 2 - Edit

For more information about the supported fields and values for `roleaccess`, see the help topic [roleaccess](#).

Additionally, you can define role accesses to custom fields from a NetSuite account. For more information, see the help topic [Restricting Access to Custom Fields](#).

Example of a `roleaccess` Structured Field

The following XML definition example is a `roleaccess` structured field embedded within a `itemcustomfield` object. In this example, the `accesslevel` for the Accountant role equals 1 and the `searchlevel` equals 1. Therefore, the role's access to the field is set to **View** and the access level through searching and reporting is set to **Run**.

```

1 <itemcustomfield scriptid="custitem_sample_itemcust">
2   <appliedtoinventory>T</appliedtoinventory>
3   <fieldtype>SELECT</fieldtype>
4   <label>SAMPLE ITEMCUSTOMFIELD</label>
5   <selectrecordtype>-112</selectrecordtype>
6   ...
7   <roleaccesses>
8     <roleaccess>
9       <accesslevel>1</accesslevel>
10      <role>ACCOUNTANT</role>
11      <searchlevel>1</searchlevel>
12    </roleaccess>
13  </roleaccesses>
14  ...
15 </itemcustomfield>

```

Example of an `itemnumbercustomfield` SDF Custom Object

The following XML definition example is for an `itemnumbercustomfield` SDF custom object. After this object is deployed to the target account, a custom item number field labeled `SAMPLE ITEMNUMBERFIELD` appears on all serial and lot number records. The `fieldtype` equals `SELECT`. Therefore, a value for `selectrecordtype` is required.

```

1 <itemnumbercustomfield scriptid="custitemnumber_record_nbr">
2   <accesslevel>2</accesslevel>
3   <appliedtoallitems>T</appliedtoallitems>
4   <appliedtogiftcerts>F</appliedtogiftcerts>
5   <appliedtolots>T</appliedtolots>
6   <appliedtoserialized>T</appliedtoserialized>
7   <applyformatting>F</applyformatting>
8   <checkspelling>F</checkspelling>
9   <defaultchecked>F</defaultchecked>
10  <defaultselection></defaultselection>
11  <defaultvalue></defaultvalue>
12  <description>Sample description for field.</description>
13  <displayheight></displayheight>
14  <displaytype>NORMAL</displaytype>
15  <displaywidth></displaywidth>
16  <dynamicdefault></dynamicdefault>
17  <encryptatrest>F</encryptatrest>
18  <fieldtype>SELECT</fieldtype>
19  <help>Enter a brief explanation of the kind of information you want entered in this field.</help>
20  <isformula>F</isformula>
21  <ismandatory>F</ismandatory>
22  <label>SAMPLE ITEMNUMBERFIELD</label>

```

```

23 <linktext></linktext>
24 <maxlength></maxlength>
25 <maxvalue></maxvalue>
26 <minvalue></minvalue>
27 <onparentdelete>NO_ACTION</onparentdelete>
28 <searchcomparefield></searchcomparefield>
29 <searchdefault></searchdefault>
30 <searchlevel>2</searchlevel>
31 <selectrecordtype>-127</selectrecordtype>
32 <showhierarchy>T</showhierarchy>
33 <showinlist>F</showinlist>
34 <sourcefilterby></sourcefilterby>
35 <sourcefrom></sourcefrom>
36 <sourcelist></sourcelist>
37 <storevalue>T</storevalue>
38 <customfieldfilters>
39   <customfieldfilter>
40     <fldcomparefield></fldcomparefield>
41     <fldfilter>STDRECORDTAXPERIODPERIODNAME</fldfilter>
42     <fldfilterchecked></fldfilterchecked>
43     <fldfiltercomparetype>LIKE</fldfiltercomparetype>
44     <fldfilternotnull>F</fldfilternotnull>
45     <fldfilternull>F</fldfilternull>
46     <fldfiltersel></fldfiltersel>
47     <fldfilterval>2013</fldfilterval>
48   </customfieldfilter>
49 </customfieldfilters>
50 <roleaccesses>
51   <roleaccess>
52     <accesslevel>1</accesslevel>
53     <role>ACCOUNTANT</role>
54     <searchlevel>1</searchlevel>
55   </roleaccess>
56 </roleaccesses>
57 </itemnumbercustomfield>

```

Custom Transaction Body Fields as XML Definitions

You can create XML definitions for custom transaction body fields as SDF custom objects using SuiteCloud Development Framework (SDF). You can either create a transactionbodycustomfield object in a SuiteCloud project or import a custom transaction body field from a NetSuite account into a SuiteCloud project. For information about creating a custom transaction body field in a NetSuite account, see the following:

- [Creating Custom Transaction Body Fields](#)
- [Field Type Descriptions for Custom Fields](#)

Add custom transaction body fields to the body of transaction records to gather information specific to your business needs.

These transaction records include:

- purchase transactions
- sales transactions
- revenue arrangements
- opportunities
- journal entries
- expense reports
- Web store checkout

- item receipts
- item fulfillments
- fulfillment requests
- store pickup fulfillments

After a `transactionbodycustomfield` object is deployed to an account, a transaction body field record is created. The field appears on the transaction records that you applied the object to.

Setting Values on the `transactionbodycustomfield` SDF Custom Object

Provide a readable script ID attribute by using the underscore followed by a meaningful name for the SDF custom object. NetSuite automatically adds a `custbody` prefix that indicates the type of field, for example, `custbody_mynewfield`.

The following fields are required for `transactionbodycustomfield` objects:

- `label` - Specifies the label for the custom field. The label is the text that your users see as the name of the custom field when it is displayed on the record.
- `fieldtype` - Specifies the type of custom field. For more information about the possible values of `fieldtype`, see the help topic [generic_customfield_fieldtype](#).
- `selectrecordtype` (conditional) - Specifies the list or record that contains the items for the custom field. This field is required only when the `fieldtype` value is equal to `SELECT` or `MULTISELECT`.

The following XML definition example is a basic `transactionbodycustomfield` object with the required fields:

```

1 <transactionbodycustomfield scriptid="custbody_mynewfield">
2 ...
3   <fieldtype>SELECT</fieldtype>
4   <label>SAMPLE TRANSACTIONBODYFIELD</label>
5   <selectrecordtype>-10</selectrecordtype>
6 ...
7 </transactionbodycustomfield>

```

For more information about possible fields for `transactionbodycustomfield` objects, see the help topic [transactionbodycustomfield](#).

If you include any SDF custom objects, files, or scripts in your fields, you must define the dependencies for them in your SuiteCloud project's manifest. For more information, see [SDF Custom Object Dependencies in SuiteApps](#).

To index a custom field to be included in global search, set the `globalsearch` field value to `T`. The `globalsearch` field can equal to `T` only if the following field conditions are met:

- `storevalue` equals to `T`
- `fieldtype` equals to one of the following:
 - `CURRENCY`
 - `FLOAT`
 - `EMAIL`
 - `TEXT`

- HELP
- URL
- INL INEHTML
- INTEGER
- PERCENT
- PHONE
- TEXTAREA



Warning: You cannot index a custom field for global search if any of your searchlevel field values equal to 0.

Embedding Structured Fields Within a transactionbodycustomfield SDF Custom Object

You can create custom filters for your custom field by embedding customfieldfilter structured fields within the transactionbodycustomfield SDF custom object. You can also customize the default access that a role has to the custom field by embedding roleaccess structured fields within the object.

For more information about the possible fields to define for these structured fields, see the following:

- [Custom Field Filters](#)
- [Role Accesses](#)

To view a full XML definition example for an transactionbodycustomfield object, see [Example of a transactionbodycustomfield SDF Custom Object](#).

Custom Field Filters

You can add custom field filters to a list/record or multiple select custom field to filter the choices available in that custom field on records and transactions. In SDF, you can use custom field filters by embedding customfieldfilter structured fields within a custom field object.



Note: Filtering can apply only when the fieldtype value of your custom field object is equal to SELECT or MULTISELECT. It does not apply to custom lists, meaning if you include customfieldfilter structured fields, your selectrecordtype value cannot equal a custom list scriptid.

customfieldfilters is the parent element and customfieldfilter is the child element. You can include multiple customfieldfilter child elements within the parent.

The fldfilter subfield specifies a standard field on the list/record that you entered for the selectrecordtype field. The fldfilter subfield is required. For more information about the supported fields and values for customfieldfilter structured fields, see the help topic [customfieldfilter](#).

When an object with a customfieldfilter structured field is deployed to a target account, the **Sourcing & Filtering** subtab of the custom field record is populated with the field values specified by the structured field. The choices available to users in a record or transaction for that custom field are tailored by the filter settings from the structured field.

Here is an example of a custom item field configuration page in NetSuite with filters set for the field:

Custom Item Field ← → List Copy to Account (Beta)

LABEL *
 PUBLISHER ID
 APP ID
 ID
 INTERNAL ID
 OWNER

DESCRIPTION
☐ MATRIX OPTION
 TYPE
 LIST/RECORD

☒ STORE VALUE ☐ USE ENCRYPTED FORMAT
☐ SHOW IN LIST
☐ GLOBAL SEARCH
☐ RECORD IS PARENT
☐ INACTIVE

Applies To Display Validation & Defaulting **Sourcing & Filtering** Access Translation History

SOURCE LIST
 SOURCE FROM
 SOURCE FILTER BY

FILTER USING *	IS CHECKED	COMPARE TYPE	COMPARE VALUE TO	VALUE IS	IS NOT EMPTY	IS EMPTY	COMPARE TO FIELD
Name		contains	paid				
		equal		<Type & tab for single value>			



Important: If you change the field type or the list/record for a field that is used for sourcing or filtering on other custom fields, all sourcing and filtering based on the field is removed.

Additionally, you can define custom field filters in NetSuite. For more information, see the help topic [Setting Filtering Criteria](#).

Example of a customfieldfilter Structured Field

The following XML definition example is a customfieldfilter structured field embedded within an itemcustomfield object whose fieldtype equals SELECT. After this object is deployed to the target account, a custom item field labeled SAMPLE ITEMCUSTOMFIELD appears on the **Main** tab of all Inventory item records. The fldfiltercomparetype equals LIKE and fldfilterval equals paid. Therefore, the field's dropdown list is filtered to display only accounts that contain "paid" in the account name.

```

1 <itemcustomfield scriptid="custitem_sample_itemcust">
2   <appliestoinventory>T</appliestoinventory>
3   <fieldtype>SELECT</fieldtype>
4   <label>SAMPLE ITEMCUSTOMFIELD</label>
5   <selectrecordtype>-112</selectrecordtype>
6   ...
7   <customfieldfilters>
8     <customfieldfilter>
9       <fldcomparefield></fldcomparefield>
10      <fldfilter>STDRECORDACCOUNTACCTNAME</fldfilter>
11      <fldfilterchecked></fldfilterchecked>
12      <fldfiltercomparetype>LIKE</fldfiltercomparetype>
13      <fldfilternotnull>F</fldfilternotnull>
14      <fldfilternull>F</fldfilternull>
15      <fldfiltersel></fldfiltersel>
16      <fldfilterval>paid</fldfilterval>
17    </customfieldfilter>
18  </customfieldfilters>
19  ...
20 </itemcustomfield>
  
```

Role Accesses

In SDF, you can control how roles access the information in custom fields by embedding roleaccess structured fields within a custom field object. The accesses that you specify determines how the custom

field can be accessed on the record or through search results and reports. This helps you to maintain the security of your business information.

roleaccesses is the parent element and roleaccess is the child element. You can include multiple roleaccess child elements within the parent.

After an object with a roleaccess structured field is deployed to a NetSuite account, the **Role** section under the **Access** subtab of the custom field record is populated with the access level values of the fields specified by the structured field.

Here is an example of a custom item field configuration page in NetSuite with access levels set for a role:

Custom Item Field

Save Cancel Reset Change ID Apply to Forms Actions

LABEL *
 SAMPLE ITEMCUSTOMFIELD
 PUBLISHER ID
 APP ID
 ID
 custitem_sample_itemcust
 INTERNAL ID
 543
 OWNER
 Larry Nelson

DESCRIPTION
 TYPE
 List/Record
 LIST/RECORD
 Account

☒ STORE VALUE ☐ USE ENCRYPTED FORMAT
☐ SHOW IN LIST
☐ GLOBAL SEARCH
☐ RECORD IS PARENT
☐ INACTIVE

Applies To Display Validation & Defaulting Sourcing & Filtering **Access** Translation History

DEFAULT ACCESS LEVEL
 Edit
 DEFAULT LEVEL FOR SEARCH/REPORTING
 Edit

Role *	ACCESS LEVEL *	LEVEL FOR SEARCH/REPORTING *
Accountant	View	Run

The following subfields are required for roleaccess structured fields:

- role - Specifies the role.
- accesslevel - Specifies the access level granted to the role for the custom field object.
Possible access level values:
 - 0 - None
 - 1 - View
 - 2 - Edit
- searchlevel - Specifies the access level granted to the role for the custom field object through search results and reports.
Possible access level values:
 - 0 - None
 - 1 - Run
 - 2 - Edit

For more information about the supported fields and values for roleaccess, see the help topic [roleaccess](#).

Additionally, you can define role accesses to custom fields from a NetSuite account. For more information, see the help topic [Restricting Access to Custom Fields](#).

Example of a roleaccess Structured Field

The following XML definition example is a roleaccess structured field embedded within a itemcustomfield object. In this example, the accesslevel for the Accountant role equals 1 and the searchlevel equals 1. Therefore, the role's access to the field is set to **View** and the access level through searching and reporting is set to **Run**.

```

1 <itemcustomfield scriptid="custitem_sample_itemcust">
2   <appliestoinventory>T</appliestoinventory>
3   <fieldtype>SELECT</fieldtype>
4   <label>SAMPLE ITEMCUSTOMFIELD</label>
5   <selectrecordtype>-112</selectrecordtype>
6   ...
7   <roleaccesses>
8     <roleaccess>
9       <accesslevel>1</accesslevel>
10      <role>ACCOUNTANT</role>
11      <searchlevel>1</searchlevel>
12    </roleaccess>
13  </roleaccesses>
14  ...
15 </itemcustomfield>

```

Example of a transactionbodycustomfield SDF Custom Object

The following XML definition example is for a transactionbodycustomfield object. After this SDF custom object is deployed to the target account, a custom transaction body field labeled SAMPLE TRANSACTIONBODYFIELD appears on the Main tab of all transaction records that the object is applied to. In this example, the object applies to purchase transactions, sales transactions, and item receipts. The fieldtype equals SELECT. Therefore, a value for selectrecordtype is required.

```

1 <transactionbodycustomfield scriptid="custbody_sample_transactionbody">
2   <accesslevel>2</accesslevel>
3   <applyformatting>F</applyformatting>
4   <bodycustomerpayment>F</bodycustomerpayment>
5   <bodydeposit>F</bodydeposit>
6   <bodydepositapplication>F</bodydepositapplication>
7   <bodyexpensereport>F</bodyexpensereport>
8   <bodyinventoryadjustment>F</bodyinventoryadjustment>
9   <bodyitemfulfillment>F</bodyitemfulfillment>
10  <bodyitemfulfillmentorder>F</bodyitemfulfillmentorder>
11  <bodyitemreceipt>T</bodyitemreceipt>
12  <bodyitemreceiptorder>F</bodyitemreceiptorder>
13  <bodyjournal>F</bodyjournal>
14  <bodyopportunity>F</bodyopportunity>
15  <bodyothertransaction>F</bodyothertransaction>
16  <bodypickingticket>F</bodypickingticket>
17  <bodyprintflag>F</bodyprintflag>
18  <bodyprintpackingslip>F</bodyprintpackingslip>
19  <bodyprintstatement>F</bodyprintstatement>
20  <bodypurchase>T</bodypurchase>
21  <bodysale>T</bodysale>
22  <bodystore>F</bodystore>
23  <bodytransferorder>F</bodytransferorder>
24  <bodyvendorpayment>F</bodyvendorpayment>
25  <checkspelling>F</checkspelling>
26  <defaultchecked>F</defaultchecked>
27  <defaultselection></defaultselection>
28  <defaultvalue></defaultvalue>
29  <description>Enter a description of this field.</description>
30  <displayheight></displayheight>
31  <displaytype>NORMAL</displaytype>
32  <displaywidth></displaywidth>
33  <dynamicdefault></dynamicdefault>
34  <encryptatrest>F</encryptatrest>
35  <fieldtype>SELECT</fieldtype>
36  <fldsize-label></fldsize-label>

```



```

37 <globalsearch>F</globalsearch>
38 <help>Enter a brief explanation of the kind of information you want entered in this field.</help>
39 <isformula>F</isformula>
40 <ismandatory>F</ismandatory>
41 <isparent>F</isparent>
42 <label>SAMPLE TRANSACTIONBODYFIELD</label>
43 <linktext></linktext>
44 <maxlength></maxlength>
45 <maxvalue></maxvalue>
46 <minvalue></minvalue>
47 <onparentdelete>SET_NULL</onparentdelete>
48 <parentsubtab></parentsubtab>
49 <searchcomparefield></searchcomparefield>
50 <searchdefault></searchdefault>
51 <searchlevel>2</searchlevel>
52 <selectrecordtype>-10</selectrecordtype>
53 <showhierarchy>T</showhierarchy>
54 <showinlist>F</showinlist>
55 <sourcefilterby></sourcefilterby>
56 <sourcefrom></sourcefrom>
57 <sourcelist></sourcelist>
58 <storevalue>T</storevalue>
59 <subtab>TRANSACTIONMAIN</subtab>
60 <customfieldfilters>
61   <customfieldfilter>
62     <fldcomparefield></fldcomparefield>
63     <fldfilter>STDITEMNAME</fldfilter>
64     <fldfilterchecked></fldfilterchecked>
65     <fldfiltercomparetype>LIKE</fldfiltercomparetype>
66     <fldfilternotnull>F</fldfilternotnull>
67     <fldfilternull>F</fldfilternull>
68     <fldfiltersel></fldfiltersel>
69     <fldfilterval>furniture</fldfilterval>
70   </customfieldfilter>
71 </customfieldfilters>
72 <roleaccesses>
73   <roleaccess>
74     <accesslevel>1</accesslevel>
75     <role>ACCOUNTANT</role>
76     <searchlevel>1</searchlevel>
77   </roleaccess>
78 </roleaccesses>
79 </transactionbodycustomfield>

```

Custom Transaction Line Fields as XML Definitions

In SuiteCloud Development Framework (SDF), you can create XML definitions of custom transaction line fields as SDF custom objects. You can either create a `transactioncolumncustomfield` object in a SuiteCloud project or import a custom transaction line field from a NetSuite account into a SuiteCloud project. For information about creating a custom transaction line field in a NetSuite account, see the following:

- [Creating Custom Transaction Line Fields](#)
- [Field Type Descriptions for Custom Fields](#)

Add custom transaction line fields to the line items of your transaction records to gather information specific to your business needs.

These transaction records include:

- expense items
- purchase items
- sale items
- store items
- revenue arrangements

- journal entries
- expense reports
- item receipts or fulfillments
- opportunity items
- time cards
- inventory adjustment
- inventory transfer
- fulfillment requests
- store pickup fulfillments

After a `transactioncolumncustomfield` object is deployed to an account, a transaction line field record is created. The field appears under a new column for line items on the transaction records that you applied the object to.

Setting Values on the `transactioncolumncustomfield` SDF Custom Object

Provide a readable script ID attribute by using the underscore followed by a meaningful name for the SDF custom object. NetSuite automatically adds a `custcol` prefix that indicates the type of field, for example, `custcol_mynewfield`.

The following fields are required for `transactioncolumncustomfield` objects:

- `label` - Specifies the label for the custom field. The label is the text that your users see as the name of the custom field when it is displayed on the record.
- `fieldtype` - Specifies the type of custom field. For more information about the possible values of `fieldtype`, see the help topic [generic_customfield_fieldtype](#).
- `selectrecordtype` (conditional) - Specifies the list or record that contains the items for the custom field. This field is required only when the `fieldtype` value is equal to `SELECT` or `MULTISELECT`.

The following XML definition example is a basic `transactioncolumncustomfield` object with the required fields:

```

1 <transactioncolumncustomfield scriptid="custcol_mynewfield">
2 ...
3   <fieldtype>SELECT</fieldtype>
4   <label>SAMPLE TRANSACTIONCOLUMN</label>
5   <selectrecordtype>-112</selectrecordtype>
6 ...
7 </transactioncolumncustomfield>

```

For more information about possible fields for `transactioncolumncustomfield` objects, see the help topic [transactioncolumncustomfield](#).

If you include any SDF custom objects, files, or scripts in your fields, you must define the dependencies for them in your SuiteCloud project's manifest. For more information, see [SDF Custom Object Dependencies in SuiteApps](#).

Embedding Structured Fields Within a `transactioncolumncustomfield` SDF Custom Object

You can create custom filters for your custom field by embedding `customfieldfilter` structured fields within the `transactioncolumncustomfield` SDF custom object. You can also customize the default access that a role has to the custom field by embedding `roleaccess` structured fields within the object.

For more information about the possible fields to define for these structured fields, see the following:

- [Custom Field Filters](#)
- [Role Accesses](#)

To view a full XML definition example for an transactioncolumncustomfield object, see [Example of a transactioncolumncustomfield SDF Custom Object](#).

Custom Field Filters

You can add custom field filters to a list/record or multiple select custom field to filter the choices available in that custom field on records and transactions. In SDF, you can use custom field filters by embedding customfieldfilter structured fields within a custom field object.

Note: Filtering can apply only when the fieldtype value of your custom field object is equal to SELECT or MULTISELECT. It does not apply to custom lists, meaning if you include customfieldfilter structured fields, your selectrecordtype value cannot equal a custom list scriptid.

customfieldfilters is the parent element and customfieldfilter is the child element. You can include multiple customfieldfilter child elements within the parent.

The fldfilter subfield specifies a standard field on the list/record that you entered for the selectrecordtype field. The fldfilter subfield is required. For more information about the supported fields and values for customfieldfilter structured fields, see the help topic [customfieldfilter](#).

When an object with a customfieldfilter structured field is deployed to a target account, the **Sourcing & Filtering** subtab of the custom field record is populated with the field values specified by the structured field. The choices available to users in a record or transaction for that custom field are tailored by the filter settings from the structured field.

Here is an example of a custom item field configuration page in NetSuite with filters set for the field:

Custom Item Field

Save Cancel Reset Change ID Apply to Forms Actions

Label: SAMPLE ITEMCUSTOMFIELD

Publisher ID: [Empty]

App ID: [Empty]

ID: custitem_sample_itemcust

Internal ID: 543

Owner: Larry Nelson

Description: [Empty]

Type: List/Record

List/Record: Account

Matrix Option: [Empty]

Store Value: [Checked] Use Encrypted Format: [Empty]

Show in List: [Empty]

Global Search: [Empty]

Record is Parent: [Empty]

Inactive: [Empty]

Applies To: Display Validation & Defaulting **Sourcing & Filtering** Access Translation History

Source List: [Empty]

Source Filter By: [Empty]

Source From: [Empty]

FILTER USING	IS CHECKED	COMPARE TYPE	COMPARE VALUE TO	VALUE IS	IS NOT EMPTY	IS EMPTY	COMPARE TO FIELD
Name		contains	paid				
		equal		<Type & tab for single value>			

Add Cancel Insert Remove

Important: If you change the field type or the list/record for a field that is used for sourcing or filtering on other custom fields, all sourcing and filtering based on the field is removed.

Additionally, you can define custom field filters in NetSuite. For more information, see the help topic [Setting Filtering Criteria](#).

Example of a customfieldfilter Structured Field

The following XML definition example is a customfieldfilter structured field embedded within an itemcustomfield object whose fieldtype equals SELECT. After this object is deployed to the target account, a custom item field labeled SAMPLE_ITEMCUSTOMFIELD appears on the **Main** tab of all Inventory item records. The fldfiltercomparetype equals LIKE and fldfilterval equals paid. Therefore, the field's dropdown list is filtered to display only accounts that contain "paid" in the account name.

```

1 <itemcustomfield scriptid="custitem_sample_itemcust">
2   <appliedtoinventory>I</appliedtoinventory>
3   <fieldtype>SELECT</fieldtype>
4   <label>SAMPLE_ITEMCUSTOMFIELD</label>
5   <selectrecordtype>-112</selectrecordtype>
6   ...
7   <customfieldfilters>
8     <customfieldfilter>
9       <fldcomparefield></fldcomparefield>
10      <fldfilter>STDRECORDACCOUNTACCTNAME</fldfilter>
11      <fldfilterchecked></fldfilterchecked>
12      <fldfiltercomparetype>LIKE</fldfiltercomparetype>
13      <fldfilternotnull>F</fldfilternotnull>
14      <fldfilternull>F</fldfilternull>
15      <fldfiltersel></fldfiltersel>
16      <fldfilterval>paid</fldfilterval>
17    </customfieldfilter>
18  </customfieldfilters>
19  ...
20 </itemcustomfield>

```

Role Accesses

In SDF, you can control how roles access the information in custom fields by embedding roleaccess structured fields within a custom field object. The accesses that you specify determines how the custom field can be accessed on the record or through search results and reports. This helps you to maintain the security of your business information.

roleaccesses is the parent element and roleaccess is the child element. You can include multiple roleaccess child elements within the parent.

After an object with a roleaccess structured field is deployed to a NetSuite account, the **Role** section under the **Access** subtab of the custom field record is populated with the access level values of the fields specified by the structured field.

Here is an example of a custom item field configuration page in NetSuite with access levels set for a role:

The following subfields are required for roleaccess structured fields:

- **role** - Specifies the role.
- **accesslevel** - Specifies the access level granted to the role for the custom field object.
Possible access level values:
 - 0 - None
 - 1 - View
 - 2 - Edit
- **searchlevel** - Specifies the access level granted to the role for the custom field object through search results and reports.
Possible access level values:
 - 0 - None
 - 1 - Run
 - 2 - Edit

For more information about the supported fields and values for roleaccess, see the help topic [roleaccess](#).

Additionally, you can define role accesses to custom fields from a NetSuite account. For more information, see the help topic [Restricting Access to Custom Fields](#).

Example of a roleaccess Structured Field

The following XML definition example is a roleaccess structured field embedded within a itemcustomfield object. In this example, the accesslevel for the Accountant role equals 1 and the searchlevel equals 1. Therefore, the role's access to the field is set to **View** and the access level through searching and reporting is set to **Run**.

```

1 <itemcustomfield scriptid="custitem_sample_itemcust">
2   <appliestoinventory>T</appliestoinventory>
3   <fieldtype>SELECT</fieldtype>
4   <label>SAMPLE ITEMCUSTOMFIELD</label>
5   <selectrecordtype>-112</selectrecordtype>
6   ...
7   <roleaccesses>
```

```

8      <roleaccess>
9        <accesslevel>1</accesslevel>
10       <role>ACCOUNTANT</role>
11       <searchlevel>1</searchlevel>
12     </roleaccess>
13   </roleaccesses>
14   ...
15 </itemcustomfield>

```

Example of a transactioncolumncustomfield SDF Custom Object

The following XML definition example is for a transactioncolumncustomfield object. After this SDF custom object is deployed to the target account, a custom transaction body field appears under a column labeled SAMPLE TRANSACTIONCOLUMN for all transaction records that the object is applied to. In this example, the object applies to inventory adjustments. The fieldtype equals SELECT. Therefore, a value for selectrecordtype is required.

```

1 <transactioncolumncustomfield scriptid="custcol_sample_transactioncolumn">
2   <accesslevel>2</accesslevel>
3   <applyformatting>F</applyformatting>
4   <colexpense>F</colexpense>
5   <colexpensereport>F</colexpensereport>
6   <colgrouponinvoices>F</colgrouponinvoices>
7   <colinventoryadjustment>T</colinventoryadjustment>
8   <colitemfulfillment>F</colitemfulfillment>
9   <colitemfulfillmentorder>F</colitemfulfillmentorder>
10  <colitemreceipt>F</colitemreceipt>
11  <colitemreceiptorder>F</colitemreceiptorder>
12  <coljournal>F</coljournal>
13  <colkititem>F</colkititem>
14  <colopportunity>F</colopportunity>
15  <colpackingslip>F</colpackingslip>
16  <colpickingticket>F</colpickingticket>
17  <colprintflag>F</colprintflag>
18  <colpurchase>F</colpurchase>
19  <colreturnform>F</colreturnform>
20  <colsale>F</colsale>
21  <colstore>F</colstore>
22  <colstorehidden>F</colstorehidden>
23  <colstorewithgroups>F</colstorewithgroups>
24  <coltime>F</coltime>
25  <coltransferorder>F</coltransferorder>
26  <defaultchecked>F</defaultchecked>
27  <defaultselection></defaultselection>
28  <defaultvalue></defaultvalue>
29  <description>Sample description for field.</description>
30  <displayheight></displayheight>
31  <displaytype>NORMAL</displaytype>
32  <displaywidth></displaywidth>
33  <dynamicdefault></dynamicdefault>
34  <encryptatrest>F</encryptatrest>
35  <fieldtype>SELECT</fieldtype>
36  <help>Enter a brief explanation of the kind of information you want entered in this field.</help>
37  <isformula>F</isformula>
38  <ismandatory>F</ismandatory>
39  <label>SAMPLE TRANSACTIONCOLUMN</label>
40  <linktext></linktext>
41  <maxlength></maxlength>
42  <maxvalue></maxvalue>
43  <minvalue></minvalue>
44  <onparentdelete>NO_ACTION</onparentdelete>
45  <searchlevel>2</searchlevel>
46  <selectrecordtype>-112</selectrecordtype>
47  <showhierarchy>T</showhierarchy>
48  <sourcefilterby></sourcefilterby>
49  <sourcefrom></sourcefrom>
50  <sourcelist></sourcelist>

```

```

51 <storevalue>T</storevalue>
52 <customfieldfilters>
53   <customfieldfilter>
54     <fldcomparefield></fldcomparefield>
55     <fldfilter>STDRECORDACCOUNTACCTNAME</fldfilter>
56     <fldfilterchecked></fldfilterchecked>
57     <fldfiltercomparetype>LIKE</fldfiltercomparetype>
58     <fldfilternotnull>F</fldfilternotnull>
59     <fldfilternull>F</fldfilternull>
60     <fldfiltersel></fldfiltersel>
61     <fldfilterval>GST</fldfilterval>
62   </customfieldfilter>
63 </customfieldfilters>
64 <roleaccesses>
65   <roleaccess>
66     <accesslevel>1</accesslevel>
67     <role>ACCOUNTANT</role>
68     <searchlevel>1</searchlevel>
69   </roleaccess>
70 </roleaccesses>
71 </transactioncolumncustomfield>

```

Custom Transaction Item Option Fields as XML Definitions

You can create XML definitions for custom transaction item option fields as SDF custom objects using SuiteCloud Development Framework (SDF). You can either create an `itemoptioncustomfield` object in a SuiteCloud project or import a custom transaction item option field from a NetSuite account into a SuiteCloud project. For information about creating a custom transaction line field in a NetSuite account, see the following:

- [Creating Custom Transaction Item Options](#)
- [Field Type Descriptions for Custom Fields](#)

Add custom transaction item option fields to the line items of your transaction records to gather information specific to your business needs.

These transaction records include:

- purchase items
- sales items
- opportunities
- web store items
- kits and assemblies

Setting Values on the `itemoptioncustomfield` Object

Provide a readable script ID attribute by using the underscore followed by a meaningful name for the object. NetSuite automatically adds a `custcol` prefix that indicates the type of field, for example, `custcol_mynewfield`.

The following fields are required for `itemoptioncustomfield` objects:

- `label` - Specifies the label for the custom field. The `label` is the text that your users see as the name of the custom field when it is displayed on the record.
- `fieldtype` - Specifies the type of custom field. For more information about the possible values of `fieldtype`, see the help topic [generic_customfield_fieldtype](#).
- `selectrecordtype` (conditional) - Specifies the list or record that contains the items for the custom field. This field is required only when the `fieldtype` value is equal to `SELECT` or `MULTISELECT`.

The following XML definition example is a basic `itemoptioncustomfield` object with the required fields:

```

1 <itemoptioncustomfield scriptid="custcol_mynewfield">
2 ...
3 <fieldtype>SELECT</fieldtype>
4 <label>SAMPLE ITEMOPTION</label>
5 <selectrecordtype>-112</selectrecordtype>
6 ...
7 </itemoptioncustomfield>

```

For more information about possible fields for `itemoptioncustomfield` objects, see the help topic [itemoptioncustomfield](#).

If you include any SDF custom objects, files, or scripts in your fields, you must define the dependencies for them in your SuiteCloud project's manifest. For more information, see [SDF Custom Object Dependencies in SuiteApps](#).

To apply the custom field to particular items, go to the object's item option configuration page. On the **Applies To** subtab, in the **Items** field, hold down the Ctrl key and click all the items you want this field to apply to, and click **Save**.

Here is an example of a custom transaction item option field configuration page that applies to specific items:

To view and edit the item option field, go to a transaction record that has the custom field applied to it and click the **Items** parent subtab. On the **Items** child subtab, click the field under the **Options** column within an entry in the sublist, and then click the **Set Options** icon (🔗) next to the field to open a popup window where you can enter a value for the item option field.

When the custom transaction item option field is included in a record, it can look like the following example:

Embedding Structured Fields Within a `itemoptioncustomfield` Object

You can create custom filters for your custom field by embedding `customfieldfilter` structured fields within the `itemoptioncustomfield` object. You can also customize the default access that a role has to the custom field by embedding `roleaccess` structured fields within the object.

For more information about the possible fields to define for these structured fields, see the following:

- [Custom Field Filters](#)
- [Role Accesses](#)

To view a full XML definition example for an `itemoptioncustomfield` object, see [Example of a `roleaccess` Structured Field](#)

Custom Field Filters

You can add custom field filters to a list/record or multiple select custom field to filter the choices available in that custom field on records and transactions. In SDF, you can use custom field filters by embedding `customfieldfilter` structured fields within a custom field object.

 **Note:** Filtering can apply only when the `fieldtype` value of your custom field object is equal to `SELECT` or `MULTISELECT`. It does not apply to custom lists, meaning if you include `customfieldfilter` structured fields, your `selectrecordtype` value cannot equal a custom list scriptid.

`customfieldfilters` is the parent element and `customfieldfilter` is the child element. You can include multiple `customfieldfilter` child elements within the parent.

The `fldfilter` subfield specifies a standard field on the list/record that you entered for the `selectrecordtype` field. The `fldfilter` subfield is required. For more information about the supported fields and values for `customfieldfilter` structured fields, see the help topic [customfieldfilter](#).

When an object with a `customfieldfilter` structured field is deployed to a target account, the **Sourcing & Filtering** subtab of the custom field record is populated with the field values specified by the structured field. The choices available to users in a record or transaction for that custom field are tailored by the filter settings from the structured field.

Here is an example of a custom item field configuration page in NetSuite with filters set for the field:

Custom Item Field

Save Cancel Reset Change ID Apply to Forms Actions

LABEL *
SAMPLE ITEMCUSTOMFIELD

PUBLISHER ID

APP ID

ID
custitem_sample_itemcust

INTERNAL ID
543

OWNER
Larry Nelson

DESCRIPTION

MATRIX OPTION

TYPE
List/Record

LIST/RECORD
Account

STORE VALUE ☒ USE ENCRYPTED FORMAT ☐

SHOW IN LIST ☐

GLOBAL SEARCH ☐

RECORD IS PARENT ☐

INACTIVE ☐

Applies To Display Validation & Defaulting **Sourcing & Filtering** Access Translation History

SOURCE LIST

SOURCE FROM

SOURCE FILTER BY

FILTER USING*	IS CHECKED	COMPARE TYPE	COMPARE VALUE TO	VALUE IS	IS NOT EMPTY	IS EMPTY	COMPARE TO FIELD
Name		contains	paid				
		equal		<Type & tab for single value>			

Add Cancel Insert Remove



Important: If you change the field type or the list/record for a field that is used for sourcing or filtering on other custom fields, all sourcing and filtering based on the field is removed.

Additionally, you can define custom field filters in NetSuite. For more information, see the help topic [Setting Filtering Criteria](#).

Example of a customfieldfilter Structured Field

The following XML definition example is a customfieldfilter structured field embedded within an itemcustomfield object whose fieldtype equals SELECT. After this object is deployed to the target account, a custom item field labeled SAMPLE ITEMCUSTOMFIELD appears on the **Main** tab of all Inventory item records. The fldfiltercomparetype equals LIKE and fldfilterval equals paid. Therefore, the field's dropdown list is filtered to display only accounts that contain "paid" in the account name.

```

1 <itemcustomfield scriptid="custitem_sample_itemcust">
2   <appliestoinventory>T</appliestoinventory>
3   <fieldtype>SELECT</fieldtype>
4   <label>SAMPLE ITEMCUSTOMFIELD</label>
5   <selectrecordtype>-112</selectrecordtype>
6   ...
7   <customfieldfilters>
8     <customfieldfilter>
9       <fldcomparefield></fldcomparefield>
10      <fldfilter>STDRECORDACCOUNTACCTNAME</fldfilter>
11      <fldfilterchecked></fldfilterchecked>
12      <fldfiltercomparetype>LIKE</fldfiltercomparetype>
13      <fldfilternotnull>F</fldfilternotnull>
14      <fldfilternull>F</fldfilternull>
15      <fldfiltersel></fldfiltersel>
16      <fldfilterval>paid</fldfilterval>
17    </customfieldfilter>
18  </customfieldfilters>
19  ...
20 </itemcustomfield>

```

Role Accesses

In SDF, you can control how roles access the information in custom fields by embedding roleaccess structured fields within a custom field object. The accesses that you specify determines how the custom

field can be accessed on the record or through search results and reports. This helps you to maintain the security of your business information.

roleaccesses is the parent element and roleaccess is the child element. You can include multiple roleaccess child elements within the parent.

After an object with a roleaccess structured field is deployed to a NetSuite account, the **Role** section under the **Access** subtab of the custom field record is populated with the access level values of the fields specified by the structured field.

Here is an example of a custom item field configuration page in NetSuite with access levels set for a role:

Custom Item Field

Save Cancel Reset Change ID Apply to Forms Actions

LABEL *
SAMPLE ITEMCUSTOMFIELD

PUBLISHER ID

APP ID

ID
custitem_sample_itemcust

INTERNAL ID
543

OWNER
Larry Nelson

DESCRIPTION

MATRIX OPTION

TYPE
List/Record

LIST/RECORD
Account

STORE VALUE ☒ USE ENCRYPTED FORMAT ☐

SHOW IN LIST ☐

GLOBAL SEARCH ☐

RECORD IS PARENT ☐

INACTIVE ☐

Applies To Display Validation & Defaulting Sourcing & Filtering **Access** Translation History

DEFAULT ACCESS LEVEL
Edit

DEFAULT LEVEL FOR SEARCH/REPORTING
Edit

Role *	ACCESS LEVEL *	LEVEL FOR SEARCH/REPORTING *
Accountant	View	Run

Add Cancel Insert Remove

The following subfields are required for roleaccess structured fields:

- role - Specifies the role.
- accesslevel - Specifies the access level granted to the role for the custom field object.
Possible access level values:
 - 0 - None
 - 1 - View
 - 2 - Edit
- searchlevel - Specifies the access level granted to the role for the custom field object through search results and reports.
Possible access level values:
 - 0 - None
 - 1 - Run
 - 2 - Edit

For more information about the supported fields and values for roleaccess, see the help topic [roleaccess](#).

Additionally, you can define role accesses to custom fields from a NetSuite account. For more information, see the help topic [Restricting Access to Custom Fields](#).

Example of a roleaccess Structured Field

The following XML definition example is a roleaccess structured field embedded within a itemcustomfield object. In this example, the accesslevel for the Accountant role equals 1 and the searchlevel equals 1. Therefore, the role's access to the field is set to **View** and the access level through searching and reporting is set to **Run**.

```

1 <itemcustomfield scriptid="custitem_sample_itemcust">
2   <appliestoinventory>T</appliestoinventory>
3   <fieldtype>SELECT</fieldtype>
4   <label>SAMPLE ITEMCUSTOMFIELD</label>
5   <selectrecordtype>-112</selectrecordtype>
6   ...
7   <roleaccesses>
8     <roleaccess>
9       <accesslevel>1</accesslevel>
10      <role>ACCOUNTANT</role>
11      <searchlevel>1</searchlevel>
12    </roleaccess>
13  </roleaccesses>
14  ...
15 </itemcustomfield>

```

Example of an itemoptioncustomfield Object

The following XML definition example is for a itemoptioncustomfield object. After this object is deployed to the target account, a custom transaction item option field labeled SAMPLE ITEMOPTION appears under the **Options** column within entries of sublists on **Items** child subtabs for all items that the object is applied to. In this example, the object applies to sale and web store transaction records. The fieldtype equals SELECT. Therefore, a value for selectrecordtype is required.

```

1 <itemoptioncustomfield scriptid="custcol_sample_itemoption">
2   <accesslevel>2</accesslevel>
3   <applyformatting>F</applyformatting>
4   <colallitems>F</colallitems>
5   <colkititem>F</colkititem>
6   <colopportunity>F</colopportunity>
7   <coloptionlabel></coloptionlabel>
8   <colpurchase>F</colpurchase>
9   <colsale>T</colsale>
10  <colstore>T</colstore>
11  <colstorehidden>F</colstorehidden>
12  <coltransferorder>F</coltransferorder>
13  <defaultchecked>F</defaultchecked>
14  <defaultselection></defaultselection>
15  <defaultvalue></defaultvalue>
16  <description>Enter a description of this field.</description>
17  <displayheight></displayheight>
18  <displaytype>NORMAL</displaytype>
19  <displaywidth></displaywidth>
20  <dynamicdefault></dynamicdefault>
21  <fieldtype>SELECT</fieldtype>
22  <help>Enter a brief explanation of the kind of information you want entered in this field.</help>
23  <includechilditems>F</includechilditems>
24  <isformula>F</isformula>
25  <ismandatory>F</ismandatory>
26  <label>SAMPLE ITEMOPTION</label>
27  <linktext></linktext>
28  <maxlength></maxlength>
29  <maxvalue></maxvalue>
30  <minvalue></minvalue>
31  <onparentdelete>NO_ACTION</onparentdelete>
32  <searchcomparefield></searchcomparefield>
33  <searchdefault></searchdefault>
34  <searchlevel>2</searchlevel>
35  <selectrecordtype>-112</selectrecordtype>
36  <showhierarchy>T</showhierarchy>

```

```

37 <sourcefilterby></sourcefilterby>
38 <sourcefrom></sourcefrom>
39 <sourcelist></sourcelist>
40 <storevalue>T</storevalue>
41 <customfieldfilters>
42   <customfieldfilter>
43     <fldcomparefield></fldcomparefield>
44     <fldfilter>STDRECORDACCOUNTACCTNAME</fldfilter>
45     <fldfilterchecked></fldfilterchecked>
46     <fldfiltercomparetype>LIKE</fldfiltercomparetype>
47     <fldfilternotnull>F</fldfilternotnull>
48     <fldfilternull>F</fldfilternull>
49     <fldfiltersel></fldfiltersel>
50     <fldfilterval>paid</fldfilterval>
51   </customfieldfilter>
52 </customfieldfilters>
53 <roleaccesses>
54   <roleaccess>
55     <accesslevel>1</accesslevel>
56     <role>SALES_MANAGER</role>
57     <searchlevel>1</searchlevel>
58   </roleaccess>
59 </roleaccesses>
60 </itemoptioncustomfield>

```

Custom Record Types as XML Definitions

Custom record types are entry forms that collect information specific to the needs of your business.

You can create and manage custom record types using SuiteCloud Development Framework (SDF). You can model a custom record type in a SuiteCloud project and deploy the project to a NetSuite account. You can either create new custom record types in your project or import existing custom record types from a NetSuite account. For more information about working with custom record types in a NetSuite account, see the help topic [Creating Custom Record Types](#).

Custom record type objects, defined by the **customrecordtype** object, are comprised of several structures, including the following:

- [Custom Record Type Object Custom Fields](#)
- [Custom Record Type Object Subtabs](#)
- [Custom Record Type Object Sublists](#)
- [Custom Record Type Object Permissions](#)
- [Custom Record Type Object Links](#)
- [Custom Record Instances](#)

Custom record types in SDF do not support the following subtabs displayed in NetSuite:

- Managers
- Translation
- Online Forms

The following example shows a basic custom record type object named "Sample Custom Record Type" with the script ID named `customrecord_sample` where all standard options are set to their default values:

```

1 <customrecordtype scriptid="customrecord_sample">
2   <accesstype>CUSTRECORDENTRYPERM</accesstype>
3   <allowattachments>T</allowattachments>
4   <allowinlinedeleting>F</allowinlinedeleting>
5   <allowinlinedetaching>T</allowinlinedetaching>
6   <allowinlineediting>F</allowinlineediting>
7   <allowmobileaccess>F</allowmobileaccess>
8   <allownumberingoverride>F</allownumberingoverride>

```

```

9      <allowquickadd>T</allowquickadd>
10     <allowquicksearch>F</allowquicksearch>
11     <allowuiaccess>T</allowuiaccess>
12     <description></description>
13     <enableidle>T</enableidle>
14     <enablekeywords>T</enablekeywords>
15     <enablemailmerge>F</enablemailmerge>
16     <enablenumbering>F</enablenumbering>
17     <enableoptimisticlocking>T</enableoptimisticlocking>
18     <enablesystemnotes>T</enablesystemnotes>
19     <hierarchical>F</hierarchical>
20     <icon></icon>
21     <iconbuiltin>T</iconbuiltin>
22     <iconindex>30</iconindex>
23     <includeinsearchmenu>T</includeinsearchmenu>
24     <incluename>T</incluename>
25     <isinactive>F</isinactive>
26     <isordered>F</isordered>
27     <numberinginit></numberinginit>
28     <numberingmindigits></numberingmindigits>
29     <numberingprefix></numberingprefix>
30     <numberingsuffix></numberingsuffix>
31     <recordname>Custom Record Type Sample</recordname>
32     <showcreationdate>F</showcreationdate>
33     <showcreationdateonlist>F</showcreationdateonlist>
34     <showid>F</showid>
35     <showlastmodified>F</showlastmodified>
36     <showlastmodifiedonlist>F</showlastmodifiedonlist>
37     <shownotes>T</shownotes>
38     <showowner>F</showowner>
39     <showownerallowchange>F</showownerallowchange>
40     <showowneronlist>F</showowneronlist>
41 </customrecordtype>

```

For more information about the standard options and what they do, see the following:

- [customrecordtype](#)
- [Entering Name and Display Settings](#)
- [Specifying Permission and UI Settings](#)
- [Configuring File and Child Record Settings](#)
- [Defining Search and Edit Settings](#)

Custom Record Type Object Icon Management

You can choose an icon to represent a custom record using the `icon`, `iconbuiltin`, and `iconindex` elements in a `customrecordtype` object.

Icons are displayed in the following places when they are specified in the SDF custom object type:

- The Create New menu on records
- The New column on list pages and list portlets
- The Recent Records menu
- The Recent Records portlet
- QuickViews

For more information about using a:

- Prebuilt icon, set the `iconbuiltin` value to `T`, and set the `iconindex` value to one of the 70 prebuilt icons in NetSuite. For information about possible values, see the help topic [generic_custom_record_icon](#).
- Custom icon, set the `iconbuiltin` value to `F`, and set the `icon` value to a File Cabinet reference. For information about how to specify file references, see [Developing SDF Custom Objects](#).

The following example shows a custom record type object that uses a custom icon located in **File Cabinet/Images/111.jpg**.

```

1 <customrecordtype scriptid="customrecord_sample">
2   ...
3   <icon>[/Images/111.jpg]</icon>
4   <iconbuiltin>F</iconbuiltin>
5   <iconindex></iconindex>
6   ...
7 </customrecordtype>

```

For more information about icon creation guidelines, see the help topic [Creating Icons for Custom Records](#).

Numbering Custom Record Type Objects

You can have numbers automatically assigned to your custom record instances for easier tracking and designation.



Warning: After you enable auto-numbering, numbers cannot be removed from records. Disabling auto-numbering in the future only prevents future records from being numbered.

Numbering custom record type object instances are controlled with the following elements:

- **enablenumbering** — Set this value to T to enable numbering.
- **numberingprefix** — Set this value to any numbers or letters that you want added before each automatically generated number.
- **numberingsuffix** — Set this value to any numbers or letters that you want added after each automatically generated number.
- **numberingmindigits** — Set this value to the number of digits you want as the minimum for automatically generated numbers, ranging from 0-20. For example, enter 5 to have the first number added as 00001.
- **numberinginit** — Set this value to the first number that you want to use to begin automatic numbering.
- **allownumberingoverride** — Set this value to enable the ability to make the number editable when you edit the custom record.

The following example shows a custom record type object that numbering, where the first instance is assigned `rec_num_00001_sandbox`, followed by `rec_num_00002_sandbox`, and so forth:

```

1 <customrecordtype scriptid="customrecord_sample">
2   ...
3   <enablenumbering>T</enablenumbering>
4   <numberingprefix>rec_num_</numberingprefix>
5   <numberingsuffix>_sandbox</numberingsuffix>
6   <numberingmindigits>5</numberingmindigits>
7   <numberinginit>1</numberinginit>
8   <allownumberingoverride>F</allownumberingoverride>
9   ...
10 </customrecordtype>

```

Custom Record Type Object Custom Fields

In SuiteCloud Development Framework (SDF), custom record custom fields are defined in a custom record type object.

Custom fields are grouped within the `customrecordcustomfields` structure. In this structure, you can create as many custom fields as needed by specifying a `customrecordcustomfield` structure for every field. The order that you define each `customrecordcustomfield` determines where they appear on the custom record in the NetSuite UI. The first `customrecordcustomfield` in the object appears toward the top of the custom record while the last `customrecordcustomfield` appears toward the bottom.

Each `customrecordcustomfield` structure contains elements that define the field. All custom record type field definitions and subtabs are supported as elements in SDF. For more information about the elements that you can specify for a custom record custom field, see the following:

- [customrecordcustomfields](#)
- [Adding Fields to Custom Record Types](#)
- [Creating a Custom Field](#)

The following example shows a custom free-form text field with access limited to the Bookkeeper role:

```

1 <customrecordtype scriptid="customrecord_sample">
2   ...
3   <customrecordcustomfields>
4     <customrecordcustomfield scriptid="custrecord_samplefield">
5       <accesslevel>2</accesslevel>
6       <allowquickadd>F</allowquickadd>
7       <applyformatting>F</applyformatting>
8       <checkspelling>T</checkspelling>
9       <defaultchecked>F</defaultchecked>
10      <defaultselection></defaultselection>
11      <defaultvalue></defaultvalue>
12      <description></description>
13      <displayheight></displayheight>
14      <displaytype>NORMAL</displaytype>
15      <displaywidth></displaywidth>
16      <dynamicdefault></dynamicdefault>
17      <encryptatrest>F</encryptatrest>
18      <fieldtype>TEXT</fieldtype>
19      <globalsearch>F</globalsearch>
20      <help></help>
21      <isformula>F</isformula>
22      <ismandatory>F</ismandatory>
23      <isparent>F</isparent>
24      <label>My Custom Field</label>
25      <linktext></linktext>
26      <maxlength></maxlength>
27      <maxvalue></maxvalue>
28      <minvalue></minvalue>
29      <onparentdelete></onparentdelete>
30      <parentsubtab></parentsubtab>
31      <rolerestrict>F</rolerestrict>
32      <searchcomparefield></searchcomparefield>
33      <searchdefault></searchdefault>
34      <searchlevel>2</searchlevel>
35      <selectrecordtype></selectrecordtype>
36      <showinlist>F</showinlist>
37      <sourcefilterby></sourcefilterby>
38      <sourcefrom></sourcefrom>
39      <sourcelist></sourcelist>
40      <storevalue>T</storevalue>
41      <subtab></subtab>
42      <roleaccesses>
43        <roleaccess>
44          <accesslevel>2</accesslevel>
45          <role>BOOKKEEPER</role>
46          <searchlevel>2</searchlevel>
47        </roleaccess>
48      </roleaccesses>
49    </customrecordcustomfield>
50  </customrecordcustomfields>
51  ...
52 </customrecordtype>

```


Note: To make a custom field available for data entry in the Quick Add portlet, set the `allowquickadd` value to T. By default, this value is set to F. For more information, see the help topic [Quick Add Portlet](#).

Applying Role-Based Access Restrictions

You can apply role-based restrictions on custom fields for particular categories such as department, class, location, employee, or subsidiary. For example, if you restrict access on an Employee SELECT field, the employee-based restriction set on the Sales Manager role record is applied to the custom record so that sales managers can see the custom record when they or their subordinates are set to the value of the custom field.

To apply role-based access restrictions on a custom record type, set the `rolerestrict` value to T, the `fieldtype` value to SELECT, and the `selectrecordtype` value to one of the following values:

- **-4** for Employee
- **-101** for Class
- **-102** for Department
- **-103** for Location
- **-117** for Subsidiary

For more information about:

- Restrictions, see the help topic [Customizing or Creating NetSuite Roles](#).
- Possible standard `selectrecordtype` values, see the help topic [generic_customfield_selectrecordtype](#).

Parent-Child Record Relationships

You can establish parent-child relationships between records when certain conditions are met in a custom field.

Set the `fieldtype` value to SELECT so that the `isparent` and `parentsubtab` elements becomes available for use. When the `isparent` value is set to T, you can specify a parent subtab by setting the `parentsubtab` value to a custom or standard subtab. For more information about possible standard subtab values, see the help topic [generic_tab_parent](#).

The following example shows a custom record custom field that is set to appear under the Message subtab of a Call record:

```

1 <customrecordtype scriptid="customrecord_sample">
2   ...
3   <customrecordcustomfields>
4     <customrecordcustomfield scriptid="custrecord_samplefield">
5       ...
6       <label>My Custom Field</label>
7       <fieldtype>SELECT</fieldtype>
8       <isparent>T</isparent>
9       <parentsubtab>CRMMESSAGE</parentsubtab>
10      <selectrecordtype>-22</selectrecordtype>
11      ...
12    </customrecordcustomfield>
13  </customrecordcustomfields>
14  ...
15 </customrecordtype>

```

Custom Record Type Object Subtabs

In SuiteCloud Development Framework (SDF), custom record subtabs are defined in a custom record type object.

Custom record subtabs are grouped within the subtabs structure. In this structure, you can create as many custom subtabs as needed by specifying a subtab structure for every subtab. The order that you define each subtab determines where they appear on the custom record in the NetSuite UI. The first subtab in the object appears toward the top of the custom record while the last subtab appears toward the bottom.

Each subtab structure contains elements that define it. Translations are not supported by SDF. For more information about the elements that you can specify for a custom record subtab, see the following:

- [subtab](#)
- [Adding Subtabs to a Custom Record](#)

The following example shows two custom subtabs where the second subtab is a child of the first subtab:

```

1 <customrecordtype scriptid="customrecord_sample">
2   ...
3   <subtabs>
4     <subtab scriptid="tab_92_1013519_539">
5       <tabparent></tabparent>
6       <tabtitle>My Main Subtab</tabtitle>
7     </subtab>
8     <subtab scriptid="tab_92_1013519_987">
9       <tabparent>[scriptid=customrecord_sample.tab_92_1013519_539]</tabparent>
10      <tabtitle>My Embedded Subtab</tabtitle>
11    </subtab>
12  </subtabs>
13  ...
14 </customrecordtype>

```

For more information about how to specify SDF custom object references, such as the one used in the tabparent element in the example, see [Developing SDF Custom Objects](#).

Custom Record Type Object Sublists

In SuiteCloud Development Framework (SDF), custom record sublists are defined in a custom record type object. Custom record sublists display saved search results on the custom record type. You must create a saved search that obtains your preferred results before you can display them on the custom record.

Custom record sublists are grouped within the recordsublists structure. In this structure, you can create as many custom sublists as needed by specifying a recordsublist structure for every sublist. The order that you define each sublist determines where they appear on the custom record in the NetSuite UI. The first sublist in the object appears toward the top of the custom record while the last sublist appears toward the bottom.

Each sublist structure contains elements that define it. For more information about the elements that you can specify for a custom record sublist, see the help topic [recordsublist](#).

The following example shows a custom sublist labelled Documents that displays the results of a saved search is named customsearch_esc_my_documents. The customrecord_sample.custrecord_samplefield custom field is the field by which the sublist is filtered:

```

1 <customrecordtype scriptid="customrecord_sample">
2   ...
3   <recordsublists>
4     <recordsublist scriptid="sublist_1_4775004_219">
5       <recorddescr>Documents</recorddescr>
6       <recordfield>[scriptid=customrecord_sample.custrecord_samplefield]</recordfield>
7       <recordsearch>[scriptid=customsearch_esc_my_documents]</recordsearch>
8       <recordtab></recordtab>
9     </recordsublist>
10  </recordsublists>
11  ...
12 </customrecordtype>

```

Custom Record Type Object Permissions

You can provide roles with access to a custom record type in SuiteCloud Development Framework (SDF) using the permissions structure in the customrecordtype SDF custom object. In this structure, you can create as many custom permissions as needed by specifying a permission structure for every permission. For more information about custom record type settings, see the help topic [Setting Up a Permissions List for a Custom Record Type](#).



Important: If you created a custom record object in version 18.1 that sets a permission on a custom role, you may receive a validation error after upgrading to the current version. To resolve this error, ensure that the referenced role contains a permission to the custom record object. For more information, see [Custom Roles as XML Definitions](#).

Each permission element requires the following information:

Tag name	Possible values	Description
permittedrole	Any standard role ID or custom role reference.	The role that can access the custom record.
permittedlevel	One of the following values: - NONE - VIEW - CREATE - EDIT - FULL	The level of access to the custom record that users of the specific role has.
restriction	One of the following values: - VIEWANDEDIT, which restricts users to viewing or editing only the records of this type that they or their subordinates created. - EDIT, which restricts users to editing only the records of this type that they or their subordinates created. They can view all records of this type.	The level of restrictions on the custom record that users of the specific role has.



Important: If a SuiteCloud project contains an SDF custom object and a custom role that reference each other, the levels of permission and restriction must match in both objects. For example, if a custom record type permits the custom role with the VIEW permission level, that custom role must set the custom record type permission level to VIEW. Projects that contain a level mismatch cannot be deployed to the NetSuite account.

In the following example, the Bookkeeper role is given view permissions to the sample custom record type:

```

1 <customrecordtype scriptid="customrecord_sample">
2   ....
3   <permissions>
4     <permission>
5       <permittedrole>BOOKKEEPER</permittedrole>
6       <permittedlevel>VIEW</permittedlevel>
7       <restriction>EDIT</restriction>
8     </permission>
9   </permissions>
10   ...
11 </customrecordtype>

```

In the following example, a custom role is given view permissions to the sample custom record type:

```

1 <customrecordtype scriptid="customrecord_sample">
2   ...
3   <permissions>
4     <permission>
5       <permittedrole>[scriptid=customrole_sample]</permittedrole>
6       <permittedlevel>VIEW</permittedlevel>
7       <restriction>EDIT</restriction>
8     </permission>
9   </permissions>
10  ...
11 </customrecordtype>

```

For information about the available fields, see the help topic [role](#).

Consider the following information for SDF SuiteApps and SuiteApp projects:

- For role permissions that exist in the target account but not in the permission field group of the customrecordtype SDF custom objects in the SDF SuiteApp (or SuiteApp project), the permissions in the target account will not be changed or removed.
- For role permissions that exist in both the permission field group of the customrecordtype SDF custom objects in the SDF SuiteApp or SuiteApp project and the target account, the permissions in the target account will be changed to the XML values for the permissions in the SDF SuiteApp or SuiteApp project.

Consider the following information for account customization projects:

- For role permissions that exist in the target account but not in the permission field group of the customrecordtype SDF custom objects in the account customization project, the permissions in the target account will not be changed or removed.
- For role permissions that exist in both the permission field group of the customrecordtype SDF custom objects in the account customization project and the target account, the permissions in the target account will be changed to the XML values for the permissions from the account customization project.

Custom Record Type Object Links

In SuiteCloud Development Framework (SDF), custom record type object links are defined in a custom record type object.

Custom record type links are grouped within the links structure. In this structure, you can create as many custom links as needed by specifying a link structure for every link. The order that you define each link determines where they appear on the link category in the NetSuite UI.

Each link structure contains elements that define it. The linkcategory element represents a combination of the Center and Section columns in the NetSuite UI. For more information about the elements that you can specify for a link, see the help topic [link](#).

The following example shows three types of links that appear in the Reports section of the Classic Center:

```

1 <customrecordtype scriptid="customrecord_sample">
2   ...
3   <links>
4     <link>
5       <linkcategory>BASICREPORTSCUSTOM</linkcategory>
6       <linklabel>Edit Samples</linklabel>
7       <linktasktype>EDIT</linktasktype>
8     </link>
9     <link>
10      <linkcategory>BASICREPORTSCUSTOM</linkcategory>
11      <linklabel>List Samples</linklabel>
12      <linktasktype>LIST</linktasktype>
13    </link>

```

```

14     <link>
15       <linkcategory>BASICREPORTSCUSTOM</linkcategory>
16       <linklabel>Search Samples</linklabel>
17       <linktasktype>SRCH</linktasktype>
18     </link>
19   </links>
20   ...
21 </customrecordtype>

```

Custom Record Instances

In SuiteCloud Development Framework (SDF), custom record instances are defined in the custom record type object. Instances of custom records, including the custom record object, can be either imported into a SuiteCloud project or downloaded from a NetSuite account. For more information, see [Downloading the XML Definition of an Object from a NetSuite Account](#).

Custom record instances are grouped within the instances structure. In this structure, each instance is recorded in its own instance structure. An instance contains standard elements, such as name and isinactive, but also contains custom elements that are named after the script IDs of the custom fields in the custom record. For more information about instance elements, see the help topic [instance](#).

Each instance has a parent element that you can use when the hierarchical element of the custom record type object is set to T.

The following example shows a single instance that was imported from a NetSuite account. The custom record contains two custom fields:

```

1  <customrecordtype scriptid="customrecord_sample">
2    ...
3    <hierarchical>T</hierarchical>
4    ...
5    <customrecordcustomfields>
6      <customrecordcustomfield scriptid="custrecord_employeename">
7        ...
8        <label>My Favorite Employee</label>
9        <fieldtype>SELECT</fieldtype>
10       <selectrecordtype>-4</selectrecordtype>
11     </customrecordcustomfield>
12     <customrecordcustomfield scriptid="custrecord_sampletextfield">
13       ...
14       <label>Sample Text Field</label>
15       <fieldtype>TEXT</fieldtype>
16     </customrecordcustomfield>
17   </customrecordcustomfields>
18   ...
19   <instances>
20     <instance scriptid="_this_instance">
21       <custrecord_employeename>[ACCOUNT_SPECIFIC_VALUE]</custrecord_employeename>
22       <custrecord_sampletextfield>Here is some sample text.</custrecord_sampletextfield>
23       <name>My first instance of a custom record.</name>
24       <isinactive>F</isinactive>
25     </instance>
26     <instance scriptid="_that_instance">
27       <custrecord_employeename>[ACCOUNT_SPECIFIC_VALUE]</custrecord_employeename>
28       <custrecord_sampletextfield>Here is some more sample text.</custrecord_sampletextfield>
29       <name>My second instance of a custom record.</name>
30       <isinactive>F</isinactive>
31       <parent>[scriptid=customrecord_sample._this_instance]</parent>
32     </instance>
33   </instances>
34   ...
35 </customrecordtype>

```

For more information about account-specific values, see [Account-Specific Values Validation in SuiteCloud Projects](#).

Custom Record Instances of Custom Segment Values

When a custom segment is applied to a custom record type, the segment field can be specified on each custom record instance. This field cannot be configured in the `customrecordcustomfieldsubrecord` structure of the custom record object, but its value can be specified in the custom record instance.

The following is an example of a custom record type instance that references a custom segment instance. The example assumes that the custom segment has a script ID of `cseg_segmentid`, and the segment is applied to the `customrecord_sample` custom record type. Additionally, the associated custom record type, `customrecord_cseg_segmentid`, should contain an instance with a script ID of `some_customsegment_instance_id`.

```

1 <customrecordtype scriptid="customrecord_sample">
2   ...
3   <instances>
4     <instance scriptid="_value1">
5       <isinactive>F</isinactive>
6       <name>My first instance</name>
7       <cseg_segmentid>[scriptid=customrecord_cseg_segmentid.some_customsegment_instance_id]</cseg_segmentid>
8     </instance>
9   </instances>
10  ...
11 </customrecordtype>

```

For more information about custom segment objects, their associated custom record type objects, and their instances, see [Custom Segments as XML Definitions](#).

Custom Segments as XML Definitions

Use custom segments to create custom classification fields for your business, such as class, department, and location. For more information about the benefits of custom segments, see the help topic [Benefits of Custom Segments](#).

You can create and manage custom segments using SuiteCloud Development Framework (SDF). You can model custom segment types and their associated custom record types in a SuiteCloud project, and deploy the project to a target NetSuite account. You can either create new custom segments and associated custom record types in your project or import existing types from a NetSuite account. For more information about working with custom segments in a NetSuite account, see the following:

- [Custom Segments](#)
- [Using the Script ID to Access Custom Segment Body, Line, and Filter By Fields](#)
- [Custom Segments in Record Searches](#)
- [Customizing a Report by Using Custom Segments](#)
- [Adding a Custom Segment to a Bundle](#)
- [Using Custom Segments in Workflows](#)
- [SuiteScript and Custom Segments](#)
- [SOAP Web Services and Custom Segments](#)
- [CSV Import and Custom Segments](#)

Custom segments have limitations, such as the transaction types that they support, and varied behavior, depending on how they are configured. In addition to these limitations, SDF only supports custom segments that use unified IDs, where the custom segment script ID is used for field IDs. For more information, see the following:

- [Transaction Types Supported by Custom Segments](#)
- [Settings that Affect Where Custom Segments are Applied](#)
- [Custom Segment Types](#)

The following sub-sections illustrate how to model custom segment objects and their associated custom record types objects in SDF by using a sample scenario. In this scenario, a business saves records of donation campaigns, campaign donors, and the transactions between the two. There are two custom records — one to represent campaigns for raising donations, and one to represent donors of a campaign. In addition, a donation is represented as a custom transaction type, and the donor field on that transaction type is represented as a transaction body custom field. A custom segment is used to represent various geographical regions, which is applied to both campaigns and donors. Another custom segment is used to represent various communication mediums, which is applied to campaigns. The campaign donor scenario code samples are located in the following sub-sections:

- [Campaign Donor Scenario Sample](#)
- [Custom Segment Object Structure](#)
- [Associated Custom Record Type Object Structure](#)

Campaign Donor Scenario Sample

The code samples in this section set up the campaign donor sample scenario. You can create the code samples in your SuiteCloud project to see how they all work together.

The following custom record type object represents the campaign custom record, `customrecord_campaign`. It is saved as `customrecord_campaign.xml` in the Objects folder of the SuiteCloud project:

```

1 <customrecordtype scriptid="customrecord_campaign">
2   <access><type>CUSTRECORDENTRYPERM</type></access>
3   <allowattachments>T</allowattachments>
4   <allowinlinedeleting>F</allowinlinedeleting>
5   <allowinlinedetaching>T</allowinlinedetaching>
6   <allowinlineediting>F</allowinlineediting>
7   <allowmobileaccess>F</allowmobileaccess>
8   <allownumberingoverride>F</allownumberingoverride>
9   <allowquickadd>T</allowquickadd>
10  <allowquicksearch>F</allowquicksearch>
11  <allowuiaccess>T</allowuiaccess>
12  <enable>T</enable>
13  <enablekeywords>T</enablekeywords>
14  <enablemailmerge>F</enablemailmerge>
15  <enablenumbering>F</enablenumbering>
16  <enableoptimisticlocking>T</enableoptimisticlocking>
17  <enablesystemnotes>T</enablesystemnotes>
18  <hierarchical>F</hierarchical>
19  <iconbuiltin>T</iconbuiltin>
20  <includeinsearchmenu>T</includeinsearchmenu>
21  <includename>T</includename>
22  <isinactive>F</isinactive>
23  <isordered>F</isordered>
24  <recordname>Campaign</recordname>
25  <showcreationdate>F</showcreationdate>
26  <showcreationdateonlist>F</showcreationdateonlist>
27  <showid>F</showid>
28  <showlastmodified>F</showlastmodified>
29  <showlastmodifiedonlist>F</showlastmodifiedonlist>
30  <shownotes>T</shownotes>
31  <showowner>F</showowner>
32  <showownerallowchange>F</showownerallowchange>
33  <showowneronlist>F</showowneronlist>
34  <customrecordcustomfields>
35    <customrecordcustomfield scriptid="custrecord_description">
36      <accesslevel>2</accesslevel>
37      <allowquickadd>F</allowquickadd>
38      <applyformatting>F</applyformatting>
39      <checkspelling>F</checkspelling>
40      <defaultchecked>F</defaultchecked>
41      <displaytype>NORMAL</displaytype>
42      <encryptatrest>F</encryptatrest>
43      <fieldtype>TEXT</fieldtype>

```

```

44     <globalsearch>F</globalsearch>
45     <isformula>F</isformula>
46     <ismandatory>F</ismandatory>
47     <isparent>F</isparent>
48     <label>Description</label>
49     <rolerestrict>F</rolerestrict>
50     <searchlevel>2</searchlevel>
51     <showinlist>F</showinlist>
52     <storevalue>T</storevalue>
53   </customrecordcustomfield>
54   <customrecordcustomfield scriptid="custrecord_start_date">
55     <accesslevel>2</accesslevel>
56     <allowquickadd>F</allowquickadd>
57     <applyformatting>F</applyformatting>
58     <checkspelling>F</checkspelling>
59     <defaultchecked>F</defaultchecked>
60     <displaytype>NORMAL</displaytype>
61     <encryptatrest>F</encryptatrest>
62     <fieldtype>DATE</fieldtype>
63     <globalsearch>F</globalsearch>
64     <isformula>F</isformula>
65     <ismandatory>F</ismandatory>
66     <isparent>F</isparent>
67     <label>Date Start</label>
68     <rolerestrict>F</rolerestrict>
69     <searchlevel>2</searchlevel>
70     <showinlist>F</showinlist>
71     <storevalue>T</storevalue>
72   </customrecordcustomfield>
73   <customrecordcustomfield scriptid="custrecord_end_date">
74     <accesslevel>2</accesslevel>
75     <allowquickadd>F</allowquickadd>
76     <applyformatting>F</applyformatting>
77     <checkspelling>F</checkspelling>
78     <defaultchecked>F</defaultchecked>
79     <displaytype>NORMAL</displaytype>
80     <encryptatrest>F</encryptatrest>
81     <fieldtype>DATE</fieldtype>
82     <globalsearch>F</globalsearch>
83     <isformula>F</isformula>
84     <ismandatory>F</ismandatory>
85     <isparent>F</isparent>
86     <label>Date End</label>
87     <rolerestrict>F</rolerestrict>
88     <searchlevel>2</searchlevel>
89     <showinlist>F</showinlist>
90     <storevalue>T</storevalue>
91   </customrecordcustomfield>
92 </customrecordcustomfields>
93 </customrecordtype>

```

The following image shows how the customrecord_campaign object appears in NetSuite after it is deployed to the account. This object only creates the name field and the three custom fields on the left. The region and campaign medium fields are custom segments that are created from other SDF custom objects:

Campaign List Search Customize

Save Cancel Reset

NAME *

☐ INACTIVE

DESCRIPTION

DATE START

DATE END

REGION
Midwest

CAMPAIGN MEDIUM

- New -
Twitter
Events
Facebook

The following custom record type object represents the donor custom record, `customrecord_donor`. It is saved as `customrecord_donor.xml` in the Objects folder of the SuiteCloud project:

```

1 <customrecordtype scriptid="customrecord_donor">
2   <accesstype>CUSTRECORDENTRYPERM</accesstype>
3   <allowattachments>T</allowattachments>
4   <allowinlinedeleting>F</allowinlinedeleting>
5   <allowinlinedetaching>T</allowinlinedetaching>
6   <allowinlineediting>F</allowinlineediting>
7   <allowmobileaccess>F</allowmobileaccess>
8   <allownumberingoverride>F</allownumberingoverride>
9   <allowquickadd>T</allowquickadd>
10  <allowquicksearch>F</allowquicksearch>
11  <allowuiaccess>T</allowuiaccess>
12  <enableidle>T</enableidle>
13  <enablekeywords>T</enablekeywords>
14  <enablemailmerge>F</enablemailmerge>
15  <enablenumbering>F</enablenumbering>
16  <enableoptimisticlocking>T</enableoptimisticlocking>
17  <enablesystemnotes>T</enablesystemnotes>
18  <hierarchical>F</hierarchical>
19  <iconbuiltin>T</iconbuiltin>
20  <includeinsearchmenu>T</includeinsearchmenu>
21  <includeinname>T</includeinname>
22  <isinactive>F</isinactive>
23  <isordered>F</isordered>
24  <recordname>Donor</recordname>
25  <showcreationdate>F</showcreationdate>
26  <showcreationdateonlist>F</showcreationdateonlist>
27  <showid>F</showid>
28  <showlastmodified>F</showlastmodified>
29  <showlastmodifiedonlist>F</showlastmodifiedonlist>
30  <shownotes>T</shownotes>
31  <showowner>F</showowner>
32  <showownerallowchange>F</showownerallowchange>
33  <showowneronlist>F</showowneronlist>
34  <customrecordcustomfields>
35    <customrecordcustomfield scriptid="custrecord_first_name">
36      <accesslevel>2</accesslevel>
37      <allowquickadd>F</allowquickadd>
38      <applyformatting>F</applyformatting>
39      <checkspelling>F</checkspelling>
40      <defaultchecked>F</defaultchecked>
41      <displaytype>NORMAL</displaytype>
42      <encryptatrest>F</encryptatrest>
43      <fieldtype>TEXT</fieldtype>
44      <globalsearch>F</globalsearch>
45      <isformula>F</isformula>
46      <ismandatory>F</ismandatory>
47      <isparent>F</isparent>
48      <label>First Name</label>
49      <rolerestrict>F</rolerestrict>
50      <searchlevel>2</searchlevel>
51      <showinlist>F</showinlist>
52      <storevalue>T</storevalue>
53    </customrecordcustomfield>
54    <customrecordcustomfield scriptid="custrecord_last_name">
55      <accesslevel>2</accesslevel>
56      <allowquickadd>F</allowquickadd>
57      <applyformatting>F</applyformatting>
58      <checkspelling>F</checkspelling>
59      <defaultchecked>F</defaultchecked>
60      <displaytype>NORMAL</displaytype>
61      <encryptatrest>F</encryptatrest>
62      <fieldtype>TEXT</fieldtype>
63      <globalsearch>F</globalsearch>
64      <isformula>F</isformula>
65      <ismandatory>F</ismandatory>
66      <isparent>F</isparent>
67      <label>Last Name</label>
68      <rolerestrict>F</rolerestrict>
69      <searchlevel>2</searchlevel>

```

```

70         <showinlist>F</showinlist>
71         <storevalue>T</storevalue>
72     </customrecordcustomfield>
73 </customrecordcustomfields>
74 </customrecordtype>

```

The following image shows how the `customrecord_donor` object appears in NetSuite after it is deployed to the account. The region field is a custom segment that is created from another SDF custom object:

The following transaction type object, `customtransaction_donation`, represents a journal of donation transactions. It is saved as `customtransaction_donation.xml` in the Objects folder of the SuiteCloud project:

```

1 <customtransactiontype scriptid="customtransaction_donation">
2   <isposting>F</isposting>
3   <isvoidable>F</isvoidable>
4   <name>Donations</name>
5   <showstatus>F</showstatus>
6   <subliststyle>JOURNAL</subliststyle>
7   <segments>
8     <classposition>NONE</classposition>
9     <departmentposition>NONE</departmentposition>
10    <locationposition>NONE</locationposition>
11  </segments>
12 </customtransactiontype>

```

The following transaction body custom field object, `custbody_donor_field`, represents a donor field that appears in the custom tab of the donation transaction type. It is saved as `custbody_donor_field.xml` in the Objects folder of the SuiteCloud project:

```

1 <transactionbodycustomfield scriptid="custbody_donor_field">
2   <accesslevel>2</accesslevel>
3   <applyformatting>F</applyformatting>
4   <bodycustomerpayment>F</bodycustomerpayment>
5   <bodycustomtransactions>[scriptid=customtransaction_donation]</bodycustomtransactions>
6   <bodydeposit>F</bodydeposit>
7   <bodyexpensereport>F</bodyexpensereport>
8   <bodyinventoryadjustment>F</bodyinventoryadjustment>
9   <bodyitemfulfillment>F</bodyitemfulfillment>
10  <bodyitemfulfillmentorder>F</bodyitemfulfillmentorder>
11  <bodyitemreceipt>F</bodyitemreceipt>
12  <bodyitemreceiptorder>F</bodyitemreceiptorder>
13  <bodyjournal>F</bodyjournal>
14  <bodyopportunity>F</bodyopportunity>
15  <bodyothertransaction>F</bodyothertransaction>
16  <bodypickingticket>F</bodypickingticket>
17  <bodyprintflag>F</bodyprintflag>
18  <bodyprintpackingslip>F</bodyprintpackingslip>
19  <bodyprintstatement>F</bodyprintstatement>
20  <bodypurchase>F</bodypurchase>
21  <bodysale>F</bodysale>
22  <bodystore>F</bodystore>
23  <bodytransferorder>F</bodytransferorder>
24  <bodyvendorpayment>F</bodyvendorpayment>

```

```

25 <checkspelling>F</checkspelling>
26 <defaultchecked>F</defaultchecked>
27 <displaytype>NORMAL</displaytype>
28 <encryptatrest>F</encryptatrest>
29 <fieldtype>SELECT</fieldtype>
30 <globalsearch>F</globalsearch>
31 <isformula>F</isformula>
32 <ismandatory>F</ismandatory>
33 <isparent>F</isparent>
34 <label>Donor</label>
35 <onparentdelete>NO_ACTION</onparentdelete>
36 <searchlevel>2</searchlevel>
37 <selectrecordtype>[scriptid=customrecord_donor]</selectrecordtype>
38 <showhierarchy>F</showhierarchy>
39 <showinlist>F</showinlist>
40 <storevalue>T</storevalue>
41 </transactionbodycustomfield>

```

The following image shows how the customtransaction_donation object appears in NetSuite after it is deployed to the account with custbody_donor_field. The region field is a custom segment that is created from another SDF custom object:

The screenshot shows the NetSuite 'Donations' form. At the top, there are 'Save', 'Cancel', and 'Reset' buttons. Below the title bar, there are tabs for 'Lines', 'Communication', and 'Custom'. The 'Custom' tab is selected, showing a 'DONOR' dropdown menu. Above this, the 'Primary Information' section includes fields for 'ENTRY NO.', 'To Be Generated', 'OUT OF BALANCE BY', 'DEBIT', 'CREDIT', 'REGION' (set to 'Midwest'), 'DATE' (9/25/2018), 'POSTING PERIOD' (Sep 2018), and 'MEMO'. At the bottom of the form, there are again 'Save', 'Cancel', and 'Reset' buttons.

For more information about the custom segment samples for this scenario, and their associated custom record types, see the following:

- [Custom Segment Object Structure](#)
- [Associated Custom Record Type Object Structure](#)

Custom Segment Object Structure

This section describes how to create and modify a custom segment object in SuiteCloud Development Framework (SDF). The samples in this section are based on the campaign donor scenario. You can create the code samples in your SuiteCloud project to see how they all work together. For more information about setting up the scenario in your SuiteCloud project, see [Campaign Donor Scenario Sample](#).



Important: SDF only supports custom segments that use unified IDs. All custom segments that are created with SDF use unified IDs, and you can only import custom segments from a NetSuite account when they use unified IDs. If you import SDF custom objects that reference legacy custom segments that do not use unified IDs, the resulting XML definitions for those SDF custom objects may contain incorrect custom segment value references. For more information about creating custom segments with unified IDs in NetSuite, see the help topic [Creating a Custom Segment](#).

Custom segments, defined by the customsegment object, are comprised of several fields and structures. In addition to the customsegment object, a customrecordtype object must be created for each custom segment to represent the associated custom record. The associated custom record defines all the possible values of the associated custom segment. Each value is represented by an instance structure. When you import a custom segment into a SuiteCloud project, the associated custom record gets imported with it. For more information about custom record type objects, see [Custom Record Types as XML Definitions](#).



Note: SuiteCloud IDE plug-in for WebStorm automatically creates an associated custom record every time you import a custom segment object.

The following is an example of a SELECT custom segment, cseg_region, that represents a geographical region. In the campaign donor scenario, a region can be applied to a donation campaign, a donor, and a donation transaction. To accomplish this, the segment is applied to the campaign custom record, donor custom record, and donation custom transaction. The file is saved as cseg_region.xml in the Objects folder of the SuiteCloud project. The associated custom record, customrecord_cseg_region, is saved as customrecord_cseg_region.xml in the Objects folder of the SuiteCloud project. For more information about associated custom records and the structure of that file, see [Associated Custom Record Type Object Structure](#).

```

1 <customsegment scriptid="cseg_region">
2   <defaultrecordaccesslevel>VIEW</defaultrecordaccesslevel>
3   <defaultsearchaccesslevel>VIEW</defaultsearchaccesslevel>
4   <defaultselection>[scriptid=customrecord_cseg_region._midwest]</defaultselection>
5   <fieldtype>SELECT</fieldtype>
6   <hasglimpact>F</hasglimpact>
7   <ismandatory>F</ismandatory>
8   <label>Region</label>
9   <recordtype>[scriptid=customrecord_cseg_region]</recordtype>
10  <valuesdisplayorder>SUBLIST</valuesdisplayorder>
11  <segmentapplication>
12    <transactionbody>
13      <applications>
14        <application>
15          <id>[scriptid=customtransaction_donation]</id>
16          <isapplied>T</isapplied>
17        </application>
18      </applications>
19    </transactionbody>
20    <transactionline>
21    </transactionline>
22    <entities>
23    </entities>
24    <crm>
25    </crm>
26    <items>
27      <subtype>BOTH</subtype>
28    </items>
29    <customrecords>
30      <applications>
31        <application>
32          <id>[scriptid=customrecord_campaign]</id>
33          <isapplied>T</isapplied>
34        </application>
35        <application>
36          <id>[scriptid=customrecord_donor]</id>

```

```

37         <isapplied>T</isapplied>
38     </application>
39 </applications>
40 </customrecords>
41 </segmentapplication>
42 <permissions>
43     <permission>
44         <recordaccesslevel>VIEW</recordaccesslevel>
45         <role>HUMAN_RESOURCES_GENERALIST</role>
46         <searchaccesslevel>VIEW</searchaccesslevel>
47         <valuemgmtaccesslevel>FULL</valuemgmtaccesslevel>
48     </permission>
49 </permissions>
50 </customsegment>

```

The following is an example of a MULTISELECT custom segment, `cseg_campaignmedium`, that represents a communication medium, such as e-mail or social media. In the campaign donor scenario, a medium can be applied to a donation campaign, so it is applied to the campaign custom record. The file is saved as `cseg_campaignmedium.xml` in the Objects folder of the SuiteCloud project. The associated custom record, `customrecord_cseg_campaignmedium`, is saved as `customrecord_cseg_campaignmedium.xml` in the Objects folder of the SuiteCloud project. For more information about associated custom records and the structure of that file, see [Associated Custom Record Type Object Structure](#).

```

1  <customsegment scriptid="cseg_campaignmedium">
2      <defaultrecordaccesslevel>VIEW</defaultrecordaccesslevel>
3      <defaultsearchaccesslevel>VIEW</defaultsearchaccesslevel>
4      <description></description>
5      <fieldtype>MULTISELECT</fieldtype>
6      <filteredby>[scriptid=cseg_region]</filteredby>
7      <hasglimpact>F</hasglimpact>
8      <help></help>
9      <ismandatory>F</ismandatory>
10     <label>Campaign Medium</label>
11     <recordtype>[scriptid=customrecord_cseg_campaignmedium]</recordtype>
12     <valuesdisplayorder>SUBLIST</valuesdisplayorder>
13     <segmentapplication>
14         <transactionbody>
15             </transactionbody>
16         <transactionline>
17             </transactionline>
18     <entities>
19         </entities>
20     <crm>
21         </crm>
22     <items>
23         <subtype>BOTH</subtype>
24     </items>
25     <customrecords>
26         <applications>
27             <application>
28                 <id>[scriptid=customrecord_campaign]</id>
29                 <isapplied>T</isapplied>
30             </application>
31         </applications>
32     </customrecords>
33 </segmentapplication>
34 </customsegment>

```

With the exception of the inactive field, custom segment objects support all primary information, account, and details fields displayed in the NetSuite UI. They support the Values, Permissions, Validation and Defaulting, and Application and Sourcing tabs. The following Application and Sourcing subtabs are supported by custom segment objects in SDF as part of the `segmentapplication` structure:

- Transactions are represented by the `transactionbody` structure.
- Transaction Columns are represented by the `transactionline` structure.
- Entities are represented by the `entities` structure.
- CRM are represented by the `crm` structure.

- Items are represented by the `items` structure.
- Custom Record Types are represented by the `customrecords` structure.

Some Application and Sourcing subtabs contain a `sourcelist` element. This element is used for dynamic defaulting, where the segment on one custom record automatically populates with the value of the segment on another custom record. For more information about dynamic defaulting and acceptable `sourcelist` values, see the following:

- [Dynamic Defaulting for Custom Segments](#)
- [segmentapplication](#)

Each supported subtab in a `segmentapplication` contains an `applications` structure. These structures specify whether the custom segment should be applied to a record type in that subtab. For example, the `customrecords` structure in the `cseg_region` example sets the equivalent of the following NetSuite UI settings:

APPLIED	RECORD TYPE	SOURCE LIST
☒	Campaign	
☐	Customer Satisfaction Survey	
☒	Donor	

Note: To modify or override an application setting in an account custom segment using SDF, you must explicitly reference the setting in the XML definition before deploying. For example, if a custom segment in an account is applied to a Campaign record type but you do not want it to apply, you must set the `isapplied` value on the Campaign **application** to F in the SuiteCloud project before deploying. If you delete the Campaign application entry from the object instead and deploy, no changes are made to the application setting in the account.

When you apply a custom segment to a transaction type or column, the segment may have GL impact. To configure GL impact on a custom segment object, the `hasglimpact` element must be set to T or F during the creation of the custom segment. This element cannot be changed after it has been specified. For more information, see the help topic [Configuring GL Impact for a Custom Segment](#).

Important: You cannot delete or uninstall a custom segment object from a SuiteApp project if the custom segment has GL impact and was already deployed to a target NetSuite account.

For more information about the `customsegment` object, see the help topic [customsegment](#).

Associated Custom Record Type Object Structure

This section describes how to create and modify an associated custom record type object in SuiteCloud Development Framework (SDF), which is required when working with custom segments. The samples in this section are based on the campaign donor scenario. You can create the code samples in your SuiteCloud project to see how they all work together. For more information about setting up the scenario in your SuiteCloud project, see [Campaign Donor Scenario Sample](#).

Each custom segment has its own associated custom record type that contains all of its possible values. When you create a custom segment in a project, you must create an associated custom record type along with it. When you import a custom segment into a SuiteCloud project, its associated custom record type is imported along with it.

Each value in the associated custom record type is represented in an instance structure. Instances are hierarchical, meaning that each instance can be parent of another instance. You can reference the script ID of a parent instance with the parent element in the child instance structure. When you deploy the associated custom record type object to a target account, the custom segment values appear in the same order as they are listed in the XML definition. Each associated custom record object supports up to 1000 instances. If your associated custom record contains more than 1000 instances, only the first 1000 are used.



Important: Associated custom record type values cannot be deleted using SDF due to the possibility of a GL impact. For example, if you delete an instance from an associated custom record type object and then deploy the object to an account, the values do not get removed from the account. You can only make instances inactive by setting their `isinactive` element to T.

Instances of an associated custom record type can be filtered using a custom element, as displayed in the `customrecord_cseg_campaignmedium` example. For more information about how to filter custom segment values, see [Custom Segment Value Filtering](#).

The following is an example of a region record type, `customrecord_cseg_region`, that is associated with the region custom segment, `cseg_region`, from the campaign donor scenario. This record contains all the possible values for the region segment field, such as New-England, Midwest, and Mid-Atlantic. Some values are expressed as parents of others. For example, Northeast is a parent of New-England. The file is saved as `customrecord_cseg_region.xml` in the Objects folder of the SuiteCloud project. For more information about the custom segment and the structure of that file, see [Custom Segment Object Structure](#).

```

1 <customrecordtype scriptid="customrecord_cseg_region">
2   <allowattachments>F</allowattachments>
3   <allowinlinedeleting>F</allowinlinedeleting>
4   <allowinlinedetaching>F</allowinlinedetaching>
5   <allowinlineediting>F</allowinlineediting>
6   <allowquickadd>F</allowquickadd>
7   <allowquicksearch>F</allowquicksearch>
8   <customsegment>[scriptid=cseg_region]</customsegment>
9   <enableidle>F</enableidle>
10  <enablekeywords>F</enablekeywords>
11  <enablemailmerge>F</enablemailmerge>
12  <enableoptimisticlocking>F</enableoptimisticlocking>
13  <enablesystemnotes>T</enablesystemnotes>
14  <iconbuiltin>T</iconbuiltin>
15  <includeinsearchmenu>F</includeinsearchmenu>
16  <showcreationdate>F</showcreationdate>
17  <showcreationdateonlist>F</showcreationdateonlist>
18  <showlastmodified>F</showlastmodified>
19  <showlastmodifiedonlist>F</showlastmodifiedonlist>
20  <shownotes>T</shownotes>
21  <showowner>F</showowner>
22  <showownerallowchange>F</showownerallowchange>
23  <showowneronlist>F</showowneronlist>
24  <customrecordcustomfields>
25    <customrecordcustomfield scriptid="custrecord_division_number">
26      <accesslevel>2</accesslevel>
27      <allowquickadd>F</allowquickadd>
28      <applyformatting>T</applyformatting>
29      <checkspelling>F</checkspelling>
30      <defaultchecked>F</defaultchecked>
31      <displaytype>NORMAL</displaytype>
32      <encryptatrest>F</encryptatrest>
33      <fieldtype>INTEGER</fieldtype>
34      <globalsearch>F</globalsearch>
35      <isformula>F</isformula>
36      <ismandatory>F</ismandatory>
37      <isparent>F</isparent>
38      <label>Division Number</label>
39      <rolerestrict>F</rolerestrict>
40      <showinlist>F</showinlist>

```

```

41     <storevalue>T</storevalue>
42   </customrecordcustomfield>
43 </customrecordcustomfields>
44 <instances>
45   <instance scriptid="_northeast">
46     <custrecord_division_number></custrecord_division_number>
47     <isinactive>F</isinactive>
48     <name>Northeast</name>
49   </instance>
50   <instance scriptid="_new_england">
51     <custrecord_division_number>1</custrecord_division_number>
52     <isinactive>F</isinactive>
53     <name>New-England</name>
54     <parent>[scriptid=customrecord_cseg_region._northeast]</parent>
55   </instance>
56   <instance scriptid="_mid_atlantic">
57     <custrecord_division_number>2</custrecord_division_number>
58     <isinactive>F</isinactive>
59     <name>Mid-Atlantic</name>
60     <parent>[scriptid=customrecord_cseg_region._northeast]</parent>
61   </instance>
62   <instance scriptid="_midwest">
63     <custrecord_division_number></custrecord_division_number>
64     <isinactive>F</isinactive>
65     <name>Midwest</name>
66   </instance>
67   <instance scriptid="_east_north_central">
68     <custrecord_division_number>3</custrecord_division_number>
69     <isinactive>F</isinactive>
70     <name>East North Central</name>
71     <parent>[scriptid=customrecord_cseg_region._midwest]</parent>
72   </instance>
73   <instance scriptid="_west_north_central">
74     <custrecord_division_number>4</custrecord_division_number>
75     <isinactive>F</isinactive>
76     <name>West North Central</name>
77     <parent>[scriptid=customrecord_cseg_region._midwest]</parent>
78   </instance>
79   <instance scriptid="_south">
80     <custrecord_division_number></custrecord_division_number>
81     <isinactive>F</isinactive>
82     <name>South</name>
83   </instance>
84   <instance scriptid="_west">
85     <custrecord_division_number></custrecord_division_number>
86     <isinactive>F</isinactive>
87     <name>West</name>
88   </instance>
89   <instance scriptid="_mountain">
90     <custrecord_division_number>8</custrecord_division_number>
91     <isinactive>F</isinactive>
92     <name>Mountain</name>
93     <parent>[scriptid=customrecord_cseg_region._west]</parent>
94   </instance>
95   <instance scriptid="_pacific">
96     <custrecord_division_number></custrecord_division_number>
97     <isinactive>F</isinactive>
98     <name>Pacific</name>
99     <parent>[scriptid=customrecord_cseg_region._west]</parent>
100 </instance>
101 </instances>
102 <links>
103   <link>
104     <linkcategory>BASICSETUPCOMPANY</linkcategory>
105     <linktasktype>LIST</linktasktype>
106   </link>
107 </links>
108 </customrecordtype>

```

The following is an example of a campaign medium record type, `customrecord_cseg_campaignmedium`, that is associated with the region custom segment, `cseg_campaignmedium`, from the campaign donor scenario. This record contains all the possible values for the campaign medium segment field, such as Events, E-

mail, and Website. The file is saved as `customrecord_cseg_campaignmedium.xml` in the Objects folder of the SuiteCloud project. For more information about the custom segment and the structure of that file, see [Custom Segment Object Structure](#).

```

1 <customrecordtype scriptid="customrecord_cseg_campaignmedium">
2   <allowattachments>F</allowattachments>
3   <allowinlinedeleting>F</allowinlinedeleting>
4   <allowinlinedetaching>F</allowinlinedetaching>
5   <allowinlineediting>F</allowinlineediting>
6   <allowquickadd>F</allowquickadd>
7   <allowquicksearch>F</allowquicksearch>
8   <customsegment>[scriptid=cseg_campaignmedium]</customsegment>
9   <enableble>F</enableble>
10  <enablekeywords>F</enablekeywords>
11  <enablemailmerge>F</enablemailmerge>
12  <enableoptimisticlocking>F</enableoptimisticlocking>
13  <enablesystemnotes>T</enablesystemnotes>
14  <iconbuiltin>T</iconbuiltin>
15  <includeinsearchmenu>F</includeinsearchmenu>
16  <showcreationdate>F</showcreationdate>
17  <showcreationdateonlist>F</showcreationdateonlist>
18  <showlastmodified>F</showlastmodified>
19  <showlastmodifiedonlist>F</showlastmodifiedonlist>
20  <shownotes>T</shownotes>
21  <showowner>F</showowner>
22  <showownerallowchange>F</showownerallowchange>
23  <showowneronlist>F</showowneronlist>
24  <instances>
25    <instance scriptid="_twitter">
26      <cseg_campaignmedium_filterby_cseg_region>
27        [scriptid=customrecord_cseg_region._northeast]|
28        [scriptid=customrecord_cseg_region._new_england]|
29        [scriptid=customrecord_cseg_region._mid_atlantic]|
30        [scriptid=customrecord_cseg_region._midwest]|
31        [scriptid=customrecord_cseg_region._east_north_central]|
32        [scriptid=customrecord_cseg_region._west_north_central]|
33        [scriptid=customrecord_cseg_region._south]|
34        [scriptid=customrecord_cseg_region._west]|
35        [scriptid=customrecord_cseg_region._mountain]|
36        [scriptid=customrecord_cseg_region._pacific]
37      </cseg_campaignmedium_filterby_cseg_region>
38      <inactive>F</inactive>
39      <name>Twitter</name>
40    </instance>
41    <instance scriptid="_events">
42      <cseg_campaignmedium_filterby_cseg_region>
43        [scriptid=customrecord_cseg_region._northeast]|
44        [scriptid=customrecord_cseg_region._new_england]|
45        [scriptid=customrecord_cseg_region._mid_atlantic]|
46        [scriptid=customrecord_cseg_region._midwest]|
47        [scriptid=customrecord_cseg_region._east_north_central]|
48        [scriptid=customrecord_cseg_region._west_north_central]|
49      </cseg_campaignmedium_filterby_cseg_region>
50      <inactive>F</inactive>
51      <name>Events</name>
52    </instance>
53    <instance scriptid="_email">
54      <cseg_campaignmedium_filterby_cseg_region>
55        [scriptid=customrecord_cseg_region._northeast]|
56        [scriptid=customrecord_cseg_region._south]|
57        [scriptid=customrecord_cseg_region._west]|
58      </cseg_campaignmedium_filterby_cseg_region>
59      <inactive>F</inactive>
60      <name>Email</name>
61    </instance>
62    <instance scriptid="_website">
63      <cseg_campaignmedium_filterby_cseg_region>
64        [scriptid=customrecord_cseg_region._west]|
65        [scriptid=customrecord_cseg_region._mountain]|
66        [scriptid=customrecord_cseg_region._pacific]
67      </cseg_campaignmedium_filterby_cseg_region>
68      <inactive>F</inactive>

```

```

69     <name>Website</name>
70   </instance>
71   <instance scriptid="_facebook">
72     <cseg_campaignmedium_filterby_cseg_region>
73       [scriptid=customrecord_cseg_region._northeast]|
74       [scriptid=customrecord_cseg_region._new_england]|
75       [scriptid=customrecord_cseg_region._mid_atlantic]|
76       [scriptid=customrecord_cseg_region._midwest]|
77       [scriptid=customrecord_cseg_region._east_north_central]|
78       [scriptid=customrecord_cseg_region._west_north_central]|
79       [scriptid=customrecord_cseg_region._south]|
80       [scriptid=customrecord_cseg_region._west]|
81       [scriptid=customrecord_cseg_region._mountain]|
82       [scriptid=customrecord_cseg_region._pacific]
83     </cseg_campaignmedium_filterby_cseg_region>
84     <isinactive>F</isinactive>
85     <name>Facebook</name>
86   </instance>
87 </instances>
88 <links>
89   <link>
90     <linkcategory>BASICSETUPCOMPANY</linkcategory>
91     <linktasktype>LIST</linktasktype>
92   </link>
93 </links>
94 </customrecordtype>

```

For more information about the `customrecordtype` object, see the help topic [customrecordtype](#).

Custom Segment Value Filtering

To filter a segment in SuiteCloud Development Framework (SDF), set the `filteredby` element in the custom segment that requires filtering. You can set this value to multiple custom segment references, as displayed in the following example:

```

1 <customsegment scriptid="cseg_campaignmedium">
2   ...
3   <filteredby>[scriptid=cseg_region]|[scriptid=cseg_other]|...</filteredby>
4   ...
5 </customsegment>

```

When filtering a custom segment by another custom segment, the instances in the associated custom record type are filtered by the other custom segment using a special custom `filterby` element. This custom element has the following syntax and can be used in any instance structure of an associated custom record type:

```

1 <customrecordtype scriptid="customrecord_cseg_SCRIPTID">
2   ...
3   <instances>
4     <instance scriptid="_SEGMENTVALUEID">
5       <SEGMENT-SCRIPTID1_filterby_SEGMENT-SCRIPTID2>
6         [scriptid=SCRIPTID-OF-INSTANCE-FROM-SEGMENT-SCRIPTID2]|
7         [scriptid=SCRIPTID-OF-INSTANCE-FROM-SEGMENT-SCRIPTID2]|
8         ...
9       </SEGMENT-SCRIPTID1_filterby_SEGMENT-SCRIPTID2>
10    </instance>
11    ...
12  </instances>
13  ...
14 </customrecordtype>

```

Replace `SEGMENT-SCRIPTID1` with the script ID of the segment that requires filtering. Replace `SEGMENT-SCRIPTID2` with the script ID of the segment that contains the instances that should filter the `SEGMENT-SCRIPTID1` instances. Set the value of this element to the SDF custom object instance script ID references

that apply. For more information about SDF custom object references, see [Specifying an SDF Custom Object Reference](#).

For example, the customrecord_cseg_campaignmedium from the campaign donor scenario uses this custom element. This associated custom record contains several possible communication medium values. However, some of these values are only applicable for specific regions. The cseg_campaignmedium_filterby_cseg_region element is used to specify the regions for each medium where they are applicable. The following snippet from the customrecord_cseg_campaignmedium shows all the regions where Events, Email, and Website are valid campaign mediums:

```

1 <customrecordtype scriptid="customrecord_cseg_campaignmedium">
2   ...
3   <instances>
4     <instance scriptid="_events">
5       <cseg_campaignmedium_filterby_cseg_region>
6         [scriptid=customrecord_cseg_region._northeast]|
7         [scriptid=customrecord_cseg_region._new_england]|
8         [scriptid=customrecord_cseg_region._mid_atlantic]|
9         [scriptid=customrecord_cseg_region._midwest]|
10        [scriptid=customrecord_cseg_region._east_north_central]|
11        [scriptid=customrecord_cseg_region._west_north_central]|
12      </cseg_campaignmedium_filterby_cseg_region>
13      <isinactive>F</isinactive>
14      <name>Events</name>
15    </instance>
16    <instance scriptid="_email">
17      <cseg_campaignmedium_filterby_cseg_region>
18        [scriptid=customrecord_cseg_region._northeast]|
19        [scriptid=customrecord_cseg_region._south]|
20        [scriptid=customrecord_cseg_region._west]|
21      </cseg_campaignmedium_filterby_cseg_region>
22      <isinactive>F</isinactive>
23      <name>Email</name>
24    </instance>
25    <instance scriptid="_website">
26      <cseg_campaignmedium_filterby_cseg_region>
27        [scriptid=customrecord_cseg_region._west]|
28        [scriptid=customrecord_cseg_region._mountain]|
29        [scriptid=customrecord_cseg_region._pacific]|
30      </cseg_campaignmedium_filterby_cseg_region>
31      <isinactive>F</isinactive>
32      <name>Website</name>
33    </instance>
34    ...
35  </instances>
36  ...
37 </customrecordtype>

```

For more information about:

- The campaign donor scenario, see [Campaign Donor Scenario Sample](#).
- How value filters work on a custom segment, see the help topic [Filtering for a Custom Segment](#).

Custom Segment Object Permissions

You can provide roles with access to a custom segment in SuiteCloud Development Framework (SDF) using the permissions structure in the segment object. In this structure, you can create as many custom permissions as needed by specifying a permission structure for every permission.

Consider the following information for SDF SuiteApps and SuiteApp projects:

- For role permissions that exist in the target account but not in the permission field group of the customsegment SDF custom objects in the SDF SuiteApp (or SuiteApp project), the permissions in the target account will not be changed or removed.

- For role permissions that exist in both the permission field group of the customsegment SDF custom objects in the SDF SuiteApp or SuiteApp project and the target account, the permissions in the target account will be changed to the XML values for the permissions in the SDF SuiteApp or SuiteApp project.

Consider the following information for account customization projects:

- For role permissions that exist in the target account but not in the permission field group of the customsegment SDF custom objects in the account customization project, the permissions in the target account will not be changed or removed.
- For role permissions that exist in both the permission field group of the customsegment SDF custom objects in the account customization project and the target account, the permissions in the target account will be changed to the XML values for the permissions from the account customization project.

In the following example, the HUMAN_RESOURCES_GENERALIST role is given View access to the Record Access Level and Search/Reporting Access Level, and Full access to the Management Access Level.

```

1 <customsegment scriptid="cseg_region">
2   ...
3   <permissions>
4     <permission>
5       <recordaccesslevel>VIEW</recordaccesslevel>
6       <role>HUMAN_RESOURCES_GENERALIST</role>
7       <searchaccesslevel>VIEW</searchaccesslevel>
8       <valuemgmtaccesslevel>FULL</valuemgmtaccesslevel>
9     </permission>
10  </permissions>
11  ...
12 </customsegment>

```

For more information about how user permissions affect custom segments, see the help topic [User Permissions for a Custom Segment](#).



Important: If a SuiteCloud project contains an SDF custom object and a custom role that reference each other, the levels of permission and restriction must match in both objects. For example, if a custom record type permits the custom role with the VIEW permission level, that custom role must set the custom record type permission level to VIEW. Projects that contain a level mismatch cannot be deployed to a target NetSuite account. For more information, see [Custom Roles as XML Definitions](#).

Custom Transaction Record Types as XML Definitions

SuiteCloud Development Framework (SDF) supports custom transaction record types that are tailored to your business needs. For use cases, see the help topic [Benefits of Custom Transaction Types](#).

Use the customtransactiontype SDF custom object to develop custom transaction types in a SuiteCloud project. You can create new custom transactions, or import existing custom transactions into your SuiteCloud project using SuiteCloud IDE plug-in.

For more information about custom transactions, see the following:

- For a list of custom transaction fields available in SDF, see the help topic [customtransactiontype](#).
- To create a custom transaction in the NetSuite user interface, see the help topic [Creating a Custom Transaction Type](#).

Consider the following information when working with custom transaction types as XML definitions:

- The following custom transaction list styles require specific scriptid prefixes:
 - For the Basic, Journal, and Header only transaction types, the scriptid prefix must be customtransaction.
 - For the Sales transaction type, the scriptid prefix must be customsale.
 - For the Purchase transaction type, the scriptid prefix must be custompurchase.

For information about those custom transaction list styles, see the help topic [Multiple Custom Transaction Styles Supported](#).

- When you create a custom transaction type, the system automatically creates a standard form for the type. You cannot create custom transaction forms for custom transactions in SDF. To use the NetSuite user interface to add custom forms, see the help topic [Adding Custom Forms for a Custom Transaction Type](#).
- SDF supports the text fields and status of a custom transaction type.
- SDF supports permissions in a custom transaction type.

When you deploy a custom transaction that includes a permissions sublist, the permissions are also updated in the target account. The following example includes a permissions sublist:

```

1 <customtransactiontype scriptid="customtransaction2">
2 ...
3   <permissions>
4     <permission>
5       <permittedlevel>NONE</permittedlevel>
6       <permittedrole>[scriptid=customrole_transactionapprover]</permittedrole>
7     </permission>
8   </permissions>
9   ...
10 </customtransactiontype>

```

- SDF does not support:
 - Instances of custom transactions
 - Sublist links in the custom transaction type
 - Fields from the Document Numbers subtab and Transaction Numbers subtab of the NetSuite user interface
 - The following fields in custom transactions: account, numberinginitupdate, linkinsertbefore, showallforms



Important: Although the Account field is required when the transaction type's List Style is Basic, the Account field does not exist in the customtransactiontype SDF custom object XML definition. The account must be specified in the Document Numbers subtab of the SDF custom object in the NetSuite user interface after installing the SuiteApp or deploying the SDF custom object to an account. To edit the Account field, see the help topic [Editing a Custom Transaction Type](#).

- To deploy custom transaction type objects to a target NetSuite account, enable the Custom Transactions feature. For more information, see the help topics [Enabling the Custom Transactions Feature](#) and [Deployment Logs for SuiteCloud Projects](#).
- Consider the following information for SDF SuiteApps and SuiteApp projects:
 - For role permissions that exist in the target account but not in the permission field group of the customtransactiontype SDF custom objects in the SDF SuiteApp (or SuiteApp project), the permissions in the target account will not be changed or removed.
 - For role permissions that exist in both the permission field group of the customtransactiontype SDF custom objects in the SDF SuiteApp or SuiteApp project and the target account, the

permissions in the target account will be changed to the XML values for the permissions in the SDF SuiteApp or SuiteApp project.

- Consider the following information for account customization projects:
 - For role permissions that exist in the target account but not in the permission field group of the customtransactiontype SDF custom objects in the account customization project, the permissions in the target account will not be changed or removed.
 - For role permissions that exist in both the permission field group of the customtransactiontype SDF custom objects in the account customization project and the target account, the permissions in the target account will be changed to the XML values for the permissions from the account customization project.

The following is an example of a customtransactiontype SDF custom object that defines a group purchase custom transaction:

```

1 <customtransactiontype scriptid="customtransaction_grouppurchase">
2   <iscredit>F</iscredit>
3   <isposting>F</isposting>
4   <isvoidable>F</isvoidable>
5   <name>Group Purchase</name>
6   <showstatus>F</showstatus>
7   <subliststyle>BASIC</subliststyle>
8   <segments>
9     <classmandatory>F</classmandatory>
10    <classposition>NONE</classposition>
11    <departmentmandatory>F</departmentmandatory>
12    <departmentposition>HEADER</departmentposition>
13    <locationmandatory>F</locationmandatory>
14    <locationposition>LINE</locationposition>
15  </segments>
16  <statuses>
17    <status scriptid="open_status">
18      <id>A</id>
19      <description>Open</description>
20      <posting>F</posting>
21    </status>
22    <status scriptid="closed_status">
23      <id>B</id>
24      <description>Closed</description>
25      <posting>T</posting>
26    </status>
27  </statuses>
28 </customtransactiontype>

```

In the example, the following fields define the position of the segment fields in the transaction:

- classmandatory
- classposition
- departmentmandatory
- departmentposition
- locationmandatory
- locationposition

For additional information about custom transactions, see the following:

- [Referencing a Custom Transaction in XML Definitions](#)
- [Custom Transaction Statuses in XML Definitions](#)
- [Custom Transaction Type Classification Fields](#)

- [segments](#)
- [Custom Transaction Type Setup](#)
- [Custom Transactions](#)
- [customtransactiontype](#)

Referencing a Custom Transaction in XML Definitions

Your users can use custom transactions in other SDF custom objects that reference the transaction.

In SuiteCloud Development Framework (SDF), custom transactions are referenced by the `scriptid` attribute value for the `customtransactiontype` element, for example, `customtransaction_grouppurchase`.

To reference a custom transaction status, append the status `scriptid` to the `customtransaction` `scriptid`, for example, `customtransaction_grouppurchase.open_status`. For more information about custom transaction statuses, see [Custom Transaction Statuses in XML Definitions](#).

Custom transaction types can be referenced by the following SDF custom objects:

- `centercategory`, using the `linkobject` field. For example:

```

1 | <centercategory scriptid="custcentercategory_adjustments_category">
2 | ...
3 |   <links>
4 |     <link>
5 |       <linklabel></linklabel>
6 |       <linkobject>[scriptid=customtransaction_fixed_asset]</linkobject>
7 |       <linktasktype>LIST</linktasktype>
8 |       <shortlist>F</shortlist>
9 |     </link>
10 |   </links>
11 | ...
12 | </centercategory>

```

- `role`, using the `permkey` field.

You can include role objects in your SuiteCloud project that grant a permission to the custom transaction type in the target account. When the custom role is in the same SuiteCloud project as the custom transaction, both objects must have matching permissions. The following example adds the `VIEW` permission to custom transaction `customtransaction2`:

```

1 | <role scriptid="customrole_sample1">
2 | ...
3 |   <permissions>
4 |     <permission>
5 |       <permkey>[customtransaction2]</permkey>
6 |       <permlevel>VIEW</permlevel>
7 |     </permission>
8 |   </permissions>
9 | ...
10 | </role>

```

- `sublist`, using a saved search that references a custom transaction.

The saved search must be defined in the NetSuite user interface. For more information, see the help topics [Defining a Saved Search](#) and [Sublists as XML Definitions](#).

- `transactionbodycustomfield`, using the `bodycustomtransactions` field. For example:

```

1 | <transactionbodycustomfield scriptid="custbody_notes">
2 | ...

```

```

3 | <bodycustomtransactions>[scriptid=customtransaction106]</bodycustomtransactions>
4 | ...
5 | </transactionbodycustomfield>

```

- `transactioncolumncustomfield`, using the `columncustomtransactions` field. For example:

```

1 | <transactioncolumncustomfield scriptid="custcol_123">
2 | ...
3 |   <columncustomtransactions>[scriptid=customtransaction108]</columncustomtransactions>
4 | ...
5 | </transactioncolumncustomfield>

```

- `workflow`, by setting a field value that is a custom transaction type status. For example:

```

1 | <workflow scriptid="customworkflow10">
2 | ...
3 |   <workflowstates>
4 |     <workflowstate scriptid="workflowstate17">
5 |       ...
6 |       <workflowactions trigger="ONENTRY">
7 |         <setfieldvalueaction scriptid="workflowaction7">
8 |           <field>STDBODYTRANSTATUS</field>
9 |           <schedulemode>DELAY</schedulemode>
10 |          <valueselect>[scriptid=customtransaction_grouppurchase.open_status]</valueselect>
11 |          <valuetype>STATIC</valuetype>
12 |        </setfieldvalueaction>
13 |      </workflowactions>
14 |    </workflowstate>
15 |  </workflowstates>
16 | </workflow>

```

For more information, see the help topic [Custom Transaction Types in Workflows](#).

In addition to the SDF custom objects described in the previous list, all custom fields can reference custom transaction types. The following example references a custom transaction in a custom field filter:

```

1 | <customfieldfilters>
2 |   <customfieldfilter>
3 |     ...
4 |     <fldfiltersel>[scriptid=customtransaction_operation]</fldfiltersel>
5 |     ...
6 |   </customfieldfilter>
7 | </customfieldfilters>

```

Custom Transaction Statuses in XML Definitions

In SuiteCloud Development Framework (SDF), you can add statuses to custom transactions for your users to track the progress of each transaction instance. For information about statuses, see the help topic [Statuses for a Custom Transaction Type](#).

To display the Status field on the custom transaction instance in the NetSuite user interface, set the `showstatus` field value to T. For information about the impact of displaying or hiding the status, see the help topic [Displaying or Hiding the Status Field for a Custom Transaction Type](#).

Note: If the custom transaction type has an application ID and was installed as part of a SuiteApp, the status sublist cannot be modified in the NetSuite user interface. You can modify the status sublist during application development using SuiteCloud Development Framework (SDF) by editing the XML definition of the SDF custom object in a SuiteCloud project.

In SDF, the status of the custom transaction can be referenced by one of the following IDs:

- The scriptid attribute value for the status field.

Use this ID for referencing the custom transaction status in an XML definition. Note that changing the scriptid attribute value signifies a new custom transaction status.

- The id field value.

Use this ID for referencing the custom transaction status in SuiteScript. The id field value must not conflict with other status field id values defined in the custom transaction. For the list of possible values, see the help topic [customtransactiontype_statuses_id](#).

After the customtransactiontype SDF custom object exists in the target account, the id field value cannot be changed. The id value can be changed when the status scriptid attribute is changed, because a scriptid change signifies a new custom transaction status.

If an instance of the custom transaction exists, you cannot modify the posting field value or remove the status. Attempting to modify the posting value or remove the status results in a validation error. For information about modifying or deleting a custom transaction status, see the help topic [Modifying or Deleting a Custom Transaction Type Status](#).

The following example defines two statuses for a custom transaction: Open and Closed.

```

1 <customtransactiontype scriptid="customtransaction1">
2   ....
3   <statuses>
4     <status scriptid="open_status">
5       <id>A</id>
6       <description>Open</description>
7       <posting>F</posting>
8     </status>
9     <status scriptid="closed_status">
10      <id>B</id>
11      <description>Closed</description>
12      <posting>T</posting>
13    </status>
14  </statuses>
15  ....
16 </customtransactiontype>

```

In the preceding example, the custom transaction status can be referenced in the following ways:

- Open status
 - Referenced in an XML definition using the scriptid open_status
 - Referenced in SuiteScript using the id field value A
- Closed status
 - Referenced in an XML definition using the scriptid closed_status
 - Referenced in SuiteScript using the id field value B

By default, custom transactions also contain the following statuses:

- voided, where the scripid is status_voided and the id value is V
- undefined, where the scriptid is status_undefined and the id value is Y

Consider the following information regarding the voided and undefined statuses:

- The voided and undefined statuses do not appear in the XML definition of the customtransactiontype object.
- The scriptid for each status can be referenced by other objects.

- The id values V and Y are reserved for the voided and undefined statuses and cannot be used for other statuses.

To reference a custom transaction status, see [Referencing a Custom Transaction in XML Definitions](#).

Reports

For more information about the SuiteCloud Development Framework (SDF) custom objects that pertain to reports, see [Financial Reports as XML Definitions](#).

Financial Reports as XML Definitions

Financial reports enable you to analyze and present your financial data using predefined layouts and statements, such as a trial balance. If you have the Financial Statements and Report Customization permissions, you can customize your report results using the Financial Report Builder in NetSuite.

You can create and manage custom financial reports only by using NetSuite. When created, you can import them into a SuiteCloud project. You can also deploy financial reports to a NetSuite account using SDF. For more information about working with financial reports in a NetSuite account, see the help topic [Financial Report Builder](#).

In SDF, financial reports exist as two separate objects that are imported or deployed together in a SuiteCloud project: reportdefinition and financiallayout.

The following is an example of the reportdefinition object:

```
1 <reportdefinition scriptid="customreport_ow_filter">
2   <definition>bNp860q863yKeTmdSzvaydsKedm86wvr6+vr6+vr6+tozLtw==</definition>
3   <dependencies>
4     <dependency>[scriptid=customlayout_2_000071_570]</dependency>
5   </dependencies>
6   <name>Custom Balance Sheet OW filter</name>
7 </reportdefinition>
```

The reportdefinition scriptid attribute and name element are respectively identical to the script ID and name of the custom report in the NetSuite account.

The following is an example of the financiallayout object:

```
1 <financiallayout scriptid="customlayout_2_000071_570">
2   <layout>+vr6+vr6+tozLtw==</layout>
3   <name>Custom Balance Sheet Layout(US) 2</name>
4 </financiallayout>
```

The financiallayout scriptid attribute and name element are respectively identical to the script ID and name of the custom report layout in the NetSuite account.



Warning: Do not manually edit the XML in SDF. Modifications to the system-generated XML may result in validation and deployment failures. You should customize financial reports with NetSuite rather than SDF.

You can reference SDF custom objects in financial reports by defining dependencies. If your report contains a dependency, you must import the SDF custom object into your SuiteCloud project to successfully deploy the project to a target NetSuite account. For more information, see dependencies.

For more information about financial reports, see the following topics:

- [reportdefinition](#)
- [financiallayout](#)
- [Financial Reports](#)
- [Accessing the Financial Report Builder](#)
- [Reporting Overview](#)

Forms

The following topics provide SuiteCloud Development Framework (SDF) information about the SDF custom objects that pertain to forms:

- [Custom Address Forms as XML Definitions](#)
- [Custom Entry Forms as XML Definitions](#)
- [Custom Code Tab and Actions Tab in XML Definitions](#)
- [Transaction Forms as XML Definitions](#)
- [Sublists as XML Definitions](#)
- [Subtabs as XML Definitions](#)

Custom Address Forms as XML Definitions

- [Account Setup for Using Custom Address Forms](#)
- [Working with the Address Template and Address Form Fields](#)
- [Example of a Custom Address Form Object](#)

[Custom Address Forms](#)

Address forms are a type of entry form that you can use to collect address information localized to the countries that your business operates in. The addressform object supports the following:

- Name
- ID
- Address Template
- Fields
- Country
- Custom Code
- Actions

For information about the custom code and actions tabs, see [Custom Code Tab and Actions Tab in XML Definitions](#)

To find address forms in a NetSuite account, go to Customizations > Forms > Address Forms. You can download and import a custom address form into your SuiteCloud project if it is not locked or part of another SuiteApp.

The following image shows a custom address form as it appears in the NetSuite UI.

Custom Address Entry Form

Save Cancel Reset Change ID Actions

NAME *
Custom Address Form

ID
custform_12_4496762_510

ADDRESS TEMPLATE
<ATTENTION>
<ADDRESSEE>
<ADDR1>
<ADDR2>
<ADDR3>
<CITY> <STATE> <ZIP>
<COUNTRY>

DESCRIPTION	SHOW	MANDATORY	DISPLAY TYPE	LABEL	COLUMN BREAK	SPACE BEFORE	SAME ROW AS PREVIOUS	TEMPLATE FIELD ID
Country	☒	☒	Normal	Country	☐		☐	COUNTRY
Attention	☒	☐	Normal	Attention	☐		☐	ATTENTION
Addressee	☒	☐	Normal	Addressee	☐		☐	ADDRESSEE
Phone	☒	☐	Normal	Phone	☐		☐	ADDRPHONE
Addr1	☒	☐	Normal	Address 1	☐		☐	ADDR1
Addr2	☒	☐	Normal	Address 2	☐		☐	ADDR2
Addr3	☐	☐	Normal	Address 3	☐		☐	ADDR3
City	☒	☐	Normal	City	☐		☐	CITY
State	☒	☐	Normal	State	☐		☐	STATE
Zip	☒	☐	Normal	Zip	☐		☒	ZIP
Province	☒	☐	Normal	Province	☐		☐	CUSTRECORD_PROVINCE
SubBuilding	☒	☐	Normal	SubBuilding	☐		☐	CUSTRECORD_SUBBUILDING
FL	☒	☐	Normal	FL	☐		☐	CUSTRECORD_FLOOR
Street Number	☒	☐	Normal	Street Number	☐		☐	CUSTRECORD_STREETNUM
Street Direction	☒	☐	Normal	Street Direction	☐		☐	CUSTRECORD_STREETDIRECTION
Street Name	☒	☐	Normal	Street Name	☐		☐	CUSTRECORD_STREETNAME
Locality	☒	☐	Normal	Locality	☐		☐	CUSTRECORD_LOCALITY
Street Type	☒	☐	Normal	Street Type	☐		☐	CUSTRECORD_STREETTYPE

Account Setup for Using Custom Address Forms

Check for the following to ensure you can work with custom address forms from your SuiteCloud project:

- The LOCATIONS Feature in the target NetSuite Account must be enabled. To enable this feature, go to Setup > Company > Enable Features. On the Company subtab, in the Classifications section, select Locations.
- Use a role with full permissions to the Custom Address Form record type. This permission is already enabled if you use the Developer role. For information about enabling permissions for the role you use with SuiteCloud projects, see [Assigning the Developer Role \(Administrator Only\)](#).
- (Optional) To add or view the countries that you can associate with a customized address form, from the NetSuite account, go to Setup > Company > Countries.

Working with the Address Template and Address Form Fields

In your SuiteCloud project, you can write address template content, set values for supported fields, and add or modify custom fields.

The following table includes information about working with certain supported elements. The complete list of supported address template fields and template field IDs is available in .

Description	Example
addressTemplate Address template content is rendered in CDATA format therefore it is not checked during project validation. The order of the displayed fields is determined by the address template.	<pre> 1 ... 2 3 <addressTemplate><![CDATA[<attention> 4 5 <addressee> 6 7 <addr1> 8 9 <addr2> 10 11 <addr3> </pre>

Description	Example
If you are not familiar with this element, go to a NetSuite account and use Customization to create your address form and address template. Then, import it into your SuiteCloud project.	<pre> 12 13 <city> 14 15 <state> 16 17 <zip> 18 19 <country>]]></addressTemplate> </pre>
countries You can use a pipe () delimited list to associate multiple countries with an address form. When you assign a country to a custom address form, it acts as a subrecord on records with an editable address. The country-specific address form appears based on the value of the Country field on the subrecord. Note: To set a value for country, you must use the valid country code. Country codes are listed in countries.	<pre> 1 ... 2 3 <countries>CA US GB</countries> </pre>
Other Custom Field Object To use an other custom field on your address form, you can reference it by script ID. For more information about references, see Specifying an SDF Custom Object Reference. Note: In the custom field object XML file, make sure that the rectype value is set to -289 so that it is applied to the address form record type. Script ID requires the object prefix and the remainder of the ID is unique and may be user defined. The object prefix is custrecord.	See Custom Other Record Fields as XML Definitions for an example.

Known Limitations of Custom Form Translations in SDF

The following are known limitations of custom form translations in SDF:

- When exporting from your source account and importing a form into your SuiteCloud project, the button label values are not included for custom forms. Currently, translation collections do not support button labels.
- To export to the SDF XML definition or import the SDF custom object into a SuiteCloud project, field labels must contain a value. If a field label has no value, validation of the SDF XML definition fails during installation. If you want to include a field that does not have a label, you can work around the SDF limitation by inserting a single space character as the translation for a field label. For example, you have fields for first name, middle name, and last name, but you only want a field label of Name to appear for the first field. You enter a label for the Name field and enter a single space for the other two fields.
- Importing custom forms into SDF:
 - When importing a form from the source account into your SuiteCloud project, the form translation is defined using the company default language, not the language of the user importing the form.
 - When deploying a custom form using SDF, the target account receives the terms as they exist in the source account. If the target account does not have translations for the default language used in the source account, then the default language in the translation collection is used.

Example of a Custom Address Form Object

The following example is a custom address form object named "Sample Address Form". It uses script ID `custaddressform_sample` and all standard options are set to their default values.

```

1 <addressForm scriptid="custaddressform_sample">
2
3   <name>Sample Address Form</name>
4
5   <mainFields>
6
7     <defaultFieldGroup>
8
9       <fields position="MIDDLE">
10
11         <field>
12
13           <id>COUNTRY</id>
14
15           <label>Country</label>
16
17           <visible>T</visible>
18
19           <mandatory>T</mandatory>
20
21           <displayType>NORMAL</displayType>
22
23           <columnBreak>F</columnBreak>
24
25           <sameRowAsPrevious>F</sameRowAsPrevious>
26
27         </field>
28
29         <field>
30
31           <id>ATTENTION</id>
32
33           <label>Attention</label>
34
35           <visible>T</visible>
36
37           <mandatory>F</mandatory>
38
39           <displayType>NORMAL</displayType>
40
41           <columnBreak>F</columnBreak>
42
43           <sameRowAsPrevious>F</sameRowAsPrevious>
44
45         </field>
46
47         <field>
48
49           <id>ADDRESSEE</id>
50
51           <label>Addressee</label>
52
53           <visible>T</visible>
54
55           <mandatory>F</mandatory>
56
57           <displayType>NORMAL</displayType>
58
59           <columnBreak>F</columnBreak>
60
61           <sameRowAsPrevious>F</sameRowAsPrevious>
62
63         </field>
64
65         <field>
66

```

```

67         <id>ADDRPHONE</id>
68
69         <label>Phone</label>
70
71         <visible>T</visible>
72
73         <mandatory>F</mandatory>
74
75         <displayType>NORMAL</displayType>
76
77         <columnBreak>F</columnBreak>
78
79         <sameRowAsPrevious>F</sameRowAsPrevious>
80
81     </field>
82
83     <field>
84
85         <id>ADDR1</id>
86
87         <label>Address 1</label>
88
89         <visible>T</visible>
90
91         <mandatory>F</mandatory>
92
93         <displayType>NORMAL</displayType>
94
95         <columnBreak>F</columnBreak>
96
97         <sameRowAsPrevious>F</sameRowAsPrevious>
98
99     </field>
100
101     <field>
102
103         <id>ADDR2</id>
104
105         <label>Address 2</label>
106
107         <visible>T</visible>
108
109         <mandatory>F</mandatory>
110
111         <displayType>NORMAL</displayType>
112
113         <columnBreak>F</columnBreak>
114
115         <sameRowAsPrevious>F</sameRowAsPrevious>
116
117     </field>
118
119     <field>
120
121         <id>ADDR3</id>
122
123         <label>Address 3</label>
124
125         <visible>F</visible>
126
127         <mandatory>F</mandatory>
128
129         <displayType>NORMAL</displayType>
130
131         <columnBreak>F</columnBreak>
132
133         <sameRowAsPrevious>F</sameRowAsPrevious>
134
135     </field>
136
137     <field>
138
139         <id>[scriptid=custrecord_barangay]</id>

```

```

140         <label>Barangay</label>
141
142         <visible>F</visible>
143
144         <mandatory>F</mandatory>
145
146         <displayType>NORMAL</displayType>
147
148         <columnBreak>F</columnBreak>
149
150         <sameRowAsPrevious>F</sameRowAsPrevious>
151
152     </field>
153
154     <field>
155
156         <id>CITY</id>
157
158         <label>City</label>
159
160         <visible>true</visible>
161
162         <mandatory>F</mandatory>
163
164         <displayType>NORMAL</displayType>
165
166         <columnBreak>F</columnBreak>
167
168         <sameRowAsPrevious>F</sameRowAsPrevious>
169
170     </field>
171
172     <field>
173
174         <id>STATE</id>
175
176         <label>State</label>
177
178         <visible>true</visible>
179
180         <mandatory>F</mandatory>
181
182         <displayType>NORMAL</displayType>
183
184         <columnBreak>F</columnBreak>
185
186         <sameRowAsPrevious>F</sameRowAsPrevious>
187
188     </field>
189
190     <field>
191
192         <id>ZIP</id>
193
194         <label>Zip</label>
195
196         <visible>true</visible>
197
198         <mandatory>F</mandatory>
199
200         <displayType>NORMAL</displayType>
201
202         <columnBreak>F</columnBreak>
203
204         <sameRowAsPrevious>true</sameRowAsPrevious>
205
206     </field>
207
208 </fields>
209
210 </defaultFieldGroup>
211
212

```



```

213     </mainFields>
214
215     <addressTemplate> <![CDATA[
216
217     <ATTENTION>
218
219     <ADDRESSEE>
220
221     <ADDRPHONE>
222
223     <ADDR1>
224
225     <ADDR2>
226
227     <ADDR3>
228
229     <CITY> <STATE> <ZIP>
230
231     <COUNTRY>
232
233     ]]>
234
235     </addressTemplate>
236
237     <country>PH|IN|US</country>
238
239 </addressForm>

```

To learn more about address forms, see the help topic [Customizing Address Forms](#).

Custom Entry Forms as XML Definitions

Custom entry forms provide you with more control over how data is collected and entered. You can use custom entry forms to capture the information for a specific business application and to maintain consistency.

To find custom entry forms using the NetSuite UI, go to Customization > Forms > Entry Forms.



Note: To work with entry forms in a SuiteCloud project, you must import custom forms from a NetSuite account.

You can edit the form that you imported and deploy the changes to your target account. To learn more about working with entryForm SDF custom objects and supported elements, see the following topics:

- [Supported Custom Entry Forms](#)
- [Custom Code Tab and Actions Tab in XML Definitions](#)
- [Sublists as XML Definitions](#)
- [Subtabs as XML Definitions](#)
- [Example of a Custom Entry Form Object](#)

For information about all supported fields, attributes, and values, see the help topic [entryForm](#) in the SDF XML Reference.

Limitation of Custom Entry Forms

Consider the following limitation when customizing entry forms in SuiteCloud Development Framework (SDF): When the original field is in the top or bottom position, the field's position cannot be changed, the field cannot be moved between tabs, and the default values for `spaceBefore` and `sameAsPrevious` cannot be modified.

Known Limitations of Custom Form Translations in SDF

The following are known limitations of custom form translations in SDF:

- When exporting from your source account and importing a form into your SuiteCloud project, the button label values are not included for custom forms. Currently, translation collections do not support button labels.
- To export to the SDF XML definition or import the SDF custom object into a SuiteCloud project, field labels must contain a value. If a field label has no value, validation of the SDF XML definition fails during installation. If you want to include a field that does not have a label, you can work around the SDF limitation by inserting a single space character as the translation for a field label. For example, you have fields for first name, middle name, and last name, but you only want a field label of Name to appear for the first field. You enter a label for the Name field and enter a single space for the other two fields.
- Importing custom forms into SDF:
 - When importing a form from the source account into your SuiteCloud project, the form translation is defined using the company default language, not the language of the user importing the form.
 - When deploying a custom form using SDF, the target account receives the terms as they exist in the source account. If the target account does not have translations for the default language used in the source account, then the default language in the translation collection is used.

Supported Custom Entry Forms

The following table shows the types of entry forms that SuiteCloud Development Framework (SDF) supports and the feature ID required by the form, if applicable.

You must reference a supported form ID when you specify the standard form value. A feature dependency must be declared in the manifest and enabled in the target account. For an example, see [Example of a Custom Entry Form Object](#).

Form Name	Form ID	Feature Dependency
Bill Of Distribution Form	BILLOFDISTRIBUTIONFORM	DISTRIBUTIONRESOURCEPLANNING
Class Mapping Form	CLASSEMENTMAPPINGFORM	MULTIBOOKCLASS
Dashboard Customer Form	DASHBOARDCUSTOMERFORM	—
Dashboard Vendor Form	DASHBOARDVENDORFORM	—
Department Mapping Form	DEPARTMENTSEGMENTMAPPINGFORM	MULTIBOOKDEPT
Distribution Network Form	DISTRIBUTIONNETWORKFORM	DISTRIBUTIONRESOURCEPLANNING
Inventory Cost Template Form	INVENTORYCOSTTEMPLATEFORM	STANDARDCOSTING
Location Costing Group Form	LOCATIONCOSTINGGROUPFORM	GROUPAVERAGECOSTING
Location Mapping Form	LOCATIONSEGMENTMAPPINGFORM	MULTIBOOKLOC
Manufacturing Cost Template Form	MANUFACTURINGCOSTTEMPLATEFORM	MFGROUTING
Manufacturing Routing Form	MANUFACTURINGROUTINGFORM	MFGROUTING
Manufacturing Task	MANUFACTURINGTASK	MFGROUTING
Shipping Partner Package Form	SHIPPINGPARTNERPACKAGEFORM	SHIPPINGPARTNERS
Shipping Partner Registration Form	SHIPPINGPARTNERREGISTRATIONFORM	SHIPPINGPARTNERS
Shipping Partner Shipment Form	SHIPPINGPARTNERSHIPMENTFORM	SHIPPINGPARTNERS
Accounting Book Form	STANDARDACCOUNTINGBOOKFORM	MULTIBOOK
Address Form	STANDARDADDRESSFORM	—
As Charged Project Revenue Rule	STANDARDASCHARGEDPROJECTREVENUERULE	CHARGEBASEDBILLINGANDADVREV REC

Form Name	Form ID	Feature Dependency
Automatic Location Assignment Configuration Form	STANDARDAUTOMATICLOCATIONASSIGNMENTCONFIGURATIONFORM	AUTOLOCATIONASSIGNMENT
Automatic Location Assignment Rule Form	STANDARDAUTOMATICLOCATIONASSIGNMENTRULEFORM	AUTOLOCATIONASSIGNMENT
Billing Account	STANDARDBILLINGACCOUNT	BILLINGACCOUNTS
Billing Rate Card Form	STANDARDBILLINGRATECARDFORM	BILLINGRATECARDS
Bill of Materials Form	STANDARDBILLOFMATERIALS	ADVANCEDBILLOFMATERIALS
Bill of Materials Revision Form	STANDARDBILLOFMATERIALSREVISION	ADVANCEDBILLOFMATERIALS
Campaign Form	STANDARDCAMPAIGNFORM	MARKETING
Case Form	STANDARDCASEFORM	SUPPORT
Charge Form	STANDARDCHARGEFORM	CHARGEBASEDBILLING
Competitor Form	STANDARDCOMPETITORFORM	SFA
Contact Form	STANDARDCONTACTFORM	—
Customer Form	STANDARDCUSTOMERFORM	—
Description Form	STANDARDDESCRIPTIONFORM	—
Discount Form	STANDARDDISCOUNTFORM	—
Employee Form	STANARDEMPLOYEEFORM	—
Entity Account Mapping Form	STANDARDENTITYACCOUNTMAPPINGFORM	COACCLASSIFICATIONMANAGEMENT
Event Form	STANARDEVENTFORM	—
Expense Amortization Event Form	STANARDEXPENSEAMORTIZATIONEVENTFORM	ADVANCEDEXPENSEMANAGEMENT
Expense Amortization Rule Form	STANARDEXPENSEAMORTIZATIONRULEFORM	ADVANCEDEXPENSEMANAGEMENT
Expense Based Charge Rule Form	STANARDEXPENSEBASEDCHARGERULEFORM	CHARGEBASEDBILLING
Expense Form	STANARDEXPENSEFORM	—
Expense Amortization Plan Form	STANARDEXPENSEPLANFORM	ADVANCEDEXPENSEMANAGEMENT
External Case Form	STANARDEXTERNALCASEFORM	SUPPORT
External Customer Form	STANARDEXTERNALCUSTOMERFORM	—
External Issue Form	STANARDEXTERNALISSUEFORM	—
External Partner Form	STANARDEXTERNALPARTNERFORM	CRM
Fair Value Formula Form	STANDARDFAIRVALUEFORMULAFORM	ADVANCEDREVENUERECOGNITION
Fair Value Price Form	STANDARDFAIRVALUEPRICEFORM	ADVANCEDREVENUERECOGNITION
Fixed Amount Project Revenue Rule	STANDARDFIXEDAMOUNTPROJECTREVENUERULE	CHARGEBASEDBILLINGANDADVREC
Fixed Date Charge Rule Form	STANDARDFIXEDDATECHARGERULEFORM	CHARGEBASEDBILLING
Foreign Currency Variance Posting Rule Form	STANDARDGAINLOSSACCOUNTMAPPINGFORM	GAINLOSSACCTMAPPING
Generic Resource Form	STANDARDGENERICRESOURCEFORM	ADVANCEDJOBS
Global Account Mapping Form	STANDARDGLOBALACCOUNTMAPPINGFORM	COACCLASSIFICATIONMANAGEMENT
Global Inventory Relationship Form	STANDARDGLOBALINVENTORYRELATIONSHIPFORM	CROSSSUBSIDIARYFULFILLMENT
Group Item Form	STANDARDGROUPITEMFORM	—
Help Desk Form	STANDARDHELPDESKFORM	HELPDESK
Inbound Shipment Form	STANDARDINBOUNDSHIPMENT	INBOUNDSHIPMENT
Inventory Detail Form	STANDARDINVENTORYDETAILFORM	ADVBINSERIALLOTMGMT
Inventory Part Form	STANDARDINVENTORYPARTFORM	—
Inventory Status	STANDARDINVENTORYSTATUS	INVENTORYSTATUS
Issue Form	STANDARDISSUEFORM	—
Item Account Mapping Form	STANDARDITEMACCOUNTMAPPINGFORM	COACCLASSIFICATIONMANAGEMENT

Form Name	Form ID	Feature Dependency
Item Demand Plan Form	STANDARDITEMDEMANDPLANFORM	ITEMDEMANDPLANNING
Item Location Configuration	STANDARDITEMLOCATIONCONFIGURATIONFORM	ENHANCEDINVENTORYLOCATION
Item Process Family Form	STANDARDITEMPROCESSFAMILYFORM	WMSSYSTEM
Item Process Group Form	STANDARDITEMPROCESSGROUPFORM	WMSSYSTEM
Item Revenue Category Form	STANDARDITEMREVENUECATEGORYFORM	ADVANCEDREVENUERECOGNITION
Item Supply Plan Form	STANDARDITEMSUPPLYPLANFORM	ITEMDEMANDPLANNING
Job Form	STANDARDJOBFORM	—
Labor Based Project Revenue Rule	STANDARDLABORBASEDPROJECTREVENUERULE	CHARGEBASEDBILLINGANDADVREV REC
Lead Form	STANDARDLEADFORM	—
Manager Group Item Form	STANDARDMANAGERGROUPITEMFORM	—
Manager Inventory Part Form	STANDARDMANAGERINVENTORYPARTFORM	—
Manager Non-Inventory Part Form	STANDARDMANAGERNONINVENTORYPARTFORM	—
Milestone Charge Rule Form	STANDARDMILESTONECHARGERULEFORM	CHARGEBASEDBILLING
Nexus	STANDARDNEXUS	ADVTAXENGINE
Non-Inventory Part Form	STANDARDNONINVENTORYPARTFORM	—
Note Form	STANDARDNOTEFORM	—
Online Inventory Detail Form	STANDARDONLINEINVENTORYDETAILFORM	ADVBINSERIALLOTMGMT
Online Job Form	STANDARDONLINEJOBFORM	—
Partner Form	STANDARDPARTNERFORM	CRM
Payment Form	STANDARDPAYMENTFORM	—
Percent Complete Project Revenue Rule	STANDARDPERCENTCOMPLETEPROJECTREVENUERULE	CHARGEBASEDBILLINGANDADVREV REC
Phone Call Form	STANDARDPHONECALLFORM	—
Planned Revenue Form	STANDARDPLANNEDREVENUEFORM	ADVANCEDREVENUERECOGNITION
Planned Standard Cost Form	STANDARDPLANNEDSTANDARDCOSTFORM	STANDARDCOSTING
Pop-up Contact Form	STANDARDPOPUPCONTACTFORM	—
Pop-up Customer Form	STANDARDPOPUPCUSTOMERFORM	—
Pop-up Employee Form	STANDARDPOPUPEMPLOYEEFORM	—
Pop-up Partner Form	STANDARDPOPUPPARTNERFORM	CRM
Pop-up Vendor Form	STANDARDPOPUPVENDORFORM	ACCOUNTING
Price Book Form	STANDARDPRICEBOOKFORM	SUBSCRIPTIONBILLING
Price Plan Form	STANDARDPRICEPLANFORM	SUBSCRIPTIONBILLING
Project Budget Form	STANDARDPROJECTBUDGETFORM	WBS
Project Intercompany Cross Charge Request Form	STANDARDPROJECTICCHARGEREQFORM	INTERCOMPANYFRAMEWORK
Project Progress Charge Rule Form	STANDARDPROJECTPROGRESSCHARGERULEFORM	CHARGEBASEDBILLING
Project Task Form	STANDARDPROJECTTASKFORM	ADVANCEDJOBS
Project Template Form	STANDARDPROJECTTEMPLATEFORM	ADVANCEDJOBS
Fixed Price Item Promotion	STANDARDPROMOTIONCODEFIXEDPRICE	STACKABLEPROMOTIONS
Promotion Code Form	STANDARDPROMOTIONCODEFORM	PROMOCODES
Free Gift Promotion	STANDARDPROMOTIONCODEFREEGIFT	STACKABLEPROMOTIONS
Item Promotion	STANDARDPROMOTIONCODEITEM	STACKABLEPROMOTIONS
Order Promotion	STANDARDPROMOTIONCODEORDER	STACKABLEPROMOTIONS

Form Name	Form ID	Feature Dependency
Shipping Promotion	STANDARDPROMOTIONCODESHIPPING	STACKABLEPROMOTIONS
Purchase Charge Rule Form	STANDARDPURCHASERULEFORM	CHARGEBASEDBILLING
Region Form	STANDARDREGIONFORM	AUTOLOCATIONASSIGNMENT
Resource Allocation Form	STANDARDRESOURCEALLOCATIONFORM	RESOURCEALLOCATIONS
Resource Group Form	STANDARDRESOURCEGROUPFORM	ADVANCEDJOBS
Revenue Element Form	STANDARDREVENUEELEMENTFORM	ADVANCEDREVENUERECOGNITION
Revenue Recognition Event Form	STANDARDREVENUERECOGNITIONEVENTFORM	ADVANCEDREVENUERECOGNITION
Revenue Recognition Plan Form	STANDARDREVENUERECOGNITIONPLANFORM	ADVANCEDREVENUERECOGNITION
Revenue Recognition Rule Form	STANDARDREVENUERECOGNITIONRULEFORM	ADVANCEDREVENUERECOGNITION
Solution Form	STANDARDSOLUTIONFORM	KNOWLEDGEBASE
Standard Cost Version Form	STANDARDSTANDARDCOSTVERSIONFORM	STANDARDCOSTING
Subscription Change Order Form	STANDARDSUBSCRIPTIONCHANGEORDER	SUBSCRIPTIONBILLING
Subscription Form	STANDARDSUBSCRIPTIONFORM	SUBSCRIPTIONBILLING
Subscription Line Form	STANDARDSUBSCRIPTIONLINEFORM	SUBSCRIPTIONBILLING
Subscription Plan Form	STANDARDSUBSCRIPTIONPLANFORM	SUBSCRIPTIONBILLING
Supply Chain Snapshot	STANDARDSUPPLYCHAINSAPSHOT	SUPPLYCHAINCONTROLTOWER
Task Form	STANDARDTASKFORM	—
Time-Based Charge Rule Form	STANDARDTIMEBASEDCHARGERULEFORM	CHARGEBASEDBILLING
Time Entry Form	STANDARDTIMEENTRYFORM	TIMESHEETS
Timesheet Form	STANDARDTIMESHEETFORM	TIMESHEETRECORDANY
Time Tracking Form	STANDARDTIMETRACKINGFORM	TIMETRACKING
Usage Form	STANDARDUSAGEFORM	SUBSCRIPTIONBILLING
Vendor Form	STANDARDVENDORFORM	ACCOUNTING
WBS Form	STANDARDWBSFORM	WBS
Bonus Form	STDBONUSFORM	VARIABLE_PAY_TBRBCT
Format Profile Form	STDFORMATPROFILEFORM	—
Student Record Form	STDSTUDENTRECORDFORM	—
Supply Chain Snapshot Simulation Form	STDSUPPLYCHAINSAPSHOTSIMULATIONFORM	SUPPLYCHAINCONTROLTOWER

Example of a Custom Entry Form Object

For project task form, the following feature dependency must be added in the manifest file:

```

1 <dependencies>
2
3   <features>
4
5     <feature required="true">ADVANCEDJOBS</feature>
6
7   </features>
8
9 </dependencies>
```

The following `custform_99` example is a customized entry based on the standard project task form. It is specified for the project task record as a preferred form and includes main fields and action bar buttons.

```

1 <entryForm scriptid="custform_99" standard="STANDARDPROJECTTASKFORM">
2
3   <name>Custom Project Task Form</name>
```

```

4
5 <recordType>PROJECTTASK</recordType>
6
7 <inactive>F</inactive>
8
9 <preferred>T</preferred>
10
11 <storedWithRecord>F</storedWithRecord>
12
13 <mainFields>
14
15   <fieldGroup scriptid="primaryinformation">
16
17     <label>Primary Information</label>
18
19     <visible>T</visible>
20
21     <showTitle>T</showTitle>
22
23     <singleColumn>F</singleColumn>
24
25     <fields position="MIDDLE">
26
27       <field>
28
29         <id>COMPANY</id>
30
31         <label>Project Name</label>
32
33         <visible>T</visible>
34
35         <mandatory>F</mandatory>
36
37         <displayType>NORMAL</displayType>
38
39         <columnBreak>F</columnBreak>
40
41         <sameRowAsPrevious>F</sameRowAsPrevious>
42
43       </field>
44
45       <field>
46
47         <id>PARENT</id>
48
49         <label>Parent Task</label>
50
51         <visible>T</visible>
52
53         <mandatory>F</mandatory>
54
55         <displayType>NORMAL</displayType>
56
57         <columnBreak>F</columnBreak>
58
59         <sameRowAsPrevious>F</sameRowAsPrevious>
60
61       </field>
62
63       <field>
64
65         <id>PRIORITY</id>
66
67         <label>Priority</label>
68
69         <visible>F</visible>
70
71         <mandatory>T</mandatory>
72
73         <displayType>NORMAL</displayType>
74
75         <columnBreak>F</columnBreak>
76

```

```

77         <sameRowAsPrevious>F</sameRowAsPrevious>
78
79     </field>
80
81     <field>
82
83         <id>ORDER</id>
84
85         <label>Insert Before</label>
86
87         <visible>T</visible>
88
89         <mandatory>F</mandatory>
90
91         <displayType>NORMAL</displayType>
92
93         <columnBreak>F</columnBreak>
94
95         <sameRowAsPrevious>F</sameRowAsPrevious>
96
97     </field>
98
99     <field>
100
101         <id>STATUS</id>
102
103         <label>Status</label>
104
105         <visible>T</visible>
106
107         <mandatory>T</mandatory>
108
109         <displayType>NORMAL</displayType>
110
111         <columnBreak>F</columnBreak>
112
113         <sameRowAsPrevious>F</sameRowAsPrevious>
114
115     </field>
116
117     <field>
118
119         <id>CONTACT</id>
120
121         <label>Contact</label>
122
123         <visible>F</visible>
124
125         <mandatory>F</mandatory>
126
127         <displayType>NORMAL</displayType>
128
129         <columnBreak>F</columnBreak>
130
131         <sameRowAsPrevious>F</sameRowAsPrevious>
132
133     </field>
134
135 </fields>
136
137 </fieldGroup>
138
139 <fieldGroup scriptid="projecttaskoverview">
140
141     <label>Project Task Overview</label>
142
143     <visible>T</visible>
144
145     <showTitle>T</showTitle>
146
147     <singleColumn>F</singleColumn>
148
149     <fields position="MIDDLE">

```

```

150
151     <field>
152
153         <id>ESTIMATEDWORK</id>
154
155         <label>Estimated Work</label>
156
157         <visible>T</visible>
158
159         <mandatory>F</mandatory>
160
161         <displayType>NORMAL</displayType>
162
163         <columnBreak>F</columnBreak>
164
165         <sameRowAsPrevious>F</sameRowAsPrevious>
166
167     </field>
168
169     <field>
170
171         <id>ACTUALWORK</id>
172
173         <label>Actual Work</label>
174
175         <visible>T</visible>
176
177         <mandatory>F</mandatory>
178
179         <displayType>NORMAL</displayType>
180
181         <columnBreak>F</columnBreak>
182
183         <sameRowAsPrevious>F</sameRowAsPrevious>
184
185     </field>
186
187     <field>
188
189         <id>REMAININGWORK</id>
190
191         <label>Remaining Work</label>
192
193         <visible>T</visible>
194
195         <mandatory>F</mandatory>
196
197         <displayType>NORMAL</displayType>
198
199         <columnBreak>F</columnBreak>
200
201         <sameRowAsPrevious>F</sameRowAsPrevious>
202
203     </field>
204
205 </fields>
206
207 </fieldGroup>
208
209 </mainFields>
210
211 <tabs>
212
213     <tab>
214
215         <id>EVENTCUSTOM</id>
216
217         <label>Custom</label>
218
219         <visible>T</visible>
220
221         <fieldGroups>
222

```



```

223         <defaultFieldGroup/>
224     </fieldGroups>
225
226     <subItems/>
227
228 </tab>
229
230 </tabs>
231
232 <actionbar>
233
234     <buttons/>
235
236     <menu>
237
238         <menuitem>
239
240             <id>copy</id>
241
242             <label>Copy</label>
243
244             <visible>T</visible>
245
246         </menuitem>
247
248         <menuitem>
249
250             <id>delete</id>
251
252             <label>Delete</label>
253
254             <visible>T</visible>
255
256         </menuitem>
257
258         <menuitem>
259
260             <id>submittedit</id>
261
262             <label>Save & Edit</label>
263
264             <visible>T</visible>
265
266         </menuitem>
267
268     </menu>
269
270 </actionbar>
271
272     <editingInList>T</editingInList>
273
274 </entryForm>
275

```

Custom Code Tab and Actions Tab in XML Definitions

In NetSuite, the Custom Code tab and Actions tab are used together to define custom buttons for custom forms. Custom buttons are a way for your users to intentionally execute client SuiteScript on custom forms. The buttons can be useful for executing a specific function when the user clicks the button on the form, such as creating a customized invoice.

You can add custom buttons to custom forms in SuiteCloud Development Framework (SDF) by specifying the `actionBar` element to define the Action tab. For information about defining `actionBar` in SDF custom objects, see [Custom Buttons on the Actions Tab in XML Definitions](#).

Each custom button runs SuiteScript from the Custom Code tab, which you can define in SDF by specifying the `customCode` element. For information about defining `customCode` in SDF custom objects, see [Custom Code Tab in XML Definitions](#).

The following SDF custom objects support the `customCode` and `actionBar` elements: `addressForm`, `entryForm`, and `transactionForm`. For information about defining those SDF custom objects, see the following topics:

- [Custom Address Forms as XML Definitions](#)
- [Custom Entry Forms as XML Definitions](#)
- [Transaction Forms as XML Definitions](#)

Custom Code Tab in XML Definitions

You can define the custom code tab by specifying the `customCode` element, which contains the `scriptFile` field that references a client SuiteScript 2.x file. The SuiteScript file can be located in the SuiteCloud Project or the File Cabinet for the target NetSuite account. Only client script files are supported for the Custom Code tab on custom forms. After deployment, the SuiteScript file is executed when the user clicks a custom button in the NetSuite UI.

Note: Although you can define the Custom Code tab without defining a custom button, the custom button is required for the user to execute the code as an action.

For information about writing valid SuiteScript for custom buttons, see the help topics [SuiteScript 2.x Client Script Type](#) and [SuiteScript 2.x User Event Script Type](#). Additional information is available in [Associating Custom Code \(Client SuiteScript\) Files With Custom Forms](#).

Important: Only SuiteScript 2.x scripts are supported for the `scriptFile` value in `customCode`. If you import a custom form into your SuiteCloud project and that form contains a `scriptFile` reference to a SuiteScript 1.0 file, the XML for the script file does not appear in the form's XML definition. For information about SuiteScript versions, see the help topic [SuiteScript Versioning Guidelines](#).

When functions are specified in the SuiteScript file, they automatically appear enabled in NetSuite UI after deployment to the target account. The functions are read-only fields and cannot be modified in the XML definition for the custom form. The following client script functions are possible:

- Page Init Function — `pageinitfunction`
- Save Record Function — `saverecordfunction`
- Validate Field Function — `validatefieldfunction`
- Field Changed Function — `fieldchangedfunction`
- Post Sourcing Function — `postsourcingfunction`
- Line Init Function — `lineinitfunction`
- Validate Line Function — `validatelinefunction`
- Validate Insert Function — `validateinsertfunction`
- Validate Delete Function — `validatedeletefunction`
- Recalc Function — `recalcfunction`

The following example defines the Custom Code tab by specifying the `customCode` element, which contains the `scriptFile` field that references the client SuiteScript file `invoice.js`. The `invoice.js` file can contain functions used by custom buttons.

```

1 <customCode>
2
3   <scriptFile>[/SuiteScript/invoice.js]</scriptFile>
4
5 </customCode>
```

The `customCode` element can be added to the following forms: `addressForm`, `entryForm`, and `transactionForm`

Custom Buttons on the Actions Tab in XML Definitions

You can add custom buttons to a custom form by defining the Actions tab using the `actionBar` element. Each custom button is defined by a `customButton` element, by specifying both a label and function. The `label` field is the name of the button in the NetSuite UI, and the `function` field is the function the button calls from the client SuiteScript file that is specified in the `scriptFile` field on the same custom form.

Note: For custom buttons to run the specified function, the function must exist in the client SuiteScript file that is defined on the Custom Code subtab on the same form. For information about defining the Custom Code subtab, see [Custom Code Tab in XML Definitions](#).

Consider the following when working with custom buttons:

- For the Custom Actions subtab to appear on the Custom Form tab after deployment, the Client SuiteScript feature must be enabled in the target account.
- If you import a custom form into your SuiteCloud project and that form contains a custom button without a `scriptFile` reference, the buttons are still defined in the form's XML definition. However, for the buttons to function on the form, the `scriptFile` reference must be added to the form's XML definition. For information about defining custom buttons on custom forms using the NetSuite UI, see the help topic [Working with Custom Buttons](#).

The following example defines two custom buttons. The first button has the label Invoice Client, and runs the function `invoiceClient`. The second button has the label Generate Report, and runs the function `generateReport`. To execute, both functions must be defined in the client SuiteScript file referenced by the `scriptFile` field for the `customCode` element in the same custom form object.

```

1 <actionBar>
2
3   <customButtons>
4
5     <customButton>
6
7       <label>Invoice Client</label>
8
9       <function>invoiceClient</function>
10
11     </customButton>
12
13     <customButton>
14
15       <label>Generate Report</label>
16
17       <function>generateReport</function>
18
19     </customButton>
20
21   </customButtons>
22
23 </actionBar>

```

The `actionBar` element can be added to the following forms: `addressForm`, `entryForm`, and `transactionForm`

Transaction Forms as XML Definitions

Users enter transactions in NetSuite using transaction forms. You can create your own custom transaction forms by customizing an existing standard transaction form. That customization may include:

rearranging and renaming fields and subtabs, hiding or disabling certain fields, hiding or renaming buttons, making specific fields required.

You can create custom transaction forms in the NetSuite application and further customize them as transactionForm objects in SuiteCloud Development Framework (SDF). Custom transaction forms cannot be created in SDF. To add custom transaction forms in your SuiteCloud project, see [Adding Custom Transaction Forms to Your SuiteCloud Project](#).

Currently SDF supports custom forms for a subset of transaction types, those that are most frequently used. For the list of supported transaction forms, see [Supported Custom Transaction Form Objects](#).

Review the following information about working with custom transaction forms in SDF:

- [Supported Custom Transaction Form Objects](#)
- [Adding Custom Transaction Forms to Your SuiteCloud Project](#)
- [Custom Transaction Form Use Cases](#)
- [Custom Transaction Form Format](#)
- [Custom Transaction Form Validation](#)
- [Limitations for Custom Transaction Forms](#)
- [Custom Transaction Form Example](#)
- [Advanced HTML/PDF Templates as XML Definitions](#)
- [Saved CSV Import Maps as XML Definitions](#)

Benefits for Working with Custom Transaction Forms

Following are some of the benefits of working with custom transaction forms in SDF:

- You can create a custom transaction form in a sandbox account, modify that form in SDF, and deploy that single form to a production account.
- You can quickly customize multiple custom transaction forms and see the changes immediately after deploying the objects to a target NetSuite account. You can continue to make further changes, quickly iterating through the process as many times as necessary.
- You can deploy the layout of a form, even when some custom fields are not yet available. Adding the fields later does not negatively impact the layout of the form. You can treat custom tabs and custom sublists similarly.
- When a custom tab exists in both the source account and target account, you can include the form by itself with a reference to the custom tab.

For reference documentation, see the help topic [transactionForm](#).

Supported Custom Transaction Form Objects

The following table lists the kinds of transaction forms that you can add to your SuiteCloud project. You can further edit the custom transaction form in SDF as needed. Currently you cannot work with custom forms for other transaction types in SDF.

Supported Object	Transaction Record Type		Standard Transaction Form	
	NetSuite	SDF	NetSuite	SDF
Blanket Purchase Order	Blanket Purchase Order	BLANKETPURCHASEORDER	Standard Blanket Purchase Order	STANDARDBLANKETPURCHASEORDER
Bill Payments	Bill Payment	VENDORPAYMENT	Standard Bill Payment	STANDARDBILLPAYMENT
Cash Refunds	Cash Refund	CASHREFUND	Standard Cash Refund	STANDARDCASHREFUND
Cash Sales	Cash Sale	CASHSALE	Standard Cash Sale	STANDARDCASHSALE
Checks	Check	CHECK	Standard Check	STANDARDCHECK

Supported Object	Transaction Record Type		Standard Transaction Form	
	NetSuite	SDF	NetSuite	SDF
Credit Card Charge	Standard Credit Card Charge	STANDARDCREDITCARDCHARGE	Credit Card	CREDITCARDCHARGE
Credit Memo	Credit Memo	CREDITMEMO	Standard Credit Memo	STANDARDCREDITMEMO
Customer Deposit	Customer Deposit	CUSTOMERDEPOSIT	Standard Customer Deposit	STANDARDCUSTOMERDEPOSIT
Customer Payments	Payment	CUSTOMERPAYMENT	Standard Customer Payment	STANDARDCUSTOMERPAYMENT
Customer Refund	Customer Refund	CUSTOMERREFUND	Standard Customer Refund	STANDARDCUSTOMERREFUND
Deposit	Deposit	DEPOSIT	Standard Deposit	STANDARDDEPOSIT
Estimates	Estimate	ESTIMATE	Standard Estimate	STANDARDESTIMATE
Expense Report	Expense Report	EXPENSEREPORT	Standard Expense Report	STANDARDEXPENSEREPORT
Fulfillment Request	Fulfillment Request	FULFILLMENTREQUEST	Standard Fulfillment Request	STANDARDFULFILLMENTREQUEST
Fulfillment Request Picking Ticket	Fulfillment Request Picking Ticket	FULFILLMENTREQUESTPICKINGTICKET	Standard Fulfillment Request Picking Ticket	STANDARDFULFILLMENTREQUESTPICKINGTICKET
Inventory Adjustment	Inventory Adjustment	INVENTORYADJUSTMENT	Standard Inventory Adjustment	STANDARDINVENTORYADJUSTMENT
Inventory Count	Inventory Count	INVENTORYCOUNT	Standard Inventory Count	STANDARDINVENTORYCOUNT
Inventory Transfer	Inventory Transfer	INVENTORYTRANSFER	Standard Inventory Transfer	STANDARDINVENTORYTRANSFER
Inventory Worksheet	Inventory Worksheet	INVENTORYWORKSHEET	Standard Inventory Worksheet	STANDARDINVENTORYWORKSHEET
Item Fulfillment	Item Fulfillment	ITEMFULFILLMENT	Standard Item Fulfillment	STANDARDITEMFULFILLMENT
Item Receipt	Item Receipt	ITEMRECEIPT	Standard Item Receipt	STANDARDITEMRECEIPT
Invoices	Invoice	INVOICE	Standard Progress Invoice Standard Service Invoice Standard Product Invoice Standard Professional Invoice Standard Finance Charge	STANDARDPROGRESSINVOICE STANDARDSERVICEINVOICE STANDARDPRODUCTINVOICE STANDARDPROFESSIONALINVOICE STANDARDFINANCECHARGE
Journal Entries	Journal	ADVINTERCOMPANYJOURNALENTRY	Standard Journal Entry	STANDARDJOURNALENTRY
Online Payments	Payment (External)	CUSTOMERPAYMENT	Standard Online Payment	STANDARDONLINEPAYMENT
Opportunity	Opportunity	OPPORTUNITY	Standard Opportunity	STANDARDOPPORTUNITY
Packing Slip	Packing Slip	PACKINGSLIP	Standard Packing Slip	STANDARDPACKINGSLIP
Picking Ticket	Picking Ticket	PICKINGTICKET	Standard Picking Ticket	STANDARDPICKINGTICKET
Purchase Contract	Purchase Contract	PURCHASECONTRACT	Standard Purchase Contract	STANDARDPURCHASECONTRACT
Purchase Orders	Purchase Order	PURCHASEORDER	Standard Purchase Order Standard Drop Ship Purchase Order	STANDARDPURCHASEORDER STANDARDDROPSHIPPURCHASEORDER
Requisition	Requisition	REQUISITION	Standard Requisition	STANDARDREQUISITION
Return Authorization - Cash	Return Authorization	RETURNAUTHORIZATIONCASH	Standard Return Authorization - Cash	STANDARDRETURNAUTHORIZATIONCASH
Return Authorization - Credit	Return Authorization	RETURNAUTHORIZATIONCREDIT	Standard Return Authorization - Credit	STANDARDRETURNAUTHORIZATIONCREDIT
Return Request - Cash	Return Authorization (External)	RETURNREQUESTCASH	Standard Return Request - Cash	STANDARDRETURNREQUESTCASH
Return Request - Credit	Return Authorization (External)	RETURNREQUESTCREDIT	Standard Return Request - Credit	STANDARDRETURNREQUESTCREDIT
Revenue Arrangement	Revenue Arrangement	REVENUEARRANGEMENT	Standard Revenue Arrangement	STANDARDREVENUEARRANGEMENT
Sales Orders	Sales Order	SALESORDER	Standard Sales Order Standard Sales Order - Progress Billing	STANDARDSALESORDER STANDARDSALESORDERPROGRESSBILLING

Supported Object	Transaction Record Type		Standard Transaction Form	
	NetSuite	SDF	NetSuite	SDF
			Standard Sales Order - Cash Sale	STANDARDSALESORDERCASHSALE
			Standard Sales Order - Invoice	STANDARDSALESORDERINVOICE
			Standard Service Invoice	STANDARDSERVICEINVOICE
	Sales Order External	SALESORDER	Standard Online Order - Invoice	STANDARDONLINEORDERINVOICE
			Standard Online Order - Cash Sale	STANDARDONLINEORDERCASHSALE
Transfer Order	Transfer Order	TRANSFERORDER	Standard Transfer Order	STANDARDTRANSFERORDER
Vendor Bill	Bill	VENDORBILL	Standard Vendor Bill	STANDARDVENDORBILL
Vendor Credit	Bill Credit	VENDORCREDIT	Standard Vendor Credit	STANDARDVENDORCREDIT
Vendor Return Authorization	Vendor Return Authorization	VENDORRETURNAUTHORIZATION	Standard Vendor Return Authorization	STANDARDVENDORRETURNAUTHORIZATION
Work Order	Work Order	WORKORDER	Standard Work Order	STANDARDWORKORDER

Adding Custom Transaction Forms to Your SuiteCloud Project

The following process describes how to work with custom transaction forms in SuiteCloud Development Framework (SDF) and add custom transaction forms to your SuiteCloud project:

- Create custom transaction forms in a NetSuite account one of the following ways:
 - Customize a transaction form in a NetSuite account. See the help topics [Creating Custom Entry and Transaction Forms](#) and [Supported Custom Transaction Form Objects](#).
 - Customize a transaction form in a NetSuite account using SuiteCloud IDE plug-in.
- Import the custom transaction forms from a NetSuite account into a SuiteCloud project.

See [Account Component Imports to SuiteCloud Projects](#).

For CLI for Java, see the help topic [importobjects](#).

For CLI for Node.js, see the help topic [object:import](#).

You can also import custom transaction forms as part of bundles. See [Conversion of a Bundle to an Account Customization Project for SDF](#).
- Customize the custom transaction form objects in SuiteCloud IDE plug-in, or a third-party IDE and SuiteCloud CLI. For the list of available fields, see the help topic [transactionForm](#).
- Validate the objects and then address errors and warnings that are returned in the validation log.

See [SuiteCloud Project Validation](#) and [Custom Transaction Form Validation](#).
- Deploy the SuiteCloud project to a target NetSuite account.

See [Deployment Logs for SuiteCloud Projects](#) and [Publishing an SDF SuiteApp](#).

Custom Transaction Form Use Cases

The following table includes the order of tasks required for some common edits you could make to custom transaction forms in SuiteCloud Development Framework (SDF).

Use Case	Details
Hide fields in a form used in another account.	- Create a custom transaction form in a NetSuite account. - Import that form into a SuiteCloud project.

Use Case	Details
	3. Hide some fields. 4. Generate dependencies in the manifest. 5. Deploy form object to a different NetSuite account.
Change field labels and reorder the fields on the form in a NetSuite account.	1. Create a custom transaction form in a NetSuite account. 2. Import that form into a SuiteCloud project. 3. Change labels on fields. 4. Reorder fields in the same field group. 5. Deploy form object to the same NetSuite account.
Include custom fields that are dependent on specific features being enabled in a NetSuite account.	1. Create a custom transaction form in NetSuite account. 2. Import that form into a SuiteCloud project. 3. Include custom fields. 4. Enable features needed for the custom fields. 5. Deploy form object to the same NetSuite account.
Set specific fields required on the form in another account.	1. Create a custom transaction form in NetSuite account. 2. Import that form into a SuiteCloud project. 3. Set some fields to be required (using the <code><mandatory></code> field). 4. Deploy form object to a different NetSuite account.

Custom Transaction Form Format

SuiteCloud Development Framework (SDF) requires the following format for custom transaction form objects:

- XML body fields or elements in forms must follow the order as exported.
- A field can be deleted if it is not required.
- Field order can only be changed within the same position. Field position is controlled by the position attribute. Positions are top, middle, and bottom.
- The scriptid attribute is lowercase.
- The reference to a standard transaction form is uppercase and in the following format:

```

1 <transactionForm scriptid="scriptid" standard="STANDARDFORM">
2
3   ...
4
5 </transactionForm>
```

The following is an example:

```

1 <transactionForm scriptid="custform_18_mstrwlf_612" standard="STANDARDONLINEORDERINVOICE">
2
```

```

3 | ...
4 |
5 | </transactionForm>

```

You can also define the Custom Code tab and Actions tab in custom transaction form objects. For more information, see [Custom Code Tab and Actions Tab in XML Definitions](#)

For more information about custom transaction form format, see the reference documentation for `transactionForm` in .

Custom Transaction Form Validation

SuiteCloud Development Framework (SDF) includes the following validations for custom transaction forms:

- The record type of the form is validated.
- Several fields and attributes are required for custom transaction forms. For more information, see the reference documentation for `transactionForm` in .
- Components of the form are validated. For example, the validation checks whether the field ID is associated with a feature. The validation returns an error if a field is controlled by a feature that is not enabled in the target account. The validation log states that the fields are either invalid or do not exist in the account. If the feature cannot be enabled in the account, the fields must be removed from the custom transaction form.
- When the default value for a field in the referenced standard transaction form is different between accounts, a validation error is returned. For example, the default value for a field is true in the referenced standard transaction form in source account but false in the target account. If the field value is read-only and must be included, the default value must be changed on the target account.
- Some validation only occurs when the object is deployed to a target NetSuite account, and not when you validate the project against a target NetSuite account. The following examples cause errors on deployment:
 - The original field has changed position. For example, the field has changed from top to bottom position.
 - The original field was in the top or bottom position, and was moved to another tab.
 - The original field is in the top or bottom position, and `spaceBefore` or `sameAsPrevious` were modified from their default values.
 - There are changes to a custom transaction form that determine the availability of certain tabs. For example, the `allowattachments` field or `enableemailmerge` field of the `custom_record_scriptid` object cannot be changed to F when it uses the tab with the `recordmedia` ID.
- For each role field specified in the `transactionForm` SDF custom object, the `id` field value must be for a role that has sufficient permissions for the transaction form.

For more information about resolving SDF validation and deployment errors, see [SuiteCloud Development Framework Error Handling](#).

Additional information is also available in [SuiteCloud Project Validation](#).

Limitations for Custom Transaction Forms

Consider the following things when customizing transaction forms in SuiteCloud Development Framework (SDF):

- Configuring custom PDF layouts in SDF is not currently supported for custom transaction forms. To customize PDF layouts for custom transaction forms, see the help topic [Customizing Transaction Form PDF Layouts](#).
- Only supported custom transaction forms appear in the list when you are importing objects from a NetSuite account into your SuiteCloud project.
- Identify fields that are not supported in target accounts and remove them.
- Ensure that all dependencies are in the manifest file.
- Renaming a form, and then deploying that form to an account that contains the original form, creates a new form. The original form is not overwritten by the renamed form.
- Only Checks and Journal Entries contain the `storedWithRecord` element in the SDF custom object XML definition. The `storedWithRecord` element can be set to T or F. For more information, see the help topic [Storing Custom Forms with Transactions](#).
- When the original field is in the top or bottom position, the field's position cannot be changed, the field cannot be moved between tabs, and the default values for `spaceBefore` and `sameAsPrevious` cannot be modified.

Known Limitations of Custom Form Translations in SDF

The following are known limitations of custom form translations in SDF:

- When exporting from your source account and importing a form into your SuiteCloud project, the button label values are not included for custom forms. Currently, translation collections do not support button labels.
- To export to the SDF XML definition or import the SDF custom object into a SuiteCloud project, field labels must contain a value. If a field label has no value, validation of the SDF XML definition fails during installation. If you want to include a field that does not have a label, you can work around the SDF limitation by inserting a single space character as the translation for a field label. For example, you have fields for first name, middle name, and last name, but you only want a field label of Name to appear for the first field. You enter a label for the Name field and enter a single space for the other two fields.
- Importing custom forms into SDF:
 - When importing a form from the source account into your SuiteCloud project, the form translation is defined using the company default language, not the language of the user importing the form.
 - When deploying a custom form using SDF, the target account receives the terms as they exist in the source account. If the target account does not have translations for the default language used in the source account, then the default language in the translation collection is used.

Linking Custom Transaction Forms

A specific transaction form can be used in SuiteCloud Development Framework (SDF) when one transaction is transformed into another transaction. For the list of valid transformations, see the help topic [Supported Transformation Types](#).

You can control which custom transaction form is used by defining the `linkedForms` element on a `transactionForm` SDF custom object. The `linkedForms` element corresponds to the Linked Forms subtab in the NetSuite UI.

Linking transaction forms as chains is a way to mirror your business workflow. For information about linking transaction forms in NetSuite, see the help topic [Linking Transaction Forms](#).

To link custom transaction forms in SDF, specify the following fields for each `linkedForm` field group:

- **form** - specifies the form, which can be one of the following:

- A standard transaction form. For example:

```
1 | <form>STANDARDPURCHASEORDER</form>
```

For a list of possible values, see the help topic [transactionform_standard](#).

- A reference to the script ID for a custom transaction form. For example:

```
1 | <form>[scriptid=custform_purchaseorder1]</form>
```

- **type** - specifies the transaction type for the form, which can be one of the following:

- The transaction type for the custom transaction form. For example:

```
1 | <type>PURCHASE_ORDER_VENDOR_RETURN_AUTHORIZATION_TRANSACTION</type>
```

For a list of possible values, see the help topic [transactionform_trantype](#).

- A reference to the script ID for a custom transaction type. For example:

```
1 | <type>[scriptid=custtransaction_purchase1]</type>
```

For information about the available fields a custom transaction type, see the help topic [customtransactiontype](#).

The following code sample defines linked transaction forms in a custom purchase order:

```
1 | <transactionForm scriptid="custform_10336" standard="STANDARDPURCHASEORDER">
2 |
3 |   <name>Custom Purchase Order</name>
4 |
5 |   <recordType>PURCHASEORDER</recordType>
6 |
7 |   ...
8 |
9 |   <linkedForms>
10 |
11 |     <linkedForm>
12 |
13 |       <type>PURCHASE_ORDER_VENDOR_RETURN_AUTHORIZATION_TRANSACTION</type>
14 |
15 |       <form>STANDARDPURCHASEORDER</form>
16 |
17 |     </linkedForm>
18 |
19 |     <linkedForm>
20 |
21 |       <type>PURCHASE_ORDER_ITEM_RECEIPT_TRANSACTION</type>
22 |
23 |       <form>[scriptid=custform_purchaseorder1]</form>
24 |
25 |     </linkedForm>
26 |
27 |   </linkedForms>
28 |
29 |
30 |
31 | </transactionForm>
```

The preceding code sample defines a `linkedForms` field group that specifies NetSuite to:

- Use the `STANDARDPURCHASEORDER` standard transaction form when converting a `PURCHASEORDER` transaction to a `PURCHASE_ORDER_VENDOR_RETURN_AUTHORIZATION_TRANSACTION` transaction.

- Use the `custform_purchaseorder1` custom transaction form when converting a `PURCHASEORDER` transaction to a `PURCHASE_ORDER_ITEM_RECEIPT_TRANSACTION` transaction.

Custom Transaction Form Field Examples

The following sections provide XML examples for custom transactionForm SDF custom object fields in SuiteCloud Development Framework (SDF):

- [Email Message Template Field](#)
- [Never Empty Field](#)
- [Roles Field](#)
- [Space Before Field](#)
- [Total Box Field](#)

Email Message Template Field

The Email Message Template field is defined by `emailMessageTemplate` in the XML definition. It specifies which transaction email template to use for the transaction type, when sending transaction emails with a PDF attachment.

The following is an XML definition example of an `emailMessageTemplate` field that references an email message template with the scripid `custemailtmpl_sales`.

```

1 <transactionForm scriptid="custform_sales1" standard="STANDARDSALESORDER">
2
3   <name>Sales Order 1</name>
4
5   <recordType>SALESORDER</recordType>
6
7   ...
8
9   <emailMessageTemplate>[scriptid=custemailtmpl_sales]</emailMessageTemplate>
10
11   ...
12
13 </transactionForm>

```

Never Empty Field

The Never Empty field is defined by `neverEmpty` in the XML definition. It specifies that at least one row must be entered for the sublist by the user when `neverEmpty` is set to T.

The following is an XML definition example of a `neverEmpty` field that is set to true:

```

1 <transactionForm scriptid="custform_sales1" standard="STANDARDSALESORDER">
2
3   <name>Sales Order 1</name>
4
5   <recordType>SALESORDER</recordType>
6
7   <inactive>F</inactive>
8
9   <preferred>T</preferred>
10
11   <mainFields>
12

```

```

13 ....
14
15     </mainFields>
16
17     <tabs>
18
19         <tab>
20
21             <id>TRANSACTIONITEMS</id>
22
23             <label>Items</label>
24
25             <visible>T</visible>
26
27             <fieldGroups>
28
29                 <defaultFieldGroup>
30
31                     <fields position="MIDDLE">
32
33 ...
34
35                     </fields>
36
37                 </defaultFieldGroup>
38
39             </fieldGroups>
40
41             <subItems>
42
43                 <subList>
44
45                     <id>ITEM</id>
46
47                     <neverEmpty>T</neverEmpty>
48
49                     <columns>
50
51 ...
52
53                     </columns>
54
55
56
57                 </subList>
58
59             </subItems>
60
61         </tab>
62
63 ...
64
65 </transactionForm>

```

In the preceding example, the `neverEmpty` is set to `T` for the `ITEM` sublist, meaning that the sublist must have a value set by the user entering data into the form.

Roles Field

The `Roles` field is defined by `roles` in the XML definition. It specifies the custom transaction form as the preferred form for specific roles. For each role field specified in the `transactionForm` SDF custom object, the corresponding `id` field value must be for a role that has sufficient permissions for the transaction form.

The following is an XML definition example of a `roles` field that defines two roles:

```

1 <transactionForm scriptid="custform_sales1" standard="STANDARD_SALESORDER">
2

```

```

3      <name>Sales Order 1</name>
4
5      <recordType>SALESORDER</recordType>
6
7      ...
8
9      <roles>
10
11         <role>
12
13             <id>[scriptid=CUSTOMROLE_1013]</id>
14
15             <preferred>false</preferred>
16
17         </role>
18
19         <role>
20
21             <id>WAREHOUSE_MANAGER</id>
22
23             <preferred>false</preferred>
24
25         </role>
26     </roles>
27
28     ...
29
30
31 </transactionForm>

```

In the preceding example, the `custform_sales1` form is the preferred custom transaction form for the `WAREHOUSE_MANAGER` role and the custom role referenced by the `scriptid` `CUSTOMROLE_1013` when performing a `SALESORDER` transaction.

Space Before Field

The Space Before field is defined by `spaceBefore` in the XML definition. It specifies to include a blank line before a field when `spaceBefore` is set to an integer value.

The following is an XML definition example of `spaceBefore` set to 1 for the `CUSTOMFORM` field:

```

1  <transactionForm scriptid="custform_demo" standard="STANDARDSALESORDER">
2
3      <name>Sales Order 1</name>
4
5      <recordType>SALESORDER</recordType>
6
7      <inactive>F</inactive>
8
9      <preferred>T</preferred>
10
11      <mainFields>
12
13          <fieldGroup scriptid="primaryinformation">
14
15              <label>Primary Information</label>
16
17              <visible>T</visible>
18
19              <showTitle>T</showTitle>
20
21              <singleColumn>F</singleColumn>
22
23              <fields position="MIDDLE">
24
25                  <field>
26
27                      <id>CUSTOMFORM</id>

```

```

28     <label>Custom Form</label>
29
30     <visible>T</visible>
31
32     <mandatory>T</mandatory>
33
34     <displayType>NORMAL</displayType>
35
36     <columnBreak>F</columnBreak>
37
38     <spaceBefore>1</spaceBefore>
39
40     <sameRowAsPrevious>F</sameRowAsPrevious>
41
42     </field>
43
44     ...
45
46     </fields>
47
48     </fieldGroup>
49
50     </mainFields>
51
52     ...
53
54     </transactionForm>
55

```

Total Box Field

The Total Box field is defined by `totalBox` in the XML definition. It specifies the fields that you want to show in the form totals.

The following is an XML definition example of a `totalBox` field:

```

1  <transactionForm scriptid="custform_sales1" standard="STANDARDSALESORDER">
2
3     <name>Sales Order 1</name>
4
5     <recordType>SALESORDER</recordType>
6
7     ...
8
9     <totalBox>
10
11         <totalBoxField>
12
13             <id>TOTAL</id>
14
15             <label>Total</label>
16
17             <visible>T</visible>
18
19         </totalBoxField>
20
21         <totalBoxField>
22
23             <id>SUBTOTAL</id>
24
25             <label>Subtotal</label>
26
27             <visible>T</visible>
28
29         </totalBoxField>
30
31     </totalBox>
32
33     ...
34

```

```
35 | </transactionForm>
```

Custom Transaction Form Example

The following is an XML definition example of a custom sales order custom transaction form object.

```
1  <transactionForm scriptid="custform_10_mstrwlf_335" standard="STANDARDSALESORDER">
2
3  <name>Custom Sales Order</name>
4
5  <recordType>SALESORDER</recordType>
6
7  <inactive>F</inactive>
8
9  <preferred>F</preferred>
10
11
12
13  <mainFields>
14
15
16
17    <fieldGroup scriptid="primaryinformation">
18
19      <label>Primary Information</label>
20
21      <visible>T</visible>
22
23      <showTitle>T</showTitle>
24
25      <singleColumn>F</singleColumn>
26
27      <fields position="MIDDLE">
28
29    </fieldGroup>
30
31
32
33    <fieldGroup scriptid="salesinformation">
34
35      <label>Sales Information</label>
36
37      <visible>T</visible>
38
39      <showTitle>T</showTitle>
40
41      <singleColumn>F</singleColumn>
42
43      <fields position="MIDDLE">
44
45        <!--List of fields is located here-->
46
47    </fieldGroup>
48
49
50
51    <fieldGroup scriptid="classification">
52
53      <label>Classification</label>
54
55      <visible>T</visible>
56
57      <showTitle>T</showTitle>
58
59      <singleColumn>F</singleColumn>
60
61      <fields position="MIDDLE">
62
63        <!--List of fields is located here-->
64
```

```

65     </fieldGroup>
66
67
68
69     <fieldGroup scriptid="intercompanymanagement">
70
71         <label>Intercompany Management</label>
72
73         <visible>T</visible>
74
75         <showTitle>T</showTitle>
76
77         <singleColumn>F</singleColumn>
78
79         <fields position="MIDDLE">
80
81     </fieldGroup>
82
83
84
85 <defaultFieldGroup>
86
87 <!--List of fields is located here-->
88
89 </defaultFieldGroup>
90
91
92
93 </mainFields>
94
95 <tabs>
96
97 </tabs>
98
99 <quickViewFields>
100
101 <!--List of fields is located here-->
102
103 </quickViewFields>
104
105 <buttons>
106
107 </buttons>
108
109 <disclaimer/>
110
111 <address/>
112
113 <allowAddMultiple>T</allowAddMultiple>
114
115 <printingType>
116
117 </printingType>
118
119 </transactionForm>

```

Sublists as XML Definitions

You can add a custom sublist to custom forms in SuiteCloud Development Framework (SDF). Custom sublists present information related to the record your user is viewing in NetSuite. The sublist is based on results from a saved search of the form record type or a related record type. For more information, see the help topic [Custom Sublists](#).

SDF supports the following sublist types: CRM, Entity, Item, Transaction.

The scriptid attribute is required for the sublist root element. The default scriptid value is custsublist.

The following fields are required for custom sublists:

Field	Description
label	Specifies the label for the sublist. The label is the text that your users see as the name of the sublist when it is displayed on the form.
savedsearch	Specifies the saved search ID. For example: <pre>1 <savedsearch>[scriptid=customsearch54]</savedsearch></pre> A field is required to filter the saved search. For example: <pre>1 <field>STDEVENTCASESTATUS</field></pre>
sublisttype	Specifies the type of sublist. Possible values are CRM, ENTITY, ITEM, TRANSACTION. The sublisttype value cannot be changed after the sublist is created.

For information about all of the available fields, see the help topic [sublist](#).



Important: Each sublist object requires a reference to a savedsearch object and a field to filter the saved search. If the referenced savedsearch object is not included in your SuiteCloud project, ensure the manifest file is updated with the dependency. If your SuiteCloud project is a SuiteApp project, it must include the savedsearch object. Only account customization projects can exclude the savedsearch object from the SuiteCloud project.

If you are including the savedsearch object in your SuiteCloud project, the XML must be generated by NetSuite. Do not edit the savedsearch object in SDF.

CRM Custom Sublist Example

The following is an XML definition example of a custom CRM sublist object. When deployed to an account, a sublist called Cases to Address is displayed on the Related Info tab of forms for case records. The saved search customsearch54 is filtered by the field STDEVENTCASESTATUS.

```

1 <sublist scriptid="custsublist_casesaddress">
2
3   <call>F</call>
4
5   <campaign>F</campaign>
6
7   <case>T</case>
8
9   <event>F</event>
10
11  <field>STDEVENTCASESTATUS</field>
12
13  <issue>F</issue>
14
15  <label>Cases to Address</label>
16
17  <savedsearch>[scriptid=customsearch54]</savedsearch>
18
19  <solution>F</solution>
20
21  <sublisttype>CRM</sublisttype>
22
23  <tab>CRMRELATEDINFO</tab>
24
25  <task>F</task>
26
27 /sublist>

```

Entity Custom Sublist Example

The following is an XML definition example of a custom Entity sublist object. When deployed to an account, a sublist called Customers to Call is displayed on the Marketing tab of forms for customer records. The saved search customsearch20 is filtered by the field STDENTITYCUSTTYPE.

```

1 <sublist scriptid="custsublist_phonecalls">
2
3   <contact>F</contact>
4
5   <customer>T</customer>
6
7   <employee>F</employee>
8
9   <field>STDENTITYCUSTTYPE</field>
10
11  <job>F</job>
12
13  <label>Customers to Call</label>
14
15  <partner>F</partner>
16
17  <savedsearch>[scriptid=customsearch20]</savedsearch>
18
19  <sublisttype>ENTITY</sublisttype>
20
21  <tab>ENTITYMARKETING</tab>
22
23  <vendor>F</vendor>
24
25 </sublist>

```

Item Custom Sublist Example

The following is an XML definition example of a custom Item sublist object. When deployed to an account, a sublist called Open Project Expenses is displayed on the Purchasing/Inventory tab of forms for othercharge records. The saved search customsearch51 is filtered by the field STDITEMPROJECTEXPENSETYPE.

```

1 <sublist scriptid="custsublist_projectexpenses">
2
3   <billofmaterials>F</billofmaterials>
4
5   <field>STDITEMPROJECTEXPENSETYPE</field>
6
7   <inventoryitem>F</inventoryitem>
8
9   <itemgroup>F</itemgroup>
10
11  <kit>F</kit>
12
13  <label>Open Project Expenses</label>
14
15  <noninventoryitem>F</noninventoryitem>
16
17  <othercharge>T</othercharge>
18
19  <savedsearch>[scriptid=customsearch51]</savedsearch>
20
21  <service>F</service>
22
23  <sublisttype>ITEM</sublisttype>
24
25  <tab>ITEMPURCHASINGINVENTORY</tab>
26
27 </sublist>

```

Transaction Custom Sublist Example

The following is an XML definition example of a custom Transaction sublist object. When deployed to an account, a sublist called Recent Payments is displayed on the Payment tab of forms for payment records. The saved search customsearch26 is filtered by the field STDBODYPAYMENTMETHOD.

```

1 <sublist scriptid="custsublist_recentpayments">
2
3   <assemblybuild>F</assemblybuild>
4
5   <deposit>F</deposit>
6
7   <expense report>F</expense report>
8
9   <field>STDBODYPAYMENTMETHOD</field>
10
11  <inventoryadjustment>F</inventoryadjustment>
12
13  <itemfulfillment>F</itemfulfillment>
14
15  <itemreceipt>F</itemreceipt>
16
17  <journal>F</journal>
18
19  <label>Recent Payments</label>
20
21  <opportunity>F</opportunity>
22
23  <payment>T</payment>
24
25  <purchase>F</purchase>
26
27  <sale>F</sale>
28
29  <savedsearch>[scriptid=customsearch26]</savedsearch>
30
31  <sublisttype>TRANSACTION</sublisttype>
32
33  <tab>TRANSACTIONPAYMENT</tab>
34
35  <vendorpayment>F</vendorpayment>
36
37 </sublist>

```

For more information about custom sublists in SDF, see the following topics:

- [Adding Custom Sublists to Forms](#)
- [Validating Custom Sublists](#)

Adding Custom Sublists to Forms

Sublists are automatically displayed in forms that correspond to the record types specified in the sublist object. For example, specifying a T value for the payment field displays the sublist on all forms for the payment record type. You do not need to modify a form to display the sublist.

Consider the following information regarding sublists and forms:

- The sublist only displays on a form when that form is used by your user to create or edit a record. The sublist does not display on the form when the form is being edited in a NetSuite account. The sublist does not exist in the XML definition of the form object.
- The tab element determines the subtab location for the sublist. For example, to display the sublist on a form's Communication tab, specify a tab value of TRANSACTIONCOMMUNICATION if the sublisttype value is TRANSACTION.
- The field element is required for the sublist to display on forms. If the field element is missing, the sublist passes validation but it does not display on the intended forms.



Tip: To assist with testing the sublist in a NetSuite account, you can enable the **Show ID Field on Sublists** preference to display the sublist ID in the form. For more information, see [SuiteCloud Development Framework Account Preferences \(SDF Developers Only\)](#).

Validating Custom Sublists

SuiteCloud Development Framework (SDF) includes the following validations for custom sublists:

- The following fields are validated: field, label, savedsearch, sublisttype, tab.
- The field value that is used to filter the saved search is valid if that field exists for the saved search, either in your SuiteCloud project or the target NetSuite account.
- The tab value is valid if it exists in the target NetSuite account.
- Saved search validation happens when the saved search is located in the target NetSuite account and referenced in the SuiteCloud project manifest. If the saved search is an XML object in your SuiteCloud project and does not exist in the target NetSuite account, the validation only occurs on deployment.
- The scriptid attribute on the sublist root element is validated.
- On deployment, the Boolean fields that represent record types are changed to false if the sublist type does not support that record type.

Subtabs as XML Definitions

Create and manage subtabs for transaction, entity, CRM, and item records in SuiteCloud Development Framework (SDF). You can model a subtab definition in a SuiteCloud project and deploy the project to a target NetSuite account. You can either create new subtabs in your project or import existing subtabs from a NetSuite account. For more information about working with subtabs in a NetSuite account, see the help topic [Creating Custom Subtabs](#).

A subtab object is used to organize custom fields on your transaction, entity, CRM, and item records. When you create a custom subtab, you must assign it a custom field. For each custom subtab, you can define an existing subtab as the parent subtab. This parent-child definition enables you to include an additional layer of information for your subtab categories.

A subtab object is comprised of the following:

- **Tab type** – You can set this element to any of the following values:
 - **Transaction** – Subtabs for cash refund, cash sale, credit memo, estimate, invoice, opportunity, purchase order, return authorization, sales order, and other transaction records.
 - **Entity** – Subtabs for customer, project, vendor, employee, other name, contact, partner, and group records.
 - **Item** – Subtabs for inventory, non-inventory, group, other charge, assembly/bill of materials, kit/package, service item and other item records.
 - **CRM** – Subtabs for task, phone call, event, case, campaign, solution, and other CRM records.
- **Title field** – You must enter the name for the subtab in the Title field.
- **Parent field** – Optionally, you can designate the subtab as a child of an existing subtab in the Parent field by creating a reference to the subtab.



Note: You cannot create a child-grandchild relationship. A custom subtab is not available to be selected as a parent for another custom subtab after you define it as a child subtab.

Your custom subtab appears on custom transaction forms when deployed to an account. The following is an XML definition example of a custom Transaction subtab. When deployed to an account, a subtab called Accounts is displayed under **Tabs** in your custom transaction form:

```

1 <subtab scriptid="custtab_accounts">
2
3   <parent>[scriptid=custtab_accountinformation]</parent>
4
5   <tabtype>TRANSACTION</tabtype>
6
7   <title>Accounts</title>
8
9 </subtab>

```

To create a custom subtab for a different tab type, follow the preceding example, but replace the tab type with an Entity, Item, or CRM record. When deployed to an account, the subtab is displayed under **Subtabs** in your custom entry form.

You can create custom entry and transaction forms in both SDF and NetSuite. For more information, see the help topic [Creating Custom Entry and Transaction Forms](#).

For the full list of available fields in the SDF custom objects mentioned in this topic, see the following topics:

- [subtab](#)
- [entryForm](#)
- [transactionForm](#)

Import and Export

The following topics provide SuiteCloud Development Framework (SDF) information about the SDF custom objects that pertain to importing and exporting data:

- [Saved CSV Import Maps as XML Definitions](#)
- [Saved Searches as XML Definitions](#)

Saved CSV Import Maps as XML Definitions

In SuiteCloud Development Framework (SDF), a saved CSV import map is a way for your users to transfer data sets from external applications into NetSuite using comma separated value (CSV) files. You can include saved CSV import maps in your SuiteCloud projects as savedcsvimport SDF custom objects. CSV import maps can be created using the NetSuite user interface, SuiteCloud IDE plug-in, and SuiteCloud CLI.

The savedcsvimport SDF custom object contains import options and CSV data mappings to record fields in NetSuite.

The savedcsvimport SDF custom object does not contain CSV data, and the CSV file can only be included in the SuiteCloud project as a File Cabinet file. The CSV data cannot be deployed to an account as record instance data. The CSV data is provided by your users when they make use of the CSV saved import. To prepare files for CSV import, see the help topic [Guidelines for CSV Import Files](#).

For information about saved CSV import maps in NetSuite, see the following topics:

- [CSV Imports Overview](#)
- [Working with Saved CSV Imports](#)
- [Setting CSV Import Preferences](#)

You should create a saved CSV import from the NetSuite user interface Import Assistant, and import the object into your SuiteCloud project. Using the Import Assistant ensures that you are mapping to valid fields. For more information, see the following:

- [Importing CSV Files with the Import Assistant](#)

- [Account Component Imports to SuiteCloud Projects](#)
- [Downloading the XML Definition of an Object from a NetSuite Account](#)

Saved CSV import maps are composed of the following:

- Import options, see [Defining Import Options for Saved CSV Import Maps in XML Definitions](#)
- File mappings, see [Defining File Mappings for Saved CSV Import Maps in XML Definitions](#)
- Record and field mappings, see [Defining Record and Field Mappings for Saved CSV Import Maps in XML Definitions](#)

For more information about saved CSV import maps, see the following:

- For permissions required by your role, see the help topic [Required Permissions for CSV Imports](#).
- For a savedcsvimport SDF custom object example, see [Saved CSV Import Maps XML Definition Example](#).

Defining Import Options for Saved CSV Import Maps in XML Definitions

In SuiteCloud Development Framework (SDF), you can define import options in savedcsvimport SDF custom objects.

The following table contains information you can use to match options in the NetSuite user interface Import Assistant with SuiteCloud Development Framework (SDF) XML elements:

Note: The table does not contain information about required elements, data types, and default values. For that information, see the help topic [savedcsvimport](#).

Import Assistant Option	XML Element	Notes
Character Encoding	charencoding	For the list of possible values, see the help topic csvimport_recordtypes . For more information, see the help topic Choose Import Character Encoding .
CSV Column Delimiter	columndelimiter	For the list of possible values, see the help topic csvimport_columndelimiter . For more information, see the help topic Choose CSV Column Delimiter .
CSV Decimal Delimiter	decimaldelimiter	For more information, see the help topic CSV Decimal Delimiter .
CSV File(s)	None	The savedcsvimport SDF custom object does not contain CSV data, and CSV file contents cannot be deployed to an account as record instance data. The CSV file is provided by your users when they make use of the CSV saved import. To prepare files for CSV import, see the help topic Guidelines for CSV Import Files .
Custom Form	entryform transactionform	For custom forms, use either the entryform or transactionform elements. For more information, see the help topic Custom Form .
Custom Multi-Select Value Delimiter	multiselectdelimiter	For more information, see the help topic Custom Multi-Select Value Delimiter .

Import Assistant Option	XML Element	Notes
Data Handling	datahandling	For more information, see the help topic Choose Data Handling for Import .
Description	description	Provides a description in the NetSuite user interface for the CSV import.
Ignore Read-Only Fields	ignorereadonly	For more information, see the help topic Ignore Read Only Fields .
Import Map Name	importname	Enter a name that can be used as a reference in the NetSuite user interface when tracking the import's progress. The name can also be used to identify the CSV import when it is rerun or edited.
Import Type	None	There is no XML element for this option. It is sufficient to specify the recordtype element.
Log System Notes for Custom Fields	logsystemnotescustfields	For more information, see the help topic Log System Notes For Custom Fields .
Overwrite Missing Fields	overwritemissingfields	For more information, see the help topic Overwrite Missing Fields .
Overwrite Sublists	overwritesublists	For more information, see the help topic Overwrite Sublists .
Prevent Duplicate Records	preventduplicates	This element requires the DUPLICATES feature to be declared as a dependency in the manifest. For more information, see the help topic Prevent Duplicate Records .
Queue Number	None	This option is not currently supported in SDF. The option is available in the Import Assistant for accounts that have a SuiteCloud Plus license. For more information, see the help topic Queue Number .
Record Type	recordtype	For the list of possible values, see the help topic csvimport_recordtypes . For information about record types supported by NetSuite, see the help topic Supported Record Types for CSV Import .
Run Server SuiteScript and Trigger Workflows	runserversuitescript	For more information, see the help topic Server Scripting and Workflow Execution .
Script ID	scriptid attribute for savedcsvimport	The prefix of the scriptid attribute value is custimport. The value must match the file name of the SDF custom object.
Use Multi-Threading	usemultithread	The option is available for accounts that have a SuiteCloud Plus license. For more information, see the help topic Use Multi-Threading .
Validate Mandatory Custom Fields	validatemandatorycustfields	For more information, see the help topic Validate Mandatory Custom Fields .

Defining File Mappings for Saved CSV Import Maps in XML Definitions

In SuiteCloud Development Framework (SDF), the default is to define file mappings for the `savedcsvimport` SDF custom object. The file mapping defines information about how the CSV file is processed during import. The CSV data is not included with the SDF custom object.

The `filemapping` element contains the `file` element, and a primary key or foreign key. For a list of fields and possible values, see the help topic [savedcsvimport](#).

The `file` element defines the NetSuite record type for the CSV file definition. For the list of possible record type values, see the help topic [csvimport_recordtypes](#). For more information about records supported by saved CSV import maps, see the help topic [Supported Record Types for CSV Import](#).

Single CSV File

When specifying field mappings for a single CSV file, a single file mapping is specified with no primary key or foreign key.

The following example defines a single file mapping for a CSV file that imports sales order data:

```
1 <filemappings>
2   <filemapping>
3     <file>SALESORDER</file>
4   </filemapping>
5 </filemappings>
```

Multiple CSV Files

When specifying field mappings for multiple CSV files, specify a file mapping for each CSV file using the `filemapping` element for the `filemappings` element.

Keys are used when there are multiple CSV files. The CSV file definition can contain a primary key for the primary CSV file, and a foreign key for the secondary CSV files. The primary key indicates how data is to be mapped between the primary and linked CSV files.

The following example defines file mappings for multiple CSV files:

```
1 <filemappings>
2   <filemapping>
3     <file>SALESORDER</file>
4     <primarykey>ID</primarykey>
5   </filemapping>
6   <filemapping>
7     <file>SALESORDER:ITEMS</file>
8     <foreignkey>Transaction ID</foreignkey>
9   </filemapping>
10 </filemappings>
```

The example defines CSV file mapping in the following way:

- The SALESORDER file mapping contains sales order record data, with the ID column specified as the primary key in the SALESORDER CSV file.
- The SALESORDER:ITEMS file mapping contains sales order item data, with the Transaction ID column specified as the foreign key in the SALESORDER:ITEMS CSV file.

For a complete `savedcsvimport` SDF custom object example, see [Saved CSV Import Maps XML Definition Example](#).

Defining Record and Field Mappings for Saved CSV Import Maps in XML Definitions

When your users import CSV files, the data is imported based on the mappings you define for records and fields. In SuiteCloud Development Framework (SDF), the record and field mappings are defined in the `savedcsvimport` SDF custom object.

To define mappings, see the following topics:

- [Record Mappings](#)
- [Field Mappings](#)

Consider the following when creating mappings:

- SuiteCloud IDE plug-in does not validate field mappings. To ensure you are mapping to valid fields, use the Import Assistant.
- It is a best practice that if a record exists in the `filemapping` it also exists in the record mapping.
- It is required that the field element be paired with either a value element or `columnreference` element.
- If you specify the `line` element for `recordmapping`, do not specify a value for the `filemapping` element.
- When mapping to the standard form for custom records, specify an empty `entryform` element and `CUSTOMRECORD` as the record type, for example:

```

1 <savedcsvimport scriptid="custimport_customrecord34">
2 ...
3   <customrecord>[customer_details]</customrecord>
4   <recordtype>CUSTOMRECORD</recordtype>
5 ...
6   <entryform/>
7 ...
8 </savedcsvimport>

```

The empty `entryform` element corresponds to the standard form as the default selection in the NetSuite user interface Import Assistant.

- SDF only permits the preferred form to be specified in a saved CSV import of a custom transaction. To map to the preferred form for a custom transaction, specify an empty `transactionform` element and `CUSTOMTRANSACTION` as the record type, for example:

```

1 <savedcsvimport scriptid="custimport_customtransaction68">
2 ...
3   <customtransaction>[scriptid=customtransaction_material_invoice]</customtransaction>
4   <recordtype>CUSTOMTRANSACTION</recordtype>
5 ...
6   <transactionform/>
7 ...
8 </savedcsvimport>

```

The empty `transactionform` element corresponds to the preferred form as the default selection in the NetSuite user interface Import Assistant.

- The Queue Number field is not supported.
- Do not reference a record using the default value. The default value is different across accounts.

Record Mappings

Record mappings are defined using a collection of `recordmapping` elements with the `recordmappings` element.

For more information about records, see the following topics:

- For the list of possible record type values, see the help topic [csvimport_recordtypes](#).
- For information about records supported by saved CSV import maps, see the help topic [Supported Record Types for CSV Import](#).

Note: During deployment, the internal IDs for records are based on values in the target account and not the values in the CSV file. If needed, you can override those IDs by using SuiteBundler to deploy the object.

The following example defines recordmappings for the SALESORDER record type and SALESORDER: ITEMS subrecord type.

```

1 <recordmappings>
2   <recordmapping>
3     <record>SALESORDER</record>
4     <fieldmappings>
5       ...
6     </fieldmappings>
7   </recordmapping>
8   <recordmapping>
9     <record>SALESORDER: ITEMS</record>
10    <fieldmappings>
11      ...
12    </fieldmappings>
13  </recordmapping>
14 </recordmappings>

```

For examples of field mappings, see [Field Mappings](#). For a complete savedcsvimport SDF custom object example, see [Saved CSV Import Maps XML Definition Example](#).

The line element is used with the recordmapping element when sourcing a list from a single file. You should use the NetSuite user interface Import Assistant to ensure the CSV columns are correctly mapped to lines. When line element is used, there is no value for the filemapping element.

The following example defines field mappings for two address lines in the customer contact address book:

```

1 <recordmapping>
2   <line>1</line>
3   <record>CUSTOMERANDCONTACT:ADDRESSBOOK:ADDRESSBOOKADDRESS</record>
4   <fieldmappings>
5     <fieldmapping>
6       <field>ADDR1</field>
7       <columnreference>
8         <column>street2</column>
9         <file>CUSTOMERANDCONTACT</file>
10        <type>NAME</type>
11      </columnreference>
12    </fieldmapping>
13    <fieldmapping>
14      <field>ADDR2</field>
15      <columnreference>
16        <column>city2</column>
17        <file>CUSTOMERANDCONTACT</file>
18        <type>NAME</type>
19      </columnreference>
20    </fieldmapping>
21  </fieldmappings>
22 </recordmapping>
23 <recordmapping>
24   <line>2</line>
25   <record>CUSTOMERANDCONTACT:ADDRESSBOOK:ADDRESSBOOKADDRESS</record>
26   <fieldmappings>
27     <fieldmapping>

```

```

28     <field>ADDR1</field>
29     <columnreference>
30         <column>street3</column>
31         <file>CUSTOMERANDCONTACT</file>
32         <type>NAME</type>
33     </columnreference>
34 </fieldmapping>
35 <fieldmapping>
36     <field>ADDR2</field>
37     <columnreference>
38         <column>city3</column>
39         <file>CUSTOMERANDCONTACT</file>
40         <type>NAME</type>
41     </columnreference>
42 </fieldmapping>
43 </fieldmappings>
44 </recordmapping>

```

Field Mappings

A field mapping is defined by the `fieldmapping` element as part of a `fieldmappings` collection, and grouped by record using the `recordmapping` element. For a list of fields and values, see the help topic [savedcsvimport](#).

For information about field mappings, see the following:

- [General CSV Field Mapping Tips](#)
- [Required Fields on Records](#)

Field mappings are defined by record type. For examples of record type mapping, see [Record Mappings](#).

Field descriptions can be found in the [SuiteTalk Schema Browser](#). Field information is also available in the NetSuite user interface form for the selected record type. You can click the field label to display help that describes the field.

To be available for mapping, the field must be both exposed to CSV import and available to you to map. You can use the NetSuite user interface Import Assistant to determine the fields that are available to you. The list of available fields can be impacted by your permissions and the features that are enabled for the target NetSuite account.

You can define a field mapping the following ways:

- Specify the default value.

In the following example, the `STDEMAIL` field is mapped to an email address as the default value.

```

1 <fieldmapping>
2   <field>STDEMAIL</field>
3   <value>jmiller@example.com</value>
4 </fieldmapping>

```

- Specify a null value.

In the following example, the `STDMEMO` field is mapped to a null value.

```

1 <fieldmapping>
2   <field>STDMEMO</field>
3   <value/>
4 </fieldmapping>

```

- Map the field to a column in a CSV file.

In the following example, the `STDPHONE` field is mapped to the Phone Number column in the `SALESORDER` CSV file.

```

1 <fieldmapping>
2   <field>STDPHONE</field>
3   <columnreference>
4     <file>SALESORDER</file>
5     <column>Phone Number</column>
6   </columnreference>
7 </fieldmapping>

```

- Map the field to a column in a CSV file with the reference type.

In the following example, the STDCUSTOMER field is mapped to the Customer EID column in the SALESORDER CSV file with a reference type of EXTERNAL_ID.

```

1 <fieldmapping>
2   <field>STDCUSTOMER</field>
3   <columnreference>
4     <file>SALESORDER</file>
5     <column>Customer EID</column>
6     <type>EXTERNAL_ID</type>
7   </columnreference>
8 </fieldmapping>

```

For a complete savedcsvimport SDF custom object example, see [Saved CSV Import Maps XML Definition Example](#).

Saved CSV Import Maps XML Definition Example

The following is a SuiteCloud Development Framework (SDF) XML definition example of a savedcsvimport SDF custom object that maps fields in a CSV file to fields on the Sales Order record type:

```

1 <savedcsvimport scriptid="custimport_salesorder">
2   <recordtype>SALESORDER</recordtype>
3   <importname>Sales Order Import</importname>
4   <datahandling>ADDUPDATE</datahandling>
5   <columndelimiter>PIPE</columndelimiter>
6   <transactionform>[scriptid=custform_csv_import]</transactionform>
7   <description>Sales Order Import</description>
8   <decimaldelimiter>COMMA</decimaldelimiter>
9   <filemappings>
10    <filemapping>
11      <file>SALESORDER</file>
12      <primarykey>ID</primarykey>
13    </filemapping>
14    <filemapping>
15      <file>SALESORDER:ITEMS</file>
16      <foreignkey>Transaction ID</foreignkey>
17    </filemapping>
18  </filemappings>
19  <recordmappings>
20    <recordmapping>
21      <record>SALESORDER</record>
22      <fieldmappings>
23        <fieldmapping>
24          <field>STDEMAIL</field>
25          <value>jmiller@example.com</value>
26        </fieldmapping>
27        <fieldmapping>
28          <field>STDMEMO</field>
29          <value/>
30        </fieldmapping>
31        <fieldmapping>
32          <field>STDPHONE</field>
33          <columnreference>
34            <file>SALESORDER</file>
35            <column>Phone Number</column>

```

```

36      </columnreference>
37    </fieldmapping>
38    <fieldmapping>
39      <field>STDCUSTOMER</field>
40      <columnreference>
41        <file>SALESORDER</file>
42        <column>Customer EID</column>
43        <type>EXTERNAL_ID</type>
44      </columnreference>
45    </fieldmapping>
46  </fieldmappings>
47 </recordmapping>
48 <recordmapping>
49   <record>SALESORDER: ITEMS</record>
50   <fieldmappings>
51     <fieldmapping>
52       <field>STDAMOUNT</field>
53       <columnreference>
54         <file>SALESORDER: ITEMS</file>
55         <column>Items Amount</column>
56       </columnreference>
57     </fieldmapping>
58   </fieldmappings>
59 </recordmapping>
60 </recordmappings>
61 </savedcsvimport>

```

Saved Searches as XML Definitions

Saved searches are supported as SDF custom objects in SuiteCloud Development Framework (SDF). A saved search is a reusable search definition that can have many advanced search filters and results display options. If you have the Publish Search permission, you can share search results with other users. Saved search results provide reporting and tracking, and can serve as the basis for business analysis and strategic decision-making.

Custom saved searches can be created and managed only by using NetSuite. After custom saved searches are created, they can be imported into a SuiteCloud project. Saved searches can also be deployed to a NetSuite account using SDF. For more information about working with saved searches in a NetSuite account, see the help topic [Saved Searches](#).

To create a saved search, you need to access a saved search definition page for a selected record type. For more information, see the help topic [Defining a Saved Search](#).

The following is an example of a saved search object that was imported from NetSuite:

```

1 <savedsearch scriptid="customsearch5">
2   <definition>966a6e804944b4865a03531372a17fa5979650f07ab72186d5287bcd3d1137a@GZC@2019.1.0.129@H4sIAAAAAAAAO1cbXO
qRhT+3pn+B4d+btMQ0PSQVgTgT4KmMR+YYhuer1FdADvbe6v78KCudTbRhe1l9mZzDiucM6+PofZw7PkiL/9Mw1bX2CcBLPomuNP01wLRuPZJij+uuaG7uDXS
+63m59/EqMkmY17DowDPwy+wYn5/BmOU3kwpX4QwbiFzERJL7/qmvuUpvNeu/3169eTCKbJlKjhyXg2bSfjT3DqJ+38Mq61CCbXXPfqqtS97Tn+FzhxoB+PP/
XGiySdTI2P8y1mPPzs/u7w8FTrd06tL7q2vnROea6XBFcapP52jvq9dh/rcahW9nvrzOeri2p03qd9Z50I4+QvGqHcxPMEe0UcxyPgE33mC+6XnX8T2qtFVnXP4EKr
Bilzkradg36dU29EvE5iM42CezuJlG2qNIerHxH2dwY49JPkmsMw7GU7d+MAyZbvPGC49khsv92xaeLHy5gstKC2h6yttZLAMP88tL0oGzg8G/X3D0YeZYk30u3g
Gu1DzDiyLZquZ6qCNlovn275pI0RrhCc7+6wGI7t7qno4EKNMVTwEAaaq5nSDrY8Cbn3lo5VsdPc1tqfU4WxoqWQ8J9sSbuyNriHbs90I0m3XrWsk+p8tL
B82wWQj/ibtJ4AQmY11ThPwbMm1XA/Ts65J8pxqAmn0b6KqhAJuaA+f0fERLYNzTWwIBDIBtA2XH5r/4YULWz85FJ+9q5/qTdzUwbF0YfmzqIEpzqJk6LmYsoK1Di
H65eTvayQf+YMKHj3JQePOcEnvhks/t7Zk3R3DzcoGSNzL0EAwciW1VdgEdL7r05Nmo3Pg4iqqI/U14Cm2qmmeyJ4ad0Yk3471IQogQx55tuQeiu+893eqAry
BqrnARIMwraz/9AKzTGZc8OR6eET90Zu4biS7TqPqntHAERsG6Rq+5iH1A4AJTz1rm41sy9pH1BUinEu0w/0jIMnGwGudwF1QDE+gI5W2hVynp5AXKxY
cob934HskjHWN5URGUuyqevomEnA2EFkensLsg1Z0gDNfcmR74Ay1I7uCC2frku2CugwGnYJjGxVLHdENa3BPJYPS+tI01FAjZGZPoX2K1SF2pxqyNkSbr
L7rU03j7x7aU6YraVumkJCLcjAwSfUTRta09eWDEeSXdwktPHhw00PtXvPtFDoKofs2W+2cpkkT9k9S7I1/dA8pj9EOZGaIipzYFquqqt/o1QDjdWHS1ceekb
hDS0F5UIUAzx7fJWGaEPoKLS2Rqz5CjzqXI7NJQJnMnoAqPx6z18XeqI0mISoDbY6yy7EEQpd9PzDC22kzqZ2N4iy4kvQZjCeFVNE9MnhQ6EcAqjdB2i
w0mbI7bTVX10bFCNis10vXj18G+jjCBj3Y63GF9L/4Qeg/hxDr/kk/4Hu7fEsftHdghSGJRC60Ntc79GknvcnqzMITjTDr1WjF8gXG2tNfcyUk7/6s
Gx8ck1Gq/d6ipuwBmew64A/PG0r91Sw+xyw/LSQ+GIA5bWdpsxIUz50450evvWgRhhtPDyBKg/TVkxQF7ck0qX7I8c1dQ9NbJYgV/qjzcZ+CJV1lPwq/ALv7mmHya
jgWm62656nRS0tw9PR7M1lhwY/PfDzjcIB/cuBEUW0jq+2GLIiBdxNG+wwwDd2gb+LHzRwnk1T5TDWkMcELTJWewUEPzi0qOVIVUrGWS9BMLPn1nAK/O/S/
Cgc6LD10K3WfyDnRjdmCbCLSPvcHQyBNnIzRZQjvUJIKmPGDbJkWyxDmtreSP0P1RGT0b8f50PmR88xG+yXQ0imQj1Ek2p3WtZrkjG0Y2jGxKsrHuTANQJJ
tunWRXzjFzXDCYyWTdyGypWmGubb45rZnVLuvmyvGN4xvGN+UfD0QnhiSzdz3YpexvU+k8526uYY/VPRLZMP1pglkU7wnr25YknG1CLjnfKOLXrDwTi
hnjMMZ2YzYrVf1Mo4tYvFPFOLm844W47qt57F1x0Y+kFYaX33Cb0C3P21XXZC/20e0F/WSoS1C9k8U7KbToSMcf5HZ/T10k3t0jbPhGxGN4xujnZKXyvd
CLVL2QKTshndMLo55j19vYxTu5QtMcmBmq5jnCod1NdLnrWz2AJTsRnZMLI55k19vYxTu1wsMLmYMQ5jng0e1nFL0LUrxgTjJv00Fsrf2w5kxeTWzi31l
Mo33Kcp12ixIEbagbP8o5/UqXqbJgVsk1X9tAd+m1JbLi/x99XvgQ0DIwFNW4PVZ5iG2YFRHfxSm605gHCEsVml8b0bjw1Q1k1VLRIjbbX0IR29/1VukL

```

```

gW78h//Sg9jGJiv1uMuC3pXy3cui3gU8g+UkV395baU5DsR5Q+uFzVcLEmXNHe6to5vL+Z5YLUsGuan+jWuaFR6yKmpFQRdg0yzZtuoP6JZmjgDNcnRr/izJdo3jD
c82tw1DI1S6bc3VayIek9bAc85bulsfVcnX76q5tRYEb8FYBtd/BF011yuC712FhrIL+ZVwzrN5UzaabzYpR1V39b/iabR3Zx+rNsR81oZ9xSyUuET3Fr1Xux9N3AxM
PIG4rvlWuyfbHamm49qa17zr4BL3sQYKeg3nqWe5BDtAMB5s7sb1jrajhp1rnJrtI+Ah21p8wt+FGYLhpOG66GDdsrjpMtw0HDenGDenzHFzYnDTcNycYdyckcXNGcN
Nw3FzjnFzThY35ww3DcFNbcbNBVncXDDcNBw3lXg3l2Rxc8lW03DcXGHcXJHFzRXTDcNxxwdaPN8hiy+w6DTd0iUuvGHhON3QIepxo2HTiEb84R1Y54Jx42HTqEc84S
1Y55px42HTiEe84TVY57Jx42HTqEf84QFZJ4pyI2HTiEh84Q1ZJ6JyI2HTqEi84R1ZJ7pyI2HTiEk84SVZJ5Jy2HTqE184TFZJ6pyU2Hj1CoyQJhNV1ganLjoV0oyQJh
NV1ganLjoV0+hEz6LWSmJjceOoWaLBBWkwWmJjceOoWaLBBWkwWmJjceOoWaLBBWkwWmJjceOoWaLBBWkwWmJjceOoWaLBBWkwWmJjceOoWaLBBWkwWmJjceOoWaLBBWkw
wWmJjv/volPRjCuF1M00jAM/DL7Bifn8GY7TvHJgEMHM6s8//QufrYINGKEAAA==</definition>
3 </savedsearch>

```



Warning: Do not manually edit the XML in SDF. Modifications to the system-generated XML may result in validation and deployment failures. You should customize saved searches with NetSuite rather than SDF.

You can reference SDF custom objects in saved searches by defining dependencies. If your saved search contains a dependency, you must import the object into your SuiteCloud project to successfully deploy the project to a target NetSuite account. For more information, see the help topic [dependencies](#).

For more information about saved searches, see the following topics:

- [savedsearch](#)
- [Saved Search Email](#)
- [Accessing a Saved Search](#)
- [Audit Trail for Saved Searches, Reports and Schedules](#)
- [Translations for Saved Searches](#)
- [Displaying Saved Search Results on Your Dashboard](#)
- [Saved Search Examples](#)

Published Dashboards and Portlets

The following topics provide SuiteCloud Development Framework (SDF) information about the SDF custom objects that pertain to published dashboards and portlets:

- [Published Dashboards as XML Definitions](#)
- [Centers as XML Definitions](#)
- [Center Categories as XML Definitions](#)
- [Center Tabs as XML Definitions](#)
- [Supported Published Dashboard Object Portlets](#)

Published Dashboards as XML Definitions

Create and manage dashboards using SuiteCloud Development Framework (SDF). You can model a published dashboard in a SuiteCloud project and deploy the project to a target NetSuite account. Published dashboards in SDF support most standard portlets, including custom portlets.

A **published dashboard** object in SDF stores settings for a collection of dashboards in the target NetSuite account. These objects are associated with a single Center and can be applied to multiple roles in that Center in a NetSuite account. Each **dashboard** in the published dashboard object controls the layout of portlets in any single Center Tab that is available in the specified Center—such as Home, Activities, Transactions, Lists, or Reports.

You can customize dashboards for multiple Center Tabs in a single published dashboard object. Publishing settings, such as applicable roles and locking, are also specified in the published dashboard object.

Customizing and deploying a published dashboard object from your SuiteCloud project to a target NetSuite account is the equivalent of going to the **Settings** portlet in a NetSuite account, clicking **Publish Dashboard**, and editing the record. All fields in this NetSuite record are supported in SDF except for the **Override Existing User's Settings** column in the **Apply to Roles** list. The following image shows some of the fields in the record that are supported by the published dashboard object:

Publish Dashboard List More

NAME *
My Published Dashboard

CENTER TYPE
Classic Center

NOTES

☐ LOCK SHORTCUTS
☐ LOCK NEW BAR

Apply To Roles **Apply To Tabs**

Please select tabs you would like to publish for this dashboard.

- Unlocked - Users can add/remove/modify dashboard content without restriction
- Locked - Users can not make any changes to their dashboard content
- Add/Move content - Users can add or move dashboard content but can not remove published content

APPLY	MODE	TAB
☐	Unlocked	Publisher Dashboard
☐	Unlocked	Home
☐	Unlocked	Project Dashboard
☐	Unlocked	Vendor Dashboard
☐	Unlocked	Customer Dashboard
☐	Unlocked	Activities

• The Name and Notes fields are for your reference. They will not be exposed to users.
• To edit a dashboard, you must log in to the center where the dashboard is published.
• If you check the **Override existing user's settings** box, the personal settings of existing users assigned to the role will be replaced with the published dashboard. If this box is cleared, only users newly assigned to the role will view the published dashboard.
• To make changes to published dashboards, update the content you display and then click List next to Publish Dashboards. Click Edit next to the dashboard you want to change. Choose to apply your changes to both existing and new users assigned to the role, or to only those newly assigned to the role. Then click **Save and Update Content**.
• If you lock Shortcuts or New Bar links, users will not be able to remove them, however they will be able to add their own, and re-order.

Published dashboard objects are represented by the publisheddashboard element. The following is an example of an empty published dashboard object:

```

1 <publisheddashboard scriptid="custpubdashboard_custom">
2   <center></center>
3   <name></name>
4   <dashboards>
5     <dashboard>
6       <centertab></centertab>
7       <layout></layout>
8       <mode></mode>
9     </dashboard>
10  </dashboards>
11  <roles>
12    <role>
13      <role></role>
14    </role>
15  </roles>
16 </publisheddashboard>

```

The center element can be set to any standard or custom Center. For information about:

- Possible standard values, see the help topic [role_centertype](#).

- Creating a custom Center in SDF, see [Centers as XML Definitions](#).

The role element specifies the roles to associate with the published dashboard. These roles must be associated with the Center specified in the published dashboard. For more information about associating roles with a custom Center in SDF, see [Center Tabs as XML Definitions](#).

For each dashboard element, portlet elements, such as calendar or trendgraph, can be specified in the leftcolumn, centercolumn, or rightcolumn elements, depending on the value specified by the layout element. The following table shows the available elements based on the dashboard layout:

Available Values for the Layout Element	Available Elements for the Layout
SINGLE_COLUMN	■ centercolumn
TWO_COLUMN	■ leftcolumn ■ centercolumn
TWO_COLUMN_RIGHT	■ centercolumn ■ rightcolumn
THREE_COLUMN	■ leftcolumn ■ centercolumn ■ rightcolumn

For more information about standard or custom portlets that can be specified in the leftcolumn, centercolumn, and rightcolumn elements, see [Supported Published Dashboard Object Portlets](#).

Published Dashboard Object Example

The following is an example of a published dashboard object for the Classic Center, known as the BASIC Center in SDF. It specifies a three-column dashboard for the Classic Center Home tab and a single-column dashboard for a Classic Center custom tab named custcentertab_basic_tab. When deployed to a NetSuite account, the dashboards are applied to any new employees that have either an accountant role or the customrole_my_basic_role custom role assigned to them in the account.

```

1 <publisheddashboard scriptid="custpubdashboard_basic">
2   <center>BASIC</center>
3   <name>Classic Center Published DB</name>
4   <dashboards>
5     <dashboard>
6       <centertab>BASICCENTERHOMEHOME</centertab>
7       <layout>THREE_COLUMN</layout>
8       <mode>UNLOCKED</mode>
9       <leftcolumn>
10        <!-- Insert left column portlet definitions here. -->
11      </leftcolumn>
12      <centercolumn>
13        <!-- Insert center column portlet definitions here. -->
14      </centercolumn>
15      <rightcolumn>
16        <!-- Insert right column portlet definitions here. -->
17      </rightcolumn>
18    </dashboard>
19    <dashboard>
20      <centertab>[scriptid=custcentertab_basic_tab]</centertab>
21      <layout>SINGLE_COLUMN</layout>
22      <mode>LOCKED</mode>
23      <centercolumn>
24        <!-- Insert center column portlet definitions here. -->
25      </centercolumn>

```



```

26     </dashboard>
27   </dashboards>
28   <roles>
29     <role>
30       <role>ACCOUNTANT</role>
31     </role>
32     <role>
33       <role>[scriptid=customrole_my_basic_role]</role>
34     </role>
35   </roles>
36 </publisheddashboard>

```



Important: You cannot use SDF to override dashboards or dashboard settings for existing users. You can apply the settings of a publisheddashboard object to existing users in an account after the object is deployed to that account. Create a new employee in that account, and assign this new user a role that is associated with the published dashboard and that has the Publish Dashboard permission. This user can then log in to NetSuite and edit the published dashboard to publish the dashboard to existing users with the **Override Existing User's Settings** box enabled.



Note: Some of the SDF supported fields are translatable. For more information, see [Translatable Fields Supported in SDF Custom Objects](#).

Script IDs of Center Links are not portable through SuiteBundler but are supported in SDF. For center links in SDF, the script ID is generated on the target account.

App ID and Publisher ID for Center Type, Center Tab, Center Category, and Center Link are not portable through Bundler but are supported in SDF.

For more information, see:

- [Publishing Dashboards](#)
- [publisheddashboard](#)

Supported Published Dashboard Object Portlets

The following portlets are supported in SuiteCloud Development Framework (SDF) by the publisheddashboard SDF custom object:

- [Analytics Portlets in SDF Custom Objects](#)
- [Calendar Portlets in SDF Custom Objects](#)
- [Custom Portlets in SDF Custom Objects](#)
- [Custom Search Portlets in SDF Custom Objects](#)
- [Event Scheduler Portlets in SDF Custom Objects](#)
- [Key Performance Indicator \(KPI\) Portlets in SDF Custom Objects](#)
- [Key Performance Indicator \(KPI\) Scorecard Portlets in SDF Custom Objects](#)
- [KPI Meter Portlets in SDF Custom Objects](#)
- [Links Portlets in SDF Custom Objects](#)
- [List Portlets in SDF Custom Objects](#)
- [My Login Audit Portlets in SDF Custom Objects](#)
- [Gantt Chart Portlets in SDF Custom Objects](#)
- [Project Info Portlets in SDF Custom Objects](#)

- Project Profitability Portlets in SDF Custom Objects
- Project Links Portlets in SDF Custom Objects
- Quick Search Portlets in SDF Custom Objects
- Recent Records Portlets in SDF Custom Objects
- Recent Reports Portlets in SDF Custom Objects
- Report Snapshot Portlets in SDF Custom Objects
- Reminders Portlets in SDF Custom Objects
- Search Form Portlets in SDF Custom Objects
- Settings Portlets in SDF Custom Objects
- Tasks Portlets in SDF Custom Objects
- Trend Graph Portlets in SDF Custom Objects

Portlets are specified in a `publisheddashboard` object. For more information about the appropriate context, see [Published Dashboards as XML Definitions](#).

Analytics Portlets in SDF Custom Objects

You can create Analytics portlets in SuiteCloud Development Framework (SDF) using the `publisheddashboard` SDF custom object. For information about the appropriate context, see [Published Dashboards as XML Definitions](#).

Analytics portlets enable you to view your workbook charts, pivot tables, and table views on your home dashboard. You can add up to 10 Analytics portlets, and each portlet can display one chart, pivot table, or table view.

Analytics portlets in SDF are represented with the `analytics` element. You can specify this element in a dashboard of a published dashboard object.

The following example defines a medium-sized Analytics portlet that displays a chart:

```

1 <analytics>
2   <portlettype>CHART</portlettype>
3   <visualization>[scriptid=custworkbook1.custchart2135234234]</visualization>
4   <name>My Chart Portlet</name>
5   <height>MEDIUM</height>
6 </analytics>

```

The following example defines an Analytics portlet that displays 20 visible rows in a pivot table:

```

1 <analytics>
2   <portlettype>PIVOT</portlettype>
3   <visualization>[scriptid=custworkbook1.custpivot2135234234]</visualization>
4   <name>My Pivot Table Portlet</name>
5   <visiblerows>20</visiblerows>
6 </analytics>

```

The following example defines an Analytics portlet that displays 20 visible rows in a table view:

```

1 <analytics>
2   <portlettype>TABLE</portlettype>
3   <visualization>[scriptid=custworkbook1.custview2135234234]</visualization>
4   <name>My Table View Portlet</name>
5   <visiblerows>20</visiblerows>
6 </analytics>

```

You can specify the following fields in an Analytics portlet:

- **Visible rows** — You can only specify the visible rows field of the Analytics portlet if it is a pivot table or a table view. Charts do not support the visible rows field.
- **Height** — You can only specify the height field of the Analytics portlet if it is a chart. Table views and pivot tables do not support the height field.

For more information, see the help topic [Setting Up the Analytics Portlet](#).

Note: The Analytics portlet only exports values from custom workbooks. It does not export values from standard workbooks. For more information about customizing workbooks, see [Custom Workbook Objects](#).

Warning: Analytics portlets are not supported by bundles. You can deploy a publisheddashboard SDF custom object containing an Analytics portlet to a NetSuite account. However, if you bundle the object, the Analytics portlet is not included in the resulting bundle.

For more information about Analytics portlets, see:

- [Workbook-based Portlets](#)
- [Workbooks](#)
- [analytics](#)

Calendar Portlets in SDF Custom Objects

You can create Calendar portlets in SuiteCloud Development Framework (SDF) using the publisheddashboard SDF custom object. For information about the appropriate context, see [Published Dashboards as XML Definitions](#).

Calendar portlets list your activities, ordered by date and time, with links to activity records.

Calendar portlets in SDF are represented with the `calendar` element. You can specify this element in a dashboard of a published dashboard object.

The following example shows a calendar portlet definition that sets the number of records to display in the agenda view to fifteen:

```

1 <calendar>
2   <numberofrecordsinagenda>15</numberofrecordsinagenda>
3   <isminimized>T</isminimized>
4   <showevents>F</showevents>
5 </calendar>

```

For more information about calendar portlets, see:

- [Calendar Portlets](#)
- [calendar](#)

Custom Portlets in SDF Custom Objects

You can create Custom portlets in SuiteCloud Development Framework (SDF) using the publisheddashboard SDF custom object. For information about the appropriate context, see [Published Dashboards as XML Definitions](#).

Custom portlets run portlet scripts on a dashboard. For more information about portlet scripts, see the help topic [SuiteScript 2.x Portlet Script Type](#).

Custom portlets in SDF are represented with the `customportlet` element. You can specify this element in a dashboard of a published dashboard object.

Custom portlet objects consist of a source element that refers to a deployment of a custom portlet script, and a parameters fragment that refers to the parameters defined in the portlet script record. The following example shows a custom portlet definition that runs a deployed portlet SuiteScript named `customscript_myportlet` that contains three parameters, one called `custscript_myportlet_title` and value **Financials**; a second one called `custscript_myportlet_something` and taking a value from a custom record; and a third one called `custscript_myportlet_checkbox` and value **T**:

```

1 <customportlet>
2   <source>[scriptid=customscript_myportlet.customdeploy_deployment]</source>
3   <parameters>
4     <parameter>
5       <id>[scriptid=customscript_myportlet.custscript_myportlet_title]</id>
6       <value>Financials</value>
7     </parameter>
8     <parameter>
9       <id>[scriptid=customscript_myportlet.custscript_myportlet_something]</id>
10      <value>[scriptid=customrecord_id.somevalue]</value>
11    </parameter>
12    <parameter>
13      <id>[scriptid=customscript_myportlet.custscript_myportlet_checkbox]</id>
14      <value>T</value>
15    </parameter>
16  </parameters>
17 </customportlet>

```

If the parameter tag is present, it must include an `id` and a `value`. The `id` must reference an existing custom field in the portlet, and the `value` must be of the type specified for that parameter. For example, if the custom field is of type **checkbox**, you must enter either **T** or **F** for the value field.



Note: There is an existing limitation in the parameter storage on the DB. This limitation implies that if the concatenation of all the parameters exceeds the maximum number of characters allowed (999 bytes), the deployment throws a generic error (An error occurred during SDF custom object update), but you don't see a validation errors for the individual parameters as they are individually meeting the limit applied to that parameter type.

For more information about custom portlets, see:

- [Custom Portlets](#)
- [customportlet](#)

Custom Search Portlets in SDF Custom Objects

You can create custom search portlets in SuiteCloud Development Framework (SDF) using the `publisheddashboard` SDF custom object. For information about the appropriate context, see [Published Dashboards as XML Definitions](#).

Custom search portlets can be configured to display saved search results on a dashboard.

Custom search portlets in SDF are represented with the `customsearch` element. You can specify this element in a dashboard of a published dashboard object.

The following example shows a custom search portlet definition that displays up to ten results from a custom saved search. The portlet redirects users to a new page that displays the results of the drilldown.

```

1 <customsearch>
2   <savedsearch>[scriptid=savedsearch_customsearch]</savedsearch>
3   <resultsize>10</resultsize>
4   <drilldown>NEW_PAGE</drilldown>
5   <charttheme>GLOBAL_THEME</charttheme>
6   <backgroundtype>GLOBAL_BACKGROUND</backgroundtype>
7   <title>My Custom Saved Search Portlet</title>
8   <allowinlineediting>T</allowinlineediting>
9 </customsearch>

```

For more information about custom search portlets, see:

- [Custom Search Portlets](#)
- [customsearch](#)

Event Scheduler Portlets in SDF Custom Objects

You can create Event Scheduler portlets in SuiteCloud Development Framework (SDF) using the `publisheddashboard` SDF custom object. For information about the appropriate context, see [Published Dashboards as XML Definitions](#).

Event Scheduler portlets in SDF are represented with the `scheduler` element. You can specify this element in a dashboard of a published dashboard object.

The following example shows an event scheduler portlet definition:

```

1 <scheduler>
2   <isminimized>F</isminimized>
3 </scheduler>

```

For more information about Event Scheduler portlets, see the following help topics:

- [Using the Event Scheduler Portlet](#)
- [scheduler](#)

Key Performance Indicator (KPI) Portlets in SDF Custom Objects

You can create KPI portlets in SuiteCloud Development Framework (SDF) using the `publisheddashboard` SDF custom object. For information about the appropriate context, see [Published Dashboards as XML Definitions](#).

KPIs provide summaries of your forecast, pipeline, orders, cases, and financial data on a dashboard. Each KPI compares data based on reports over different time ranges. You can reference standard KPIs or create custom KPIs based on saved searches.

KPI portlets in SDF are represented with the `keyperformanceindicators` element. You can specify this element in a dashboard of a published dashboard object.

The following example shows a KPI portlet definition that sets up three distinct KPIs—Bank Balance, Closed Cases, Utilization. Each KPI contains child elements to represent the included data, such as date or period ranges and comparisons.

```

1 <keyperformanceindicators>
2   <cachedata>F</cachedata>
3   <kpis>
4     <kpi>

```

```

5      <compare>T</compare>
6      <comparedaterange>LAST_MONTH_TO_DATE</comparedaterange>
7      <compareperiodrange>LAST_PERIOD</compareperiodrange>
8      <daterange>TODAY</daterange>
9      <headline>F</headline>
10     <kpi>BANKBAL</kpi>
11     <periodrange>THIS_PERIOD</periodrange>
12   </kpi>
13   <kpi>
14     <compare>T</compare>
15     <comparedaterange>LAST_WEEK</comparedaterange>
16     <daterange>THIS_WEEK</daterange>
17     <headline>F</headline>
18     <highlightif>ALWAYS</highlightif>
19     <kpi>CLOSEDCASES</kpi>
20   </kpi>
21   <kpi>
22     <daterange>LAST_MONTH</daterange>
23     <headline>F</headline>
24     <kpi>UTILIZATION</kpi>
25     <periodrange>THIS_PERIOD</periodrange>
26   </kpi>
27 </kpis>
28 </keyperformanceindicators>

```

The `kpi` element supports several child elements to configure KPIs but some of these child elements are not supported by all KPI types. In addition, some child elements function differently depending on whether the Accounting Periods feature is enabled in the target NetSuite account. For example, KPIs that define a `periodrange` element must always have a `daterange` defined, too. If the Accounting Periods feature is disabled in the account, only the `daterange` element is used. If the Accounting Periods feature is enabled, the `periodrange` or `daterange` element is used according to the **Report by Period** preference.

Due to these caveats and the varied configurations for each portlet, SDF performs extensive validation on KPI portlets when they are deployed to a target NetSuite account. This validation ensures that the child elements used for any KPI type is supported.

Note: Some KPI fields may contain or require account-specific values. These values cannot be specified or imported into a SuiteCloud project. These values can only be specified in the portlet using the NetSuite UI after the published dashboard is deployed to a target account.

For more information about KPI portlets, see:

- [Key Performance Indicators Portlet](#)
- [keyperformanceindicators](#)

Key Performance Indicator (KPI) Scorecard Portlets in SDF Custom Objects

You can create KPI Scorecard portlets in SuiteCloud Development Framework (SDF) using the `publisheddashboard` SDF custom object. For information about the appropriate context, see [Published Dashboards as XML Definitions](#).

KPI Scorecard portlets in SDF are represented with the `kpireport` element. You can specify this element in a dashboard of a published dashboard object.

The following example shows a KPI scorecard portlet with defined Orientation (KPIs on Left, meaning that the scorecard lists KPI labels on the left and date ranges across the top), followed report (for example, Financial Ratios), Restrict To label (either All, My Team, or Only Mine), Show Date Row (to indicate whether to include explicit date ranges as the first row), and Show Trend Graph Icon (to indicate whether you want the KPI Scorecard portlet to include icons that can be clicked to open popup trend graphs of KPI data):

```

1 <kpireport>
2   <isminimized>F</isminimized>
3   <orientation>KPIs on Left</orientation>
4   <reportid>-2</reportid>
5   <restrict>All</restrict>
6   <showdates>T</showdates>
7   <trend>T</trend>
8 </kpireport>

```

For more information about KPI Scorecard portlets, see the following help topics:

- [KPI Scorecards](#)
- [KPI Scorecard Portlet](#)
- [kpireport](#)

KPI Meter Portlets in SDF Custom Objects

You can create KPI meter portlets in SuiteCloud Development Framework (SDF) using the `publisheddashboard` SDF custom object. For information about the appropriate context, see [Published Dashboards as XML Definitions](#).

KPI meter portlets display data for one of the KPIs included in a KPI portlet on the same dashboard or a different KPI.

KPI meter portlets in SDF are represented with the `kpmeter` element. You can specify this element in a dashboard of a published dashboard object.

The following example shows a KPI meter portlet definition that illustrates the Bank Balance KPI data in a meter:

```

1 <kpmeter>
2   <kpi>BANKBAL</kpi>
3 </kpmeter>

```

For more information about KPI meter portlets, see:

- [KPI Meter Portlets](#)
- [kpmeter](#)

Links Portlets in SDF Custom Objects

You can create links portlets in SuiteCloud Development Framework (SDF) using the `publisheddashboard` SDF custom object. For information about the appropriate context, see [Published Dashboards as XML Definitions](#).

Links portlets in SDF are represented with the `tasklinks` element. You can specify this element in a dashboard of a published dashboard object. The Links portlet supports the following portlets:

- Reports Links portlets
- Activities Links portlets

The following example shows a links portlet definition:

```

1 <tasklinks>
2   <isminimized>F</isminimized>

```

```
3 | </tasklinks>
```

For more information about Links portlets, see the following help topics:

- [The Reports Page](#)
- [The Activities Page](#)
- [tasklinks](#)

List Portlets in SDF Custom Objects

You can create list portlets in SuiteCloud Development Framework (SDF) using the `publisheddashboard` SDF custom object. For information about the appropriate context, see [Published Dashboards as XML Definitions](#).

List portlets display records of a specific record type. You can choose a standard record type, such as employees, activities, transactions, or a custom record type. Additionally, you can use this portlet to display saved search results related to the record type.

List portlets in SDF are represented with the `list` element. You can specify this element in a dashboard of a published dashboard object.

The following example shows a list portlet definition that displays a list of up to 25 customer records:

```
1 | <list>
2 |   <type>CUSTOMER</type>
3 |   <size>25</size>
4 |   <allowinlineediting>F</allowinlineediting>
5 | </list>
```

For more information about list portlets, see:

- [List Portlets](#)
- [list](#)

My Login Audit Portlets in SDF Custom Objects

You can create My Login Audit portlets in SuiteCloud Development Framework (SDF) using the `publisheddashboard` SDF custom object. For information about the appropriate context, see [Published Dashboards as XML Definitions](#).

The My Login Audit portlet shows information about your login activity.

My Login Audit portlets in SDF are represented with the `lastlogin` element. You can specify this element in a dashboard of a published dashboard object.

The following example shows My Login Audit portlet definition with various types of logins and other information (for example last login, last mobile login, and date of password expiration).

```
1 | <lastlogin>
2 |   <isminimized>F</isminimized>
3 |   <showcurrentuilogin>F</showcurrentuilogin>
4 |   <showcurrentuiloginip>F</showcurrentuiloginip>
5 |   <showfailuresthismonth>F</showfailuresthismonth>
6 |   <showfailuresthisweek>F</showfailuresthisweek>
7 |   <showfailurestoday>T</showfailurestoday>
8 |   <showlastfailureip>F</showlastfailureip>
```



```

9      <showlastfailuretime>F</showlastfailuretime>
10     <showlastlogin>T</showlastlogin>
11     <showlastloginip>F</showlastloginip>
12     <showlastmobilelogin>T</showlastmobilelogin>
13     <showlastmobileloginip>F</showlastmobileloginip>
14     <showlastotherservicelogin>F</showlastotherservicelogin>
15     <showlasttuilogin>F</showlasttuilogin>
16     <showlasttuiloginip>F</showlasttuiloginip>
17     <showlastwebservicelogin>F</showlastwebservicelogin>
18     <showlastwebserviceloginip>F</showlastwebserviceloginip>
19     <showmobilethismonth>F</showmobilethismonth>
20     <showmobilethisweek>F</showmobilethisweek>
21     <showmobiletoday>F</showmobiletoday>
22     <showpasswordchanged>T</showpasswordchanged>
23     <showpasswordexpiration>T</showpasswordexpiration>
24     <showsuccessthismonth>F</showsuccessthismonth>
25     <showsuccessthisweek>F</showsuccessthisweek>
26     <showsuccesstoday>T</showsuccesstoday>
27     <showwebservicethismonth>F</showwebservicethismonth>
28     <showwebservicethisweek>F</showwebservicethisweek>
29     <showwebservicetoday>F</showwebservicetoday>
30 </lastlogin>

```

For more information about My Login Audit portlets, see the following help topics:

- [My Login Audit Portlet](#)
- [lastlogin](#)

Gantt Chart Portlets in SDF Custom Objects

You can create Gantt Chart portlets in SuiteCloud Development Framework (SDF) using the `publisheddashboard` SDF custom object. For information about the appropriate context, see [Published Dashboards as XML Definitions](#).

Gantt Chart portlets displays graphical representation of tasks for your project.

Gantt Chart portlets in SDF are represented with the `pdgantttchart` element. You can specify this element in a dashboard of a published dashboard object.

The following example shows a Gantt Chart portlet definition:

```

1 <pdgantttchart>
2   <isminimized>F</isminimized>
3 </pdgantttchart>

```

For more information about Gantt Chart portlets, see the following help topics:

- [Gantt Chart](#)
- [pdgantttchart](#)

Project Info Portlets in SDF Custom Objects

You can create Project Info portlets in SuiteCloud Development Framework (SDF) using the `publisheddashboard` SDF custom object. For information about the appropriate context, see [Published Dashboards as XML Definitions](#).

Project Info portlets shows information about your project such as project name, project manager, customer, or status, based on the project record. You can also use this portlet to edit the project information.

Project Info portlets in SDF are represented with the `pdinfo` element. You can specify this element in a dashboard of a published dashboard object.

The following example shows a project info portlet definition:

```
1 | <pdinfo>
2 |   <isminimized>F</isminimized>
3 | </pdinfo>
```

- [Creating a Project Record](#)
- [pdinfo](#)

Project Profitability Portlets in SDF Custom Objects

You can create Project Profitability portlets in SuiteCloud Development Framework (SDF) using the `publisheddashboard` SDF custom object. For information about the appropriate context, see [Published Dashboards as XML Definitions](#).

Project Profitability portlets (or P&L portlets) display a calculation of your project's profit and other information such as cost, revenue, or margin.

Project Profitability portlets in SDF are represented with the `pdp1` element. You can specify this element in a dashboard of a published dashboard object.

The following example shows a project profitability portlet definition:

```
1 | <pdp1>
2 |   <isminimized>F</isminimized>
3 | </pdp1>
```



Important: To add Project Profitability portlets to your dashboard, ensure that you enable **Advanced Project Profitability** feature on your account first. For more information, see the help topic [Advanced Project Profitability](#).

For more information about Project Profitability portlets, see the following help topics:

- [Project Profitability Report](#)
- [pdp1](#)

Project Links Portlets in SDF Custom Objects

You can create Project Links portlets in SuiteCloud Development Framework (SDF) using the `publisheddashboard` SDF custom object. For information about the appropriate context, see [Published Dashboards as XML Definitions](#).

Project Links portlets display icon links to other features on a project, such as Gantt Chart, Project Task or Expenses.

Project Links portlets in SDF are represented with the `pdquicklinks` element. You can specify this element in a dashboard of a published dashboard object.

The following example shows a project links portlet definition:

```
1 | <pdquicklinks>
```

```

2 | <isminimized>F</isminimized>
3 | </pdquicklinks>

```

For more information about Project Links portlets, see the following help topics:

- [Project Management](#)
- [pdquicklinks](#)

Quick Search Portlets in SDF Custom Objects

You can create Quick Search portlets in SuiteCloud Development Framework (SDF) using the `publisheddashboard` SDF custom object. For information about the appropriate context, see [Published Dashboards as XML Definitions](#).

Quick Search portlets search for general records or transaction records from a dashboard.

Quick search portlets in SDF are represented with the `quicksearch` element. You can specify this element in a dashboard of a published dashboard object.

The following example shows a quick search portlet definition that searches over general records, such as customers, campaigns, and vendors. The default search type is set to campaign records, and cash refund transaction types are searched.

```

1 | <quicksearch>
2 |   <searchtype>GENERIC</searchtype>
3 |   <defaultgeneraltype>CAMPAIGN</defaultgeneraltype>
4 |   <defaulttransactiontype>CASH_REFUND</defaulttransactiontype>
5 | </quicksearch>

```



Important: Quick search portlet objects in SDF cannot search custom transactions.

For more information about quick search portlets, see:

- [Quick Search Portlet](#)
- [quicksearch](#)

Recent Records Portlets in SDF Custom Objects

You can create Recent Records portlets in SuiteCloud Development Framework (SDF) using the `publisheddashboard` SDF custom object. For information about the appropriate context, see [Published Dashboards as XML Definitions](#).

Recent Records portlets in SDF are represented with the `recentrecords` element. You can specify this element in a dashboard of a published dashboard object.

The following example shows a recent records portlet definition:

```

1 | <recentrecords>
2 |   <isminimized>F</isminimized>
3 | </recentrecords>

```

For more information about Recent Records portlets, see the following help topics:

- [Recent Records Portlet](#)

- [Using the Recent Records Menu](#)
- [recentrecords](#)

Recent Reports Portlets in SDF Custom Objects

You can create Recent Reports portlets in SuiteCloud Development Framework (SDF) using the `publisheddashboard` SDF custom object. For information about the appropriate context, see [Published Dashboards as XML Definitions](#).

Recent Reports portlets in SDF are represented with the `recentreports` element. You can specify this element in a dashboard of a published dashboard object.

The following example shows a recent reports portlet definition:

```
1 <recentreports>
2   <isminimized>F</isminimized>
3 </recentreports>
```

For more information about Recent Reports portlets, see the following help topics:

- [The Reports Page](#)
- [recentreports](#)

Report Snapshot Portlets in SDF Custom Objects

You can create Report Snapshot portlets in SuiteCloud Development Framework (SDF) using the `publisheddashboard` SDF custom object. For information about the appropriate context, see [Published Dashboards as XML Definitions](#).

Report Snapshot portlets can show summaries of some reports that are available to you.

Report Snapshot portlets in SDF are represented with the `ehnsnapshots` element. You can specify this element in a dashboard of a published dashboard object.

The following example shows a Report Snapshot portlet definition that displays campaign activity report of the current month:

```
1 <ehnsnapshots>
2   <backgroundtype>GLOBAL_BACKGROUND</backgroundtype>
3   <charttheme>GLOBAL_THEME</charttheme>
4   <charttype>AREA</charttype>
5   <color></color>
6   <comparedaterange></comparedaterange>
7   <comparedaterange2></comparedaterange2>
8   <comparedranges></comparedranges>
9   <compareperiodrange></compareperiodrange>
10  <custom>F</custom>
11  <customsnapshot></customsnapshot>
12  <daterange>THIS_MONTH_TO_DATE</daterange>
13  <graphlayout>COLUMN</graphlayout>
14  <hierarchylevel>3</hierarchylevel>
15  <isgraph>T</isgraph>
16  <isminimized>F</isminimized>
17  <isnewdaterange></isnewdaterange>
18  <listtop>TOP_5</listtop>
19  <orderby></orderby>
20  <orderdesc>T</orderdesc>
21  <periodrange>THIS_PERIOD</periodrange>
22  <snapshot>CAMPAIGN_ACTIVITY</snapshot>
```

```

23 <topx>TOP_5</topx>
24 </enhsnapshots>

```

For more information about Report Snapshot portlets, see the following help topics:

- [Report Snapshot Portlets](#)
- [enhsnapshots](#)

Reminders Portlets in SDF Custom Objects

You can create Reminders portlets in SuiteCloud Development Framework (SDF) using the `publisheddashboard` SDF custom object. For information about the appropriate context, see [Published Dashboards as XML Definitions](#).

A Reminders portlet displays important tasks in NetSuite, such as things that are past due or soon due to be completed. You can define a reminders portlet in a published dashboard using the `reminders` element.

You can define the following kinds of reminders:

- **System-defined reminders**, are reminders provided by NetSuite for frequently needed use-cases. Some system-defined reminders require that the `days` field be specified in the reminder definition.

For the list of available standard reminders that can be specified in SDF, see the help topics [reminders_standard_reminders_with_days](#) and [reminders_standard_reminders_without_days](#). For the equivalent NetSuite user interface names with descriptions, see the help topic [System-Defined Reminders Table](#).

Valid system-defined reminders in SDF depend on the user's role and the features enabled in the target NetSuite account. To determine the NetSuite features that need to be added to the project manifest, consult the SuiteCloud Development Framework XML Reference or validation log.

The following example defines a reminder that displays the number of direct deposits the user needs to approve. Clicking this reminder in NetSuite takes the user to the Approve Direct Deposit Payments page filtered to list all payments pending approval.

```

1 <reminder>
2   <id>DIRECTDEPOSITSTOAPPROVE</id>
3 </reminder>

```

- **Custom reminders**, are reminders defined by you. Custom reminders can be useful when the reminders provided by NetSuite do not meet all of your users' needs. Custom reminders are based on the results of a saved search.



Important: Each custom reminder requires a reference to a savedsearch object. If the referenced savedsearch object is not included in your SuiteCloud project, ensure the manifest file is updated with the dependency. If your SuiteCloud project is a SuiteApp project, it must include the savedsearch object. Only account customization projects can exclude the savedsearch object from the SuiteCloud project.

If you are including the savedsearch object in your SuiteCloud project, the XML must be generated by NetSuite. Do not edit the savedsearch object in SDF.

Before a saved search can be used in the custom reminder, the **Available for Reminders** box must be enabled on that saved search. For more information, see the help topics [Defining a Saved Search](#) and [Creating Custom Reminders](#).

The following example defines a customer reminder that references a saved search object:

```

1 <reminder>
2   <id>[scriptid=customsearch_emps_kwolfe]</id>
3 </reminder>

```

For each reminder, highlighting can be defined using the `highlightingrules` element. For the list of possible color values, see the help topic [reminders_highlighting_rules_colors](#).

The following example defines a reminder that is highlighted red when there are 5 or more employees with authorization expiring in the next 15 days.

```

1 <reminder>
2   <days>15</days>
3   <id>EMPLOYEESWITHEXPIRINGAUTHORIZATION</id>
4   <highlightingrules>
5     <rule>
6       <color>RED</color>
7       <greaterthanorequalto>5</greaterthanorequalto>
8     </rule>
9   </highlightingrules>
10 </reminder>

```

The following global options can be specified for all reminders in the portlet:

- To display the reminder in the reminders portlet even when there is nothing to remind, set `showzerorests` to `T`.
- To minimize the reminder in the reminders portlet, set `isminimized` to `T`.

Reminders can be specified in the following locations of the reminders portlet:

- Adding reminders to the headline element displays the reminders in the upper part of the reminders portlet and with a more prominent layout.

The following example defines a reminder in the headline location:

```

1 <headline>
2   <reminder>
3     <days>7</days>
4     <id>EMPLOYEESWITHUPCOMINGBIRTHDAY</id>
5   </reminder>
6 </headline>

```

- Adding reminders to the other element displays the reminders in the lower part of the reminders portlet with the standard layout.

The following example defines a reminder in the other location:

```

1 <other>
2   <reminder>
3     <id>[scriptid=customsearch_emps_kwolfe]</id>
4   </reminder>
5 </other>

```

The following example defines a Reminders portlet with several reminders as part of a published dashboard object:

```

1 <publisheddashboard scriptid="custpubdashboard_reminders">
2   <center>ACCOUNTCENTER</center>
3   <locknewbar>F</locknewbar>
4   <lockshortcuts>F</lockshortcuts>
5   <name>Reminders</name>
6   <notes/>

```

```

7   <roles>
8     <role>
9       <role>HUMAN_RESOURCES_GENERALIST</role>
10    </role>
11  </roles>
12  <dashboards>
13    <dashboard>
14      <centertab>BASICCENTERHOMEHOME</centertab>
15      <layout>THREE_COLUMN</layout>
16      <mode>UNLOCKED</mode>
17      <leftcolumn>
18        <reminders>
19          <isminimized>F</isminimized>
20          <showzeroreults>T</showzeroreults>
21          <headline>
22            <reminder>
23              <days>5</days>
24              <id>EMPLOYEESWITHUPCOMINGANNIVERSARY</id>
25              <highlightingrules>
26                <rule>
27                  <color>BLUE</color>
28                  <greaterthanorequalto>1</greaterthanorequalto>
29                </rule>
30              </highlightingrules>
31            </reminder>
32            <reminder>
33              <days>7</days>
34              <id>EMPLOYEESWITHUPCOMINGBIRTHDAY</id>
35            </reminder>
36          </headline>
37          <other>
38            <reminder>
39              <id>[scriptid=customsearch_emps_kwolfe]</id>
40            </reminder>
41          </other>
42        </reminders>
43      </leftcolumn>
44    </dashboard>
45  </dashboards>
46 </publisheddashboard>

```

Search Form Portlets in SDF Custom Objects

You can create search form portlets in SuiteCloud Development Framework (SDF) using the `publisheddashboard` SDF custom object. For information about the appropriate context, see [Published Dashboards as XML Definitions](#).

When you use a saved search as your preferred search form, you can reference the saved search in a search form portlet on a dashboard.

Search form portlets in SDF are represented with the `searchform` element. You can specify this element in a dashboard of a published dashboard object.

The following example shows a search form portlet definition that references a custom saved search that was set as a preferred search in the NetSuite account:

```

1  <searchform>
2    <savedsearch>[scriptid=savedsearch_customsearch]</savedsearch>
3  </searchform>

```

For more information about search form portlets and preferred search forms, see the following:

- [Defining a Saved Search as a Preferred Search Form](#)
- [Search Form Portlet](#)
- [searchform](#)

Settings Portlets in SDF Custom Objects

You can create settings portlets in SuiteCloud Development Framework (SDF) using the `publisheddashboard` SDF custom object. For information about the appropriate context, see [Published Dashboards as XML Definitions](#).

Settings portlets in SDF are represented with the `settings` element. You can specify this element in a dashboard of a published dashboard object.

The following example shows a settings portlet definition:

```
1 <settings>
2   <isminimized>F</isminimized>
3 </settings>
```

For more information about Settings portlets, see the following help topics:

- [Settings Portlet](#)
- [settings](#)

Tasks Portlets in SDF Custom Objects

You can create tasks portlets in SuiteCloud Development Framework (SDF) using the `publisheddashboard` SDF custom object. For information about the appropriate context, see [Published Dashboards as XML Definitions](#).

Tasks portlets in SDF are represented with the `tasklinks` element and are predefined by a task record type. You can specify this element in a dashboard of a published dashboard object.

The following example shows a tasks portlet definition:

```
1 <tasklinks>
2   <isminimized>F</isminimized>
3 </tasklinks>
```

For more information about Tasks portlets, see the following help topics:

- [Tasks Portlet](#)
- [list](#)

Trend Graph Portlets in SDF Custom Objects

You can create trend graph portlets in SuiteCloud Development Framework (SDF) using the `publisheddashboard` SDF custom object. For information about the appropriate context, see [Published Dashboards as XML Definitions](#).

Trend graph portlets show the differences in KPIs data over time. Up to five trend graph portlets can be added to a dashboard.

Trend graph portlets in SDF are represented with the `trendgraph` element. You can specify this element in a dashboard of a published dashboard object.

The following example shows a trend graph portlet definition that displays monthly ESTIMATES KPI data in an area chart:

```
1 <trendgraph>
```



```

2      <defaultcharttype>AREA</defaultcharttype>
3      <kpi>ESTIMATES</kpi>
4      <trendtype>MONTHLY</trendtype>
5      <showmovingaverage>T</showmovingaverage>
6      <movingaverageperiod>2</movingaverageperiod>
7      <showlastdatapoint>T</showlastdatapoint>
8      <includezeroonyaxis>F</includezeroonyaxis>
9      <charttheme>GLOBAL_THEME</charttheme>
10     <backgroundtype>GLOBAL_BACKGROUND</backgroundtype>
11     <customseriescolor>#ABCDEF</customseriescolor>
12 </trendgraph>

```

For more information about trend graph portlets, see:

- [Trend Graph Portlets](#)
- [trendgraph](#)

Scripting

The following topics provide SuiteCloud Development Framework (SDF) information about the SDF custom objects that pertain to scripting:

- [User Event Scripts as XML Definitions](#)
- [Scheduled Scripts as XML Definitions](#)
- [SDF Installation Scripts as XML Definitions](#)
- [Workflows as XML Definitions](#)
- [Translation Collections as XML Definitions](#)
- [Map/Reduce Script Objects as XML Definitions](#)
- [Portlet Scripts as XML Definitions](#)
- [Client Scripts as XML Definitions](#)
- [Mass Update Scripts as XML Definitions](#)
- [Suitelet Scripts as XML Definitions](#)
- [RESTlet Scripts as XML Definitions](#)
- [Integration Records as XML Definitions](#)
- [Single Page Applications as XML Definitions](#)
- [Custom Tool Scripts as XML Definitions](#)

User Event Scripts as XML Definitions

You can import user event scripts from a target NetSuite account into your SuiteCloud projects. User event scripts are triggered on the NetSuite server when a user performs certain actions on a record. These actions are called user events and can occur when a user creates, loads, updates, copies, deletes or submits a record. For more information about user event scripts, see the help topic [SuiteScript 2.x User Event Script Type](#).

You can create a user event script for a NetSuite account using SuiteCloud Development Framework (SDF) by creating a `usereventscript` SDF custom object in a SuiteCloud project and then deploying it to a target account. You can also import a user event script from a NetSuite account into your SuiteCloud project.

For information about importing SDF custom objects, files, and scripts from a NetSuite account into your SuiteCloud project, see [Account Component Imports to SuiteCloud Projects](#).

Components of a Script Object

There are three components that are required for the script object:

1. **The Scriptid Attribute:** Provide a readable scriptid attribute for the script record and scriptdeployment structure by adding an underscore to the default value followed by a meaningful name for the object. The script record's scriptid attribute must start with a customscript_ prefix and each scriptdeployment structure's scriptid attribute must start with a customdeploy_ prefix.
2. **The Script Record:** Represented in XML by the object. The object contains several elements that define it, including the object name and a reference to the location of the script file.
3. **The Script Deployment Record:** Represented in XML by the scriptdeployments structure. A script object may contain multiple script deployments.

Components of a User Event Script Object

The user event script object has two important components that allow it to work:

- **The Script Record-** Represented in XML by the usereventscript SDF custom object. This object contains several elements that define it, including the name of the user event script object and a reference to the location of the user event script file. The following example shows a usereventscript object named "User Event Example" that references a user event script file named "UserEventScript.js".

```

1 <usereventscript scriptid="customscript_userevent_example">
2   <name>User Event Example</name>
3   <notifyowner>T</notifyowner>
4   <scriptfile>[/SuiteScripts/UserEventScript.js]</scriptfile>
5   <scriptdeployments>
6     <scriptdeployment scriptid="customdeploy_userevent_example">
7       <isdeployed>T</isdeployed>
8       <loglevel>DEBUG</loglevel>
9       <recordtype>[customrecord_userevent_example]</recordtype>
10      <status>TESTING</status>
11    </scriptdeployment>
12  </scriptdeployments>
13 </usereventscript>

```

- **The Script Deployment Record-** Represented by the scriptdeployments structure. It is contained within the usereventscript object where you can define certain values such as the record type and status. The following example shows the recordtype element referencing a custom record named "customrecord_userevent_example". The status element has a value set to "TESTING". This example only has one script deployment, but a user event script object may contain multiple script deployments.

```

1 <usereventscript scriptid="customscript_userevent_example">
2   <name>User Event Example</name>
3   <notifyowner>T</notifyowner>
4   <scriptfile>[/SuiteScripts/UserEventScript.js]</scriptfile>
5   <scriptdeployments>
6     <scriptdeployment scriptid="customdeploy_userevent_example">
7       <isdeployed>T</isdeployed>
8       <loglevel>DEBUG</loglevel>
9       <recordtype>[customrecord_userevent_example]</recordtype>
10      <status>TESTING</status>
11    </scriptdeployment>
12  </scriptdeployments>
13 </usereventscript>

```

For more information about:

- `usereventscript` object fields and possible values, see the help topic [usereventscript](#).
- `scriptdeployment` structure fields and possible values, see the help topic [scriptdeployment](#).
- Creating a generic script record in a NetSuite account, see the help topic [Creating a Script Record](#).
- Creating a deployment record in a NetSuite account, see the help topic [Deploying a Script by using the Deployments Sublist](#).

Setting Values for a Script Object

Each script object has a set of values that are required to successfully validate its script record and script deployment record.

The following elements are required to successfully validate your script record:

- `name` —Provide a meaningful name for your script record.
- `scriptfile` —Reference the appropriate script file. This must be a JavaScript file (.js).

The following element is required to successfully validate your script deployments:

- `status` —Provide a value for the status element. The default value is TESTING. The only values accepted for all scripts except scheduled scripts are:
 - RELEASED: The script will run in the accounts of all specified audience members.
 - TESTING: The script will execute for the script owner and specified audience.

The remaining possible values for the status element are:

- COMPLETED
- INPROGRESS
- INQUEUE
- NOTSCHEDULED
- SCHEDULED

Setting Values for a User Event Script Object

Provide a readable `scriptid` attribute for the `usereventscript` object and `scriptdeployment` structure by adding an underscore to the default value followed by a meaningful name for the object. The `usereventscript` object's `scriptid` attribute must start with a "customscript_" prefix and each `scriptdeployment` structure's `scriptid` attribute must start with a "customdeploy_" prefix.

The following elements are required to successfully validate a `usereventscript` object:

- `name`- Provide a meaningful name for your script record.
- `scriptfile`- Reference the appropriate script file. This must be a JavaScript file (.js).

The following elements are required to successfully validate a `scriptdeployment` structure:

- `recordtype`- Reference the appropriate record within your deployment script so that your user event script deploys to the correct record.
- `status`- Provide a value for the status element. The default value is TESTING. Possible values are:
 - COMPLETED
 - INPROGRESS
 - INQUEUE
 - NOTSCHEDULED

- RELEASED
- SCHEDULED
- TESTING

Example of a User Event Script Object

The following example shows a `usereventscript` object containing multiple `scriptdeployments`.

```

1 <usereventscript scriptid="customscript_userevent_example">
2   <name>User Event Example</name>
3   <notifyowner>T</notifyowner>
4   <scriptfile>[/SuiteScripts/UserEventScript.js]</scriptfile>
5   <scriptdeployments>
6     <scriptdeployment scriptid="customdeploy_userevent_example">
7       <isdeployed>T</isdeployed>
8       <loglevel>DEBUG</loglevel>
9       <recordtype>[customrecord_userevent_example]</recordtype>
10      <status>TESTING</status>
11    </scriptdeployment>
12    <scriptdeployment scriptid="customdeploy_test1">
13      <isdeployed>T</isdeployed>
14      <loglevel>DEBUG</loglevel>
15      <recordtype>[customrecord_test1]</recordtype>
16      <status>TESTING</status>
17    </scriptdeployment>
18    <scriptdeployment scriptid="customdeploy_test2">
19      <isdeployed>T</isdeployed>
20      <loglevel>DEBUG</loglevel>
21      <recordtype>[customrecord_test2]</recordtype>
22      <status>TESTING</status>
23    </scriptdeployment>
24  </scriptdeployments>
25 </usereventscript>

```

Scheduled Scripts as XML Definitions

Scheduled scripts are represented in SuiteCloud Development Framework (SDF) by `scheduledscript` objects. For information about available fields, see the reference documentation for the `scheduledscript` element in .

To create a `scheduledscript` object, perform one the following procedures:

Procedure	Details
In SuiteCloud IDE, create a scheduled script in a NetSuite account.	For information about scheduled scripts, see the help topic SuiteScript 2.x Scheduled Script Type .
In the NetSuite user interface, create a scheduled script in a NetSuite account.	See the following topics: ■ Creating a Script Record ■ Scheduled Script Deployment Record Import the <code>scheduledscript</code> object into your SuiteCloud project. See Account Component Imports to SuiteCloud Projects.
In SuiteCloud plug-ins and extensions, create a scheduled script using a template.	See the following topics: ■ Creating a SuiteScript File in SuiteCloud Extension for Visual Studio Code ■ Creating an XML Definition for an SDF Custom Object in SuiteCloud IDE Plug-in for WebStorm

Procedure	Details
	 Tip: You can modify the NetSuite XML definition template used for scheduledscript objects. See  SuiteCloud IDE Plug-in for Eclipse Guide .

For information about working with scheduled scripts in SDF, see the following topics:

- [Defining a Recurring Scheduled Script Deployment](#)
- [Scheduled Script Example](#)

Defining a Recurring Scheduled Script Deployment

Scripts are scheduled in SuiteCloud Development Framework (SDF) using recurrence fields on the scriptdeployment field in the scheduledscript object.

Additional information about scheduled script deployment can be found in [Scheduled Script Submission](#).

SDF includes the following validations for recurrence fields:

- The date and time format must be specified in the correct format. For information about date and time formats, see [Date and Time Formats in SDF Custom Objects](#).
- Only a single recurrence field can be specified for each script deployment.

The following table contains descriptions for the recurrence fields. For a list of possible values, see the reference documentation for **recurrence** in the scheduledscript element in .

Recurrence Fields	Description
daily	Schedules the script to deploy every x number of days, with a start date and end date. The event can repeat.
everyweekday	Schedules the script to deploy every week day, with a start date and end date. The event can repeat.
monthly	Schedules the script to deploy every x number of months, with a start date and end date. The event can repeat.
monthlydayofweek	Schedules the script to deploy on a day of the week every x number of months, with a start date and end date. The day of the week can be specified based on its placement. For example, first, second, third, fourth, or last. The event can repeat.
single	Schedules the script to deploy as a single event with a start date and start time. The event can repeat.
weekly	Schedules the script to deploy every x number of weeks, with a start date and end date. Specific days of the week are Boolean fields. The event can repeat.
yearly	Schedules the script to deploy every year with a start date and end date. The event can repeat.
yearlydayofweek	Schedules the script to deploy every year based on a day of the week, with a start date and end date. The day of the week can be specified based on its placement. For example, first, second, third, fourth, or last. The event can repeat.

Recurrence Examples

The following table contains examples of fields used for recurring schedule events.

Recurrence Field	Example	Description
daily	<pre> 1 <recurrence> 2 <daily> 3 <everyxdays>3</everyx 4 days> 5 <startdate>2020-08-27</ 6 startdate> 7 <starttime>00:00:05Z</ 8 starttime> 9 <repeat>PT12H</repeat> 10 <enddate>2021-08-27Z</ 11 enddate> 12 </daily> 13 </recurrence> </pre>	Schedules the script to deploy every 3 days, starting August 27, 2020 at 12:00 a.m Eastern Standard Time (North America). The script deployment repeats after 12 hours. The recurrence ends August 27, 2021.
everyweekday	<pre> 1 <recurrence> 2 <everyweekday> 3 <startdate>2020-09-27</ 4 startdate> 5 <starttime>00:00:00Z</ 6 starttime> 7 <repeat>PT15M</repeat> 8 <enddate>2021-10-24Z</ 9 enddate> 10 </everyweekday> 11 </recurrence> </pre>	Schedules the script to deploy every weekday, starting September 27, 2020 at 12:00 a.m. The script deployment repeats after 15 minutes. The recurrence ends October 24, 2021.
monthly	<pre> 1 <recurrence> 2 <monthly> 3 <everyxmonths>1</everyx 4 months> 5 <startdate>2020-12-21</ 6 startdate> 7 <starttime>00:00:00Z</ 8 starttime> 9 <repeat>PT30M</repeat> 10 <enddate>2021-06-21Z</ 11 enddate> 12 </monthly> 13 </recurrence> </pre>	Schedules the script to deploy every month, starting December 21, 2020 at 12:00 a.m. The script deployment repeats after 30 minutes. The recurrence ends June 21, 2021.
monthlydayofweek	<pre> 1 <recurrence> 2 <monthlydayofweek> 3 <orderofweek>LAST</ 4 orderofweek> 5 <dayofweek>FRIDAY</ 6 dayofweek> 7 <everyxmonths>2</everyx 8 months> 9 <startdate>2020-09-15</ 10 startdate> 11 <starttime>00:00:00Z</ 12 starttime> 13 <repeat>PT8H</repeat> 14 <enddate>2021-08-16Z</ 15 enddate> 16 </monthlydayofweek> 17 </recurrence> </pre>	Schedules the script to deploy the last Friday of every second month. The start date is September 15, 2020 at 12:00 a.m. The script deployment repeats after 8 hours. The recurrence ends August 16, 2021.
single	<pre> 1 <recurrence> 2 <single> 3 <startdate>2020-08-27</ 4 startdate> 5 <starttime>00:00:00Z</ 6 starttime> </pre>	Schedules the script to deploy on July 27, 2020 at 12:00 a.m. The script deployment repeats after 2 hours.

Recurrence Field	Example	Description
	<pre> 5 <repeat>PT2H</repeat> 6 </single> 7 </recurrence> </pre>	
weekly	<pre> 1 <recurrence> 2 <weekly> 3 <everyxweeks>1</ everyxweeks> 4 <monday>F</monday> 5 <tuesday>T</tuesday> 6 <wednesday>F</wednesday> 7 <thursday>T</thursday> 8 <friday>T</friday> 9 <saturday>F</saturday> 10 <sunday>T</sunday> 11 <startdate>2020-11-27</ startdate> 12 <starttime>00:00:00Z</ starttime> 13 <repeat>PT1H</repeat> 14 <enddate>2021-10-27Z</ enddate> 15 </weekly> 16 </recurrence> </pre>	Schedules the script to deploy every week on Tuesday, Thursday, Friday, and Sunday. The start date is November 27, 2020 at 12:00 a.m. The script deployment repeats after 1 hour. The recurrence ends October 27, 2021.
yearly	<pre> 1 <recurrence> 2 <yearly> 3 <startdate>2020-06-18</ startdate> 4 <starttime>00:00:00Z</ starttime> 5 <repeat>PT30M</repeat> 6 <enddate>2025-06-18Z</ enddate> 7 </yearly> 8 </recurrence> </pre>	Schedules the script to deploy every year starting June 18, 2020 at 12:00 a.m. The script deployment repeats after 30 minutes. The recurrence ends June 18, 2025.
yearlydayofweek	<pre> 1 <recurrence> 2 <yearlydayofweek> 3 <orderofweek>THIRD</ orderofweek> 4 <dayofweek>MONDAY</ dayofweek> 5 <startdate>2020-07-25</ startdate> 6 <starttime>00:00:00Z</ starttime> 7 <repeat>PT12H</repeat> 8 <enddate>2021-08-27Z</ enddate> 9 </yearlydayofweek> 10 </recurrence> </pre>	Schedules the script to deploy the Monday of every third week, starting July 25, 2020 at 12:00 a.m. The script deployment repeats after 12 hours. The recurrence ends August 27, 2021.

Scheduled Script Example

The following is an XML definition example of a scheduled script object in SuiteCloud Development Framework (SDF).

The scheduled script object contains a script deployment that is scheduled to deploy the script on the second Monday of every month, starting October 10, 2020 at 1:00 a.m. The recurrence ends April 10, 2021.

For more information about scheduled scripts, see the help topic [SuiteScript 2.x Scheduled Script Type](#).

```
1 | <scheduledscript scriptid="customscript_tps_scheduled_script">
```

```

2 <defaultfunction>http</defaultfunction>
3 <description></description>
4 <isinactive>F</isinactive>
5 <name>TPS Scheduled Script</name>
6 <notifyadmins>F</notifyadmins>
7 <notifyemails></notifyemails>
8 <notifyowner>T</notifyowner>
9 <scriptfile>[/SuiteScripts/TPSscheduledScript.js]</scriptfile>
10 <scriptdeployments>
11   <scriptdeployment scriptid="customdeploy1">
12     <isdeployed>T</isdeployed>
13     <loglevel>AUDIT</loglevel>
14     <status>SCHEDULED</status>
15     <title>TPS Scheduled Script</title>
16     <recurrence>
17       <monthlydayofweek>
18         <dayofweek>MONDAY</dayofweek>
19         <enddate>2021-04-10</enddate>
20         <everyxmonths>1</everyxmonths>
21         <orderofweek>SECOND</orderofweek>
22         <repeat></repeat>
23         <startdate>2020-10-10</startdate>
24         <starttime>01:00:00Z</starttime>
25       </monthlydayofweek>
26     </recurrence>
27   </scriptdeployment>
28 </scriptdeployments>
29 </scheduledscript>

```

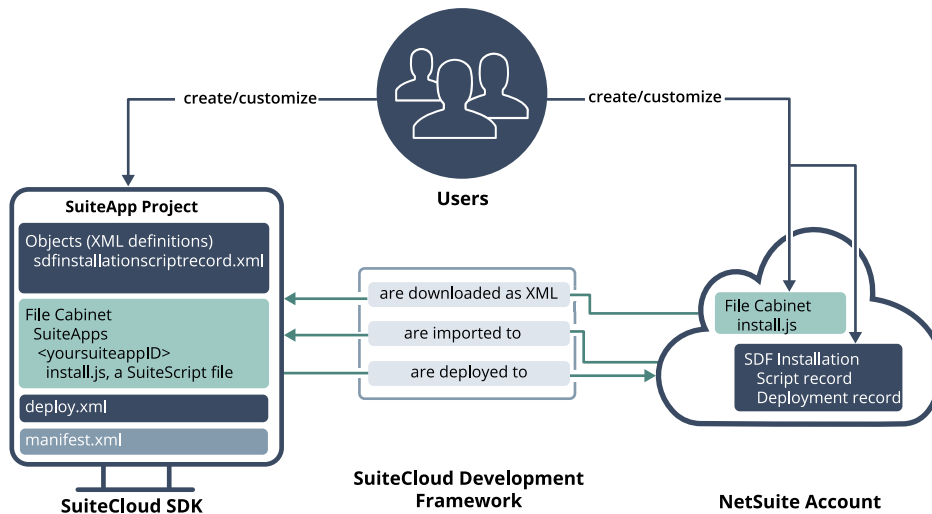
SDF Installation Scripts as XML Definitions

Executing Installation Scripts

You can use SuiteCloud Development Framework (SDF) to create scripts that run while SuiteApp deployments are installed to a target NetSuite account. These types of scripts are called SuiteScript 2.x SDF installation scripts. You can use SDF installation scripts to customize installation. For example, to do the following tasks on a target NetSuite account:

- Verify the account and run your own validations.
- Check for required permissions and features prior to installation. For an example, see [SDF Installation Script Example of Validating an Account Preference Before Installation](#).
- Stop installation when a requirement is missing in the account.
- Migrate data that is required by an application. For an example, see [SDF Installation Script Example of Customizing a SuiteApp Update](#).

The SDF installation script records and deployments are represented in SDF as `sdfinstallationscript` objects. The SuiteScript files referenced by a `sdfinstallationscript` object are stored in the FileCabinet folder.



Note: For SDF installation scripts and SuiteCloud project types:

- SuiteApp projects only support SDF installation scripts.
- Account customization projects do not support SDF installation scripts.

To use SDF installation scripts, do the following tasks in your project:

- In the File Cabinet folder, include a SuiteScript file as a SDF installation script type. For more information about SuiteScript and using this script type, see the help topic [SuiteScript 2.x SDF Installation Script Type](#).
- In the Objects folder, the sdfinstallationscript XML object represents the script record. It can also contain deployment records as XML substructures. For information about available fields, see the reference documentation for the sdfinstallationscript element in .
- In the deploy file, include a run element and script element containing the following:
 - a path to script object
 - a reference to the script deployment

For an example, see [Deploy File that Runs SDF Installation Script Example](#). For information about reference format, see [Syntax Guidelines and Reference Formats for SDF Custom Objects](#).

 - From the deploy file, specify when you want the script to run per the order of appearance in the deploy file. For example, by placing your SDF installation script reference at the end of the deploy file, you instruct the script to run after the SuiteApp is installed. For more information, see [Customizing SuiteApp Project Deployment using SDF Installation Scripts](#).

For more information about using SDF installation scripts in a SuiteApp project, see the following topics:

- [SDF Installation Script Example of Validating an Account Preference Before Installation](#)
- [Handling Exceptions and Monitoring SDF Installation Scripts](#)

Defining a Script Record and Deployment for an SDF Installation Script

In SuiteCloud Development Framework (SDF), you can create new scripts and script records in your SuiteApp project or import existing scripts and records from a NetSuite account. To perform these tasks, see the following topics:

- From your SuiteApp project, to create a new SDF installation script, see the help topic [SuiteScript 2.x SDF Installation Script Type](#).
- From a NetSuite account, to add a new SDF installation script, see the help topic [Creating a Script Record](#).
- From your SDF SuiteApp project, to add a new or existing SDF installation script, see:
 - [Creating a SuiteScript File in SuiteCloud Extension for Visual Studio Code](#).
 - [Creating an XML Definition for an SDF Custom Object in SuiteCloud IDE Plug-in for WebStorm](#).
 - [Account Component Imports to SuiteCloud Projects](#).

Note: To use the NetSuite XML definition template for `sdfinstallationscript` objects, see [SuiteCloud IDE Plug-in for Eclipse Guide](#). The `sdfinstallationscript` object includes a reference to the SuiteScript file and information about the file (through script record and script deployment definitions). The SuiteScript content is not part of the XML definition.

The following image shows the SDF installation script record as it appears in the NetSuite UI.

The screenshot displays the NetSuite interface for a script record. At the top, the title 'Script' is followed by buttons for 'Edit', 'Back', 'Deploy Script', and an 'Actions' dropdown. The record details are as follows:

TYPE	DESCRIPTION
SDF Installation	
NAME	OWNER
Install Script Demo	IDE IDE
ID	☐ INACTIVE
customscript_install_script	
API VERSION	
2.0	

Below the details is a tabbed interface with 'Scripts' selected. The 'SCRIPT FILE' section shows 'preview checkfeature_fail.js' with 'download' and 'Edit' links, and a checked 'RUN FUNCTION' checkbox. The 'Custom Plug-In Types' section shows a table with the header 'CUSTOM PLUG-IN TYPE' and the message 'No records to show.' At the bottom, there are buttons for 'Edit', 'Back', 'Deploy Script', and an 'Actions' dropdown.

The following image shows an SDF installation deployment record as it appears in the NetSuite UI.

Script

Edit

Back

Deploy Script

Actions

Type

SDF Installation

Name

SDF Installation

ID

customscript_install_script

API Version

2.1

Execute as version

2.1

Description

Owner

☐ Inactive

Scripts

Parameters

Unhandled Errors

Execution Log

Deployments

System Notes

Title	ID	Status	Log Level
Demo Deployment	customdeploy1	Testing	Debug

Note: The status field is set to Testing, meaning that the script deploys for the deployment record owner only. Set the status to Released when you want others to be able to run the script.

For more information about SuiteScript deployment and script records, see the following:

- [Setting Script Deployment Status](#)
- [SuiteScript Developer Guide](#)
- [SDF Installation Script Example of Customizing a SuiteApp Update](#)
- [SuiteScript 2.x](#)
- [SuiteScript 2.1](#)
- [SuiteScript Versioning Guidelines](#)

SDF Installation Script Example of Validating an Account Preference Before Installation

The following components are needed to trigger the SuiteCloud Development Framework (SDF) installation script for this example.

- [SDF Installation SuiteScript File Example](#)
- [SDF Installation Script Object Example](#)
- [Deploy File that Runs SDF Installation Script Example](#)
- [Deployment Log Example](#)

For a more advanced example, see [SDF Installation Script Example of Customizing a SuiteApp Update](#).

SDF Installation SuiteScript File Example

The following example shows how to use SuiteScript with a SuiteCloud Development Framework (SDF) installation script. This installation script determines whether the Email Employee on Approvals preference in the General Settings of your target account is enabled

It performs the to the following tasks:

- The Email Employee on Approvals is a box in the UI so the `getPreference` method returns a Boolean value, indicating the value set for this preference in the target account.
- If the Email Employee on Approvals preference is not enabled, a message is thrown that will appear in scripting log and in IDE console and deployment of the SuiteApp fails.

- If the Email Employee on Approvals preference is enabled, project deployment continues and you can run more scripts and tasks on the installation when needed.
- [SDF Installation Script Object Example](#)
- [Deploy File that Runs SDF Installation Script Example](#)

For information about SuiteScript API used and the runtime module, see:

- [User.getPreference\(options\)](#)
- [N/runtime Module](#)

The `checkpreference_fail.js` script example that follows is an SDF installation script that uses SuiteScript to ensure the Email Employee on Approvals preference is enabled in your target account before installing your SuiteCloud project. The script must be in your SuiteApp project, saved in the File cabinet as a .JS file, and referenced as a dependency in the manifest file. To run the script, there must be a corresponding SDF installation script record and deployment (`sdfinstallationobject`) in the project and you must also include a valid run tag in the project deploy file. For more information, see the following topics:



Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType SDFInstallationScript
4   */
5   define(['N/runtime'], (runtime) => {
6     const run = (scriptContext) => {
7       function checkPrerequisites() {
8         if (!runtime.getCurrentUser().getPreference({name: 'emailemployeeonapproval'})) {
9           throw 'The Email Employee on Approvals preference must be enabled. ' +
10             'SDF INSTALL SCRIPT MESSAGE: Please enable the preference and try again.';
11         }
12       }
13     }
14     return {run}
15   });

```

This installation script checks in the General Preferences of your target account if the Email Employee on Approvals preference is enabled. The following image shows the preference as it appears in the NetSuite UI.

The screenshot shows the 'General Preferences' window in NetSuite. On the right side, under the 'Log System Notes on Update Only' section, the 'Email Employee on Approvals' checkbox is checked and highlighted with a red rectangle. Other visible preferences include 'Number of Rows in List Segments' set to 50, 'Maximum Entries in Dropdowns' set to 200, and 'Default Customer Type' set to Company.

SDF Installation Script Object Example

The following example is an XML definition of the SuiteCloud Development Framework (SDF) installation script that executes the `checkpreference_fail.js` SuiteScript in [SDF Installation Script Example of Validating an Account Preference Before Installation](#).

```

1  <sdfinstallationscript scriptid="customscript_install_script">
2    <name>Install Script Demo</name>
3    <notifyowner>T</notifyowner>
4    <notifyuser>F</notifyuser>
5    <scriptfile>[/SuiteApps/com.netsuite.example/checkpreference_fail.js]</scriptfile>
6    <scriptdeployments>
7      <scriptdeployment scriptid="customdeploy1">
8        <isdeployed>T</isdeployed>
9        <loglevel>ERROR</loglevel>
10       <status>RELEASED</status>
11       <title>Demo Deployment</title>
12     </scriptdeployment>
13   </scriptdeployments>
14 </sdfinstallationscript>

```

To run the script referenced in this definition, you must update the SuiteApp project deploy file. See [Deploy File that Runs SDF Installation Script Example](#).

For more information about SDF installation script, see the help topic [SuiteScript 2.x SDF Installation Script Type](#).

Deploy File that Runs SDF Installation Script Example

The following deploy file specifies to run the SuiteCloud Development Framework (SDF) installation script when the SuiteApp project is deployed.

```

1  <deploy>
2    <run>
3      <script>
4        <path>~/Objects/customscript_install_script.xml</path>
5        <deployment>customdeploy1</deployment>
6      </script>
7    </run>
8    <files>
9      <path>~/FileCabinet/SuiteApps/com.example/*</path>
10    </files>
11    <objects>
12      <path>~/Objects/*</path>
13    </objects>
14  </deploy>

```

Deployment Log Example

The following SuiteCloud IDE console output shows successful deployment of the components required to start processing of a SuiteCloud Development Framework (SDF) installation script:

- the SDF installation script, `checkpreference_fail` SuiteScript file
- the script record definition, `customscript_install_script` object
- the deployment definition, `customscript_install_script.customdeploy1` element

```

1  ...
2  Validate account settings -- Success
3  Validate Custom Objects against the Account -- Success
4  Info -- Current installed version [1.0.0]
5  Begin deployment -- com.netsuite.example(1.0.0)
6  Upload file -- ~/FileCabinet/SuiteApps/com.netsuite.example/checkpreference_fail.js
7  Create object -- customscript_install_script (sdfinstallationscript)

```

```

8 | Creat object -- customscript_install_script.customdeploy1 (scriptdeployment)
9 | Run installation script -- ~Objects/customscript_install_script.xml
10 | ...

```

For more deployment output samples, see [Handling Exceptions and Monitoring SDF Installation Scripts](#).

SDF Installation Script Example of Customizing a SuiteApp Update

The following example shows all the coding components that are required in a SuiteApp to run a SuiteCloud Development Framework (SDF) installation script. During deployment, the SDF installation script migrates an older version of the SuiteApp in the target account to the newer version, for upgrading a SuiteApp from 4.0 to 4.1.

The following components are needed to run the SDF installation script:

- [SDF Installation SuiteScript File Example](#)
- [SDF Custom Objects in SuiteApps Example](#)
- [Deploy File that Executes the SDF Installation Script Example](#)

For another SDF Installation script example, see [SDF Installation Script Example of Validating an Account Preference Before Installation](#).

SDF Installation SuiteScript File Example

The following code sample is the SuiteCloud Development Framework (SDF) installation script that is saved in the File Cabinet folder and named `migrate.js`. When the `migrate.js` SuiteScript file is triggered by deployment of SuiteApp v4.1 to an account with SuiteApp v4.0, it creates a “My New Field” transaction body field on the custom transaction record named “My Vendor Payment” and sets a default value for instances in the account.

These are the tasks that occur when `migrate.js` is executed:

- Accesses the `fromVersion` and `toVersion` parameters to get the current SuiteApp version installed to the account and the SuiteApp version to be installed when the SDF SuiteApp is deployed. If the account version is 4.0.0, the `migrate` function will run.
- The `migrate` function sets up the account for the 4.1.0 version by updating transaction records with a new custom transaction body field and setting it to a default value based on options from an existing custom transaction type.
- The `migrate` function runs the following:
 - Creates an ad-hoc search that finds instances of the My Vendor Payment record. Uses an `each` function to iterate through the results.
 - During the iteration, each My Vendor Payment instance is loaded and the value of My New Field (`custbody_install_script`) is set to the value of the `custscript1` parameter (also know as script custom field). That parameter is `val_a` of the `customlist_transaction_field` list. My New Field takes values from the same list (`customlist_transaction_field`).

```

1 | /**
2 |  * @NApiVersion 2.x
3 |  * @NScriptType SDFInstallationScript
4 |  */
5 | define(['N/record', 'N/search', 'N/runtime'], function(record, search, runtime) {
6 |     function migrate(context) {
7 |         if (context.fromVersion === '4.0.0' && context.toVersion === '4.1.0') {
8 |             var tSearch = search.create({
9 |                 type: search.Type.TRANSACTION,
10 |                 columns: [{
11 |                     name: 'internalid'

```

```

12     }},
13     filters: [{
14         name: 'recordtype',
15         operator: search.Operator.IS,
16         values: ['customtransaction_install_script']
17     }]
18 });
19 tSearch.run().each(function(result) {
20     var tranInternalId = result.getValue({name: 'internalid'});
21     var tran = record.load({
22         type: 'customtransaction_install_script',
23         id: tranInternalId
24     });
25     var fieldValue = runtime.getCurrentScript().getParameter({name: 'custscript1'});
26     tran.setValue({
27         fieldId: 'custbody_install_script',
28         value: fieldValue
29     });
30     tran.save();
31 })
32 }
33 }
34 return {
35     run: function run(context) {
36         migrate(context);
37     }
38 };
39 });

```

SDF Custom Objects in SuiteApps Example

The following sdfinstallationscript object is as an XML definition that is kept in the Objects folder of the project. It defines the SuiteCloud Development Framework (SDF) installation record and the deployment for the migrate.js SDF installation script that migrates accounts with SuiteApp 4.0 to SuiteApp 4.1.

In the custscript1 field definition, the customlist_transaction_field record type is selected, which is a custom list holding the following values: val_a, val_b, val_c. The script deployment definition specifies val_a for the custscript1 script field (also known as a script parameter).

```

1 <sdfinstallationscript scriptid="customscript_install_script">
2   <description>This is my example sdf installation script</description>
3   <isinactive>F</isinactive>
4   <name>Install Script Demo</name>
5   <notifyadmins>F</notifyadmins>
6   <notifyemails></notifyemails>
7   <notifyowner>T</notifyowner>
8   <notifyuser>F</notifyuser>
9   <scriptfile>[/SuiteApps/com.example/migrate.js]</scriptfile>
10  <scriptcustomfields>
11    <scriptcustomfield scriptid="custscript1">
12      <accesslevel>2</accesslevel>
13      <applyformatting>F</applyformatting>
14      <checkspelling>F</checkspelling>
15      <defaultchecked>F</defaultchecked>
16      <displaytype>NORMAL</displaytype>
17      <fieldtype>SELECT</fieldtype>
18      <isformula>F</isformula>
19      <ismandatory>F</ismandatory>
20      <label>test</label>
21      <onparentdelete>NO_ACTION</onparentdelete>
22      <searchlevel>2</searchlevel>
23      <selectrecordtype>[scriptid=customlist_transaction_field]</selectrecordtype>
24      <storevalue>T</storevalue>
25    </scriptcustomfield>
26  </scriptcustomfields>
27  <scriptdeployments>
28    <scriptdeployment scriptid="customdeploy1">
29      <custscript1>[scriptid=customlist_transaction_field.val_a]</custscript1>
30      <isdeployed>T</isdeployed>
31      <loglevel>ERROR</loglevel>

```

```

32     <status>RELEASED</status>
33     <title>Demo Deployment</title>
34   </scriptdeployment>
35 </scriptdeployments>
36 </sdfinstallationscript>

```

The following XML definition is a custom transaction type which the new field is created on. The name of the transaction type is "My Vendor Payment" and this object is contained in v4.0 of the SuiteApp. The transaction body field is new for the upgraded SuiteApp (version 4.1) that the SDF installation script is verifying is installed to the target account.

```

1 <customtransactiontype scriptid="customtransaction_install_script">
2   <name>My Vendor Payment</name>
3   <subliststyle>BASIC</subliststyle</name>
4 </customtransactiontype>

```

The following XML definition is a list record containing values that can be set on the "My New Field" custom transaction body field, on the "My Vendor Payment" custom transaction.

```

1 <customlist scriptid="customlist_transaction_field">
2   <inactive>F</inactive>
3   <isordered>T</isordered>
4   <name>test script param </name>
5   <customvalues>
6     <customvalue scriptid="val_a">
7       <inactive>F</inactive>
8       <value>A</value>
9     </customvalue>
10    <customvalue scriptid="val_b">
11      <inactive>F</inactive>
12      <value>B</value>
13    </customvalue>
14    <customvalue scriptid="val_c">
15      <inactive>F</inactive>
16      <value>C</value>
17    </customvalue>
18  </customvalues>
19 </customlist>

```

The following XML definition is the new transaction body field that is part of the v4.1 SuiteApp. The "My New Field" field can be set to values defined in the "customlist_transaction_field" object and it appears on the Custom subtab of the My Vendor Payment.

```

1 <transactionbodycustomfield scriptid="custbody_install_script">
2   <accesslevel>2</accesslevel>
3   <displaytype>NORMAL</displaytype>
4   <fieldtype>SELECT</fieldtype>
5   <label>My New Field</label>
6   <storevalue>T</storevalue>
7   <selectrecordtype>[scriptid=customlist_transaction_field]</selectrecordtype>
8   <bodycustomtransactions>[scriptid=customtransaction_install_script]</bodycustomtransactions>
9 </transactionbodycustomfield>

```

Deploy File that Executes the SDF Installation Script Example

The following SuiteCloud Development Framework (SDF) XML example is the deploy file that deploys the objects in the v4 folder to the target account, followed by v41 folder objects. The SDF installation script executes at the end of deployment.

```

1 <deploy>
2   <files>
3     <path>~/FileCabinet/SuiteApps/com.example/*</path>
4   </files>
5   <objects>

```



```

6      <path>~/Objects/v4/*</path>
7    </objects>
8  </objects>
9    <path>~/Objects/v41/*</path>
10  </objects>
11  <run>
12    <script>
13      <path>~/Objects/v4/customscript_install_script.xml</path>
14      <deployment>customdeploy1</deployment>
15    </script>
16  </run>
17 </deploy>

```

The following example of the updated transaction record as it appears in the NetSuite UI after installing SuiteApp v4.1:

My Vendor Payment 🔍

1

Edit Back Actions

Primary Information

ENTRY NO. 1	DATE 11/19/2018
AMOUNT 1.00	POSTING PERIOD
CURRENCY USA	MEMO
EXCHANGE RATE 1.00	

Classification

SUBSIDIARY
Parent Company

Lines Communication System Information **Custom** GL Impact

MY NEW FIELD
A

Edit Back Actions

Workflows as XML Definitions

Create and manage workflows using SuiteCloud Development Framework (SDF). You can model a workflow definition in a SuiteCloud project and deploy the project to a target NetSuite account. You can either create new workflows in your project or import existing workflows from a NetSuite account. For information about working with workflows in a NetSuite account, see:

- [Required Permissions for SuiteFlow](#)
- [Account Component Imports to SuiteCloud Projects](#)

Start designing a workflow SDF custom object by specifying definition fields. These fields contain basic information about the workflow and are similar to the properties specified on the workflow definition page, such as the name, description, record types, and release status. For more information about all available definition fields and how they are defined in SDF, see the following:

- [workflow](#)
- [Workflow Definition Page](#)
- [Workflow Base Record](#)

- [Execution Contexts and Workflows](#)
- [Context Type Examples](#)

SDF supports both event-based and scheduled workflow types. The workflow type determines which fields should be specified in the workflow object.

Basic Definition Fields

There are some basic fields that are used to define a workflow. The following example defines them for a workflow used for approvals:

```

1 <workflow scriptid="customworkflow_approvals">
2 ...
3   <isinactive>F</isinactive>
4   <islogenabled>T</islogenabled>
5   <keephistory>ALWAYS</keephistory>
6   <name>Approval Workflow</name>
7   <recordtypes>[scriptid=customrecord_approval_routing]</recordtypes>
8   <releasestatus>RELEASED</releasestatus>
9   <runasadmin>F</runasadmin>
10 ...
11 </workflow>

```

In the preceding example, the workflow is set to generate an execution log when the workflow runs, and to always keep the history by saving the logs. The workflow is active and its release status is set to run for all users. The workflow does not require administrator permission to run.

The following fields provide the basic definition of a workflow:

- **isinactive** – The **isinactive** field specifies that the workflow does not initiate at any time. The default value is F. For more information, see the help topic [Inactivating a Workflow](#).
- **islogenabled** – The **islogenabled** field specifies whether the workflow generates a workflow execution log when the workflow runs. The default value is F.
- **keephistory** – The **keephistory** field specifies when the workflow instance, workflow history, and workflow logs are saved for a workflow. Options include ALWAYS, NEVER, and ONLYWHENTESTING. The default value is ONLYWHENTESTING. For more information, see the help topic [Disabling History for a Workflow](#).
- **recordtypes** – All workflows must have a base record type, specified using the **recordtypes** field. For Customer, Item, and Transaction records, you can also specify a sub type. For more information, see the help topic [Workflow Base Record](#).
- **releasestatus** – You can define the release status using the **releasestatus** field, which is a property that controls how the workflow can or cannot be initiated. That property can be useful when you want to test the workflow, release it to users, stop it from being run, or suspend its execution. The default value is NOTINITIATING.

You can specify the following release statuses:

- **NOTINITIATING** - The workflow no longer runs in testing or released mode, but still appears on the Workflow list page. New workflow instances cannot be initiated, but existing instances are still active and execute their actions and transitions. For more information, see the help topic [Viewing Existing Workflows](#).



Note: The NOTINITIATING status has replaced the NOTRUNNING status. Although NOTINITIATING is used for new objects created in NetSuite, the NOTRUNNING status is still supported in SDF for backward compatibility.

- **RELEASED** - The workflow to runs for all users. For more information, see the help topics [Defining the Workflow Audience](#) and [Users Who Can Create and View Workflows](#).

- **SUSPENDED** - Disables all workflow execution. When you set the `releasestatus` of a workflow to `SUSPENDED`, the following are true:
 - No new instances of the suspended workflow are created.
 - No existing instances of the suspended workflow are executed.
 - None of the following are executed when they exist in a suspended workflow: actions, transitions, and scheduled workflows.
 - It is not possible to initiate a suspended workflow from SuiteScript, or by using the Initiate Workflow or Mass Update actions in SuiteFlow.

To resume a suspended workflow, change the `releasestatus` to `RELEASED`. Suspended workflows resume from the point the workflow was at when it was suspended.

- **TESTING** - Tests a workflow. For more information, see the help topic [Workflow Execution Log](#).

For more information about the `releasestatus` property and its statuses, see the help topic [Release Status for Workflow Instances](#).

- `runasadmin` – The `runasadmin` field specifies that the workflow runs as a user with administrator permissions. The default value is `F`.



Note: Only administrators should use the `runasadmin` field. When deploying a workflow to an account with a non-administrator role, the `runasadmin` field is ignored.

Event-Based Workflows

Event-based workflows are created by specifying most of the workflow object fields that have an `init` prefix, such as `initoncreate`, `initonvieworupdate`, or `initcondition`. These fields are similar to the properties specified on the workflow definition page. For more information about all available event-based fields, see the following:

- [Workflow Triggers](#)
- [Triggers Reference](#)
- [Workflow Conditions](#)
- [Defining a Condition with the Condition Builder](#)
- [Defining a Condition with Formulas](#)
- [Condition Examples](#)
- [Workflow Event Types](#)
- [Event Types Reference](#)
- [Examples of Event Types](#)

The following example shows a workflow object that is set to run before a user submits a new Estimate record:

```

1 <workflow scriptid="customworkflow1">
2   <name>My custom workflow</name>
3   <recordtypes>ESTIMATE</recordtypes>
4   <initoncreate>T</initoncreate>
5   <initonvieworupdate>F</initonvieworupdate>
6   <inittriggertype>BEFORESUBMIT</inittriggertype>
7   ...
8 </workflow>
```

The `initcondition` workflow object field supports formulas, expressions, and conditions created with the visual builder in the workflow condition builder. You can define complex conditions and expressions using

record field value comparisons. These conditions can be used with workflow initiation, workflow actions, and workflow transitions.

The `initcondition` element is composed of the following:

- A formula element that defines the condition. The formula element can contain a formula, expression, or condition.
- The type of condition, which can be one of the following:
 - FORMULA – denoting a formula to define the condition. For more information, see the help topic [Defining a Condition with Formulas](#).
 - VISUAL_BUILDER – denoting a condition or expression defined by the visual builder in NetSuite. For more information, see the help topic [Defining a Condition with the Condition Builder](#).
- If the formula element contains a condition or expression, it is followed by a parameters element containing a list of parameters that assign values to the fields in the formula. When defining the parameters:
 - Each name element must correspond to a field name in the formula. Field names are encapsulated by double quotation marks (") in the formula.
 - Each value element must be a valid value for the corresponding field.
 - The record type can be specified for the field using `selectrecordtype`.

The following example shows a visual builder expression used in the initiation statement of a workflow object:

```

1  ...
2  <initcondition>
3    <formula><![CDATA["User" IN ("Current User") AND "Alt. Phone" = '555-123-1234' AND "Role" IN ("Role1","Role2","Role3") AND
4    "Days Overdue" = 100 AND "Account" = "Old Record:Account"]]</formula>
5    <type>VISUAL_BUILDER</type>
6    <parameters>
7      <parameter>
8        <name>User</name>
9        <value>STDUSERUSER</value>
10     </parameter>
11     <parameter>
12       <name>Current User</name>
13       <selectrecordtype>-4</selectrecordtype>
14       <value>@CURRENTUSER@</value>
15     </parameter>
16     <parameter>
17       <name>Alt. Phone</name>
18       <value>STDENTITYALTPHONE</value>
19     </parameter>
20     <parameter>
21       <name>Role</name>
22       <value>STDENTITYACCESSROLE</value>
23     </parameter>
24     <parameter>
25       <name>Role1</name>
26       <selectrecordtype>-118</selectrecordtype>
27       <value>ADMINISTRATOR</value>
28     </parameter>
29     <parameter>
30       <name>Role2</name>
31       <selectrecordtype>-118</selectrecordtype>
32       <value>CEO</value>
33     </parameter>
34     <parameter>
35       <name>Role3</name>
36       <selectrecordtype>-118</selectrecordtype>
37       <value>[appid=com.samples.myapp, scriptid=customrole_sdfdeveloper]</value>
38     </parameter>
39     <parameter>
40       <name>Days Overdue</name>
41       <value>STDENTITYDAYSOVERDUE</value>

```

```

41     </parameter>
42     <parameter>
43         <name>Account</name>
44         <value>STDENTITYACCOUNTNUMBER</value>
45     </parameter>
46     <parameter>
47         <name>Old Record:Account</name>
48         <value>@OLDRECORD@:STDENTITYACCOUNTNUMBER</value>
49     </parameter>
50 </parameters>
51 </initcondition>
52 ...

```

Rather than use SDF to create complex conditions and expressions in workflow objects, you should import workflows created in an account with such conditions and expressions defined using visual builder. The following example shows a workflow object created using the tutorial in [Creating Your First Workflow](#). The workflow initiates when an Opportunity record is created and the user saves the opportunity with a status of In Discussion, In Negotiation, In Progress, or Qualified. An action is triggered to go to the Opportunities page.

```

1 <workflow scriptid="customworkflow_opportunity">
2   <description></description>
3   <initcontexts></initcontexts>
4   <initeventtypes></initeventtypes>
5   <initoncreate>T</initoncreate>
6   <initonviewupdate>F</initonviewupdate>
7   <initssavedsearchcondition></initssavedsearchcondition>
8   <inittriggertype>BEFORELOAD</inittriggertype>
9   <isinactive>F</isinactive>
10  <islogenabled>T</islogenabled>
11  <keephistory>T</keephistory>
12  <name>First Workflow</name>
13  <recordtypes>OPPORTUNITY</recordtypes>
14  <releasestatus>TESTING</releasestatus>
15  <runasadmin>F</runasadmin>
16  <initcondition>
17    <formula><![CDATA["Opportunity/Estimate Status" IN ("Entity Status1","Entity Status2","Entity Status3","Entity
18    Status4")]]></formula>
19  </initcondition>
20  <type>VISUAL_BUILDER</type>
21  <parameters>
22    <parameter>
23      <name>Opportunity/Estimate Status</name>
24      <value>STDBODYENTITYSTATUS</value>
25    </parameter>
26    <parameter>
27      <name>Entity Status1</name>
28      <selectrecordtype>-104</selectrecordtype>
29      <value>In Discussion</value>
30    </parameter>
31    <parameter>
32      <name>Entity Status2</name>
33      <selectrecordtype>-104</selectrecordtype>
34      <value>In Negotiation</value>
35    </parameter>
36    <parameter>
37      <name>Entity Status3</name>
38      <selectrecordtype>-104</selectrecordtype>
39      <value>In Progress</value>
40    </parameter>
41    <parameter>
42      <name>Entity Status4</name>
43      <selectrecordtype>-104</selectrecordtype>
44      <value>Qualified</value>
45    </parameter>
46  </parameters>
47  </initcondition>
48  <workflowstates>
49    <workflowstate scriptid="workflowstate1">
50      <description></description>
51      <donotexitworkflow>F</donotexitworkflow>

```

```

50     <name>State 1: Entry</name>
51     <positionx>243</positionx>
52     <positiony>133</positiony>
53     <workflowactions triggerType="ONENTRY">
54         <setfieldmandatoryaction scriptid="workflowaction1">
55             <clienttriggerfields></clienttriggerfields>
56             <conditionsavedsearch></conditionsavedsearch>
57             <contexttypes></contexttypes>
58             <eventtypes></eventtypes>
59             <field>STDBODYTITLE</field>
60             <isinactive>F</isinactive>
61             <ismandatory>T</ismandatory>
62             <initcondition>
63                 <formula></formula>
64                 <type>VISUAL_BUILDER</type>
65             </initcondition>
66         </setfieldmandatoryaction>
67     </workflowactions>
68     <workflowtransitions>
69         <workflowtransition scriptid="workflowtransition1">
70             <buttonaction></buttonaction>
71             <conditionsavedsearch></conditionsavedsearch>
72             <contexttypes></contexttypes>
73             <eventtypes></eventtypes>
74             <scheduleddelay></scheduleddelay>
75             <scheduletimeunit></scheduletimeunit>
76             <tostate>[scriptid=customworkflow1.workflowstate2]</tostate>
77             <triggerType>AFTERSUBMIT</triggerType>
78             <waitforworkflow></waitforworkflow>
79             <waitforworkflowstate></waitforworkflowstate>
80             <initcondition>
81                 <formula></formula>
82                 <type>VISUAL_BUILDER</type>
83             </initcondition>
84         </workflowtransition>
85     </workflowtransitions>
86 </workflowstate>
87 <workflowstate scriptid="workflowstate2">
88     <description></description>
89     <donotexitworkflow>F</donotexitworkflow>
90     <name>State 2: See Opportunities</name>
91     <positionx>473</positionx>
92     <positiony>133</positiony>
93     <workflowactions triggerType="ONENTRY">
94         <gotopageaction scriptid="workflowaction2">
95             <conditionsavedsearch></conditionsavedsearch>
96             <contexttypes></contexttypes>
97             <eventtypes></eventtypes>
98             <isinactive>F</isinactive>
99             <targetpage>LIST_TRAN_OPPTY</targetpage>
100             <initcondition>
101                 <formula></formula>
102                 <type>VISUAL_BUILDER</type>
103             </initcondition>
104         </gotopageaction>
105     </workflowactions>
106 </workflowstate>
107 </workflowstates>
108 </workflow>

```

When you use SDF to modify expressions in workflow objects, consider the following:

- During the import into a SuiteCloud project, parameter fields with account specific values have the value set to [ACCOUNT_SPECIFIC_VALUE] in the workflow object definition. For deployment to succeed when **Action on Account-Specific Values** is set to ERROR, change the value to one that is valid for either a custom field in the project or a field in the target account. For more information, see [Account-Specific Values Validation in SuiteCloud Projects](#).
- Removing the initcondition block from a workflow object in SDF does not remove that condition from that object in the target account during deployment. To delete the condition from the object, specify an empty formula. For example:

```

1  ...
2  <initcondition>
3    <formula></formula>
4    <type>VISUAL_BUILDER</type>
5  </initcondition>
6  ...

```

- Validation errors are returned when the `initcondition` block is defined with any of the following:
 - A parameter exists in the formula field but there is no corresponding parameter name.
 - The formula is invalid, containing a syntax error.
 - The parameter's assigned value is incorrect for its field.
 - The value requires a feature that it is not compatible with the target NetSuite account, and consequently cannot be added to the manifest. If the feature can be set, the required feature is provided in the validation log so that it can be added to the manifest.
 - Parameter fields with a value of `[ACCOUNT_SPECIFIC_VALUE]` when **Action on Account-Specific Values** is set to `ERROR`.
- Validation is only performed on supported record types. If the parameter is a field for an unsupported `selectrecordtype` value, the parameter cannot be validated.
- Only valid fields for the record type specified in `recordtypes` can be specified as parameters. Specifying fields for other record types returns a reference key error on deployment.
- All system fields are deployed to the target account in English. All custom fields are deployed to the target account in the language defined for the field object in the SuiteCloud project.

For more information about event-based workflows and creating them with the NetSuite UI, see the following:

- [Initiating a Workflow on an Event](#)
- [Creating a Workflow](#)

Scheduled Workflows

Scheduled workflows are created by specifying the `initsavedsearchfilter` field, `initsavedsearchcondition` field, and `recurrence` field group in the workflow object. Use the `initsavedsearchfilter` and `initsavedsearchcondition` fields to specify the saved search on which to run the workflow object. Use the `recurrence` field group to schedule the workflow's execution. You can indicate a repeating schedule by specifying the `startdate` and `enddate` fields that are available in any recurrence selection.

Note: When you remove the `enddate` element from a recurrence in your project and deploy it to a target account, any `enddate` value for that recurrence that already exists in the account is preserved. The `enddate` value is only deleted when you change the recurrence type in the project and deploy it to the target account.

For more information about scheduled workflow fields, see the following:

- [workflow](#)
- [recurrence](#)

The following example shows a workflow object that runs a custom search every weekday at 7:00am from mid-September until mid-October:

```

1 | <workflow scriptid="customworkflow1">

```

```

2      <name>My custom workflow</name>
3      <recordtypes>CUSTOMER</recordtypes>
4      ...
5      <initsearchcondition>[scriptid=customsearch1]</initsearchcondition>
6      <initsearchfilter>[scriptid=customsearch1]</initsearchfilter>
7      <recurrence>
8          <everyweekday>
9              <startdate>2020-09-14</startdate>
10             <enddate>2020-10-14</enddate>
11             <starttime>07:00:00Z</starttime>
12         </everyweekday>
13     </recurrence>
14     ...
15 </workflow>

```



Note: The format of the time and date values is based on the UTC standard and is ISO compliant. For more information about how to format time and date values, see [Date and Time Formats in SDF Custom Objects](#).

For more information scheduled workflows and creating them with the NetSuite UI, see the following:

- [Initiating a Workflow on a Schedule](#)
- [Scheduling a Workflow](#)

Workflow Custom Fields

You can create custom fields in either an event-based workflow or a scheduled workflow. In a workflow object, custom fields are grouped within the workflowcustomfields structure. In this structure, you can create as many custom fields as needed by specifying a workflowcustomfield structure for each one. The elements that are available in a workflowcustomfield are similar to the properties that you can specify in the fields subtab of the workflow definition page. For more information about the available elements that you can specify for a custom field, see the following:

- [workflowcustomfield](#)
- [Workflow Definition Page Fields Subtab](#)
- [Workflow Custom Fields](#)
- [Creating and Using Workflow Fields](#)

The following example shows a workflow object that defines two custom fields:

```

1 <workflow scriptid="customworkflow1">
2   <name>My custom workflow</name>
3   <recordtypes>LEAD</recordtypes>
4   ...
5   <workflowcustomfields>
6     <workflowcustomfield scriptid="custworkflow1">
7       <applyformatting>F</applyformatting>
8       <defaultchecked>F</defaultchecked>
9       <displaytype>NORMAL</displaytype>
10      <fieldtype>SELECT</fieldtype>
11      <label>Webinar. Response</label>
12      <selectrecordtype>[scriptid=customrecordtype1]</selectrecordtype>
13      <storevalue>T</storevalue>
14    </workflowcustomfield>
15    <workflowcustomfield scriptid="custworkflow2">
16      <applyformatting>F</applyformatting>
17      <defaultchecked>F</defaultchecked>
18      <displaytype>NORMAL</displaytype>
19      <fieldtype>SELECT</fieldtype>
20      <label>CaseStudy. Response</label>
21      <selectrecordtype>[scriptid=customrecordtype1]</selectrecordtype>
22      <storevalue>T</storevalue>

```



```

23     </workflowcustomfield>
24   </workflowcustomfields>
25   ...
26 </workflow>

```

For information about how to create workflow states, see [Workflow State Creation](#).

Workflow State Creation

In SuiteCloud Development Framework (SDF), workflow states are defined in a workflow SDF custom object. Workflow states are grouped within the workflowstates structure. In this structure, you can create as many states as needed by specifying a workflowstate structure for each state. Each workflowstate structure contains elements and structures to define its workflow state fields, transitions that lead to other states, and actions. For more information about the available elements that you can specify for a workflow state, see the following:

- [workflowstate](#)
- [Workflow States](#)
- [Workflow State Actions](#)
- [Workflow Custom Fields](#)
- [Creating and Using State Fields](#)
- [States Reference](#)

The following example shows a workflow object that contains a series of states, with basic elements defined for each state:

```

1 <workflow scriptid="customworkflow1">
2   <name>My custom workflow</name>
3   <recordtypes>LEAD</recordtypes>
4   ...
5   <workflowstates>
6     <workflowstate scriptid="workflowstate_entry">
7       <name>State 1: Entry</name>
8       <donotexitworkflow>F</donotexitworkflow>
9       <positionx>213</positionx>
10      <positiony>33</positiony>
11      ...
12    </workflowstate>
13    <workflowstate scriptid="workflowstate_approvalpending">
14      <name>State 2: Pending Approval</name>
15      <donotexitworkflow>F</donotexitworkflow>
16      <positionx>213</positionx>
17      <positiony>123</positiony>
18      ...
19    </workflowstate>
20    <workflowstate scriptid="workflowstate_approved">
21      <name>State 3a: Approved</name>
22      <donotexitworkflow>F</donotexitworkflow>
23      <positionx>213</positionx>
24      <positiony>213</positiony>
25      ...
26    </workflowstate>
27    <workflowstate scriptid="workflowstate_rejected">
28      <name>State 3b: Rejected</name>
29      <donotexitworkflow>F</donotexitworkflow>
30      <positionx>213</positionx>
31      <positiony>303</positiony>
32      ...
33    </workflowstate>
34  </workflowstates>
35  ...
36 </workflow>

```

Workflow State Transitions

In SuiteCloud Development Framework (SDF), state transitions are defined in the workflow state that they derive from. Transitions are grouped within the workflowtransitions structure of the workflow state. In this structure, you can create as many transitions as needed by specifying a workflowtransition structure for each transition. Each workflowtransition structure contains elements to define the destination state, when the transition should occur, and other basic properties. For more information about the available elements that you can specify for a workflow transition, see the following:

- [workflowtransition](#)
- [Workflow Transitions](#)
- [Creating a Transition](#)
- [Scheduling a Transition](#)
- [Transitions Examples](#)

The following example shows a workflow object that contains three states, where one state contains transitions to the other states depending on a button action:

```

1  <workflow scriptid="customworkflow1">
2    <name>My custom workflow</name>
3    <recordtypes>LEAD</recordtypes>
4    ...
5    <workflowstates>
6      ...
7      <workflowstate scriptid="workflowstate_approvalpending">
8        <name>State 2: Pending Approval</name>
9        ...
10     <workflowtransitions>
11       <workflowtransition scriptid="workflowtransition_approveclick">
12         <buttonaction>
13           [scriptid=customworkflow_approvals.workflowstate_approvalpending.workflowaction_approvebutton]
14         </buttonaction>
15         <conditionsavedsearch></conditionsavedsearch>
16         <contexttypes></contexttypes>
17         <eventtypes></eventtypes>
18         <scheduleddelay></scheduleddelay>
19         <scheduletimeunit></scheduletimeunit>
20         <tostate>[scriptid=customworkflow_approvals.workflowstate_approved]</tostate>
21         <triggertype></triggertype>
22         <waitforworkflow></waitforworkflow>
23         <waitforworkflowstate></waitforworkflowstate>
24         <initcondition>
25           <formula></formula>
26           <type>VISUAL_BUILDER</type>
27         </initcondition>
28       </workflowtransition>
29       <workflowtransition scriptid="workflowtransition_rejectclick">
30         <buttonaction>
31           [scriptid=customworkflow_approvals.workflowstate_approvalpending.workflowaction_rejectbutton]
32         </buttonaction>
33         <conditionsavedsearch></conditionsavedsearch>
34         <contexttypes></contexttypes>
35         <eventtypes></eventtypes>
36         <scheduleddelay></scheduleddelay>
37         <scheduletimeunit></scheduletimeunit>
38         <tostate>[scriptid=customworkflow_approvals.workflowstate_rejected]</tostate>
39         <triggertype></triggertype>
40         <waitforworkflow></waitforworkflow>
41         <waitforworkflowstate></waitforworkflowstate>
42         <initcondition>
43           <formula></formula>
44           <type>VISUAL_BUILDER</type>
45         </initcondition>
46       </workflowtransition>
47     </workflowtransitions>
48   </workflowstate>

```

```

49     <workflowstate scriptid="workflowstate_approved">
50       <name>State 3a: Approved</name>
51       ...
52     </workflowstate>
53     <workflowstate scriptid="workflowstate_rejected">
54       <name>State 3b: Rejected</name>
55       ...
56     </workflowstate>
57   </workflowstates>
58   ...
59 </workflow>

```

Workflow State Actions

In SuiteCloud Development Framework (SDF), state actions are grouped in the `workflowactions` structure of a workflow state. The `workflowactions` element requires a `triggertype` attribute to indicate when the group of actions execute. Each action defined in the structure has its own nested structure and a unique element name, such as `addbuttonaction`, `sendemailaction`, and `setfieldvalueaction`.

All action types share common action properties, triggers, and conditions. The names of these elements may vary depending on the type of action you specify.



Important: Workflow actions are subrecords, so the deletion rules vary during deployment based on the SuiteCloud project type that the workflow is in. For more information, see [Differences Between SuiteCloud Projects](#).

For more information about the available elements that you can specify in various workflow actions, see the following:

- [workflowactions](#)
- [Workflow Actions](#)
- [Working with Actions](#)
- [Workflow Actions Overview](#)
- [Action Examples](#)

The following example shows a workflow state that contains two `addbuttonaction` actions:

```

1 <workflow scriptid="customworkflow1">
2   <name>My custom workflow</name>
3   <recordtypes>LEAD</recordtypes>
4   ...
5   <workflowstates>
6     ...
7     <workflowstate scriptid="workflowstate_approvalpending">
8       <name>State 2: Pending Approval</name>
9       ...
10      <workflowactions triggertype="BEFORELOAD">
11        <addbuttonaction scriptid="workflowaction_approvebutton">
12          <checkconditionbeforeexecution>T</checkconditionbeforeexecution>
13          <conditionsavedsearch></conditionsavedsearch>
14          <contexttypes></contexttypes>
15          <eventtypes></eventtypes>
16          <isinactive>F</isinactive>
17          <label>Approve</label>
18          <saverecordfirst>F</saverecordfirst>
19          <initcondition>
20            <formula><![CDATA[{salesrep.supervisor.id}={me}]]></formula>
21            <type>FORMULA</type>
22          </initcondition>
23        </addbuttonaction>
24        <addbuttonaction scriptid="workflowaction_rejectbutton">

```

```

25         <checkconditionbeforeexecution>T</checkconditionbeforeexecution>
26         <conditionsavedsearch></conditionsavedsearch>
27         <contexttypes></contexttypes>
28         <eventtypes></eventtypes>
29         <isinactive>F</isinactive>
30         <label>Reject</label>
31         <saverecordfirst>F</saverecordfirst>
32         <initcondition>
33             <formula><![CDATA[{salesrep.supervisor.id}={me}]]></formula>
34             <type>FORMULA</type>
35         </initcondition>
36         </addbuttonaction>
37     </workflowactions>
38 </workflowstate>
39 ...
40 </workflowstates>
41 ...
42 </workflow>

```

You can create Sublist Action Groups in workflows for transactions when you work with them in SDF. A Sublist Action Group is a group of actions assigned to a workflow state that is to be executed on each line of a sublist. For more information, see the help topic [Using Sublist Action Groups](#).

SDF supports the following Sublist Action Groups:

- Create Record
- Return User Error
- Send Email
- Set Field Value

The following is an example of a workflow sublist action group as it is defined in a workflow SDF custom object:

```

1 <workflow scriptid="customworkflow_sample">
2     ...
3     <workflowsublistactiongroup scriptid="workflowaction123">
4         <conditionsavedsearch/>
5         <contexttypes/>
6         <eventtypes/>
7         <isinactive>F</isinactive>
8         <scheduledelay/>
9         <schedulemode>DELAY</schedulemode>
10        <schedulerecurrence/>
11        <scheduletimeofday/>
12        <scheduletimeunit/>
13        <sublist>ITEM</sublist>
14        <initcondition>
15            <formula/>
16            <type>VISUAL_BUILDER</type>
17        </initcondition>
18        <setfieldvalueaction scriptid="workflowaction456">
19            <field>STDBODYSHIPZIP</field>
20            <isinactive>F</isinactive>
21            <valuechecked>F</valuechecked>
22            <valuedate/>
23            <valuefield/>
24            <valueformula/>
25            <valuejoinfield/>
26            <valueselect/>
27            <valuetext>SDC</valuetext>
28            <valuetype>STATIC</valuetype>
29            <initcondition>
30                <formula/>
31                <type>VISUAL_BUILDER</type>
32            </initcondition>
33        </setfieldvalueaction>
34    </workflowsublistactiongroup>
35    ...

```

```
36 | </workflow>
```

Create Line Action

You can define a create line action using `createlineaction`. The action adds a new line to a sublist and can be inserted as the first or last line in the sublist. You can also specify default values for line fields, and the values can be static or dynamically defined.

Use `createlineaction` when lines need to be added to a sublist at certain points in the workflow. The `createlineaction` element can be added to a workflow state as an individual action using `workflowactions`, and as part of an action group using `workflowactiongroup`. The `createlineaction` element can be specified in workflow definitions on record types that contain a sublist supported by SuiteFlow.

For more information about:

- Using a create line action in the NetSuite UI, see the help topic [Create Line Action](#).
- Using actions in a workflow, see the help topic [Working with Actions](#).
- Setting field values in actions, see the help topic [Setting Field Values in Action Definitions](#).

The following example shows a workflow state that contains a create line action that adds the field `STDCOLISCLOSED` as a new line in the ITEM sublist when the workflow advances to the Rejected state:

```
1 <workflow scriptid="customworkflow245">
2   <name>Sales Order Basic Approval</name>
3   <recordtypes>SALESORDER</recordtypes>
4   <releasestatus>RELEASED</releasestatus>
5   ...
6   <workflowstates>
7     ...
8     <workflowstate scriptid="workflowstate8">
9       <description></description>
10      <donotexitworkflow>F</donotexitworkflow>
11      <name>Rejected</name>
12      <positionx>463</positionx>
13      <positiony>223</positiony>
14      ...
15      <workflowactions trigger="ONENTRY">
16        <createlineaction scriptid="workflowaction41">
17          <conditionsavedsearch></conditionsavedsearch>
18          <contexttypes></contexttypes>
19          <eventtypes></eventtypes>
20          <isinactive>F</isinactive>
21          <position>AFTERLASTLINE</position>
22          <sublist>ITEM</sublist>
23          <initcondition>
24            <formula></formula>
25            <type>VISUAL_BUILDER</type>
26          </initcondition>
27          <fieldsettings>
28            <fieldsetting>
29              <targetfield>STDCOLISCLOSED</targetfield>
30              <valuechecked>T</valuechecked>
31              <valuedate></valuedate>
32              <valuefield>STDBODYCOMPLETE</valuefield>
33              <valueformula></valueformula>
34              <valuejoinfield></valuejoinfield>
35              <valueselect></valueselect>
36              <valuetext></valuetext>
37            </fieldsetting>
38          </fieldsettings>
39        </createlineaction>
40        ...
41      </workflowactions>
42    </workflowstate>
43    ...
```

```

44     </workflowstates>
45 </workflow>

```

Workflow Action Parameter Sections

Some action types let you specify a parameter fields. In the NetSuite UI, these fields are located in the Parameters section. For example, the following is an image of the Parameters section of an Initiate Workflow action in NetSuite:

FIELD	VALUE	JOIN	VALUE FIELD	FORMULA
a				
b				
c				

In SDF, parameter fields are grouped in a structure of the workflow action. The name of this structure depends on the action type. You can create parameter fields in SDF for the following actions:

- Create Record Action Types. For information, see the help topics [fieldsettings](#) and [Creating and Subscribing to a Record](#).
- Custom Action Types. For information, see the help topic [parametersettings](#).
- Go to Record Action Types. For information, see the help topic [fieldsettings](#).
- Initiate Workflow Action Types. For information, see the help topic [workflowfieldsettings](#).
- Transform Record Action Types. For information, see the help topic [fieldsettings](#).

Workflow Object Example

The following example shows three SDF custom objects that can be used together in a SuiteCloud project for SuiteCloud Development Framework (SDF). The project consists of a custom list object, a transaction body custom field object, and a workflow object. When deployed to a target NetSuite account, the workflow allows supervisors to approve or reject estimates entered by their sales representatives. This example is based on a similar workflow sample in the SuiteFlow Help. For more information about this workflow and how to implement it using SuiteFlow instead of SDF, see the help topic [Estimate Approval Routing Workflow](#).

The following workflow object concepts are shown in the example code:

- Event-based workflows
- Triggers
- States
- Transitions
- Actions, specifically `setfieldvalueaction`, `addbuttonaction`, and `sendemailaction`

The following file, when saved under `/Objects/customlist_approvalstatuses.xml` in a SuiteCloud project, specifies a custom list that stores all possible values of an Estimate record—approved, rejected, and pending approval.

```

1 <customlist scriptid="customlist_approvalstatuses">
2   <description></description>
3   <isinactive>F</isinactive>
4   <isordered>T</isordered>

```

```

5      <name>Approval Statuses</name>
6      <customvalues>
7          <customvalue scriptid="val_51131_mstrwlf_705">
8              <isinactive>F</isinactive>
9              <value>Pending Approval</value>
10         </customvalue>
11         <customvalue scriptid="val_51132_mstrwlf_378">
12             <isinactive>F</isinactive>
13             <value>Approved</value>
14         </customvalue>
15         <customvalue scriptid="val_51133_mstrwlf_156">
16             <isinactive>F</isinactive>
17             <value>Rejected</value>
18         </customvalue>
19     </customvalues>
20 </customlist>

```

The following file, when saved under **/Objects/custbody_approvalstatus.xml** in a SuiteCloud project, specifies an Approval Status transaction body custom field. The workflow sets this field to one of the possible values specified by the `customlist_approvalstatuses` custom list.

```

1  <transactionbodycustomfield scriptid="custbody_approvalstatus">
2      <accesslevel>2</accesslevel>
3      <applyformatting>F</applyformatting>
4      <bodyassemblybuild>F</bodyassemblybuild>
5      <bodycustomerpayment>F</bodycustomerpayment>
6      <bodydeposit>F</bodydeposit>
7      <bodyexpensereport>F</bodyexpensereport>
8      <bodyinventoryadjustment>F</bodyinventoryadjustment>
9      <bodyitemfulfillment>F</bodyitemfulfillment>
10     <bodyitemfulfillmentorder>F</bodyitemfulfillmentorder>
11     <bodyitemreceipt>F</bodyitemreceipt>
12     <bodyitemreceiptorder>F</bodyitemreceiptorder>
13     <bodyjournal>F</bodyjournal>
14     <bodyopportunity>F</bodyopportunity>
15     <bodyothertransaction>F</bodyothertransaction>
16     <bodypickingticket>F</bodypickingticket>
17     <bodyprintflag>F</bodyprintflag>
18     <bodyprintpackingslip>F</bodyprintpackingslip>
19     <bodyprintstatement>F</bodyprintstatement>
20     <bodypurchase>F</bodypurchase>
21     <bodysale>T</bodysale>
22     <bodystore>F</bodystore>
23     <bodyvendorpayment>F</bodyvendorpayment>
24     <checkspelling>F</checkspelling>
25     <defaultchecked>F</defaultchecked>
26     <displaytype>NORMAL</displaytype>
27     <encryptatrest>F</encryptatrest>
28     <fieldtype>SELECT</fieldtype>
29     <globalsearch>F</globalsearch>
30     <isformula>F</isformula>
31     <ismandatory>F</ismandatory>
32     <isparent>F</isparent>
33     <label>Custom Approval Status</label>
34     <onparentdelete>NO_ACTION</onparentdelete>
35     <searchlevel>2</searchlevel>
36     <selectrecordtype>[scriptid=customlist_approvalstatuses]</selectrecordtype>
37     <showhierarchy>F</showhierarchy>
38     <showinlist>F</showinlist>
39     <storevalue>T</storevalue>
40 </transactionbodycustomfield>

```

The following file, when saved under **/Objects/customworkflow_approvals.xml** in a SuiteCloud project, specifies the Estimate Approval Routing workflow. This workflow initiates when a sales representative saves a new estimate record. The workflow checks the estimate to determine whether the sales representative has a supervisor in their employee record. If they do not have a supervisor, the estimate status is approved and an email is sent to the sales representative. If the sales representative has a supervisor, the estimate status changes to Pending Approval, and two buttons become visible to their supervisor. When the supervisor clicks either of the buttons that appear on the estimate record,

that status of the estimate changes to either Approved or Rejected, and an email is sent to the sales representative. For more information, see the help topic [Estimate Approval Routing Workflow](#).

```

1 <workflow scriptid="customworkflow_approvals">
2   <description></description>
3   <initcontexts></initcontexts>
4   <initeventtypes></initeventtypes>
5   <initoncreate>T</initoncreate>
6   <initonviewupdate>F</initonviewupdate>
7   <initssavedsearchcondition></initssavedsearchcondition>
8   <inittriggertype>BEFORESUBMIT</inittriggertype>
9   <isinactive>F</isinactive>
10  <islogenabled>T</islogenabled>
11  <keephistory>T</keephistory>
12  <name>Estimate Approval Routing</name>
13  <recordtypes>ESTIMATE</recordtypes>
14  <releasestatus>TESTING</releasestatus>
15  <runasadmin>F</runasadmin>
16  <initcondition>
17    <formula></formula>
18    <type>VISUAL_BUILDER</type>
19  </initcondition>
20  <workflowstates>
21    <workflowstate scriptid="workflowstate_entry">
22      <description></description>
23      <donotexitworkflow>F</donotexitworkflow>
24      <name>State 1: Entry</name>
25      <positionx>213</positionx>
26      <positiony>33</positiony>
27      <workflowactions triggerType="ONENTRY">
28        <setfieldvalueaction scriptid="workflowaction_setpendingvalue">
29          <clienttriggerfields></clienttriggerfields>
30          <conditionsavedsearch></conditionsavedsearch>
31          <contexttypes></contexttypes>
32          <eventtypes></eventtypes>
33          <field>[scriptid=custbody_approvalstatus]</field>
34          <isinactive>F</isinactive>
35          <scheduleddelay></scheduleddelay>
36          <schedulemode>DELAY</schedulemode>
37          <schedulerecurrence></schedulerecurrence>
38          <scheduledtimeofday></scheduledtimeofday>
39          <scheduledtimeunit></scheduledtimeunit>
40          <valuechecked>F</valuechecked>
41          <valuedate></valuedate>
42          <valuefield></valuefield>
43          <valueformula></valueformula>
44          <valuejoinfield></valuejoinfield>
45          <valueselect></valueselect>
46          <valuetext></valuetext>
47          <valuetype>STATIC</valuetype>
48          <initcondition>
49            <formula></formula>
50            <type>VISUAL_BUILDER</type>
51          </initcondition>
52        </setfieldvalueaction>
53      </workflowactions>
54      <workflowtransitions>
55        <workflowtransition scriptid="workflowtransition_hassupervisor">
56          <buttonaction></buttonaction>
57          <conditionsavedsearch></conditionsavedsearch>
58          <contexttypes></contexttypes>
59          <eventtypes></eventtypes>
60          <scheduleddelay></scheduleddelay>
61          <scheduledtimeunit></scheduledtimeunit>
62          <tostate>[scriptid=customworkflow_approvals.workflowstate_approvalpending]</tostate>
63          <triggertype>ONENTRY</triggertype>
64          <waitforworkflow></waitforworkflow>
65          <waitforworkflowstate></waitforworkflowstate>
66          <initcondition>
67            <formula><![CDATA[{supervisor} is not null]]></formula>
68            <type>FORMULA</type>
69          </initcondition>

```



```

70     </workflowtransition>
71     <workflowtransition scriptid="workflowtransition_isasupervisor">
72         <buttonaction></buttonaction>
73         <conditionsavedsearch></conditionsavedsearch>
74         <contexttypes></contexttypes>
75         <eventtypes></eventtypes>
76         <scheduleddelay></scheduleddelay>
77         <scheduletimeunit></scheduletimeunit>
78         <tostate>[scriptid=customworkflow_approvals.workflowstate_approved]</tostate>
79         <triggertype>ONENTRY</triggertype>
80         <waitforworkflow></waitforworkflow>
81         <waitforworkflowstate></waitforworkflowstate>
82         <initcondition>
83             <formula></formula>
84             <type>VISUAL_BUILDER</type>
85         </initcondition>
86     </workflowtransition>
87 </workflowtransitions>
88 </workflowstate>
89 <workflowstate scriptid="workflowstate_approvalpending">
90     <description></description>
91     <donotexitworkflow>F</donotexitworkflow>
92     <name>State 2: Pending Approval</name>
93     <positionx>213</positionx>
94     <positiony>123</positiony>
95     <workflowactions triggertype="BEFORELOAD">
96         <addbuttonaction scriptid="workflowaction_approvebutton">
97             <checkconditionbeforeexecution>T</checkconditionbeforeexecution>
98             <conditionsavedsearch></conditionsavedsearch>
99             <contexttypes></contexttypes>
100             <eventtypes></eventtypes>
101             <isinactive>F</isinactive>
102             <label>Approve</label>
103             <savererecordfirst>F</savererecordfirst>
104             <initcondition>
105                 <formula><![CDATA[{salesrep.supervisor.id}={me}]]></formula>
106                 <type>FORMULA</type>
107             </initcondition>
108         </addbuttonaction>
109         <addbuttonaction scriptid="workflowaction_rejectbutton">
110             <checkconditionbeforeexecution>T</checkconditionbeforeexecution>
111             <conditionsavedsearch></conditionsavedsearch>
112             <contexttypes></contexttypes>
113             <eventtypes></eventtypes>
114             <isinactive>F</isinactive>
115             <label>Reject</label>
116             <savererecordfirst>F</savererecordfirst>
117             <initcondition>
118                 <formula><![CDATA[{salesrep.supervisor.id}={me}]]></formula>
119                 <type>FORMULA</type>
120             </initcondition>
121         </addbuttonaction>
122     </workflowactions>
123     <workflowtransitions>
124         <workflowtransition scriptid="workflowtransition_approveclick">
125             <buttonaction>[scriptid=customworkflow_approvals.workflowstate_approvalpending.workflowaction_approvebutton]</buttonac
tion>
126             <conditionsavedsearch></conditionsavedsearch>
127             <contexttypes></contexttypes>
128             <eventtypes></eventtypes>
129             <scheduleddelay></scheduleddelay>
130             <scheduletimeunit></scheduletimeunit>
131             <tostate>[scriptid=customworkflow_approvals.workflowstate_approved]</tostate>
132             <triggertype></triggertype>
133             <waitforworkflow></waitforworkflow>
134             <waitforworkflowstate></waitforworkflowstate>
135             <initcondition>
136                 <formula></formula>
137                 <type>VISUAL_BUILDER</type>
138             </initcondition>
139         </workflowtransition>
140         <workflowtransition scriptid="workflowtransition_rejectclick">

```

```

141      <buttonaction>[scriptid=customworkflow_approvals.workflowstate_approvalpending.workflowaction_rejectbutton]</buttonac
tion>
142      <conditionsavedsearch></conditionsavedsearch>
143      <contexttypes></contexttypes>
144      <eventtypes></eventtypes>
145      <scheduledelay></scheduledelay>
146      <scheduletimeunit></scheduletimeunit>
147      <tostate>[scriptid=customworkflow_approvals.workflowstate_rejected]</tostate>
148      <triggertype></triggertype>
149      <waitforworkflow></waitforworkflow>
150      <waitforworkflowstate></waitforworkflowstate>
151      <initcondition>
152        <formula></formula>
153        <type>VISUAL_BUILDER</type>
154      </initcondition>
155    </workflowtransition>
156  </workflowtransitions>
157 </workflowstate>
158 <workflowstate scriptid="workflowstate_approved">
159   <description></description>
160   <donotexitworkflow>F</donotexitworkflow>
161   <name>State 3a: Approved</name>
162   <positionx>213</positionx>
163   <positiony>213</positiony>
164   <workflowactions triggertype="ONENTRY">
165     <setfieldvalueaction scriptid="workflowaction_setapprovevalue">
166       <clienttriggerfields></clienttriggerfields>
167       <conditionsavedsearch></conditionsavedsearch>
168       <contexttypes></contexttypes>
169       <eventtypes></eventtypes>
170       <field>[scriptid=custbody_approvalstatus]</field>
171       <isinactive>F</isinactive>
172       <scheduledelay></scheduledelay>
173       <schedulemode>DELAY</schedulemode>
174       <schedulerecurrence></schedulerecurrence>
175       <scheduletimeofday></scheduletimeofday>
176       <scheduletimeunit></scheduletimeunit>
177       <valuechecked>F</valuechecked>
178       <valuedate></valuedate>
179       <valuefield></valuefield>
180       <valueformula></valueformula>
181       <valuejoinfield></valuejoinfield>
182       <valueselect></valueselect>
183       <valuetext></valuetext>
184       <valuetype>STATIC</valuetype>
185       <initcondition>
186         <formula></formula>
187         <type>VISUAL_BUILDER</type>
188       </initcondition>
189     </setfieldvalueaction>
190     <sendemailaction scriptid="workflowaction_sendapprovalemail">
191       <attachmentfield></attachmentfield>
192       <attachmentfile></attachmentfile>
193       <attachmentjoinfield></attachmentjoinfield>
194       <attachmenttype>SPECIFIC</attachmenttype>
195       <body>Estimate {number} has been approved by your supervisor.</body>
196       <conditionsavedsearch></conditionsavedsearch>
197       <contexttypes></contexttypes>
198       <eventtypes></eventtypes>
199       <includeformat></includeformat>
200       <includerecordlink>T</includerecordlink>
201       <includetransaction>F</includetransaction>
202       <isinactive>F</isinactive>
203       <recipientbccemail></recipientbccemail>
204       <recipientccemail></recipientccemail>
205       <recipientemail></recipientemail>
206       <recipientfield>STDBODYSALESREP</recipientfield>
207       <recipientjoinfield></recipientjoinfield>
208       <recipienttype>FIELD</recipienttype>
209       <scheduledelay></scheduledelay>
210       <schedulemode>DELAY</schedulemode>
211       <schedulerecurrence></schedulerecurrence>
212       <scheduletimeofday></scheduletimeofday>

```

```

213     <scheduledtimeunit></scheduledtimeunit>
214     <senderfield>STDBODYEMPLOYEE</senderfield>
215     <senderjoinfield></senderjoinfield>
216     <sendertype>FIELD</sendertype>
217     <subject>Your Estimate Has Been Approved</subject>
218     <template></template>
219     <usetemplate>F</usetemplate>
220     <initcondition>
221         <formula></formula>
222         <type>VISUAL_BUILDER</type>
223     </initcondition>
224 </sendemailaction>
225 </workflowactions>
226 </workflowstate>
227 <workflowstate scriptid="workflowstate_rejected">
228     <description></description>
229     <donotexitworkflow>F</donotexitworkflow>
230     <name>State 3b: Rejected</name>
231     <positionx>213</positionx>
232     <positiony>303</positiony>
233     <workflowactions triggertype="ONENTRY">
234         <setfieldvalueaction scriptid="workflowaction_setrejectvalue">
235             <clienttriggerfields></clienttriggerfields>
236             <conditionsavedsearch></conditionsavedsearch>
237             <contexttypes></contexttypes>
238             <eventtypes></eventtypes>
239             <field>[scriptid=custbody_approvalstatus]</field>
240             <isinactive>F</isinactive>
241             <scheduleddelay></scheduleddelay>
242             <schedulemode>DELAY</schedulemode>
243             <schedulerecurrence></schedulerecurrence>
244             <scheduledtimeofday></scheduledtimeofday>
245             <scheduledtimeunit></scheduledtimeunit>
246             <valuechecked>F</valuechecked>
247             <valuedate></valuedate>
248             <valuefield></valuefield>
249             <valueformula></valueformula>
250             <valuejoinfield></valuejoinfield>
251             <valueselect></valueselect>
252             <valuetext></valuetext>
253             <valuetype>STATIC</valuetype>
254             <initcondition>
255                 <formula></formula>
256                 <type>VISUAL_BUILDER</type>
257             </initcondition>
258         </setfieldvalueaction>
259         <sendemailaction scriptid="workflowaction_sendrejectionemail">
260             <attachmentfield></attachmentfield>
261             <attachmentfile></attachmentfile>
262             <attachmentjoinfield></attachmentjoinfield>
263             <attachmenttype>SPECIFIC</attachmenttype>
264             <body>Estimate {number} has been rejected by your supervisor.</body>
265             <conditionsavedsearch></conditionsavedsearch>
266             <contexttypes></contexttypes>
267             <eventtypes></eventtypes>
268             <includeformat></includeformat>
269             <includerecordlink>T</includerecordlink>
270             <includetransaction>F</includetransaction>
271             <isinactive>F</isinactive>
272             <recipientbccemail></recipientbccemail>
273             <recipientccemail></recipientccemail>
274             <recipientemail></recipientemail>
275             <recipientfield>STDBODYSALESREP</recipientfield>
276             <recipientjoinfield></recipientjoinfield>
277             <recipienttype>FIELD</recipienttype>
278             <scheduleddelay></scheduleddelay>
279             <schedulemode>DELAY</schedulemode>
280             <schedulerecurrence></schedulerecurrence>
281             <scheduledtimeofday></scheduledtimeofday>
282             <scheduledtimeunit></scheduledtimeunit>
283             <senderfield>STDBODYEMPLOYEE</senderfield>
284             <senderjoinfield></senderjoinfield>
285             <sendertype>FIELD</sendertype>

```

```

286     <subject>Your Estimate Has Been Rejected</subject>
287     <template></template>
288     <usetemplate>F</usetemplate>
289     <initcondition>
290         <formula></formula>
291         <type>VISUAL_BUILDER</type>
292     </initcondition>
293     </sendemailaction>
294 </workflowactions>
295 </workflowstate>
296 </workflowstates>
297 </workflow>

```

Translation Collections as XML Definitions

Translation Collections and XLIFF Strings

A Translation Collection stores translation strings with their translations in NetSuite. A Translation Collection is defined in a SuiteCloud project for SuiteCloud Development Framework (SDF) by the following:

- translationcollection objects – define the Translation Collections, and are composed of the object fields and translation strings. Each translationcollection object can reference multiple XLIFF files. For more information, see [Translation Collection Object Fields in XML Definitions](#).
- XLIFF (.xlf or .xliff) formatted files – which are optional files that individually contain the translated strings for a target language. Each XLIFF file is associated with a single translationcollection object, has a single target language, and contains translations for that target language. For more information, see [Translation Collection Strings in SuiteCloud Projects](#).

The XLIFF files are located separately from the translationcollection object in the SuiteCloud project, providing a convenient way to add translation strings received from your translators. The XLIFFs can also be imported into the Translation Collection in NetSuite separately. Using XLIFF files provided by your translators eliminates the need to manually manipulate the strings.

For information about creating and managing Translation Collections in NetSuite, see the help topic [Translation Collections Overview](#).

To reference translation collections in SDF custom objects, see [Translatable Fields on SDF Custom Objects](#)

To interact with Translation Collections programmatically with SuiteScript, see the help topic [N/translation Module](#).

Consider the following information when working with Translation Collections as XML definitions:

- [Importing Translation Collections into SuiteCloud Projects](#)
- [Deploying SuiteCloud Projects Containing Translation Collections](#)
- [Including Translation Collections in SDF SuiteApps](#)

Importing Translation Collections into SuiteCloud Projects

Consider the following information when you are importing Translation Collections into a SuiteCloud project:

- You can import existing translationcollection objects from an account by selecting custcollection as the search criteria. When importing translationcollection objects into a SuiteCloud project, the corresponding XLIFF files are not created.

- When you use the **Import Bundle Components** action to import a Translation Collection into an account customization project (ACP), the XLIFF files are not created with the translationcollection objects. For more information, see [Conversion of a Bundle to an Account Customization Project for SDF](#).
- When you use the **Convert to SuiteCloud Project** action from a bundle in the NetSuite UI, previously imported XLIFF files are included for the translationcollection objects. The XLIFF files are located in a Translations subdirectory of the ZIP file. For information about converting a bundle to a SuiteCloud project, see [Converting a Bundle into an Account Customization Project from a NetSuite Account](#).

You can use the NetSuite UI to export the XLIFF files from the NetSuite account and then add those XLIFF files to your SuiteCloud project. For more information, see the help topic [Exporting a Translation Collection](#).

Deploying SuiteCloud Projects Containing Translation Collections

Consider the following information when deploying a SuiteCloud project containing Translation Collections:

- Translation Collections are created in the NetSuite account based on fields in the translationcollection object.
- The XLIFF (.xlf or .xliff) files are imported into NetSuite and their strings are added to their corresponding Translation Collections. For XLIFF file requirements, see [Translation Collection Strings in SuiteCloud Projects](#).



Warning: When the Translation Collection already exists in the target account, XLIFF files in the SuiteCloud project overwrite the target account's existing strings during deployment. There is no conflict detection when deploying Translation Collection strings. Ensure the XLIFF files contain the correct version of the strings. The previous XLIFF import information is located on the **Import Jobs** subtab of the Manage Translations page.

- Removing a term (translation string) from a translationcollection object in an SDF SuiteApp project removes both that term and its translated strings from the Translation Collection in the target NetSuite account.
- Removing a translation from the XLIFF file does not remove that translation from the target account if the translation already exists at the time of deployment.
- Removing a Translation Collection from an ACP project does not remove that Translation Collection from the target account on subsequent deployments.
- XLIFF files are imported into the target NetSuite account in the order that they are specified in the deploy file.

Including Translation Collections in SDF SuiteApps

Consider the following information when including Translation Collections in SDF SuiteApps:

- Removing a Translation Collection from the SDF SuiteApp project removes that collection from the NetSuite account when the SuiteApp is installed as an upgrade. The Translation Collection's strings are also removed.
- Translation Collections are removed from the NetSuite account when the SDF SuiteApp is uninstalled. The corresponding translation strings are also removed.
- When an SDF SuiteApp deployment locks a Translation Collection in the target account, subsequent deployments can still update that Translation Collection including its strings.

Translation Collection Object Fields in XML Definitions

In SuiteCloud Development Framework (SDF), the `translationcollection` SDF custom object defines the Translation Collection by including the following information:

- Translation Collection record metadata - such as `scriptid`, `name`, `defaultlanguage`, and optional `description`.
- Translation strings - which are the list of strings that are being translated. Each string contains the following attributes and fields:
 - `scriptid` – is the attribute for the string field, and is used as an identifier for the translation string. This `scriptid` attribute value is also known as the term or key. If an XLIFF file is included in a SuiteCloud project, this value is also used as the `id` attribute value for the `trans-unit` field.
 - `defaulttranslation` – specifies the default translation used for the term. This translation is in the same language as the default language.
 - `description` (optional) – provides a description for the term. This description is exported to the XLIFF and helps translation vendors understand the context of the term for accurate translation. The term's description is the when, where, and why users see the term's string in the application.

For information about all of the available fields, see the help topic [translationcollection](#).

Consider the following information when defining `translationcollection` objects:

- The filename and `scriptid` for a `translationcollection` object begins with the prefix `custcollection_`.
- The string `scriptid` attribute term can be lowercase alphanumeric with underscores. Duplicate terms cannot be specified within the same Translation Collection.
- String values for terms are not required, and can be added later from the NetSuite UI. Although string values cannot be stored in the `translationcollection`, they can be included in XLIFF files in the SuiteCloud project. For more information, see [Translation Collection Strings in SuiteCloud Projects](#).
- The `defaultlanguage` value is set at the time the `translationcollection` is created in an account. Subsequent deployments cannot change the default language for a Translation Collection. The `defaultlanguage` value must be one from the list of static values listed in [translationcollection_defaultlanguage](#).
- The name and description fields can be translated. To translate the name and description fields, reference terms from the same translation collection. For information about working with translatable fields in SDF custom objects, see [Translatable Fields on SDF Custom Objects](#).

The following example defines a `translationcollection` object with a user-readable name of Accommodation Collection and the `scriptid` identifier `custcollection_accommodation`:

```

1 <translationcollection scriptid='custcollection_accommodation'>
2   <name>Accommodation Collection</name>
3   <defaultlanguage>en-US</defaultlanguage>
4   <description>This Translation Collection contains terms for the Hotel customization.</description>
5   <strings>
6     <string scriptid='front_desk'>
7       <defaulttranslation>front desk</defaulttranslation>
8       <description>The front desk of a hotel</description>
9     </string>
10    <string scriptid='accommodation'>
11      <defaulttranslation>accommodation</defaulttranslation>
12    </string>
13  </strings>
14 </translationcollection>

```

In the preceding example, the object defines translated strings for a custom record that is used for hotel accommodation information. The default language is United States English, and the `defaulttranslation`

value for each term is also in United States English. The first term is `front_desk`, which has a the default translation "front desk".

Translation Collection Strings in SuiteCloud Projects

In SuiteCloud Development Framework (SDF), translated terms are stored in the SuiteCloud project as Translation Collection strings in XLIFF formatted files. Each `translationcollection` object in the SuiteCloud project can optionally have one or more XLIFF (.xlf or .xliff) formatted file. A separate XLIFF file is used for each language.

Your translators can provide you with translated strings in XLIFF formatted files. The following example contains Spanish language translations for the `custcollection_accommodation` Translation Collection:

```

1 <?xml version="1.0" encoding="UTF-8" standalone="yes"?>
2 <xliff xmlns="urn:oasis:names:tc:xliff:document:1.2" version="1.2">
3   <file original="netsuite-database" source-language="en-US" target-language="es-ES" datatype="database">
4     <body>
5       <group collection="custcollection_accommodation">
6         <trans-unit id="FRONT_DESK">
7           <source>front desk</source>
8           <target>mostrador</target>
9           <note from="developer">The front desk of a hotel</note>
10        </trans-unit>
11        <trans-unit id="ACCOMMODATION">
12          <source>accommodation</source>
13          <target>alojamiento</target>
14          <note from="developer">The type of reservation.</note>
15        </trans-unit>
16      </group>
17    </body>
18  </file>
19 </xliff>

```

Each XLIFF file must meet the following requirements:

- An XLIFF file can only be associated with a single `translationcollection` object, have a single target language, contain translations for that target language.
- Each XLIFF file must meet the XLIFF 1.2 Specification.
- Each XLIFF file must be UTF-8 encoded.
- Each XLIFF file must specify a target language.
- For each XLIFF file, the target language must be different than the source language.
- All XLIFF files must be located in the Translations directory at the root of the SuiteCloud project. If the Translations directory is needed and does not exist, create it. The Translations directory is not required when there are no XLIFF files are included with the `translationcollection` objects in the SuiteCloud project.

The path to the XLIFF file must be added to the `deploy.xml` file. The asterisk (*) wildcard can be specified in the path to specify multiple files, but files in the Translation directory without the .xlf and .xliff file extensions are not included in deployment and do not appear in the deployment log. The following example adds the path to an XLIFF file containing Spanish language translations for the `custcollection_accommodation` Translation Collection:

```

1 <deploy>
2 ...
3   <translationimports>
4     <path>~/Translations/accommodation_es_ES.xlf</path>
5   </translationimports>
6 ...

```

```
7 | </deploy>
```

Consider the following information regarding translation string validation:

- SuiteCloud project validation includes validation for translation string length. A validation error is returned when the string does not satisfy field length constraints for the field referencing the string. Consider the following information regarding string length validation:
 - When a translation string is referenced by multiple SDF custom objects, the length constraint is derived from both the minimum length and maximum length allowed for all fields on all objects referencing that string.
For example, if field A has a maximum length of 50 and field B has a maximum length of 20, then the following is true for a translation string with length of 30:
 - Field A referencing the translation string passes validation.
 - Field B referencing the translation string returns a validation error stating that the maximum length is 20.
 - The string length validation is run on all translation strings in the SuiteApp regardless of the default language of the account.
 - The string length validation is run as an account specific validation due to the re-use of terms, which includes object references that exist in the account and do not exist in the SuiteApp.
- SuiteCloud project validation includes validation of translation strings for fields with uniqueness constraints. A validation error is returned when the uniqueness constraint for a field is not satisfied by the translation strings in the default language. Consider the following information regarding field uniqueness constraint validation:
 - The field uniqueness constraint validation compares the source string against both the name of the field of objects in the account and the value in the translation collection.
 - The field uniqueness constraint validation for translation strings is account-specific and only validated for the default language.

The XLIFF files can be omitted from the SuiteCloud project and added later in the target NetSuite account. For more information, see the following:

- [Importing Translations into a Translation Collection](#)
- [Working with Translation Collection Strings](#)

If no XLIFF files exist in the SuiteCloud project when it is deployed to an account, the files are still automatically created when exporting the Translation Collection from the account using the NetSuite UI. For more information, see the help topic [Exporting a Translation Collection](#).

Map/Reduce Script Objects as XML Definitions

A map/reduce script record and its deployments are represented in SuiteCloud Development Framework (SDF) by a `mapreducescript` SDF custom object. Map/reduce scripts are a server side script type in SuiteScript used for processing a large number of records or a large amount of data. For more information about using map/reduce scripts, see the help topic [SuiteScript 2.x Map/Reduce Script Type](#) in the SuiteScript documentation.

For information about working with a map/reduce script object from your SuiteCloud project, see the following topics:

- [Structure of a Map/Reduce Object](#)
- [Referencing a Script File in a mapreducescript SDF Custom Object](#)

- [Defining a Map/Reduce Script Deployment in XML](#)
- [Setting a Concurrency Limit on Your Map/Reduce Script Deployment in SDF](#)
- [Referencing a Custom Plug-in in a mapreducescript SDF Custom Object](#)
- [Referencing a Custom Script Field in a mapreducescript SDF Custom Object](#)
- [Example of a mapreducescript SDF Custom Object](#)

Structure of a Map/Reduce Object

The map/reduce script object is comprised of script record elements, a reference to the map/reduce script file, and optionally includes script deployments, custom fields, and plug-in structures. The following example shows the structure of a map/reduce script object.

Note: Map/reduce script IDs require the prefix 'customscript_' followed by your unique ID. For more information about prefixes, see [SDF Custom Object File Structure](#).

```

1 <mapreducescript scriptid="customscript_mysuitescript">
2   <isinactive>F</isinactive>
3   <name>NAME</name>
4   <notifyowner>T</notifyowner>
5   <scriptfile>[/FolderPath/FileName.js]</scriptfile>
6   <scriptdeployments>
7     <scriptdeployment>
8       ...
9     </scriptdeployment>
10  </scriptdeployments>
11  <customplugintypes>
12    <plugintype>
13      ...
14    </plugintype>
15  </customplugintypes>
16  ...
17  <scriptcustomfield>
18    <scriptcustomfield>
19      ...
20    </scriptcustomfield>
21  </scriptcustomfields>
22 </mapreducescript>

```

For information about the fields and values supported for this object, see the following topics:

- [mapreducescript](#)
- [scriptdeployment](#)

Manifest Requirements

To deploy map/reduce script objects to an account, Server Side Scripting must be enabled. For more information, see [Feature Dependencies for SDF Custom Objects in SuiteCloud Projects](#) and [Enabling SuiteCloud Development Framework in the Target NetSuite Account \(Administrator Only\)](#).

Defining a Map/Reduce Script Deployment in XML

As part of developing map/reduce scripts in SuiteCloud Development Framework (SDF), to execute the map/reduce script in the target account, you need to include script deployment values in your mapreducescript SDF custom object. To learn about each deployment option, see the help topic [Map/Reduce Script Deployment Record](#).

You can nest a `scriptdeployments` structure within your `mapreducescript` SDF custom object:

```

1 <mapreducescript scriptid="customscript_mr">
2   ...
3   <scriptdeployments>
4     <scriptdeployment scriptid="customdeploy_123890">
5       <title>Audit Deploy</title>
6       <isdeployed>T</isdeployed>
7       <loglevel>AUDIT</loglevel>
8       <runasrole>ADMINISTRATOR</runasrole>
9       <status>NOTSCHEDULED</status>
10      <yieldaftermins>60</yieldaftermins>
11    </scriptdeployment>
12  </scriptdeployments>
13  ...
14 </mapreducescript>

```

You can also import or download existing script deployments from a NetSuite account.

For more information, see [Account Component Imports to SuiteCloud Projects](#) and [Downloading the XML Definition of an Object from a NetSuite Account](#).

The `scriptdeployments` structure can hold multiple `scriptdeployment` elements for defining script deployments.

```

1 <mapreducescript scriptid="customscript_mr">
2   ...
3   <scriptdeployments>
4     <scriptdeployment scriptid="customdeploy_123">
5       <isdeployed>T</isdeployed>
6       <loglevel>AUDIT</loglevel>
7       <runasrole></runasrole>
8       <status>SCHEDULED</status>
9       <yieldaftermins>10</yieldaftermins>
10    </scriptdeployment>
11    <scriptdeployment scriptid="customdeploy_sdf_project_test">
12      <isdeployed>T</isdeployed>
13      <loglevel>DEBUG</loglevel>
14      <runasrole></runasrole>
15      <status>NOTSCHEDULED</status>
16      <yieldaftermins>60</yieldaftermins>
17    </scriptdeployment>
18  </scriptdeployments>
19  ...
20 </mapreducescript>

```

Setting a Concurrency Limit on Your Map/Reduce Script Deployment in SDF

As part of supporting map/reduce scripts in SuiteCloud Development Framework (SDF), if the `scriptdeployments` SDF custom object XML definition in your SuiteCloud project specifies a concurrency value that exceeds the limit available in the target account, the concurrency limit value is adjusted to the account limit when your project is deployed to the account.

If the account concurrency limit increases in your target account, the concurrency value limit is updated when the source value is larger. This happens automatically to the map/reduce record that was deployed through SDF. Adjustment by SDF no longer occurs when the concurrency field of deployment record is edited from another context such as the UI.

For example, if you set concurrency value to 10 in your map/reduce object and deploy the SuiteCloud project to an account with a concurrency limit of 5, the deployment record in the account is set to 5. If you then download the map/reduce object as XML or import it to a SuiteCloud project, the concurrency value is unchanged. It remains set to 10.

Note: Concurrency limits are specific to the account and SuiteCloud Plus licenses.

The following example shows a Map/Reduce script deployment with a concurrency limit of five:

```

1 <mapreducescript scriptid="customscript_mr">
2   ...
3   <scriptdeployments>
4     <scriptdeployment scriptid="customdeploy_123">
5       <title>My Script Deployment</title>
6       <isdeployed>T</isdeployed>
7       <loglevel>AUDIT</loglevel>
8       <runasrole></runasrole>
9       <status>NOTSCHEDULED</status>
10      <concurrencylimit>5</concurrencylimit>
11    </scriptdeployment>
12  </scriptdeployments>
13  ...
14 </mapreducescript>

```

For more information, see the following topics:

- [scriptdeployment](#)
- [mapreducescript](#)
- [Concurrency Limit](#)

Referencing a Script File in a mapreducescript SDF Custom Object

In SuiteCloud Development Framework (SDF), the mapreducescript SDF custom object is a definition of the script record and deployment record. In addition, your object needs a reference to the map/reduce script file to execute a script in your target account. Map/reduce script files need to be kept in the File Cabinet and saved as a JS file.

You must include the script in your project's File Cabinet folder and reference it from the mapreducescript SDF custom object. The following example shows MapReduceScript.js referenced from the customscript_mr SDF custom object:

```

1 <mapreducescript scriptid="customscript_mr">
2   ...
3   <scriptfile>[/SuiteScripts/MapReduceScript.js]</scriptfile>
4   ...
5 </mapreducescript>

```

For more information about SDF reference format rules, see [Specifying a Custom File or Script Reference](#).

Referencing a Custom Plug-in in a mapreducescript SDF Custom Object

As part of supporting map/reduce scripts in SuiteCloud Development Framework (SDF), to specify the custom plug-in type in a mapreducescript SDF custom object, use the customplugintypes and plugintype elements to reference your custom plug-in. For example:

```

1 <mapreducescript scriptid="customscript_mr">
2   ...
3   <customplugintypes>
4     <plugintype>
5       <plugintype>[scriptid=customscript_doc_signing]</plugintype>

```

```

6      </plugintype>
7    </customplugintypes>
8    ...
9  </mapreducescript>

```

For information about custom plug-ins:

- [Custom Plug-ins as XML Definitions](#)
- [plugintype](#)

Referencing a Custom Script Field in a mapreducescript SDF Custom Object

As part of supporting map/reduce scripts in SuiteCloud Development Framework (SDF), to specify the script parameters in a mapreducescript object, use the customscriptfields structure and elements. For example:

```

1  <mapreducescript scriptid="customscript_mr">
2    ...
3    <scriptcustomfields>
4      <scriptcustomfield scriptid="custscript1">
5        <accesslevel>2</accesslevel>
6        <applyformatting>F</applyformatting>
7        <checkspelling>F</checkspelling>
8        <defaultchecked>F</defaultchecked>
9        <displaytype>NORMAL</displaytype>
10       <fieldtype>TEXT</fieldtype>
11       <isformula>F</isformula>
12       <ismandatory>F</ismandatory>
13       <label>test</label>
14       <onparentdelete>NO_ACTION</onparentdelete>
15       <searchlevel>2</searchlevel>
16       <storevalue>T</storevalue>
17     </scriptcustomfield>
18   </scriptcustomfields>
19   ...
20 </mapreducescript>

```

For more information about custom script fields in SuiteScript, see the help topic [Creating Script Parameters](#).

Example of a mapreducescript SDF Custom Object

The following example defines a mapreducescript SDF custom object:

```

1  <mapreducescript scriptid="customscript_mr">
2    <isinactive>F</isinactive>
3    <name>NAME</name>
4    <notifyowner>T</notifyowner>
5    <scriptfile>[/SuiteScripts/MapReduceScript.js]</scriptfile>
6    <customplugintypes>
7      <plugintype>
8        <plugintype>[scriptid=customscript_doc_signing]</plugintype>
9      </plugintype>
10   </customplugintypes>
11   <scriptcustomfields>
12     <scriptcustomfield scriptid="custscript1">
13       <accesslevel>2</accesslevel>
14       <applyformatting>F</applyformatting>
15       <checkspelling>F</checkspelling>
16       <defaultchecked>F</defaultchecked>
17       <displaytype>NORMAL</displaytype>
18       <fieldtype>TEXT</fieldtype>

```

```

19      <isformula>F</isformula>
20      <ismandatory>F</ismandatory>
21      <label>test</label>
22      <onparentdelete>NO_ACTION</onparentdelete>
23      <searchlevel>2</searchlevel>
24      <storevalue>T</storevalue>
25    </scriptcustomfield>
26  </scriptcustomfields>
27  <scriptdeployments>
28    <scriptdeployment scriptid="customdeploy_123">
29      <isdeployed>T</isdeployed>
30      <loglevel>AUDIT</loglevel>
31      <runasrole></runasrole>
32      <status>NOTSCHEDULED</status>
33      <yieldaftermins>60</yieldaftermins>
34    </scriptdeployment>
35  </scriptdeployments>
36 </mapreducescript>

```

Portlet Scripts as XML Definitions

You can import portlet scripts from a target NetSuite account into your SuiteCloud projects. The portlet scripts can be used to dictate the behaviors and functionality of a custom portlet in the NetSuite dashboard. The portlet script object contains definitions provided by the portlet scripts.

The following portlet types are supported by SuiteCloud Development Framework (SDF):

- LIST
- FORM
- HTML
- LINKS

The custom portlet script object can be created in your SuiteCloud project and deployed to a target NetSuite account. A custom portlet script can also be imported from a NetSuite account into your SuiteCloud project. You can use a SuiteCloud project to create a full custom portlet workflow. This can be done by performing the following tasks:

- Create a published dashboard object that includes a custom portlet. For more information, see [Published Dashboards as XML Definitions](#)
- Use SuiteScript to create a portlet script that controls the behavior of the custom portlet. For more information, see the help topic [SuiteScript 2.x Portlet Script Type](#)
- Add a portlet script object to create a script record for the portlet script, and apply it to the custom portlet. For more information, see [Setting Values for a Portlet Script Object](#)

For additional information, see:

- [Account Component Imports to SuiteCloud Projects](#)
- [Supported Published Dashboard Object Portlets](#)
- [Custom Portlets](#)
- [Dashboard Portlets](#)

Components of a Script Object

There are three components that are required for the script object:

1. **The Scriptid Attribute:** Provide a readable scriptid attribute for the script record and scriptdeployment structure by adding an underscore to the default value followed by a

meaningful name for the object. The script record's `scriptid` attribute must start with a `customscript_` prefix and each `scriptdeployment` structure's `scriptid` attribute must start with a `customdeploy_` prefix.

2. **The Script Record:** Represented in XML by the `object`. The object contains several elements that define it, including the object name and a reference to the location of the script file.
3. **The Script Deployment Record:** Represented in XML by the `scriptdeployments` structure. A script object may contain multiple script deployments.

Components of a Portlet Script Object

The following components allow the portlet script object to work:

- **The Script Record:** Represented in XML by the portlet script object. The object contains several elements that define it, including the name of the portlet script object, a reference to the location of the portlet script file, and the portlet type.

The following example shows a portlet script object named "Portlet Test 1". The portlet script object also defines the portlet type as "HTML", and references a portlet script file named "Portlet_test1.js".

```

1 <portlet scriptid="customscript_porlet_test1">
2   <defaultfunction>testPortlet</defaultfunction>
3   <name>Portlet Test 1</name>
4   <notifyowner>T</notifyowner>
5   <portlettype>HTML</portlettype>
6   <scriptfile>[/SuiteScripts/Portlet_test1.js]</scriptfile>
7   <scriptdeployments>
8     <scriptdeployment scriptid="customdeploy_portlet_test1">
9       <isdeployed>T</isdeployed>
10      <loglevel>DEBUG</loglevel>
11      <status>TESTING</status>
12      <title>Portlet Test 1</title>
13    </scriptdeployment>
14  </scriptdeployments>
15 </portlet>

```

- **The Script Deployment Record:** Represented in XML by the `scriptdeployments` structure. It is contained within the portlet script object where you can define certain values such as the title of your script deployment, and the status.

The following example shows a `scriptdeployments` structure with the title "Portlet Test 1" and a status value set to "TESTING". This example only has one script deployment, but a portlet script object may contain multiple script deployments.

```

1 <portlet scriptid="customscript_porlet_test1">
2   <defaultfunction>testPortlet</defaultfunction>
3   <name>Portlet Test 1</name>
4   <notifyowner>T</notifyowner>
5   <portlettype>HTML</portlettype>
6   <scriptfile>[/SuiteScripts/Portlet_test1.js]</scriptfile>
7   <scriptdeployments>
8     <scriptdeployment scriptid="customdeploy_portlet_test1">
9       <isdeployed>T</isdeployed>
10      <loglevel>DEBUG</loglevel>
11      <status>TESTING</status>
12      <title>Portlet Test 1</title>
13    </scriptdeployment>
14  </scriptdeployments>
15 </portlet>

```

- **The Portlet Script:** Can be imported or created using SuiteScript. The portlet script file must be stored in the File Cabinet of your SuiteCloud project.

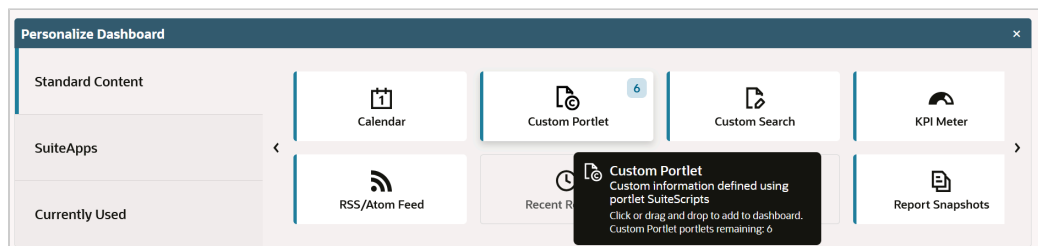
The following example shows a Simple Form portlet script named `formPortlet.js`.

```

1  /**
2  * @ApiVersion 2.x
3  * @NScriptType Portlet
4  */
5  define(['N/search'],
6      function(search) {
7          function render(params) {
8              var portlet = params.portlet;
9              portlet.title = 'Simple Form Portlet';
10             var fld = portlet.addField({
11                 id: 'text',
12                 type: 'text',
13                 label: 'Text'
14             });
15             fld.updateLayoutType({
16                 layoutType: 'normal'
17             });
18             fld.updateBreakType({
19                 breakType: 'startcol'
20             });
21             portlet.setSubmitButton({
22                 url: 'http://httpbin.org/post',
23                 label: 'Submit',
24                 target: '_top'
25             });
26         }
27         return {
28             render: render
29         };
30     });

```

- **The Custom Portlet:** Can be created in the NetSuite UI, or in a SuiteCloud project. When the custom portlet is created, you can reference it in the custom script object.



For more information about:

- Creating and deploying a portlet script, see the help topic [Creating and Deploying a Portlet Script](#).
- Creating a portlet script file, see the help topic [SuiteScript 2.x Portlet Script Type](#)
- Using the portlet script file referenced in the example, see the help topic [Simple Form Portlet Script Sample](#)
- Creating a custom portlet in the NetSuite UI, see the help topic [Custom Portlets](#)
- Creating a custom portlet in a SuiteCloud project, see [Published Dashboards as XML Definitions](#).

Setting Values for a Script Object

Each script object has a set of values that are required to successfully validate its script record and script deployment record.

The following elements are required to successfully validate your script record:

- **name** —Provide a meaningful name for your script record.

- **scriptfile** —Reference the appropriate script file. This must be a JavaScript file (.js).

The following element is required to successfully validate your script deployments:

- **status** —Provide a value for the status element. The default value is TESTING. The only values accepted for all scripts except scheduled scripts are:
 - RELEASED: The script will run in the accounts of all specified audience members.
 - TESTING: The script will execute for the script owner and specified audience.

The remaining possible values for the status element are:

- COMPLETED
- INPROGRESS
- INQUEUE
- NOTSCHEDULED
- SCHEDULED

Setting Values for a Portlet Script Object

Provide a readable scriptid attribute for the portlet script object and scriptdeployment structure by adding an underscore to the default value, followed by a meaningful name for the object. The portlet object's scriptid attribute must start with a "customscript_" prefix and each scriptdeployment structure's scriptid attribute must start with a "customdeploy_" prefix.

The following elements are required to successfully validate your custom portlet object:

- **name**- Provide a meaningful name for your portlet object.
- **scriptfile**- Reference the appropriate portlet script file. This must be a JavaScript file (.js).
- **portlettype**- Define which type of portlet script is being referenced. This element can be set to the following values:
 - FORM
 - HTML
 - LIST
 - LINKS

For more information about portlet script types, see:

- [SuiteScript 2.x Portlet Script Type](#)
- [SuiteScript 2.x Portlet Script Entry Points and API](#)

The following elements are required to successfully validate your script deployments:

- **Title**- Provide a meaningful title for your script deployment record.
- **Status**- Provide a value for the status element. The default value is TESTING. Possible values are:
 - COMPLETED
 - INPROGRESS
 - INQUEUE
 - NOTSCHEDULED
 - RELEASED
 - SCHEDULED

□ TESTING

For more information about:

- portlet object fields and possible values, see the help topic [portlet](#).
- scriptdeployments structure fields and possible values, see the help topic [scriptdeployment](#).

Example of a Portlet Script Object

The following example shows the contents of the SuiteScript/formPortlet.js file that is referenced in the portlet script object:

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Portlet
4   */
5   define(['N/search'],
6     function(search) {
7       function render(params) {
8         var portlet = params.portlet;
9         portlet.title = 'Simple Form Portlet';
10        var fld = portlet.addField({
11          id: 'text',
12          type: 'text',
13          label: 'Text'
14        });
15        fld.updateLayoutType({
16          layoutType: 'normal'
17        });
18        fld.updateBreakType({
19          breakType: 'startcol'
20        });
21        portlet.setSubmitButton({
22          url: 'http://httpbin.org/post',
23          label: 'Submit',
24          target: '_top'
25        });
26      }
27      return {
28        render: render
29      };
30    });

```

The following example shows a custom portlet script object named “Portlet Object Example”. The object indicates the portlet type is Simple Form and references the Simple Form portlet script named “formPortlet.js”.

```

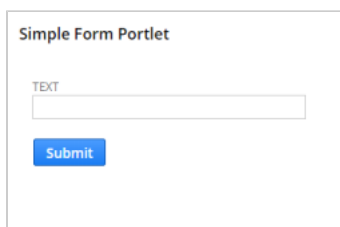
1  <portlet scriptid="customscript_portletobject_example">
2    <defaultfunction>testPortlet</defaultfunction>
3    <name>Portlet Object Example</name>
4    <notifyowner>T</notifyowner>
5    <notifyadmins>F</notifyadmins>
6    <portlettype>FORM</portlettype>
7    <scriptfile>[/SuiteScripts/formPortlet.js]</scriptfile>
8    <scriptdeployments>
9      <scriptdeployment scriptid="customdeploy_portletobject_example">
10        <isdeployed>T</isdeployed>
11        <allemployees>T</allemployees>
12        <allroles>T</allroles>
13        <dashboardapp>F</dashboardapp>
14        <loglevel>DEBUG</loglevel>
15        <status>TESTING</status>
16        <title>Portlet Object Example</title>
17      </scriptdeployment>
18    </scriptdeployments>

```

19 | `</portlet>`

Note: When you have successfully deployed your portlet script object to a target NetSuite account, you must create your custom portlet either in the NetSuite UI or in your SuiteCloud project. Now, you can go to your dashboard to display your custom portlet. For information, see the help topics [Custom Portlets](#) and [Adding a Portlet to a Dashboard](#).

The following image shows how your custom portlet would appear in your dashboard after successfully deploying it to a target NetSuite account:



Client Scripts as XML Definitions

You can import client scripts from a target NetSuite account into your SuiteCloud projects. Client Scripts are generally used to validate user-entered data, and to auto-populate fields or sublists when triggered by various form events.

There are 12 client event types that can trigger the execution of your client scripts:

- `pageInit`
- `saveRecord`
- `validateField`
- `fieldChanged`
- `postSourcing`
- `lineInit`
- `validateLine`
- `recalc`
- `validateInsert`
- `validateDelete`
- `localizationContextEnter`
- `localizationContextExit`

You can create a client script object in your SuiteCloud project and deploy it to a target NetSuite account. A client script can also be imported from a NetSuite account into your SuiteCloud project.

For more information, see:

- [SuiteScript 2.x Client Script Type](#)
- [Using the currentRecord Module in Client Scripts](#)
- [SuiteScript 2.x Client Script Entry Points and API](#)
- [Account Component Imports to SuiteCloud Projects](#)

Components of a Script Object

There are three components that are required for the script object:

1. **The Scriptid Attribute:** Provide a readable scriptid attribute for the script record and scriptdeployment structure by adding an underscore to the default value followed by a meaningful name for the object. The script record's scriptid attribute must start with a customscript_ prefix and each scriptdeployment structure's scriptid attribute must start with a customdeploy_ prefix.
2. **The Script Record:** Represented in XML by the object. The object contains several elements that define it, including the object name and a reference to the location of the script file.
3. **The Script Deployment Record:** Represented in XML by the scriptdeployments structure. A script object may contain multiple script deployments.

Components of a Client Script Object

The following components allow the portlet object to work.

- **The Script Record:** Represented in XML by the clientscript object. The object contains several elements that define it, including the name of the clientscript object, and a reference to the location of the client script file. The following example shows a clientscript object named "Client Script Example" that references a client script file named "ClientScriptExample.js".

```

1 <clientscript scriptid="customscript_clientscript_example">
2   <isinactive>T</isinactive>
3   <name>Client Script Example</name>
4   <notifyowner>T</notifyowner>
5   <scriptfile>[/SuiteScripts/ClientScriptExample.js]</scriptfile>
6   <scriptdeployments>
7     <scriptdeployment scriptid="customdeploy_clientscript_example">
8       <isdeployed>T</isdeployed>
9       <loglevel>DEBUG</loglevel>
10      <recordtype>TASK</recordtype>
11      <status>TESTING</status>
12    </scriptdeployment>
13  </scriptdeployments>
14 </clientscript>

```

- **The Script Deployment Record:** Represented in XML by the scriptdeployments structure. It is contained within the clientscript object where you can define certain values such as the recordtype and status. the following example shows the recordtype element referencing a custom record named "TASK". The status element is set to "TESTING". This example only has one script deployment, but a client script object may contain multiple script deployments.

```

1 <clientscript scriptid="customscript_clientscript_example">
2   <isinactive>T</isinactive>
3   <name>Client Script Example</name>
4   <notifyowner>T</notifyowner>
5   <scriptfile>[/SuiteScripts/ClientScriptExample.js]</scriptfile>
6   <scriptdeployments>
7     <scriptdeployment scriptid="customdeploy_clientscript_example">
8       <isdeployed>T</isdeployed>
9       <loglevel>DEBUG</loglevel>
10      <recordtype>TASK</recordtype>
11      <status>TESTING</status>
12    </scriptdeployment>
13  </scriptdeployments>
14 </clientscript>

```

For more information about:

- The steps needed to run your client script in the NetSuite UI, see the help topic [SuiteScript 2.x Client Script Type](#)
- Client script samples, see the help topic [Client Script Samples](#)
- Attaching your client script to a form, see the help topic [Attaching a Client Script to a Form](#)
- Creating a generic script record in a NetSuite account, see the help topic [Creating a Script Record](#)
- Creating a deployment record in a NetSuite account, see the help topic [Deploying a Script by using the Deployments Sublist](#)

Setting Values for a Script Object

Each script object has a set of values that are required to successfully validate its script record and script deployment record.

The following elements are required to successfully validate your script record:

- **name** —Provide a meaningful name for your script record.
- **scriptfile** —Reference the appropriate script file. This must be a JavaScript file (.js).

The following element is required to successfully validate your script deployments:

- **status** —Provide a value for the status element. The default value is TESTING. The only values accepted for all scripts except scheduled scripts are:
 - RELEASED: The script will run in the accounts of all specified audience members.
 - TESTING: The script will execute for the script owner and specified audience.

The remaining possible values for the status element are:

- COMPLETED
- INPROGRESS
- INQUEUE
- NOTSCHEDULED
- SCHEDULED

Setting Values for a Client Script Object

Provide a readable scriptid attribute for the clientscript object and scriptdeployment structure by adding an underscore to the default value followed by a meaningful name for the object. The clientscript object's scriptid attribute must start with a "customscript_" prefix and each scriptdeployment structure's scriptid attribute must start with a "customdeploy_" prefix.

The following elements are required to successfully validate a clientscript object:

- **name**- Provide a meaningful name for your script record.
- **scriptfile**- Reference the appropriate script file. This must be a JavaScript file (.js).

The following elements are required to successfully validate a scriptdeployment structure:

- **recordtype**- Reference the appropriate record within your deployment script so that your client script deploys to the correct record.
- **status**- Provide a value for the status element. The default value is TESTING. Possible values are:
 - COMPLETED

- INPROGRESS
- INQUEUE
- NOTSCHEDULED
- RELEASED
- SCHEDULED
- TESTING

For more information about:

- `clientscript` object fields and possible values, see the help topic [clientscript](#)
- `scriptdeployments` structure fields and possible values, see the help topic [scriptdeployment](#)

Example of a Client Script Object

The following example shows the client script file named “ClientScriptExample.js” that is being referenced in the client script object. This client script is using the `pageInit` function to display a title and message before the form is first loaded.

```

1  /**
2   * @ApiVersion 2.x
3   * @NScriptType ClientScript
4   */
5
6   define(['N/ui/dialog'],
7     function(dialog) {
8       function helloWorld() {
9         var options = {
10           title: 'Hello!',
11           message: 'Hello, world!'
12         };
13
14         try {
15           dialog.alert(options);
16           log.debug({
17             title: 'Success',
18             details: 'Alert displayed successfully'
19           });
20         } catch (e) {
21           log.error({
22             title: e.name,
23             details: e.message
24           });
25         }
26       }
27
28       return {
29         pageInit: helloWorld
30       };
31     }
32 );

```

The following example shows a client script object referencing the “ClientScriptExample.js” file:

```

1  <clientscript scriptid="customscript_clientscript_example">
2    <isinactive>F</isinactive>
3    <name>Client Script Example</name>
4    <notifyowner>T</notifyowner>
5    <scriptfile>[/SuiteScripts/ClientScriptExample.js]</scriptfile>
6    <scriptdeployments>
7      <scriptdeployment scriptid="customdeploy_clientscript_example">
8        <isdeployed>T</isdeployed>

```

```

9      <loglevel>DEBUG</loglevel>
10     <recordtype>TASK</recordtype>
11     <status>TESTING</status>
12   </scriptdeployment>
13 </scriptdeployments>
14 </clientscript>

```

Integration Records as XML Definitions

SuiteCloud Development Framework (SDF) supports referencing integration records from a SuiteCloud project by defining the integration SDF custom object. The integration record itself is not stored in the SuiteCloud project. Instead, the integration record is represented by an integration SDF custom object. The scriptid attribute in an integration SDF custom object acts as a unique identifier for the integration record in SDF SuiteApps. This ID provides a way to auto-install the integration record in a NetSuite account as part of the SDF SuiteApp installation, and remove the integration record when the SDF SuiteApp is uninstalled. For more information about the scriptid attribute, see the help topic [integration](#). For information about auto-installation, see the help topic [Auto-Installation of Integration Records](#).



Note: You cannot create an integration record using SDF. The integration SDF custom object represents the integration record by its scriptid attribute value, but the SDF custom object does not contain any fields or data. To create an integration record in a NetSuite account, see the help topic [Creating an Integration Record](#).

When deploying an SDF project (or installing an SDF SuiteApp) that contains an integration SDF custom object, a validation error is not returned when an integration record with that script ID already exists in the target account. Instead, the integration record is not deployed (or installed) to the target account because it already exists and does not need to be created.

The following example defines an integration SDF custom object that references the script ID for an integration record:

```

1 <integration scriptid="custinteg_9f89c84a669f">
2 </integration>

```

Mass Update Scripts as XML Definitions

You can import mass update scripts from a target NetSuite account into your SuiteCloud projects for SuiteCloud Development Framework (SDF). Mass update scripts let you perform custom updates to fields that are unavailable through general mass updates. You can also customize mass update scripts to run complex calculations across many records. For more information, see the help topic [SuiteScript 2.x Mass Update Script Type](#).

The mass update script object can be created in your SuiteCloud project and deployed to a target NetSuite account. Mass update scripts can also be imported from a NetSuite account into your SuiteCloud project. For information about importing SDF custom objects, files, and scripts from a NetSuite account into your SuiteCloud project, see [Account Component Imports to SuiteCloud Projects](#).

For information about working with a mass update script object from your SuiteCloud project, see the following topics:

- [Components of a Script Object](#)
- [Components of a Mass Update Script Object](#)

- [Setting Values for a Script Object](#)
- [Setting Values for a Mass Update Script Object](#)
- [Example of a Mass Update Script Object](#)

Components of a Script Object

There are three components that are required for the script object:

1. **The Scriptid Attribute:** Provide a readable scriptid attribute for the script record and scriptdeployment structure by adding an underscore to the default value followed by a meaningful name for the object. The script record's scriptid attribute must start with a customscript_ prefix and each scriptdeployment structure's scriptid attribute must start with a customdeploy_ prefix.
2. **The Script Record:** Represented in XML by the object. The object contains several elements that define it, including the object name and a reference to the location of the script file.
3. **The Script Deployment Record:** Represented in XML by the scriptdeployments structure. A script object may contain multiple script deployments.

Components of a Mass Update Script Object

The following components allow the massupdatescript to work:

- **The Script Record:** Represented in XML by the massupdatescript object. The object contains several elements that define it, including the name of the massupdatescript object, and a reference to the location of the mass update script file. The following example shows a massupdatescript object named "Mass Update Test" that references a mass update script file named "MassUpdateScripttest.js".

```

1 <massupdatescript scriptid="customscript_massupdate_test">
2   <defaultfunction>testMassUpdate</defaultfunction>
3   <name>Mass Update Test</name>
4   <notifyowner>T</notifyowner>
5   <scriptfile>[/SuiteScripts/MassUpdateScripttest.js]</scriptfile>
6   <scriptdeployments>
7     <scriptdeployment scriptid="customdeploy_massupdate_test">
8       <isdeployed>T</isdeployed>
9       <loglevel>DEBUG</loglevel>
10      <recordtype>[customrecord_massupdate_test]</recordtype>
11      <status>TESTING</status>
12    </scriptdeployment>
13  </scriptdeployments>
14 </massupdatescript>

```

- **The Script Deployment Record:** Represented in XML by the scriptdeployments structure. It is contained within the massupdatescript object where you can define values such as the recordtype and status. The following example shows the recordtype element referencing a custom record named "customrecord_massupdate_test". The status element has a value set to "TESTING". This example only has one script deployment, but a mass update script object may contain multiple script deployments.

```

1 <massupdatescript scriptid="customscript_massupdate_test">
2   <defaultfunction>testMassUpdate</defaultfunction>
3   <name>Mass Update Test</name>
4   <notifyowner>T</notifyowner>
5   <scriptfile>[/SuiteScripts/MassUpdateScripttest.js]</scriptfile>
6   <scriptdeployments>
7     <scriptdeployment scriptid="customdeploy_massupdate_test">
8       <isdeployed>T</isdeployed>
9       <loglevel>DEBUG</loglevel>
10      <recordtype>[customrecord_massupdate_test]</recordtype>

```

```

11     <status>TESTING</status>
12   </scriptdeployment>
13 </scriptdeployments>
14 </massupdatescript>

```

For more information about:

- The available massupdatescript object fields and possible values, see the help topic [massupdatescript](#).
- The available scriptdeployment structure fields and possible values, see the help topic [scriptdeployment](#).
- Running a Mass Update Script using the NetSuite UI, see the help topic [SuiteScript 2.x Mass Update Script Type](#).
- Creating a generic script record in a NetSuite account, see the help topic [Creating a Script Record](#).
- Creating a deployment record in a NetSuite account, see the help topic [Deploying a Script by using the Deployments Sublist](#).

Setting Values for a Script Object

Each script object has a set of values that are required to successfully validate its script record and script deployment record.

The following elements are required to successfully validate your script record:

- **name** —Provide a meaningful name for your script record.
- **scriptfile** —Reference the appropriate script file. This must be a JavaScript file (.js).

The following element is required to successfully validate your script deployments:

- **status** —Provide a value for the status element. The default value is TESTING. The only values accepted for all scripts except scheduled scripts are:
 - RELEASED: The script will run in the accounts of all specified audience members.
 - TESTING: The script will execute for the script owner and specified audience.

The remaining possible values for the status element are:

- COMPLETED
- INPROGRESS
- INQUEUE
- NOTSCHEDULED
- SCHEDULED

Setting Values for a Mass Update Script Object

Provide a readable scriptid attribute for the massupdatescript object and scriptdeployment structure by adding an underscore to the default value followed by a meaningful name for the object. The massupdatescript object's scriptid attribute must start with a "customscript_" prefix and each scriptdeployment structure's scriptid attribute must start with a "customdeploy_" prefix.

The following elements are required to successfully validate a massupdatescript object:

- **name** —Provide a meaningful name for your script record.
- **scriptfile** —Reference the appropriate script file. This must be a JavaScript file (.js).

The following elements are required to successfully validate a scriptdeployment structure:

- **recordtype** —Reference the appropriate record within your deployment script so that your mass update script deploys to the correct record.
- **status** —Provide a value for the status element. The default value is TESTING. Possible values are:
 - **RELEASED**: The script will run in the accounts of all specified audience members.
 - **TESTING**: The script will execute for the script owner and specified audience.

Example of a Mass Update Script Object

The following example shows a mass update script object referencing the `MassUpdateScripttest.js` file. The object's script record is named `Mass Update Test` with a `scriptid` attribute of `customscript_massupdate_test`.

```

1 <massupdatescript scriptid="customscript_massupdate_test">
2   <defaultfunction>testMassUpdate</defaultfunction>
3   <name>Mass Update Test</name>
4   <notifyowner>T</notifyowner>
5   <scriptfile>[/SuiteScripts/MassUpdateScripttest.js]</scriptfile>
6   <scriptdeployments>
7     <scriptdeployment scriptid="customdeploy_massupdate_test">
8       <isdeployed>T</isdeployed>
9       <loglevel>DEBUG</loglevel>
10      <recordtype>[customrecord_massupdate_test]</recordtype>
11      <status>TESTING</status>
12    </scriptdeployment>
13  </scriptdeployments>
14 </massupdatescript>

```

The following example shows a mass update script file named `MassUpdateScripttest.js` that is being referenced in the mass update script object. This script implements a mass update to set the probability field of all existing records to 61%.

```

1 /**
2  *@NApiVersion 2.x
3  *@NScriptType MassUpdateScript
4  */
5 define(['N/record'],
6   function(record) {
7     function each(params) {
8       // Set the probability to 61%
9       var recOpportunity = record.load({
10         type: params.type,
11         id: params.id
12       });
13       recOpportunity.setValue('probability', 61);
14       recOpportunity.save();
15     }
16     return {
17       each: each
18     };
19   });

```

Suitelet Scripts as XML Definitions

You can import suitelet script objects from a target NetSuite account into your SuiteCloud projects for SuiteCloud Development Framework (SDF). Suitelets are extensions of the SuiteScript API that give developers the ability to build custom NetSuite pages and backend logic. Suitelets are server-side scripts that operate in a request-response model. They are invoked by HTTP GET or POST requests to system generated URLs. For more information, see the help topic [SuiteScript 2.x Suitelet Script Type](#).

The suitelet script object can be created in a SuiteCloud project and deployed to a target NetSuite account. Suitelet scripts can also be imported from a target NetSuite account into a SuiteCloud project. For information about importing SDF custom objects, files, and scripts from a target NetSuite account into a SuiteCloud project, see [Account Component Imports to SuiteCloud Projects](#).

For information about working with a suitelet script object from your SuiteCloud project, see the following topics:

- [Components of a Script Object](#)
- [Components of a Suitelet Script Object](#)
- [Setting Values for a Script Object](#)
- [Setting Values for a Suitelet Script Object](#)
- [Example of a Suitelet Script Object](#)

For additional information about suitelet scripts, see:

- [SuiteScript 2.x Suitelet Script Type](#)
- [SuiteScript 2.x Suitelet Script Reference](#)
- [How Suitelet Scripts are Executed](#)
- [Suitelet Script Deployment Page](#)

Components of a Script Object

There are three components that are required for the script object:

1. **The Scriptid Attribute:** Provide a readable scriptid attribute for the script record and scriptdeployment structure by adding an underscore to the default value followed by a meaningful name for the object. The script record's scriptid attribute must start with a customscript_ prefix and each scriptdeployment structure's scriptid attribute must start with a customdeploy_ prefix.
2. **The Script Record:** Represented in XML by the object. The object contains several elements that define it, including the object name and a reference to the location of the script file.
3. **The Script Deployment Record:** Represented in XML by the scriptdeployments structure. A script object may contain multiple script deployments.

Components of a Suitelet Script Object

The following components allow the suitelet script object to work.

- **The Script Record:** Represented in XML by the suitelet script object. The object contains several elements that define it, including the name of the suitelet script object, and a reference to the location of the suitelet script file.

The following example shows a suitelet script object named Suitelet Script Test that references a suitelet script file named Suitelet.js and has the script id customscript_suitelettest.

```

1 <suitelet scriptid="customscript_suitelettest">
2   <name>Suitelet Script Test</name>
3   <notifyowner>T</notifyowner>
4   <scriptfile>[/SuiteScripts/Suitelet.js]</scriptfile>
5   <scriptdeployments>
6     <scriptdeployment scriptid="customdeploy_suitelettest">
7       <isdeployed>T</isdeployed>
8       <loglevel>DEBUG</loglevel>
9       <status>TESTING</status>

```

```

10     <title>SuiteletScriptTest</title>
11   </scriptdeployment>
12 </scriptdeployments>
13 </suitelet>

```

- **The Script Deployment Record:** Represented in XML by the scriptdeployments structure. It is contained within the suitelet script object where you can define certain values such as the title, status and event type for your script deployments.

The following example shows two scriptdeployments structures with the titles SuiteletScriptTest and SuiteletScriptTest2, and a status value set to TESTING.

```

1 <suitelet scriptid="customscript_suitelettest">
2   <name>Suitelet Script Test</name>
3   <notifyowner>T</notifyowner>
4   <scriptfile>/SuiteScripts/Suitelet.js</scriptfile>
5   <scriptdeployments>
6     <scriptdeployment scriptid="customdeploy_suitelettest">
7       <isdeployed>T</isdeployed>
8       <loglevel>DEBUG</loglevel>
9       <status>TESTING</status>
10      <title>SuiteletScriptTest</title>
11    </scriptdeployment>
12  </scriptdeployments>
13 </suitelet>

```

For more information about:

- Suitelet Scripts, see the help topic [SuiteScript 2.x Suitelet Script Type](#)
- Deploying a Suitelet Script, see the help topic [Suitelet Script Deployment Page](#)
- Creating a generic script record in a NetSuite account, see the help topic [Creating a Script Record](#).
- Creating a deployment record in a NetSuite account, see the help topic [Deploying a Script by using the Deployments Sublist](#).

Setting Values for a Script Object

Each script object has a set of values that are required to successfully validate its script record and script deployment record.

The following elements are required to successfully validate your script record:

- **name** —Provide a meaningful name for your script record.
- **scriptfile** —Reference the appropriate script file. This must be a JavaScript file (.js).

The following element is required to successfully validate your script deployments:

- **status** —Provide a value for the status element. The default value is TESTING. The only values accepted for all scripts except scheduled scripts are:
 - RELEASED: The script will run in the accounts of all specified audience members.
 - TESTING: The script will execute for the script owner and specified audience.

The remaining possible values for the status element are:

- COMPLETED
- INPROGRESS
- INQUEUE
- NOTSCHEDULED
- SCHEDULED

Setting Values for a Suitelet Script Object

Provide a readable `scriptid` attribute for the suitelet script object and `scriptdeployment` structure by adding an underscore to the default value, followed by a meaningful name for the object. The suitelet script object's `scriptid` attribute must start with a "customscript_" prefix and each `scriptdeployment` structure's `scriptid` attribute must start with a "customdeploy_" prefix.

The following elements are required to successfully validate your suitelet script object:

- **name** —Provide a meaningful name for your script record.
- **scriptfile** —Reference the appropriate script file. This must be a JavaScript file (.js).

The following elements are required to successfully validate your script deployments:

- **title** —Provide a meaningful title for your script deployment record.
- **status** —Provide a value for the status element. The default value is **TESTING**. Possible values are:
 - **RELEASED**: The script will run in the accounts of all specified audience members.
 - **TESTING**: The script will execute for the script owner and specified audience.

The following element is optional to successfully validate your suitelet script object:

eventtype —This element is optional. You can use the `<eventtype>` tag in your script deployment record to specify a script execution context at the same time of script deployment. For a suitelet script object, the available values for the `<eventtype>` tag are **GET** and **POST**. If you leave your `<eventtype>` tag blank, your project will still deploy and only execute the event type or event types that are specified in the suitelet script file referenced in your script deployment record.

 **Note:** Warning - If you specify an event type, your deployed script will execute only on that specific event regardless of the event types that are specified in your suitelet script file. For example, if you set the value of your event type to **GET**, your script will be restricted from running on any event other than **GET** request.

For information about:

- The additional fields specifically used for a suitelet script deployment record, see the help topic [Suitelet Script Deployment Page](#).
- The available suitelet object fields and possible values, see the help topic [suitelet](#).
- The available `scriptdeployment` structure fields and possible values, see the help topic [scriptdeployment](#).
- Creating a generic script record in a NetSuite account, see the help topic [Creating a Script Record](#).
- Creating a deployment record in a NetSuite account, see the help topic [Deploying a Script by using the Deployments Sublist](#).

Example of a Suitelet Script Object

The following example shows a suitelet script object named `Suitelet Script Test` that references a suitelet script file named `Suitelet.js` and has the script id `customscript_suitelettest`.

```

1 <suitelet scriptid="customscript_suitelettest">
2   <name>Suitelet Script Test</name>
3   <notifyowner>T</notifyowner>
4   <scriptfile>[/SuiteScripts/Suitelet.js]</scriptfile>
5   <scriptdeployments>
6     <scriptdeployment scriptid="customdeploy_suitelettest">
```

```

7      <isdeployed>T</isdeployed>
8      <loglevel>DEBUG</loglevel>
9      <status>TESTING</status>
10     <eventtype><eventtype>
11     <title>SuiteletScriptTest</title>
12   </scriptdeployment>
13 </scriptdeployments>
14 <scriptdeployments>
15   <scriptdeployment scriptid="customdeploy_suitelettest2">
16     <isdeployed>T</isdeployed>
17     <loglevel>DEBUG</loglevel>
18     <status>TESTING</status>
19     <eventtype><eventtype>
20     <title>SuiteletScriptTest2</title>
21   </scriptdeployment>
22 </scriptdeployments>
23 </suitelet>

```

The following example shows a suitelet script file named `Suitelet.js` that is referenced in the suitelet script object. This sample is of a basic suitelet that shows an empty page with a Hello World header. For information about writing more complex scripts such as suitelets used to create custom pages, forms, lists, assistants and backend suitelets, see the help topic [SuiteScript 2.x Suitelet Script Type Code Samples](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5  define([], function() {
6    function onRequest(context) {
7      var html = '<html><body><h1>Hello World</h1></body></html>';
8      context.response.write(html);
9      context.response.setHeader({
10        name: 'Custom-Header-Demo',
11        value: 'Demo'
12      });
13    }
14
15    return {
16      onRequest: onRequest
17    };
18  });

```

RESTlet Scripts as XML Definitions

RESTlets are generally used to transfer data from another system into NetSuite or to extract data from NetSuite. RESTlets can also be used in combination with other scripts to customize the behavior of a page in NetSuite. RESTlets are SuiteScripts that can be called by an external application or from NetSuite. When called, the RESTlet executes, and in some cases returns a value to the calling application.

The RESTlet script object can be created in a SuiteCloud project for SuiteCloud Development Framework (SDF) and deployed to a target NetSuite account. RESTlet scripts can also be imported from a target NetSuite account into a SuiteCloud project. For information about importing SDF custom objects, files, and scripts from a target NetSuite account into a SuiteCloud project, see [Account Component Imports to SuiteCloud Projects](#).

For information about working with a RESTlet script object from your SuiteCloud project, see the following topics:

- [Components of a Script Object](#)
- [Components of a RESTlet Script Object](#)
- [Setting Values for a Script Object](#)
- [Setting Values for a RESTlet Script Record](#)

- [Example of a RESTlet Script Object](#)

For information about working with RESTlet scripts, see the following topics:

- [SuiteScript 2.x RESTlet Script Type](#)
- [SuiteScript 2.x RESTlet Reference](#)
- [SuiteScript 2.x RESTlet Script and Request Examples](#)
- [SuiteScript 2.x RESTlet Script Entry Points and API](#)

Components of a Script Object

There are three components that are required for the script object:

1. **The Scriptid Attribute:** Provide a readable scriptid attribute for the script record and scriptdeployment structure by adding an underscore to the default value followed by a meaningful name for the object. The script record's scriptid attribute must start with a customscript_ prefix and each scriptdeployment structure's scriptid attribute must start with a customdeploy_ prefix.
2. **The Script Record:** Represented in XML by the object. The object contains several elements that define it, including the object name and a reference to the location of the script file.
3. **The Script Deployment Record:** Represented in XML by the scriptdeployments structure. A script object may contain multiple script deployments.

Components of a RESTlet Script Object

The following components are required for a RESTlet script object to work:

- **The Script Record:** Represented in XML by the RESTlet script object. The object contains several elements that define it, including the name of the RESTlet script object, and a reference to the location of the RESTlet script file.

The following example shows a RESTlet script object named Restlet Example that references a RESTlet script file named RestletExample.js and has the script id customscript_restletexample.

```

1 <Restlet scriptid="customscript_restletexample">
2   <isinactive>T</isinactive>
3   <name>Restlet Example</name>
4   <notifyowner>T</notifyowner>
5   <scriptfile>[/SuiteScripts/RestletExample.js]</scriptfile>
6   <scriptdeployments>
7     <scriptdeployment scriptid="customdeploy_restletexample">
8       <isdeployed>T</isdeployed>
9       <loglevel>DEBUG</loglevel>
10      <status>TESTING</status>
11      <title>Restlet Example</title>
12    </scriptdeployment>
13  </scriptdeployments>
14 </Restlet>

```

- **The Script Deployment Record:** Represented in XML by the scriptdeployments structure. It is contained within the RESTlet script object where you can define certain values such as the title and status for your script deployments.

The following example shows a scriptdeployments structure with the title RestletExample and a status value set to TESTING.

```

1 <Restlet scriptid="customscript_Restletexample">

```

```

2      <isinactive>T</isinactive>
3      <name>RESTlet Example</name>
4      <notifyowner>T</notifyowner>
5      <scriptfile>[/SuiteScripts/RestletExample.js]</scriptfile>
6      <scriptdeployments>
7          <scriptdeployment scriptid="customdeploy_restletexample">
8              <isdeployed>T</isdeployed>
9              <loglevel>DEBUG</loglevel>
10             <status>TESTING</status>
11             <title>Restlet Example</title>
12          </scriptdeployment>
13      </scriptdeployments>
14  </Restlet>

```

For more information about:

- RESTlet scripts, see the help topic [SuiteScript 2.x RESTlet Script Type](#)
- Deploying a RESTlet, see the help topic [Deploying a RESTlet](#)
- Creating a generic script record in a NetSuite account, see the help topic [Creating a Script Record](#).
- Creating a deployment record in a NetSuite account, see the help topic [Deploying a Script by using the Deployments Sublist](#).

Setting Values for a Script Object

Each script object has a set of values that are required to successfully validate its script record and script deployment record.

The following elements are required to successfully validate your script record:

- name —Provide a meaningful name for your script record.
- scriptfile —Reference the appropriate script file. This must be a JavaScript file (.js).

The following element is required to successfully validate your script deployments:

- status —Provide a value for the status element. The default value is TESTING. The only values accepted for all scripts except scheduled scripts are:
 - RELEASED: The script will run in the accounts of all specified audience members.
 - TESTING: The script will execute for the script owner and specified audience.

The remaining possible values for the status element are:

- COMPLETED
- INPROGRESS
- INQUEUE
- NOTSCHEDULED
- SCHEDULED

Setting Values for a RESTlet Script Record

Provide a readable scriptid attribute for the RESTlet script object and scriptdeployment structure by adding an underscore to the default value, followed by a meaningful name for the object. The RESTlet script object's scriptid attribute must start with a "customscript_" prefix and each scriptdeployment structure's scriptid attribute must start with a "customdeploy_" prefix.

The following elements are required to successfully validate your RESTlet script object:

- name —Provide a meaningful name for your script record.

- **scriptfile** —Reference the appropriate script file. This must be a JavaScript file (.js).

The following elements are required to successfully validate your script deployments:

- **title** —Provide a meaningful title for your script deployment record.
- **status** —Provide a value for the status element. The default value is TESTING. Possible values are:
 - **RELEASED**: The script will run in the accounts of all specified audience members.
 - **TESTING**: The script will execute for the script owner and specified audience.

Example of a RESTlet Script Object

The following example shows a RESTlet script object named `Restlet Example` that references a RESTlet script file named `RestletExample.js` and has the script id `customscript_restletexample`.

```

1 <Restlet scriptid="customscript_restletexample">
2   <isinactive>T</isinactive>
3   <name>Restlet Example</name>
4   <notifyowner>T</notifyowner>
5   <scriptfile>[/SuiteScripts/RestletExample.js]</scriptfile>
6   <scriptdeployments>
7     <scriptdeployment scriptid="customdeploy_restletexample">
8       <isdeployed>T</isdeployed>
9       <loglevel>DEBUG</loglevel>
10      <status>TESTING</status>
11      <title>Restlet Example</title>
12    </scriptdeployment>
13  </scriptdeployments>
14 </Restlet>

```

The following example shows a RESTlet script file named `RestletExample.js` that is referenced in the RESTlet script object. This is an example of a RESTlet script that can create multiple contact records when it has been called. For more examples of RESTlet scripts, see the help topic [SuiteScript 2.x RESTlet Script and Request Examples](#).

```

1 /**
2  * @NApiVersion 2.x
3  * @NScriptType restlet
4  */
5 define(['N/record'], function(record) {
6   return {
7     post : function(restletBody) {
8       var restletData = restletBody.data;
9       for ( var contact in restletData) {
10        var objRecord = record.create({
11          type : record.Type.CONTACT,
12          isDynamic : true
13        });
14        var contactData = restletData[contact];
15        for ( var key in contactData) {
16          if (contactData.hasOwnProperty(key)) {
17            objRecord.setValue({
18              fieldId : key,
19              value : contactData[key]
20            });
21          }
22        }
23        var recordId = objRecord.save({
24          enableSourcing : false,
25          ignoreMandatoryFields : false
26        });
27        log.debug(recordId);
28      }
29    }
30  }

```



```
31 | });
```

Secrets as XML Definitions

You can store, manage, and reference API secrets securely in NetSuite at Setup > Company > Preferences > API Secrets. You can then reference these secrets in SuiteCloud Development Framework (SDF), preventing the need for plaintext secrets in scripts. API secrets include hashes, passwords, keys, and other secrets for managing digital authentication credentials. Secrets up to 1,000,000 characters are accepted. Secret SDF custom objects can be imported in a SuiteCloud project if the Available in SuiteApp box is checked.

For information about creating API secrets, see the help topic [Creating Secrets](#). For the SuiteCloud Development Framework XML Reference information about secret SDF custom objects, see the help topic [secret](#).

Secrets are represented in SDF by a secret SDF custom object that references the API secret. In SDF, you need to create an SDF custom object XML file with the prefix `custsecret_`, that contains the root tag `<secret>` and script ID with the prefix `custsecret_`. For more information, see [SDF Custom Object and File Development in SuiteCloud Projects](#).

API secret owners can set criteria to limit access for other users or allow access for a specific SuiteApp. A secret can only be referenced by its script ID, and the password value cannot be displayed. If you need to share a password across multiple SuiteApps, you must create multiple secrets with the same password.

Only SuiteScript 2.x APIs can use the secret. For a list of supported SuiteScript 2.x modules, see the help topic [Supported SuiteScript 2.x modules](#).

Consider the following information when including secret SDF custom objects in your SuiteCloud project:

- A secret SDF custom object cannot be included in more than one SuiteApp.
- A secret can only be updated in the same account where it was created. This applies for an SDF SuiteApp installed to a target account and a SuiteCloud project deployed to a target account.
- Ensure that you do not enter a secret in an account that will be deleted in the future.

Secret Object Example

The following is an XML definition example of a secret SDF custom object.

The secret SDF custom object `secret` creates a new reference to a script `custsecret_example_id` that was created in a NetSuite account.

```
1 | <secret scriptid="custsecret_example_id"/>
```

Single Page Applications as XML Definitions

A single page application (SPA) is a web application or site that loads a single HTML page and dynamically updates that page when the user interacts with it.

In NetSuite, you can use SPAs to build a customized user interface. SPAs are created as part of SuiteApp projects created using the SuiteCloud Development Framework (SDF). SPAs are defined in SDF as a `singlepageapp` SDF custom object.

To see the list of SPAs deployed in your NetSuite account, go to Customization > Scripting > Single Page Applications.

For information about working with SPAs, see the following help topics:

- [Single Page Applications](#)
- [Components and Structure of Single Page Applications](#)
- [Single Page Application Creation And Development](#)
- [Single Page Application Management](#)

Take note of the following guidelines when creating the XML definition file for a `singlepageapp` SDF custom object:

- Each SPA implementation in a SuiteApp project must have its own SPA object definition XML file stored in the `Objects` folder.
- The specified value for `scriptid` in the XML definition must match the file name of the SPA object definition file.
- The paths specified in the object definition are relative to the `src/FileCabinet/` folder. Take note of the following:
 - The target location for the SPA folder must be inside `src/FileCabinet/SuiteApps/` `<ApplicationID>/` (where the transpiled files will be saved after running the build). In the XML definition, you do not need to include `src/FileCabinet` in the path.
 - The path specified for the SPA client script (`clientscriptfile`) and SPA server script (`serverscriptfile`) must point to a location inside the SPA folder.
 - The assets folder is optional. If you provided a folder path in the `assetsfolder` field of the SPA object definition, then you also need to have the assets folder in the SPA folder.

For information about the project structure for SuiteApps with SPAs, see the help topic [SuiteApp Project Structure for Single Page Applications](#).

- You can set up additional configurations for the SPA, such as the log level, execute as role, audience, and links.

For information about the attributes and fields in a `singlepageapp` SDF custom object, see the help topic [singlepageapp](#).

For a complete example showing all the supported fields, see [Single Page Application XML Definition Example](#).

The basic XML definition for an SPA object in a SuiteApp looks like the following:

```

1 <singlepageapp scriptid="custspa_helloworld">
2   <name>Hello World</name>
3   <description>This is a sample SuiteApp that uses NetSuite UIF components.</description>
4   <url>helloworld-suiteapp-spa</url>
5   <folder>[/SuiteApps/com.netsuite.spa/helloworld/]</folder>
6   <clientscriptfile>[/SuiteApps/com.netsuite.spa/helloworld/SpaClient.js]</clientscriptfile>
7   <serverscriptfile>[/SuiteApps/com.netsuite.spa/helloworld/SpaServer.js]</serverscriptfile>
8   <assetsfolder>[/SuiteApps/com.netsuite.spa/helloworld/assets/]</assetsfolder>
9 </singlepageapp>

```

For information about errors related to SPAs in SDF SuiteApp projects, see the help topic [Troubleshooting Single Page Applications](#).

Single Page Application XML Definition Example

The following shows a sample XML definition for a single page application (SPA) object in a SuiteApp project named **SPA Example** with the following information:

- The script ID is `custspa_appdashboard`, and the file name for the object definition file is `custspa_appdashboard.xml` (not shown in the sample).

- The `SpaClient.js` and `SpaServer.js` script files are located at: `/SuiteApps/com.example.spa/app-dashboard/` folder (under `src/FileCabinet`).
- The SPA object runs on the following URL: `https://<AccountID>.app.netsuite.com/spa-app/com.example.spa/app-dashboard`
- The SPA audience includes all internal roles (because `<audienceallroles>T</audienceallroles>` is set) and the two external roles in the `<audienceceroles>` list.
- The object definition includes all optional fields supported for the SPA object.

For information about SPA SDF custom objects in SuiteApp projects, see [Single Page Applications as XML Definitions](#) and [singlepageapp](#).

```

1 <singlepageapp scriptid="custspa_appdashboard">
2   <name>SPA Example</name>
3   <description>This is a sample SPA SuiteApp that uses NetSuite UIF.</description>
4   <url>app-dashboard</url>
5   <folder>[/SuiteApps/com.example.spa/app-dashboard/]</folder>
6   <clientscriptfile>[/SuiteApps/com.example.spa/app-dashboard/SpaClient.js]</clientscriptfile>
7   <serverscriptfile>[/SuiteApps/com.example.spa/app-dashboard/SpaServer.js]</serverscriptfile>
8   <assetsfolder>[/SuiteApps/com.example.spa/app-dashboard/assets/]</assetsfolder>
9   <loglevel>DEBUG</loglevel>
10  <audienceallroles>T</audienceallroles>
11  <audienceceroles>CUSTOMER_CENTER|EMPLOYEE_CENTER</audienceceroles>
12  <executeas></executeas>
13  <links>
14    <link>
15      <linkcategory>BASICSETUPCUSTOMIZATION</linkcategory>
16      <linklabel>App Dashboard</linklabel>
17    </link>
18  </links>
19  <notifyuser>F</notifyuser>
20  <notifyowner>T</notifyowner>
21  <notifyadmins>F</notifyadmins>
22  <notifyemails>testemail@test.com</notifyemails>
23 </singlepageapp>

```

Custom Tool Scripts as XML Definitions

Custom tool scripts let you build NetSuite tools that external AI clients can invoke through the NetSuite AI Connector Service. With these scripts, you can retrieve data, trigger actions, or perform most SuiteScript-supported operations from natural language prompts in your AI client.

Custom tools are implemented as part of SuiteCloud projects created using the SuiteCloud Development Framework (SDF). In SDF, custom tools are defined as a `tool` SDF custom object.

For information about developing custom tools for the NetSuite AI Connector Service, see the following help topics:

- [NetSuite AI Connector Service](#)
- [Creating Custom Tools for the NetSuite AI Connector Service](#)
- [SuiteScript 2.1 Custom Tool Script Type](#)

Take note of the following guidelines when creating the XML definition file for custom tool SDF objects:

- Each custom tool script in the SDF project must have a corresponding `tool` object definition XML file in the Objects folder.
- The specified value for `scriptid` in the XML definition must match the file name of the custom tool object definition file.
- The permissions set in the custom tool object definition determine the visibility of the tool in the AI client. If the user has all of the permissions defined for the tool, the tool will be visible to them.

However, access to data and the ability to run tool actions still depend on the user's NetSuite role and permissions. Users can only access data and perform operations that their NetSuite role allows, regardless of tool visibility.

For information about the attributes and fields in a tool SDF custom object, see the help topic [tool](#).

Example of a Custom Tool Script Object

This sample shows a custom tool object named Report Tool that references a script file named `report.js` and a JSON schema file named `report_schema.json`.

The object definition XML file is named `customtool_reports.xml`.

```

1 <tool scriptid="customtool_reports">
2   <name>Report Tool</name>
3   <scriptfile>[/SuiteApps/com.netsuite.tools/tools/report.js]</scriptfile>
4   <rpcschema>[/SuiteApps/com.netsuite.tools/tools/report_schema.json]</rpcschema>
5   <exposeto3rdpartyagents>T</exposeto3rdpartyagents>
6   <permissions>
7     <permission>
8       <permkey>LIST_EMAILTEMPLATE</permkey>
9       <permlevel>FULL</permlevel>
10    </permission>
11    <permission>
12      <permkey>ADMN_CRMLIST</permkey>
13      <permlevel>VIEW</permlevel>
14    </permission>
15  </permissions>
16 </tool>

```

Plug-ins

The following topics provide SuiteCloud Development Framework (SDF) information about the SDF custom objects that pertain to plug-ins:

- [Custom Plug-ins as XML Definitions](#)
- [Custom GL Lines Plug-ins as XML Definitions](#)
- [Email Capture Plug-ins as XML Definitions](#)
- [Financial Institution Connectivity Plug-ins as XML Definitions](#)
- [Financial Institution Parser Plug-ins as XML Definitions](#)
- [Bank Statement Parser Plug-ins as XML Definitions](#)
- [Dataset Builder Plug-ins as XML Definitions](#)
- [Workbook Builder Plug-ins as XML Definitions](#)

Custom Plug-ins as XML Definitions

You can use SuiteCloud Development Framework (SDF) to create custom plug-ins that customize functionality in NetSuite. For information about how custom plug-ins are used, see the help topic [Custom Plug-in Overview](#).

For information about working with custom plug-ins in SDF, see the following topics.

- [Creating a Custom Plug-in in an XML Definition](#)

- [Instantiating a Custom Plug-in Script in an XML Definition](#)
- [Validating a Custom Plug-in in an XML Definition](#)
- [Custom Plug-in Object Examples](#)

Creating a Custom Plug-in in an XML Definition

The following process describes creating a custom plug-in for use in SuiteCloud Development Framework (SDF):

1. Create a custom plug-in type. In SDF, that is represented by a `plugintype` object.
See [Creating a Custom Plug-in Type in an XML Definition](#).
2. Create alternate custom plug-in implementations for that type. In SDF, those are represented by `pluginimplementation` objects.
See [Creating a Custom Plug-in Implementation in an XML Definition](#).
3. Instantiate the custom plug-in in a SuiteScript script.
See the help topic [Instantiating a Custom Plug-in Script in SuiteScript 2.x](#).
4. Validate the SDF custom objects and then deploy them to a NetSuite account.
See [Validating a Custom Plug-in in an XML Definition](#) and [Deployment Logs for SuiteCloud Projects](#).
5. Set the active implementation in a NetSuite account.
See the help topic [Managing Custom Plug-in Implementations](#).

 **Tip:** To bundle the custom plug-in, see the help topic [Bundling a Custom Plug-in](#).

Creating a Custom Plug-in Type in an XML Definition

In SuiteCloud Development Framework (SDF), custom plug-in types are represented by `plugintype` objects.

To create a `plugintype` object, perform one of the following procedures:

Procedure	Details
In SuiteCloud IDE, create a custom plug-in implementation in a NetSuite account .	See the help topic Creating the Default Implementation for a Custom Plug-in .
In the NetSuite interface, create a custom plug-in type in a NetSuite account.	See the help topic Creating the Default Implementation for a Custom Plug-in . Import the <code>plugintype</code> object into your SuiteCloud project. See Account Component Imports to SuiteCloud Projects .
In the SuiteCloud Extension or in the SuiteCloud IDE, create a custom plug-in implementation using a template.	See the help topic Creating a SuiteScript File in SuiteCloud Extension for Visual Studio Code . See the help topic Creating an XML Definition for an SDF Custom Object in SuiteCloud IDE Plug-in for WebStorm .

Custom Plug-in Type Required Fields

The following table contains fields that are required for custom `plugintype` objects. For a list of all available fields, see the reference documentation for the `plugintype` element in .

Note: The `scriptid` attribute is required for the `plugintype` element. The `scriptid` attribute value must match the filename excluding the file extension.

For SuiteScript 1.0 custom plug-in types:

- The `class` field is required. The `class` field value represents the custom plugin. Format the class name in Pascal case (PascalCase). Use the class name to instantiate the implementation you want to use in your custom plug-in script. For more information, see the help topic [Instantiating a Custom Plug-in Script in SuiteScript 1.0](#).
- If you are using a library, use the `libraries` element to add library files that support your implementation.

Field	Type	Description
<code>deploymentmodel</code>	single-select list	The number of custom plug-in type implementations that an administrator can activate at one time. Possible values are: ■ <code>ALLOW_MULTIPLE</code> indicates two or more implementations of the interface can be activated at the same time within an account. ■ <code>ALLOW_SINGLE</code> indicates only one implementation of the interface can be activated at any time within an account.
<code>name</code>	string	A user-friendly name for the custom plug-in type. This name is seen by solution implementors when they create alternate implementations of the plug-in type. The name is also seen by administrators when they activate or deactivate implementations of this type.
<code>scriptfile</code>	filereference	The script file (.js) that contains the default implementation for the custom plug-in type.
<code>status</code>	single-select list	The status of the custom plug-in type. Possible values are <code>RELEASED</code> and <code>TESTING</code> . Ensure the value is set to <code>RELEASED</code> prior to bundling the custom plug-in. See the help topic Bundling a Custom Plug-in .

Custom Plug-in Type Methods

Specify the functions from the script file using the `methods` element.

Functions are specified using the following fields:

Field	Type	Description
<code>method</code>	string	The name of a function defined in the interface. Do not include parentheses or parameters.
<code>description</code>	string	A description for the method.

For an example of a `plugintype` object that contains methods, see [Custom Plug-in Type Object Example](#).

Creating a Custom Plug-in Implementation in an XML Definition

In SuiteCloud Development Framework (SDF), custom plug-in implementations are represented by `pluginimplementation` objects. A custom plug-in type (`plugintype`) can have multiple custom plug-in implementations. Custom plug-ins can also be referred to as alternate implementations of the custom plug-in type.

To create a pluginimplementation object, perform one of the following procedures.

Procedure	Details
In SuiteCloud IDE, create a custom plug-in implementation in a NetSuite account.	See the help topic Creating a Custom Plug-in Alternate Implementation .
In the NetSuite interface, create a custom plug-in implementation in a NetSuite account.	See the help topic Creating a Custom Plug-in Alternate Implementation . Import the pluginimplementation object into your SuiteCloud project. See Account Component Imports to SuiteCloud Projects .

Custom Plug-in Implementation Required Fields

The following table contains fields are required for custom pluginimplementation objects. For a list of all available fields, see the reference documentation for the pluginimplementation element in .

Note: The scriptid attribute is required for the pluginimplementation element. The scriptid attribute value must match the filename excluding the file extension.

For SuiteScript 1.0 custom plug-in implementations, the libraries element is available. Use that element to add library files that support your implementation.

Field	Type	Description
customplugintype	single-select list	The scriptid attribute value for the plugintype object that the pluginimplementation object is implementing. In other words, the ID for the custom plug-in type that the custom plug-in implementation is implementing. After the first time the custom plug-in implementation is deployed, its custom plug-in type cannot be changed.
name	string	A user-friendly name for the custom plug-in implementation. The name is seen by administrators when they activate or deactivate the custom plug-in implementation.
scriptfile	filereference	The script file (.js) that contains an alternate implementation for the custom plug-in type. The SuiteScript version of a custom plug-in implementation must match the SuiteScript version of its plug-in type. For example, a SuiteScript 2.x custom plug-in type must only have SuiteScript 2.x custom plug-in implementations.
status	single-select list	The status of the custom plug-in type. Possible values are RELEASED and TESTING. Ensure the value is set to RELEASED prior to bundling the custom plug-in. See the help topic Bundling a Custom Plug-in .

For an example of a pluginimplementation object, see [Custom Plug-in Implementation Object Example](#).

Instantiating a Custom Plug-in Script in an XML Definition

To instantiate a custom plug-in script implementation in your code in SuiteCloud Development Framework (SDF), use the plugin.loadImplementation function. For more information, see the help topic [Instantiating a Custom Plug-in Script in SuiteScript 2.x](#).

To specify the custom plug-in type in a `usereventscript` object, use the `customplugintypes` and `plugintype` fields. For example:

```

1 <usereventscript scriptid="customscript_docsign_suitescript_user_event">
2 ...
3 <customplugintypes>
4   <plugintype>
5     <plugintype[scriptid=customscript_doc_signing]</plugintype>
6   </plugintype>
7 </customplugintypes>
8 ...
9 </usereventscript>

```

For additional information, see the `usereventscript` element in .

Validating a Custom Plug-in in an XML Definition

SuiteCloud Development Framework (SDF) includes the following validations for custom plug-ins:

- The `scriptid` attribute is required for the `plugintype` and `pluginimplementation` elements. The `scriptid` attribute value must match the filename and exclude the file extension.
- For `plugintype` objects, each method field must contain a unique value.
- The `class` field is required for SuiteScript 1.0 custom plug-ins. The `class` field is not used for SuiteScript 2.x custom plug-ins. For information about SuiteScript versions, see the help topic [SuiteScript Versioning Guidelines](#).
- The SuiteScript version of a custom plug-in implementation must match the SuiteScript version of its plug-in type. For example, a SuiteScript 2.x custom plug-in type must only have SuiteScript 2.x custom plug-in implementations.
- After the first time the custom plug-in implementation is deployed, its custom plug-in type cannot be changed.

Custom Plug-in Object Examples

The following two examples are provided for custom plug-ins in SuiteCloud Development Framework (SDF):

- [Custom Plug-in Type Object Example](#)
- [Custom Plug-in Implementation Object Example](#)

Custom Plug-in Type Object Example

The following is an XML definition example of a custom plug-in type object in SuiteCloud Development Framework (SDF).

The object defines the custom plug-in type `customscript_doc_signing` that has the user-friendly name Document Signing. The object has a default implementation of the plug-in specified in `documentSign.js` using the methods `getSignature` and `addApproval`. Alternate implementations of those methods can be specified in a custom plug-in implementation object.

```

1 <plugintype scriptid="customscript_doc_signing">
2   <deploymentmodel>ALLOW_MULTIPLE</deploymentmodel>
3   <loglevel>DEBUG</loglevel>

```



```

4      <name>Document Signing</name>
5      <scriptfile>[/SuiteScripts/documentSign.js]</scriptfile>
6      <status>RELEASED</status>
7      <methods>
8          <method>
9              <method>getSignature</method>
10             <description>Gets the signature for the document.</description>
11         </method>
12         <method>
13             <method>addApproval</method>
14             <description>Adds the approval for the document.</description>
15         </method>
16     </methods>
17 </pluginintype>

```

Custom Plug-in Implementation Object Example

The following is an XML definition example of a custom plug-in implementation object in SDF.

The object defines an alternate implementation of the customscript_doc_signing custom plug-in type. The alternate implementation is customscript_doc_signing_implementation with a user-friendly name of Alternate Document Signing. The alternate implementation is defined in altDocumentSign.js.

```

1 <pluginimplementation scriptid="customscript_doc_signing_implementation">
2   <customplugintype>[scriptid=customscript_doc_signing]</customplugintype>
3   <isinactive>F</isinactive>
4   <loglevel>DEBUG</loglevel>
5   <name>Alternate Document Signing</name>
6   <notifyadmins>F</notifyadmins>
7   <notifyowner>T</notifyowner>
8   <notifyuser>F</notifyuser>
9   <scriptfile>[/SuiteScripts/altDocumentSign.js]</scriptfile>
10  <status>RELEASED</status>
11 </pluginimplementation>

```

Custom GL Lines Plug-ins as XML Definitions

You can create or import Custom GL Lines plug-in implementations using SuiteCloud Development Framework (SDF). You can create or import Custom GL Lines plug-in implementations in a SuiteCloud project to modify the general ledger impact of standard and custom transactions. The Custom GL Lines Plug-in enables you to comply with a wide range of global accounting standards by applying custom transaction logic that adds lines to the NetSuite GL Impact page. The Custom GL Lines Plug-in can set and read custom segment values from custom and standard lines, and from transaction records. The default values for column segments are sourced from the body. The Custom GL Lines Plug-in can change this value, even for segments that are applied only to the body.

For information about how Custom GL Lines plug-ins are used, see the help topic [Custom GL Lines Plug-in Overview](#).

You can create and manage a Custom GL Lines plug-in implementation in a SuiteCloud project and deploy it to a target NetSuite account. You can also import a Custom GL Lines plug-in implementation from a NetSuite account into a SuiteCloud project. For more information about importing objects, see [Account Component Imports to SuiteCloud Projects](#).

Customizing a Custom GL Lines plug-in implementation object from your SuiteCloud project is the equivalent of editing or making a new Custom GL Lines plug-in implementation from the NetSuite UI. For example, select Customization > Plug-ins > Plug-in Implementation > New from a NetSuite account.

The following is a screenshot of a Custom GL Lines plug-in implementation record in the NetSuite UI:

Custom GL Lines plug-in implementations in SDF are represented with the `<customglplugin>` object. To define this object, see the following sections:

- [Create a Plug-in Implementation Script File](#)
- [Custom GL Lines Plug-in Implementation Required Fields](#)
- [The Libraries \(Scripts\) Structure](#)

Create a Plug-in Implementation Script File

You need to have a Custom GL Lines plug-in implementation script either in a NetSuite account, or in the same SuiteCloud project as your XML object. For more information, see the help topics [Plug-in Implementation Script File Creation for SuiteScript 1.0](#) and [SuiteScript Files in SuiteCloud Projects](#).

Custom GL Lines Plug-in Implementation Required Fields

The following table contains elements that are required for `customglplugin` objects. For a list of all available elements, see the reference documentation for the `customglplugin` element in .

Element	Type	Description
customglplugin	single-select list	The element that defines the Custom GL Lines plug-in implementation object. This element requires a unique <code>scriptid</code> attribute value that defines the object.  Note: The <code>scriptid</code> attribute is required for the <code>customglplugin</code> element. The <code>scriptid</code> attribute value must match the filename excluding the file extension.
name	string	A user-friendly name for the Custom GL Lines plug-in implementation. The name is seen by administrators when they activate or deactivate the Custom GL Lines plug-in implementation.

Element	Type	Description
scriptfile	filereference	The script file (.js) that contains the implementation for the Custom GL Lines plug-in.
status	single-select list	The status of the custom plug-in type. Possible values are RELEASED and TESTING. Ensure the value is set to RELEASED prior to bundling the Custom GL Line plug-in. See the help topic Bundle the Plug-in Implementation .

The Libraries (Scripts) Structure

The <customglplugin> object contains a <libraries> structure to represent the script subrecord.

Element	Type	Description
libraries	structure	This structure contains all the utility script files or supporting library files that your plug-in implementation script file requires to run.
library	structure	This structure represents a single utility script file or supporting library file. You can specify multiple library structures.
scriptfile	filereference	This element takes a file path reference to a utility script file or supporting library file. For example: <pre> 1 <customglplugin scriptid="customscript_customglplugin"> 2 ... 3 <libraries> 4 <library> 5 <scriptfile>[/SuiteScripts/utilityScript.js]</scriptfile> 6 </library> 7 </libraries> 8 ... 9 </customglplugin> </pre> Files in the SuiteBundles and SuiteScripts folders of your File Cabinet can be selected as library files.

Custom GL Lines Plug-in Implementation Object Example

The following is an XML definition example of an Email Capture plug-in implementation object in a SuiteCloud project.

The Email Capture plug-in implementation object, customscript_emailcaptureplugin, creates a new plug-in that is in TESTING status. Debugging messages are logged when the script is run. The owner of the object is notified whenever an error is thrown. The customscript_emailCapturePlugin.js script is called when NetSuite receives an email sent to an email address associated with the Email Capture plug-in implementation.

```

1 <customglplugin scriptid="customscript_emailCapturePlugin">
2   <name>Custom GL Lines plug-in</name>
3   <description></description>
4   <status>TESTING</status>
5   <loglevel>DEBUG</loglevel>
6   <inactive>F</inactive>
7   <notifyadmins>F</notifyadmins>
8   <notifyemails></notifyemails>
9   <notifyowner>T</notifyowner>

```

```

10 <notifyuser>F</notifyuser>
11 <runasrole></runasrole>
12 <scriptfile>[/SuiteScripts/customscript_emailCapturePlugin.js]</scriptfile>
13 </customglplugin>

```

For more information, see `customglplugin` element in .

Email Capture Plug-ins as XML Definitions

You can create or import Email Capture plug-in implementations using SuiteCloud Development Framework (SDF). You can create or import Email Capture plug-in implementations in a SuiteCloud project to customize how NetSuite handles email logic. Use the Email Capture plug-in to define NetSuite behavior based on the properties and contents of an email message. When NetSuite receives an email sent to an email address associated with an implementation of the plug-in, NetSuite executes the business logic defined in the implementation. For example, you can use SuiteScript and an implementation of the Email Capture plug-in to process invoices, automate task creation and assignment, or automate escalations through email.

For information about how Email Capture plug-ins are used, see the help topic [Email Capture Plug-in Overview](#).

You can create and manage an Email Capture plug-in implementation in a SuiteCloud project and deploy it to a NetSuite account. You can also import an Email Capture plug-in implementation from a NetSuite account into a SuiteCloud project. For more information about importing objects, see [Account Component Imports to SuiteCloud Projects](#).

Customizing an Email Capture plug-in implementation object from your SuiteCloud project is the equivalent of editing or making a new Email Capture plug-in implementation from the NetSuite UI. For example, select **Customization > Plug-ins > Plug-in Implementation > New** from a NetSuite account.

The following is an image of an Email Capture plug-in implementation record in Edit mode in the NetSuite UI:

Plug-In Implementation ← → List Copy to Account (Beta) More

Actions

TYPE: Email Capture
 NAME: Email Capture
 ID: customscript_emailcapturetest
 API VERSION: 1.0

STATUS: Released
 LOG LEVEL: Debug
 EXECUTE AS ROLE: Current Role

DESCRIPTION:
 OWNER: Wolfe, K ☐ INACTIVE

Scripts | Unhandled Errors | Execution Log | System Notes

IMPLEMENTATION: email capture (test)

Libraries

LIBRARY SCRIPT FILE:

Actions

Email Capture plug-in implementations in SDF are represented with the <emailcaptureplugin> object. To define this object, see the following sections:

- [Create a Plug-in Implementation Script File](#)
- [Email Capture Plug-in Implementation Required Fields](#)
- [The Libraries \(Scripts\) Structure](#)

Create a Plug-in Implementation Script File

You need to have an Email Capture plug-in implementation script either in a NetSuite account, or in the same SuiteCloud project as your XML object. For more information, see the help topics [Create a Plug-in Implementation Script File](#) and [SuiteScript Files in SuiteCloud Projects](#).

Email Capture Plug-in Implementation Required Fields

The following table contains elements that are required for emailcaptureplugin objects. For a list of all available elements, see the reference documentation for the emailcaptureplugin element in .

Element	Type	Description
emailcaptureplugin	single-select list	The element that defines the Email Capture plug-in Implementation object. This element requires a unique scriptid attribute value that defines the object.  Note: The scriptid attribute is required for the emailcaptureplugin element. The scriptid attribute value must match the filename excluding the file extension.
name	string	A user-friendly name for the Email Capture plug-in implementation. The name is seen by administrators when they activate or deactivate the Email Capture plug-in implementation.
scriptfile	filereference	The script file (.js) that contains the implementation for the Email Capture plug-in.
status	single-select list	The status of the custom plug-in type. Possible values are RELEASED and TESTING. Ensure the value is set to RELEASED prior to bundling the Email Capture plug-in. See the help topic Bundle the Plug-in Implementation .

The Libraries (Scripts) Structure

The <emailcaptureplugin> object contains a <libraries> structure to represent the script subrecord.

Element	Type	Description
libraries	structure	This structure contains all the utility script files or supporting library files that your plug-in implementation script file requires to run.
library	structure	This structure represents a single utility script file or supporting library file. You can specify multiple library structures.

Element	Type	Description
scriptfile	filereference	This element takes a file path reference to a utility script file or supporting library file. For example: <pre> 1 <emailcaptureplugin scriptid="customscript_emailcaptureplugin"> 2 ... 3 <libraries> 4 <library> 5 <scriptfile>[/SuiteScripts/utilityScript.js]</scriptfile> 6 </library> 7 </libraries> 8 ... 9 </emailcaptureplugin> </pre> Files in the SuiteBundles and SuiteScripts folders of your File Cabinet can be selected as library files.

Email Capture Plug-in Implementation Object Example

The following is an XML definition example of an Email Capture plug-in implementation object in a SuiteCloud project.

The Email Capture plug-in implementation object, `customscript_emailcaptureplugin`, creates a new plug-in that is in TESTING status. Debugging messages are logged when the script is run. The owner of the object is notified whenever an error is thrown. The `customscript_emailCapturePlugin.js` script is called when NetSuite receives an email sent to an email address associated with the Email Capture plug-in implementation.

```

1 <emailcaptureplugin scriptid="customscript_emailCapturePlugin">
2   <name>Email Capture plug-in</name>
3   <description></description>
4   <status>TESTING</status>
5   <loglevel>DEBUG</loglevel>
6   <isinactive>F</isinactive>
7   <notifyadmins>F</notifyadmins>
8   <notifyemails></notifyemails>
9   <notifyowner>T</notifyowner>
10  <notifyuser>F</notifyuser>
11  <runasrole></runasrole>
12  <scriptfile>[/SuiteScripts/customscript_emailCapturePlugin.js]</scriptfile>
13 </emailcaptureplugin>

```

For more information, see `emailcaptureplugin` element in .

Financial Institution Connectivity Plug-ins as XML Definitions

Note: Financial Institution Connectivity Plug-in objects in SuiteCloud Development Framework (SDF) only support SuiteScript 2.0. SuiteScript 2.1 is not supported.

You can create and manage a Financial Institution Connectivity Plug-in implementation in a SuiteCloud project and deploy it to a target NetSuite account. The Financial Institution Connectivity Plug-in enables you to directly connect to financial institutions and automatically receive bank data imports, daily. For information about how these plug-ins are used, see the help topic [Financial Institution Connectivity Plug-in Interface Overview](#).

You can also import a Financial Institution Connectivity Plug-in implementation from a NetSuite account into a SuiteCloud project. For more information about importing objects, see [Account Component Imports to SuiteCloud Projects](#).

Customizing a plug-in implementation object from your SuiteCloud project is the equivalent of editing or making a new plug-in implementation from the NetSuite UI. For example, select **Customization > Plug-ins > Plug-in Implementation > New** from a NetSuite account.

To create a Financial Institution Connectivity Plug-in implementation in a SuiteCloud project, you must create a plug-in implementation script file. For details, see [Create a Financial Institution Connectivity Plug-in Implementation Script File](#).

To run a Financial Institution Connectivity Plug-in, you must configure and save a format profile record in the NetSuite account that uses the plug-in implementation. Invoking the plug-in is only possible in the NetSuite UI, since SDF does not support format profile records.

For details on running the plug-in, see the help topic [Conditions for Running a Financial Institution Connectivity Plug-in](#). For information about format profile setup, see the help topic [Creating Format Profiles for Bank Reconciliation](#).



Note: If you have the administrator role, you can uninstall an SDF SuiteApp containing a Financial Institution Connectivity Plug-in implementation that is already used in a format profile. This is true even if the format profile has been used to import data. After you uninstall the SDF SuiteApp, NetSuite does the following:

- Removes the plug-in reference from the format profile. The **Connectivity Method** field in the format profile is changed to Manual Import.
- Inactivates the format profile. You can reactivate the format profile if it contains a parser plug-in, but the record will be configured for manual import unless you change the connectivity method.



Important: Format profiles configured to import expense data do not currently support manual import. Before reactivating a Corporate Card Expenses format profile, ensure that you configure the record for automatic import. See the help topic [Creating Format Profiles for Expense Reporting](#).

If the format profile does not contain a parser plug-in, you must select one before you can reactivate the format profile. See the help topic [Modifying Format Profiles](#). If you no longer need the format profile, you should delete it. See the help topic [Deleting a Format Profile](#).

Financial Institution Connectivity Plug-in implementations in SDF are represented with the `ficonnectivityplugin` object. To define this SDF custom object, see the following sections:

- [Create a Financial Institution Connectivity Plug-in Implementation Script File](#)
- [Financial Institution Connectivity Plug-in Implementation Required Fields](#)

Create a Financial Institution Connectivity Plug-in Implementation Script File

You need to have a Financial Institution Connectivity Plug-in implementation script either in the target account, or in the same SuiteCloud project as your SDF custom object. For more information, see the help topics [Creating a Financial Institution Connectivity Plug-in Script File](#) and [SuiteScript Files in SuiteCloud Projects](#).

Financial Institution Connectivity Plug-in Implementation Required Fields

The following table contains fields that are required for `ficonnectivityplugin` objects. For a list of all available fields, see the help topic [ficonnectivityplugin](#).

Element	Type	Description
name	string	A user-friendly name for the Financial Institution Connectivity Plug-in implementation. The name is seen by administrators when they activate or deactivate the Financial Institution Connectivity Plug-in implementation. This field value can be up to 40 characters long.
scriptfile	filereference	The script file (.js) that contains the implementation for the Financial Institution Connectivity Plug-in. This field must reference a .js file.

Financial Institution Connectivity Plug-in Implementation Object Example

The following is an XML definition example of a Financial Institution Connectivity Plug-in implementation object in a SuiteCloud project.

The Financial Institution Connectivity Plug-in implementation object, `customscript1`, creates a new plug-in that is in TESTING status. Debugging messages are logged when the script is run. The owner of the object is notified whenever an error is thrown. The `customscript1.js` script is called when NetSuite receives a request to retrieve bank data from a financial institution.

Note: Each `ficonnectivityplugin` SDF custom object requires a `scriptfile` field reference to a SuiteScript file that implements interface functions. For more information, see the help topic [Creating a Financial Institution Connectivity Plug-in Script File](#). This script file can be in the target account or in the same SuiteCloud project as your plug-in implementation object.

```

1 <ficonnectivityplugin scriptid="customscript1">
2   <description></description>
3   <isinactive>F</isinactive>
4   <loglevel>DEBUG</loglevel>
5   <name>Your connectivity</name>
6   <notifyadmins>F</notifyadmins>
7   <notifyemails></notifyemails>
8   <notifyowner>T</notifyowner>
9   <notifyuser>F</notifyuser>
10  <runasrole></runasrole>
11  <scriptfile>[/SuiteApps/com.netsuite.yourapp/src/parser/your_connectivity.js]</scriptfile>
12  <status>TESTING</status>
13 </ficonnectivityplugin>

```

For more information about the fields in the preceding example, see the help topic [ficonnectivityplugin](#).

Financial Institution Parser Plug-ins as XML Definitions

Note: Financial Institution Parser Plug-in objects in SuiteCloud Development Framework (SDF) only support SuiteScript 2.0. SuiteScript 2.1 is not supported.

You can create and manage a Financial Institution Parser Plug-in implementation in a SuiteCloud project and deploy it to a target NetSuite account. The Financial Institution Parser Plug-in interface enables you to

create and upload your own parsers to parse financial data for different purposes. The plug-in interface lets you translate data received from a third party into data that can be used in the bank reconciliation or expense report process.

This plug-in interface is intended to replace the Bank Statement Parser Plug-in interface, by providing a more general platform for parsing financial data. As of NetSuite 2020.2, you can parse and retrieve the following:

- **Bank and credit card data for matching and reconciliation** – Automatically or manually import data from financial institutions into NetSuite for use in the downstream matching and reconciliation feature.
- **Corporate card charges for expense reporting** – Automatically import data from corporate credit card vendors such as American Express into NetSuite for use in the downstream expense reporting feature.

For information about using Financial Institution Parser Plug-ins, see the help topic [Financial Institution Parser Plug-in Interface Overview](#).

You can also import a Financial Institution Parser Plug-in implementation from a NetSuite account into a SuiteCloud project. For details about importing objects, see [Account Component Imports to SuiteCloud Projects](#).

Customizing a plug-in implementation object from your SuiteCloud project is the equivalent of editing or making a new plug-in implementation from the NetSuite UI. For example, select Customization > Plug-ins > Plug-in Implementation > New from a NetSuite account.

To create a Financial Institution Parser Plug-in implementation in a SuiteCloud project, you must create a plug-in implementation script file. For details, see [Create a Financial Institution Parser Plug-in Implementation Script File](#).

To run a Financial Institution Parser Plug-in, you must configure and save a format profile in the NetSuite account that uses the plug-in implementation. Invoking a Financial Institution Parser Plug-in is only possible in the NetSuite UI, since SDF does not support format profile records.

For details on running this plug-in, see the help topic [Conditions for Running a Financial Institution Parser Plug-in](#). For information about format profile setup, see the help topic [Creating Format Profiles for Bank Reconciliation](#).

Note: If you have the administrator role, you can uninstall an SDF SuiteApp containing a Financial Institution Parser Plug-in implementation that is already used in a format profile. This is true even if the format profile has been used to import data. After you uninstall the SDF SuiteApp, NetSuite does the following:

- Removes the plug-in reference from the format profile. The **Transaction Parser** field in the format profile is cleared.
- Inactivates the format profile. To reactivate the format profile, you first need to select a new parser plug-in. See the help topic [Modifying Format Profiles](#). If you no longer need the format profile, you should delete it. See the help topic [Deleting a Format Profile](#).

Financial Institution Parser Plug-in implementations in SDF are represented with the `fiparserplugin` object. To define this object, see the following sections:

- [Create a Financial Institution Parser Plug-in Implementation Script File](#)
- [Financial Institution Parser Plug-in Implementation Required Fields](#)

Create a Financial Institution Parser Plug-in Implementation Script File

You need to have a Financial Institution Parser Plug-in implementation script either in the target account, or in the same SuiteCloud project as your XML object. For more information, see the help topics [Creating a Financial Institution Parser Plug-in Script File](#) and [SuiteScript Files in SuiteCloud Projects](#).

Financial Institution Parser Plug-in Implementation Required Fields

The following table contains fields that are required for `fiparserplugin` objects. For a list of all available fields, see the help topic [fiparserplugin](#).

Element	Type	Description
name	string	A user-friendly name for the Financial Institution Parser Plug-in implementation. The name is seen by administrators when they activate or deactivate the Financial Institution Parser Plug-in implementation. This field value can be up to 40 characters long.
scriptfile	filereference	The script file (.js) that contains the implementation for the Financial Institution Parser Plug-in. This field must reference a .js file.

Financial Institution Parser Plug-in Implementation Object Example

The following is an XML definition example of a Financial Institution Parser Plug-in implementation object in a SuiteCloud project.

The Financial Institution Parser Plug-in implementation object, `customscript_fiprol`, creates a new plug-in named Financial Institution Parser 1 that references the SuiteScript file `connectionprofile1.js`. This plug-in is in TESTING status and debugging messages are logged when the SuiteScript file is run. The owner of the object is notified whenever an error is thrown. The `customscript_fiprol.js` script is called when NetSuite receives a request to parse financial data.

Note: Each `fiparserplugin` SDF custom object requires a `scriptfile` field reference to a SuiteScript file that implements interface functions. For more information, see the help topic [Creating a Financial Institution Parser Plug-in Script File](#). This script file can be in the target account or in the same SuiteCloud project as your plug-in implementation object.

```

1 <fiparserplugin scriptid="customscript_fiprol">
2   <description></description>
3   <isinactive>F</isinactive>
4   <loglevel>DEBUG</loglevel>
5   <name>Financial Institution Parser 1</name>
6   <notifyadmins>F</notifyadmins>
7   <notifyemails></notifyemails>
8   <notifyowner>T</notifyowner>
9   <notifyuser>F</notifyuser>
10  <runasrole></runasrole>
11  <scriptfile>[/SuiteApps/connectionprofile1.js]</scriptfile>
12  <status>TESTING</status>
13 </fiparserplugin>

```

For more information about the fields in the preceding example, see the help topic [fiparserplugin](#).

Bank Statement Parser Plug-ins as XML Definitions

Note: Bank Statement Parser Plug-in objects in SuiteCloud Development Framework (SDF) only support SuiteScript 2.0.

You can create and manage a Bank Statement Parser Plug-in implementation in a SuiteCloud project and deploy it to a target NetSuite account. The Bank Statement Parser Plug-in enables you to create and upload your own parsers for different bank and credit card statement formats and store them in NetSuite. The plug-in translates the information from unsupported file formats into account statements or transactions that NetSuite can understand and use. This enables you to import other file formats and then use downstream matching and reconciliation features.

For information about how these plug-ins are used, see the help topic [Bank Statement Parser Plug-in Interface Overview](#).

You can also import a Bank Statement Parser Plug-in implementation from a NetSuite account into a SuiteCloud project. For more information about importing objects, see [Account Component Imports to SuiteCloud Projects](#).

Customizing a plug-in implementation object from your SuiteCloud project is the equivalent of editing or making a new plug-in implementation from the NetSuite UI. For example, select **Customization > Plug-ins > Plug-in Implementation > New** from a NetSuite account.

To create a Bank Statement Parser Plug-in implementation in a SuiteCloud project, you must create a plug-in implementation script file. For details, see [Create a Bank Statement Parser Plug-in Implementation Script File](#).

To run a Bank Statement Parser Plug-in, you must do the following:

1. Configure a format profile record that uses a Bank Statement Parser Plug-in implementation. See the help topic [Creating Format Profiles for Bank Reconciliation](#).
2. Import your bank data into the NetSuite account. See the help topic [Bank Data Import](#).

Note: You can only invoke the Bank Statement Parser Plug-in from the NetSuite UI.

Bank Statement Parser Plug-in implementations in SDF are represented with the `bankstatementparserplugin` object. To define this object, see the following sections:

- [Create a Bank Statement Parser Plug-in Implementation Script File](#)
- [Bank Statement Parser Plug-in Implementation Required Fields](#)

Create a Bank Statement Parser Plug-in Implementation Script File

You need to have a Bank Statement Parser Plug-in implementation script either in the target account, or in the same SuiteCloud project as your XML object. For more information, see the help topics [Creating a Bank Statement Parser Plug-in Script File](#) and [SuiteScript Files in SuiteCloud Projects](#).

Bank Statement Parser Plug-in Implementation Required Fields

The following table contains fields that are required for `bankstatementparserplugin` objects. For a list of all available fields, see the help topic [bankstatementparserplugin](#).

Element	Type	Description
name	string	A user-friendly name for the Bank Statement Parser Plug-in implementation. The name is seen by administrators when they activate or deactivate the Bank Statement Parser Plug-in implementation. This field value can be up to 40 characters long.
scriptfile	filereference	The script file (.js) that contains the implementation for the Bank Statement Parser Plug-in. This field must reference a .js file.

Bank Statement Parser Plug-in Implementation Object Example

The following is an XML definition example of a Bank Statement Parser Plug-in implementation object in a SuiteCloud project.

The Bank Statement Parser Plug-in implementation object, `customscript1`, creates a new plug-in that is in TESTING status. Debugging messages are logged when the script is run. The owner of the object is notified whenever an error is thrown. The `customscript1.js` script is called when NetSuite receives a request to parse bank data.

Note: Each `bankstatementparserplugin` SDF custom object requires a `scriptfile` field reference to a SuiteScript file that implements interface functions. For more information, see the help topic [Creating a Bank Statement Parser Plug-in Script File](#). This script file can be in the target account or in the same SuiteCloud project as your plug-in implementation object.

```

1 <bankstatementparserplugin scriptid="customscript1">
2   <description></description>
3   <isinactive>F</isinactive>
4   <loglevel>DEBUG</loglevel>
5   <name>Your Parser</name>
6   <notifyadmins>F</notifyadmins>
7   <notifyemails></notifyemails>
8   <notifyowner>T</notifyowner>
9   <notifyuser>F</notifyuser>
10  <runasrole></runasrole>
11  <scriptfile>[/SuiteApps/com.netsuite.yourapp/src/parser/your_parser.js]</scriptfile>
12  <status>TESTING</status>
13 </bankstatementparserplugin>

```

For more information about the fields in the preceding example, see the help topic [bankstatementparserplugin](#).

Dataset Builder Plug-ins as XML Definitions

Note: Dataset Builder Plug-in objects in SuiteCloud Development Framework (SDF) only support SuiteScript 2.x.

You can create and manage a Dataset Builder Plug-in implementation in a SuiteCloud project and deploy it to a target NetSuite account. The Dataset Builder Plug-in enables you to create custom datasets that fit your business needs. You can override the `createDataset()` function and create a dataset that contains a specific subset of data or a dataset that uses custom logic. For example, you can create a dataset

based on user-specific or role-specific information. The default behavior in the N/dataset module does not permit you to do this, which makes the Dataset Builder Plug-in a good choice for these use cases. Datasets you create using the N/dataset module are static and cannot use dynamic information such as the current role, current user, and so on.

For information about using Dataset Builder Plug-ins, see the help topic [Dataset Builder Plug-in Interface Overview](#).

You can also import a Dataset Builder Plug-in implementation from a NetSuite account into a SuiteCloud project. For more information about importing objects, see [Account Component Imports to SuiteCloud Projects](#).

Customizing a plug-in implementation object from your SuiteCloud project is the equivalent of editing or making a new plug-in implementation from the NetSuite UI. For example, select Customization > Plug-ins > Plug-in Implementation > New from a NetSuite account.

To create a Dataset Builder Plug-in implementation in a SuiteCloud project, you must create a plug-in implementation script file. For details, see the help topic [Creating a Dataset Builder Plug-in Script File](#).

Dataset Builder Plug-in implementations in SDF are represented with the `datasetbuilderplugin` object. To define this object, see the following sections:

- [Create a Dataset Builder Plug-in Implementation Script File](#)
- [Dataset Builder Plug-in Implementation Required Fields](#)

Create a Dataset Builder Plug-in Implementation Script File

You need to have a Dataset Builder Plug-in implementation script either in the target account, or in the same SuiteCloud project as your XML object. For more information, see the help topics [Creating a Dataset Builder Plug-in Script File](#) and [SuiteScript Files in SuiteCloud Projects](#).

Dataset Builder Plug-in Implementation Required Fields

The following table contains fields that are required for `datasetbuilderplugin` objects. For a list of all available fields, see the help topic [datasetbuilderplugin](#).

Element	Type	Description
name	string	A user-friendly name for the Dataset Builder Plug-in implementation. The name is seen by administrators when they activate or deactivate the Dataset Builder Plug-in implementation. This field value can be up to 40 characters long.
scriptfile	filereference	The script file (.js) that contains the implementation for the Dataset Builder Plug-in. This field must reference a .js file.

Dataset Builder Plug-in Implementation Object Example

The following is an XML definition example of a Dataset Builder Plug-in implementation object in a SuiteCloud project.

The Dataset Builder Plug-in implementation object, `customscript_dataset`, creates a new plug-in that is in `TESTING` status. Debugging messages are logged when the script is run. The owner of the object is

notified whenever an error is thrown. The `datasetbuilder.js` script is called when NetSuite receives a request to create a custom dataset.

Note: Each `datasetbuilderplugin` SDF custom object requires a `scriptfile` field reference to a SuiteScript file that implements interface functions. For more information, see the help topic [Creating a Dataset Builder Plug-in Script File](#). This script file can be in the target account or in the same SuiteCloud project as your plug-in implementation object.

```

1 <datasetbuilderplugin scriptid="customscript_dataset">
2   <description></description>
3   <isinactive>F</isinactive>
4   <loglevel>DEBUG</loglevel>
5   <name>My Dataset Builder</name>
6   <notifyadmins>F</notifyadmins>
7   <notifyemails/>
8   <notifyowner>T</notifyowner>
9   <notifyuser>F</notifyuser>
10  <runasrole/>
11  <scriptfile>[/SuiteApps/datasetbuilder.js]</scriptfile>
12  <status>TESTING</status>
13 </datasetbuilderplugin>

```

For more information about the fields in the preceding example, see the help topic [datasetbuilderplugin](#).

Workbook Builder Plug-ins as XML Definitions

Note: Workbook Builder Plug-in objects in SuiteCloud Development Framework (SDF) only support SuiteScript 2.0 and SuiteScript 2.1.

You can create and manage a Workbook Builder Plug-in implementation in a SuiteCloud project and deploy it to a target NetSuite account. The Workbook Builder Plug-in enables you to create custom workbooks that fit your business needs. You can override the `createWorkbook()` function and create a workbook that contains a specific subset of data or a workbook that uses custom logic. For example, you can create a workbook based on user-specific or role-specific information. The default behavior in the N/workbook module does not permit you to do this, which makes the Workbook Builder Plug-in a good choice for these use cases. Workbooks you create using the N/workbook module are static and cannot use dynamic information such as the current role, current user, and so on.

For information about using Workbook Builder Plug-ins, see the help topic [Workbook Builder Plug-in Interface Overview](#).

You can also import a Workbook Builder Plug-in implementation from a NetSuite account into a SuiteCloud project. For more information about importing objects, see [Account Component Imports to SuiteCloud Projects](#).

Customizing a plug-in implementation object from your SuiteCloud project is the equivalent of editing or making a new plug-in implementation from the NetSuite UI. For example, select **Customization > Plug-ins > Plug-in Implementation > New** from a NetSuite account.

To create a Workbook Builder Plug-in implementation in a SuiteCloud project, you must create a plug-in implementation script file. For details, see the help topic [Creating a Workbook Builder Plug-in Script File](#).

Workbook Builder Plug-in implementations in SDF are represented with the `workbookbuilderplugin` object. To define this object, see the following sections:

- [Create a Workbook Builder Plug-in Implementation Script File](#)

■ Workbook Builder Plug-in Implementation Required Fields

Create a Workbook Builder Plug-in Implementation Script File

You need to have a Workbook Builder Plug-in implementation script either in the target account, or in the same SuiteCloud project as your XML object. For more information, see the help topics [Creating a Workbook Builder Plug-in Script File](#) and [SuiteScript Files in SuiteCloud Projects](#).

Workbook Builder Plug-in Implementation Required Fields

The following table contains fields that are required for workbookbuilderplugin objects. For a list of all available fields, see the help topic [workbookbuilderplugin](#).

Element	Type	Description
name	string	A user-friendly name for the Workbook Builder Plug-in implementation. The name is seen by administrators when they activate or deactivate the Workbook Builder Plug-in implementation. This field value can be up to 40 characters long.
scriptfile	filereference	The script file (.js) that contains the implementation for the Workbook Builder Plug-in. This field must reference a .js file.

Workbook Builder Plug-in Implementation Object Example

The following is an XML definition example of a Workbook Builder Plug-in implementation object in a SuiteCloud project.

The Workbook Builder Plug-in implementation object, `customscript_workbook`, creates a new plug-in that is in TESTING status. Debugging messages are logged when the script is run. The owner of the object is notified whenever an error is thrown. The `workbookbuilder.js` script is called when NetSuite receives a request to create a custom workbook.



Note: Each workbookbuilderplugin SDF custom object requires a scriptfile field reference to a SuiteScript file that implements interface functions. For more information, see the help topic [Creating a Workbook Builder Plug-in Script File](#). This script file can be in the target account or in the same SuiteCloud project as your plug-in implementation object.

```

1 <workbookbuilderplugin scriptid="customscript_workbook">
2   <description></description>
3   <isinactive>F</isinactive>
4   <loglevel>DEBUG</loglevel>
5   <name>My Workbook Builder</name>
6   <notifyadmins>F</notifyadmins>
7   <notifyemails/>
8   <notifyowner>T</notifyowner>
9   <notifyuser>F</notifyuser>
10  <runasrole/>
11  <scriptfile>[/SuiteApps/workbookbuilder.js]</scriptfile>
12  <status>TESTING</status>
13 </workbookbuilderplugin>

```

For more information about the fields in the preceding example, see the help topic [workbookbuilderplugin](#).

Centers and Tabs

The following topics provide SuiteCloud Development Framework (SDF) information about SDF custom objects that pertain to centers and tabs:

- [KPI Scorecards as XML Definitions](#)
- [KPI Comparisons in KPI Scorecards](#)
- [KPI Formulas in KPI Scorecards](#)
- [Date Ranges and Periods in KPI Scorecards](#)
- [Highlighting in KPI Scorecards](#)
- [KPI Scorecards XML Definition Example](#)

Centers as XML Definitions

You can create and manage custom centers in a NetSuite account by creating a center object in a SuiteCloud project and deploying the project to a target account. Additionally, you can import centers from a NetSuite account into a SuiteCloud project.

The following example shows a custom center object with a defined label:

```
1 <center scriptid="custcenter_custom">
2   <label>My Custom Center</label>
3 </center>
```

In addition to centers, you can create and manage the following objects in SDF:

- **Center categories** — You can add custom categories to standard, built-in NetSuite tabs or custom tabs. For more information, see [Center Categories as XML Definitions](#).
- **Center tabs** — You can designate which portlets are available to the center, and which roles have access to the center. For more information, see [Center Tabs as XML Definitions](#).

For more information about centers, see:

- [Custom Centers](#)
- [center](#)

Center Categories as XML Definitions

You can create and manage custom center categories in a NetSuite account by creating a centercategory object in a SuiteCloud project and deploying the project to a target account. Additionally, you can import center categories from a NetSuite account into a SuiteCloud project.

You can add custom categories to either standard, built-in NetSuite tabs or custom tabs. After creating custom categories for standard tabs or custom tabs, you can then add links to each category. Center links can take users to NetSuite pages, custom records, Suitelets, or external websites. For more information about center links, see the help topics [Creating Center Links](#), [centerlink](#), and [link](#).

The following example shows a custom centercategory object that makes the custcentercategory_custom center category available in the custcentertab_custom center tab. The LIST_ACTIVITY link appears on the custcentercategory_custom center category and takes users to the Activity List page within NetSuite:

```
1 <centercategory scriptid="custcentercategory_custom">
2   <center>[scriptid=custcenter_custom]</center>
3   <centertab>[scriptid=custcentertab_custom]</centertab>
4   <label>My Custom Center Category</label>
5   <links>
```



```

6      <link>
7          <linkid>LIST_ACTIVITY</linkid>
8          <linklabel>Activity List</linklabel>
9          <shortlist>F</shortlist>
10     </link>
11 </links>
12 </centercategory>

```

In addition to center categories, you can create and manage the following objects in SDF:

- **Centers** — You can create custom centers that you apply to custom roles. For more information, see [Centers as XML Definitions](#).
- **Center tabs** — You can designate which portlets are available to the center, and which roles have access to the center. For more information, see [Center Tabs as XML Definitions](#).

Known Limitations of Custom Center Translations in SDF

The following are known limitations of custom center translations in SDF:

- If the Label field for the custom center link is left blank for the default company language, validation fails in the SDF XML definition. In the SDF XML definition, if the field is blank, NetSuite cannot insert the core string translation of the object referenced by the Custom Center link. For centerlink SDF custom object fields, see the help topic [centerlink](#).
- When installing or deploying a SuiteCloud project or SuiteApp on the target account, the XML definition uses the translation of the company default language. If the XML definition does not contain a translation for the company default language, the XML definition uses the default company language defined in the translation collection XML definition. The language setting of the person performing the import is not used.

For more information about center categories, see:

- [Creating Center Categories](#)
- [centercategory](#)

Center Tabs as XML Definitions

Create and manage custom center tabs in a NetSuite account by creating a centertab object in a SuiteCloud project and deploying the project to a target account. Additionally, you can import center tabs from a NetSuite account into a SuiteCloud project.

Use a center tab to associate centers with roles. Additionally, you can designate portlets that should appear on a dashboard when users click the tab.

Note: Custom center tabs always include five trend graph portlets. These portlets can be accessed by any dashboard created for the custom center tab. Even though these portlets are not represented in the custom center tab XML definition, NetSuite includes them as part of the custom center tab object when it is deployed to a target account.

The following example shows a custom centertab object that makes the custcenter_mycenter center available to all roles:

```

1 <centertab scriptid="custcentertab_custom">
2   <allroles>T</allroles>
3   <center>[scriptid=custcenter_mycenter]</center>
4   <label></label>
5   <portlets>
6     <portlet scriptid="">
7       <portlet></portlet>
8       <portletcolumn>1</portletcolumn>

```

```

9      <isportletshown>T</isportletshown>
10    </portlet>
11  </portlets>
12 </centertab>

```

In addition to center tabs, you can create and manage the following objects in SDF:

- **Centers** — You can create custom centers that you apply to custom roles. For more information, see [Centers as XML Definitions](#).
- **Center categories** — You can add custom categories to standard, built-in NetSuite tabs or custom tabs. For more information, see [Center Categories as XML Definitions](#).

For more information about center tabs, see:

- [Creating Center Tabs](#)
- [centertab](#)

KPI Scorecards as XML Definitions

KPI scorecards compare complex relationships among multiple KPIs over multiple date ranges or accounting periods. Use KPI scorecards to store your selection of performance indicators, mathematical formulas, time periods, and date ranges. Each KPI scorecard object supports several standard KPIs and up to ten custom KPIs that are based on saved searches. For more information about KPI scorecards, see the following:

- [Key Performance Indicators](#)
- [KPI Scorecards](#)

Create and manage KPI scorecards using SuiteCloud Development Framework (SDF). You can model a KPI scorecard definition in a SuiteCloud project and deploy the project to a target NetSuite account. For more information about working with KPI scorecards in a NetSuite account, see the following:

- [Enabling the KPI Scorecards Feature](#)
- [Creating a KPI Scorecard](#)
- [Account Component Imports to SuiteCloud Projects](#)

KPI scorecards are referenced by the `kpiscorecard` root element in SDF. The following is a sample KPI scorecard object that contains some basic information:

```

1 <kpiscorecard scriptid="custkpiscorecard_sample">
2   <name>Sample KPI Scorecard</name>
3   <useperiods>F</useperiods>
4   <description>This is a sample KPI scorecard.</description>
5   <custom>
6     <kpi1>SAVEDSEARCH1</kpi1>
7     <kpi2>SAVEDSEARCH2</kpi2>
8     <kpi3>SAVEDSEARCH3</kpi3>
9     <kpi4>SAVEDSEARCH4</kpi4>
10    <kpi5>SAVEDSEARCH5</kpi5>
11    <kpi6>SAVEDSEARCH6</kpi6>
12    <kpi7>SAVEDSEARCH7</kpi7>
13    <kpi8>SAVEDSEARCH8</kpi8>
14    <kpi9>SAVEDSEARCH9</kpi9>
15    <kpi10>SAVEDSEARCH10</kpi10>
16   </custom>
17   <audience>
18     <allroles>T</allroles>
19     <allpartners>F</allpartners>
20     <allemployees>F</allemployees>
21   </audience>
22   ...
23 </kpiscorecard>

```

Use the `useperiods` Boolean element to determine whether the KPI scorecard should use periods instead of date ranges. When periods are used, accounting periods should be set up in a NetSuite account. You should set the **Report by Period** preference to **All Reports** on the **Analytics** subtab of the Set Preferences page.

The custom structure is required when using custom KPIs in your formulas and comparisons. Each element should reference the saved search ID that is used as the basis for each custom KPI. Each element is labeled from **kpi1** through **kpi10**, supporting up to ten saved search references. A KPI scorecard object supports up to ten custom KPIs.

The audience structure is optional. Use this structure to define which roles can display the KPI scorecard in their KPI scorecard portlets.

In addition to these information fields, KPI scorecards can have the following structures:

■ **kpis**

The `kpis` structure encapsulates all the KPIs that you want to compare. Alternatively, you can use this structure to set up formula expressions. For more information about setting up the `kpis` structure, see the following:

- [KPI Comparisons in KPI Scorecards](#)
- [KPI Formulas in KPI Scorecards](#)

■ **ranges**

The `ranges` structure defines multiple date ranges or periods that apply to all scorecard data. For more information, see [Date Ranges and Periods in KPI Scorecards](#).

■ **highlightings**

The `highlightings` structure sets highlighting options to make results more visible in the KPI scorecard portlet. For more information, see [Highlighting in KPI Scorecards](#).

After deploying a KPI scorecard object from a SuiteCloud project to the target NetSuite account, you can reference it from a published dashboard through NetSuite. For more information, see the help topic [Adding a KPI Scorecard Portlet to a Dashboard](#).



Important: KPI scorecard objects cannot be referenced or configured for use in any published dashboard objects that are managed through a SuiteCloud project.

For more information about the KPI scorecard object, see the help topic [kpiscorecard](#).

Example

The following is an example of a KPI scorecard object. This scorecard creates a matrix involving two key performance indicators, COGS vs Bank Balance and Activities vs Previous, and two ranges, THIS FISCAL HALF and LAST YEAR VS. THIS FISCAL YEAR.

```

1 <kpiscorecard scriptid="custkpiscorecard_sample">
2   <name>Sample KPI</name>
3   <useperiods>F</useperiods>
4   <kpis>
5     <kpi>
6       <kpi>COGS</kpi>
7       <comparevalue>BANKBAL</comparevalue>
8       <comparewithprevious>F</comparewithprevious>
9       <hidden>F</hidden>
10      <invertcomparison>F</invertcomparison>
11    </kpi>
12    <kpi>
13      <kpi>CUSTOM</kpi>
14      <comparewithprevious>T</comparewithprevious>
15      <comparisontype>RATIO_PERCENT</comparisontype>

```

```

16         <hidden>F</hidden>
17         <invertcomparison>T</invertcomparison>
18     </kpi>
19 </kpis>
20 <ranges>
21     <range>
22         <comparewithprevious>F</comparewithprevious>
23         <invertcomparison>T</invertcomparison>
24         <range>THIS_FISCAL_HALF</range>
25     </range>
26     <range>
27         <comparevalueto>THIS_FISCAL_YEAR</comparevalueto>
28         <comparewithprevious>F</comparewithprevious>
29         <comparisontype>SUM</comparisontype>
30         <invertcomparison>F</invertcomparison>
31         <range>LAST_YEAR</range>
32     </range>
33 </ranges>
34 <custom>
35     <kpi1>ACTIVITIES</kpi1> <!-- Saved Search associated with the first CUSTOM KPI -->
36 </custom>
37 <highlightings>
38     <highlighting>
39         <backgroundcolor>000000</backgroundcolor>
40         <bold>T</bold>
41         <condition>ALWAYS</condition>
42         <foregroundcolor>FF0000</foregroundcolor>
43         <headline>T</headline>
44         <icon>DOLLAR_SIGN_3</icon>
45         <kpiindex>1</kpiindex>
46         <rangeindex>1</rangeindex>
47     </highlighting>
48 </highlightings>
49 <audience>
50     <allemployees>F</allemployees>
51     <allpartners>F</allpartners>
52     <allroles>T</allroles>
53 </audience>
54 </kpiscorecard>

```

KPI Comparisons in KPI Scorecards

The `kpis` structure in a `kpiscorecard` can encapsulate all the KPIs that you want to compare.



Note: In addition to KPI comparisons, the `kpis` structure can encapsulate formulas that calculate KPI comparisons. For more information, see [KPI Formulas in KPI Scorecards](#).

Each KPI portlet reference in the `kpis` structure is represented by its own `kpi` structure. The following is an example of the `kpis` structure:

```

1  ...
2  <kpis>
3      <kpi>
4          <kpi>COGS</kpi>
5          <comparevalueto>BANKBAL</comparevalueto>
6          <comparewithprevious>F</comparewithprevious>
7          <hidden>F</hidden>
8          <invertcomparison>F</invertcomparison>
9      </kpi>
10     <kpi>
11         <kpi>CUSTOM</kpi>
12         <comparewithprevious>T</comparewithprevious>
13         <comparisontype>RATIO_PERCENT</comparisontype>
14         <hidden>F</hidden>
15         <invertcomparison>T</invertcomparison>
16     </kpi>
17 </kpis>
18  ...

```

You can set the value of the `kpi` element in each `kpi` structure to a KPI ID, such as COGS. By setting this value to a standard or custom KPI, you can perform comparisons using the `comparevalueto`, `comparewithprevious`, and `comparisontype` elements. For more information about valid KPI IDs, see the help topic [KPI IDs Available for Scorecard Formulas](#).

The value of the `comparevalueto` element can reference a KPI ID.

Set the `comparewithprevious` Boolean element to T to compare the result of the KPI comparison with the comparison defined by the `kpi` structure above it.

The `comparisontype` element can be set to one of the following values:

- **RATIO_ABSOLUTE:** $KPI2 / KPI1$
- **RATIO_PERCENT:** $100 * (KPI2 / KPI1)$
- **SUM:** $KPI1 + KPI2$
- **VARIANCE_ABSOLUTE:** $KPI2 - KPI1$
- **VARIANCE_PERCENT:** $100 * (KPI2 - KPI1) / KPI1$



Warning: KPI scorecards do not support results that are unions of two comparisons. If you enter a KPI comparison and a date range comparison, the result is displayed as N/A. To work around this limitation, try defining the comparison as a formula instead. For example, instead of entering **ESTIMATES** for `kpi`, **SALES** for `comparevalueto`, and **VARIANCE_PERCENT** for `comparisontype`, enter **FORMULAPERCENT** for `kpi` and `{SALES}/{QUOTA}` for `formula`. See the help topic [Defining KPI Scorecard Formulas](#).

Set the `invertcomparison` Boolean element to T when you want to invert the operation used to calculate the comparison type.

For more information about `kpi` elements, see the help topic [kpi](#).

KPI Formulas in KPI Scorecards

The `kpis` structure in a `kpiscorecard` can encapsulate formulas that calculate KPI comparisons.



Note: In addition to KPI formulas, the `kpis` structure can encapsulate all the KPI performance indicators that you want to compare. For more information, see [KPI Comparisons in KPI Scorecards](#).

Each formula reference in the `kpis` structure is represented by the `kpi` structure. The following is an example of the `kpis` structure:

```

1  ...
2  <kpis>
3    <kpi>
4      <kpi>FORMULAPERCENT</kpi>
5      <formula>{SALES}/{QUOTA}</formula>
6      <lessismore>F</lessismore>
7      <hidden>F</hidden>
8    </kpi>
9  </kpis>
10 ...

```

Set the `kpi` element to any of the following values to define a formula expression:

- **FORMULACURRENCY**
- **FORMULANUMERIC**
- **FORMULAPERCENT**

Use the formula element to define the formula expression. These formulas are similar to those used in Excel. They can include functions and KPI IDs. For more information about valid KPI IDs, see the help topic [KPI IDs Available for Scorecard Formulas](#).



Warning: KPI scorecards do not support results that are unions of two comparisons. If you enter a KPI comparison and a date range comparison, the result is displayed as N/A. To work around this limitation, try defining the comparison as a formula instead. For example, instead of entering **ESTIMATES** for **kpi**, **SALES** for **comparevalueto**, and **VARIANCE_PERCENT** for **comparisontype**, enter **FORMULAPERCENT** for **kpi** and **{SALES}/{QUOTA}** for **formula**. See the help topic [Defining KPI Scorecard Formulas](#).

Set the `<lessismore` Boolean element to T if lower values from the calculated result are more preferable than higher values. When this element is set to T, downward arrows are green and upward arrows are red in the KPI scorecard.

Set the `hidden` Boolean element to T to hide the results of the formula from the KPI scorecard.

Set the `label` value to the text that should appear as a row or column header in the KPI scorecard portlet.

For more information about `kpi` elements, see the help topic [kpi](#).

Date Ranges and Periods in KPI Scorecards

The `ranges` structure in a `kpiscorecard` defines the date ranges or periods to use in KPI data calculations. The `useperiods` Boolean element of the `kpiscorecard` determines whether periods should be used instead of date ranges.

Each date range or period in the `ranges` structure is represented by the `range` structure. The following is an example of a `ranges` structure that uses date ranges instead of periods:

```

1  ...
2  <useperiods>F</useperiods>
3  ...
4  <ranges>
5    <range>
6      <range>THIS_FISCAL_HALF</range>
7      <comparewithprevious>F</comparewithprevious>
8      <invertcomparison>T</invertcomparison>
9    </range>
10   <range>
11     <range>LAST_YEAR</range>
12     <comparevalueto>THIS_FISCAL_YEAR</comparevalueto>
13     <comparewithprevious>F</comparewithprevious>
14     <comparisontype>SUM</comparisontype>
15     <invertcomparison>F</invertcomparison>
16   </range>
17 </ranges>
18 ...

```

Use the `range` and `comparevalueto` elements to reference the specific ranges that you want to compare. For more information about valid ranges, see:

- [kpi_ranges_daterange](#)
- [kpi_ranges_daterange_report](#)
- [kpi_ranges_period](#)
- [kpi_ranges_daterange_or_period](#)

Set the `comparewithprevious` Boolean element to T when you want to compare the result of the range comparison with the range comparison above it.

Set the `comparisontype` element to any of the following values to define the type of relationship or operation to perform:

- **RATIO_ABSOLUTE:** $KPI2 / KPI1$
- **RATIO_PERCENT:** $100 * (KPI2 / KPI1)$
- **SUM:** $KPI1 + KPI2$
- **VARIANCE_ABSOLUTE:** $KPI2 - KPI1$
- **VARIANCE_PERCENT:** $100 * (KPI2 - KPI1) / KPI1$

Set the `invertcomparison` Boolean element to `T` to invert the operation used to calculate the comparison type.

Set the `label` value to the text that should appear as a row or column header in the KPI scorecard portlet.

For more information about range elements, see the help topic [range](#).

Highlighting in KPI Scorecards

The `highlightings` structure in a `kpiscorecard` configures how rows and comparisons should be highlighted or colorized in the KPI scorecard portlet. Highlighting makes results more visible.

Each highlighting option in the `highlightings` structure is represented by the `highlighting` structure. The following is an example of a `highlightings` structure that uses a single highlighting option:

```

1  ...
2  <highlightings>
3    <highlighting>
4      <kpiindex>1</kpiindex>
5      <condition>ALWAYS</condition>
6      <threshold>2.0</threshold>
7      <rangeindex>1</rangeindex>
8      <backgroundcolor>000000</backgroundcolor>
9      <bold>T</bold>
10     <foregroundcolor>FF0000</foregroundcolor>
11     <headline>T</headline>
12     <icon>DOLLAR_SIGN_3</icon>
13   </highlighting>
14 </highlightings>
15 ...

```

Set the `kpiindex` value to the KPI that the highlighting should apply to. This is an integer-based value that's determined by how the KPIs appear in the `kpi` structure of the `kpiscorecard`, starting with the number 1. By default, the highlighting applies to all KPIs.

Set the `condition` element to any of the following values to indicate how the result should be compared to the `threshold` value:

- ALWAYS
- GREATER_THAN
- LESS_THAN
- MAGNITUDE_GREATER_THAN
- MAGNITUDE_LESS_THAN

Set the `threshold` element to a floating point value. The highlighting takes effect when the criteria of the **condition** is met against the **threshold** value.

Set the `rangeindex` value to a range. This is an integer-based value that's determined by how the ranges appear in the `ranges` structure of the `kpiscorecard`, starting with the number 1. This range determines whether highlighting should be applied when the comparison with the threshold is performed. By default, the highlighting applies to all ranges.

Set any of the following elements to specify the highlighting that should be applied when the criteria of the **condition** is met against the **threshold** value:

- **icon:** Displays an icon with the highlighted results. For more information about possible icon values, see the help topic [kpiscorecards_highlight_icons](#).
- **foregroundColor:** Changes the text color of the highlighted result. This value must be set to a valid 6-digit hexadecimal RGB value between #000000 and #FFFFFF, where #000000 represents black and #FFFFFF represents white. The # prefix is optional.
- **backgroundColor:** Changes the background color of the highlighted result. This value must be set to a valid 6-digit hexadecimal RGB value between #000000 and #FFFFFF, where #000000 represents black and #FFFFFF represents white. The # prefix is optional.
- **bold:** Set this Boolean value to T to bold the text of the highlighted result.
- **headline:** Set this Boolean value to T to display the highlighted result as a headline, in large bold text at the top of the KPI scorecard portlet.

For more information about highlighting elements, see the help topic [highlighting](#).

KPI Scorecards XML Definition Example

The following code sample is an example of a kpiscorecard SDF custom object with the name Inventory Performance Indicators. This scorecard creates a matrix involving two key performance indicators, Average Inventory and Inventory, one formula Current Ratio, and three ranges, THIS MONTH, LAST MONTH and % CHANGE.

```

1 <kpiscorecard scriptid="custkpiscorecard_inventory_performance_indicators">
2   <description>Inventory Performance Indicators</description>
3   <name>Inventory Performance Indicators</name>
4   <useperiods>F</useperiods>
5   <custom>
6     <kpi1></kpi1>
7     <kpi10></kpi10>
8     <kpi2></kpi2>
9     <kpi3></kpi3>
10    <kpi4></kpi4>
11    <kpi5></kpi5>
12    <kpi6></kpi6>
13    <kpi7></kpi7>
14    <kpi8></kpi8>
15    <kpi9></kpi9>
16  </custom>
17  <audience>
18    <allemployees>F</allemployees>
19    <allpartners>F</allpartners>
20    <allroles>T</allroles>
21  </audience>
22  <kpis>
23    <kpi>
24      <comparevalueto></comparevalueto>
25      <comparewithprevious>F</comparewithprevious>
26      <comparisontype></comparisontype>
27      <hidden>F</hidden>
28      <invertcomparison>F</invertcomparison>
29      <kpi>AVERAGEINVENTORY</kpi>
30      <label></label>
31    </kpi>
32    <kpi>
33      <comparevalueto></comparevalueto>
34      <comparewithprevious>F</comparewithprevious>
35      <comparisontype></comparisontype>
36      <hidden>F</hidden>
37      <invertcomparison>F</invertcomparison>
38      <kpi>INVENTORY</kpi>
39      <label></label>
40    </kpi>
41    <kpi>
42      <formula>({BANKBAL}+{RECEIVABLES}+{OTHERCURRENTASSET})/({PAYABLES}+{CREDITCARDBAL}+{OTHERCURRENTLIAB})</formula>
43      <hidden>F</hidden>

```



```

44 <kpi>FORMULANUMERIC</kpi>
45 <label>Current Ratio</label>
46 <lessismore>F</lessismore>
47 </kpi>
48 </kpi>
49 <ranges>
50 <range>
51 <comparevalueto></comparevalueto>
52 <comparewithprevious>F</comparewithprevious>
53 <comparisontype></comparisontype>
54 <invertcomparison>F</invertcomparison>
55 <label></label>
56 <range>THIS_MONTH</range>
57 </range>
58 <range>
59 <comparevalueto></comparevalueto>
60 <comparewithprevious>F</comparewithprevious>
61 <comparisontype></comparisontype>
62 <invertcomparison>F</invertcomparison>
63 <label></label>
64 <range>LAST_MONTH</range>
65 </range>
66 <range>
67 <comparevalueto></comparevalueto>
68 <comparewithprevious>T</comparewithprevious>
69 <comparisontype>VARIANCE_PERCENT</comparisontype>
70 <invertcomparison>F</invertcomparison>
71 <label>% Change</label>
72 <range>THIS_MONTH</range>
73 </range>
74 </ranges>
75 <highlightings>
76 <highlighting>
77 <backgroundcolor></backgroundcolor>
78 <bold>T</bold>
79 <condition>ALWAYS</condition>
80 <foregroundcolor>3366ff</foregroundcolor>
81 <headline>F</headline>
82 <icon></icon>
83 <kpiindex>1</kpiindex>
84 <rangeindex>ALL</rangeindex>
85 <threshold></threshold>
86 </highlighting>
87 </highlightings>
88 </kpiscorecard>

```

The following screenshot shows how the Inventory Performance Indicators KPI scorecard looks to your users in a target NetSuite account:

Inventory Performance Indicators			
INDICATOR	THIS MONTH	LAST MONTH	% CHANGE
Average Inventory	\$0	\$0	N/A
Inventory	\$9,383,933	\$9,383,933	0.00%
Current Ratio	3.82	3.82	0.00%

Users and Roles in SDF

The following topics provide SuiteCloud Development Framework (SDF) information about the SDF custom objects that pertain to users and roles:

- Roles and Permissions as an XML Definition
- Custom Roles as XML Definitions

- [Providing Roles with Access to a Custom Field in SDF](#)



Note: For general information about users and roles in NetSuite, see the help topic [NetSuite Access Overview](#).

Roles and Permissions as an XML Definition

The following topics describe how to create and customize roles and permissions in SuiteCloud projects for SuiteCloud Development Framework (SDF):

- [Custom Roles as XML Definitions](#)
- [Custom Record Type Object Permissions](#)
- [Providing Roles with Access to a Custom Field in SDF](#)

Custom Roles as XML Definitions

You can create and manage roles in a NetSuite account using SuiteCloud Development Framework (SDF). You can create a role definition in a SuiteCloud project and deploy the project to a target account. Role definitions in SDF can reference standard and custom record permissions.

The following role fields and sublists are not supported by SDF and can only be defined in the NetSuite UI:

- Subsidiaries field
- Forms sublist
- Searches sublist
- Dashboard sublist

The following example shows a basic custom role object named “Sample Role” with the script ID named “customrole_sample”:

```

1 <role scriptid="customrole_sample">
2   <centertype>ACCOUNTCENTER</centertype>
3   <issalesrole>F</issalesrole>
4   <issupportrole>F</issupportrole>
5   <iswebseviceonlyrole>F</iswebseviceonlyrole>
6   <employeeerstriction>NONE</employeeerstriction>
7   <employeeviewingallowed>F</employeeviewingallowed>
8   <restricttimeandexpenses>F</restricttimeandexpenses>
9   <restrictip>F</restrictip>
10  <coreadminpermission>F</coreadminpermission>
11  <restrictbydevice>F</restrictbydevice>
12  <name>Sample Role</name>
13  <permissions>
14    <permission>
15      <permkey></permkey>
16      <permlevel></permlevel>
17      <restriction></restriction>
18    </permission>
19  </permissions>
20 </role>

```

The centertype element determines which tabs and links are available for groups of similar roles. For more information about possible centertype values, see the following:

- [role_centertype](#)
- [center](#)

In the permissions structure, the permkey element should be the script ID of the object type that you want to apply a permission to. The permlevel element is the level of access that you want to grant on that object type. The restriction element is the level of restriction to place on that object type.

Note: The restriction element is only applicable to permissions that reference SDF custom objects.

You can customize additional role options using the NetSuite UI. For more information about creating a custom role using the NetSuite UI, see the help topic [Changing Custom Roles](#).

Important: If a SuiteCloud project contains an SDF custom object and a custom role that reference each other, the levels of permission and restriction must match in both objects. For example, if a custom record type permits the custom role with the VIEW permission level, that custom role must set the custom record type permission level to VIEW. Projects that contain a level mismatch cannot be deployed to a target NetSuite account.

For more information about the custom role object, see the help topic [role](#).

Custom Role Restrictions

You can apply any type of restriction to a custom role in SDF. Restrictions define the set of records that users of an assigned role can view or edit for specific record types such as employee, location, department, and class.

As part of the custom role object, the following elements can be used to apply restrictions on users of the assigned role:

- The `restrictbydevice` element provides access to the role only from the devices listed at Setup > Company > Setup Tasks > Company Information.
- The `restrictip` element provides access to the role only from the IP addresses listed at Setup > Company > Setup Tasks > Company Information.
- The `restricttimeandexpenses` element prevents users with this role from entering expense reports or time transactions on behalf of users other than their subordinates.
- The `employeerestriction` element determines whether to restrict role access to transaction, customer, and employee records, based on values in the employee, sales rep, and supervisor fields on these records. The associated `employeeviewingallowed` element indicates whether the role can view employee records other than their own and their subordinates.

In addition to these elements, you can create or modify a `recordrestrictions` structure to apply restrictions on the location, department, and class records. This structure can contain multiple `recordrestriction` structures, where each one represents a unique record type restriction. You cannot have multiple `recordrestriction` structures defining the restrictions of a single record type. A `recordrestriction` contains the following elements:

- The `segment` element is the record type that you want to restrict on the role, such as LOCATION, DEPARTMENT, or CLASS.
- The `restrictions` element indicates the type of restriction to place on the segment or record type.
- The `viewingallowed` element indicates whether users with this role can view instances of record type.
- The `itemsrestricted` element applies the restrictions to item records, in addition to transaction, employee, and partner records.

The following is an example of a custom role object in SDF that uses restriction fields:

```

1 <role scriptid="customrole_restrictsample">
2   <centertype>ACCOUNTCENTER</centertype>
3   <name>Role Restriction Sample</name>
4   ...
5   <restrictbydevice>F</restrictbydevice>
6   <restrictip>T</restrictip>
7   <restricttimeandexpenses>T</restricttimeandexpenses>
8   <employeerestriction>UNASSIGNED</employeerestriction>
9   <employeeviewingallowed>T</employeeviewingallowed>

```

```

10  ...
11  <recordrestrictions>
12    <recordrestriction>
13      <segment>LOCATION</segment>
14      <restriction>OWNONLY</restriction>
15      <itemrestricted>T</itemrestricted>
16      <viewingallowed>T</viewingallowed>
17    </recordrestriction>
18    <recordrestriction>
19      <segment>DEPARTMENT</segment>
20      <restriction>DEFAULTTOWN</restriction>
21      <itemrestricted>F</itemrestricted>
22      <viewingallowed>F</viewingallowed>
23    </recordrestriction>
24    <recordrestriction>
25      <segment>CLASS</segment>
26      <restriction>UNASSIGNED</restriction>
27      <itemrestricted>T</itemrestricted>
28      <viewingallowed>F</viewingallowed>
29    </recordrestriction>
30  </recordrestrictions>
31 </role>

```



Note: When restricting record types, such as LOCATION or DEPARTMENT, you may need to modify the manifest file in your project to indicate that their corresponding features are required in the target account.

For more information, see:

- [Permissions and Restrictions](#)
- [Customizing or Creating NetSuite Roles](#)

Custom Role Example

The following example shows a custom role definition. When deployed to an account, this definition creates or modifies the custom role, SDF Developer, that enables several permissions required for SDF development. The role includes permission to a custom record, customrecord_sample.

```

1  <role scriptid="customrole_sdfdeveloper">
2    <centertype>ACCOUNTCENTER</centertype>
3    <issalesrole>F</issalesrole>
4    <issupportrole>F</issupportrole>
5    <iswebserviceonlyrole>F</iswebserviceonlyrole>
6    <employeeerstriction>NONE</employeeerstriction>
7    <employeeviewingallowed>F</employeeviewingallowed>
8    <restricttimeandexpenses>F</restricttimeandexpenses>
9    <restrictip>F</restrictip>
10   <restrictbydevice>F</restrictbydevice>
11   <name>SDF_Developer</name>
12   <permissions>
13     <permission>
14       <permkey>ADMI_WORKFLOW</permkey>
15       <permlevel>FULL</permlevel>
16     </permission>
17     <permission>
18       <permkey>ADMI_WEBSERVICES</permkey>
19       <permlevel>FULL</permlevel>
20     </permission>
21     <permission>
22       <permkey>ADMI_MANAGE_OWN_OAUTH_TOKENS</permkey>
23       <permlevel>FULL</permlevel>
24     </permission>
25     <permission>
26       <permkey>ADMI_CUSTOMSCRIPT</permkey>
27       <permlevel>FULL</permlevel>
28     </permission>
29     <permission>
30       <permkey>ADMI_APP_DEPLOYMENT</permkey>

```

```

31     <permlevel>FULL</permlevel>
32 </permission>
33 <permission>
34     <permkey>ADMI_SAVEDASHBOARD</permkey>
35     <permlevel>FULL</permlevel>
36 </permission>
37 <permission>
38     <permkey>ADMI_CUSTOTHERFIELD</permkey>
39     <permlevel>FULL</permlevel>
40 </permission>
41 <permission>
42     <permkey>ADMI_KPIREPORT</permkey>
43     <permlevel>FULL</permlevel>
44 </permission>
45 <permission>
46     <permkey>ADMI_ENABLEFEATURES</permkey>
47     <permlevel>FULL</permlevel>
48 </permission>
49 <permission>
50     <permkey>LIST_EMAILTEMPLATE</permkey>
51     <permlevel>FULL</permlevel>
52 </permission>
53 <permission>
54     <permkey>LIST_FILECABINET</permkey>
55     <permlevel>FULL</permlevel>
56 </permission>
57 <permission>
58     <permkey>ADMI_CUSTTRANSACTION</permkey>
59     <permlevel>FULL</permlevel>
60 </permission>
61 <permission>
62     <permkey>ADMI_CUSTFORM</permkey>
63     <permlevel>FULL</permlevel>
64 </permission>
65 <permission>
66     <permkey>ADMI_CUSTTRANFIELD</permkey>
67     <permlevel>FULL</permlevel>
68 </permission>
69 <permission>
70     <permkey>ADMI_CUSTFIELDTAB</permkey>
71     <permlevel>FULL</permlevel>
72 </permission>
73 <permission>
74     <permkey>ADMI_CUSTOMSUBLIST</permkey>
75     <permlevel>FULL</permlevel>
76 </permission>
77 <permission>
78     <permkey>ADMI_SUBLIST</permkey>
79     <permlevel>FULL</permlevel>
80 </permission>
81 <permission>
82     <permkey>ADMI_CUSTRECORD</permkey>
83     <permlevel>FULL</permlevel>
84 </permission>
85 <permission>
86     <permkey>ADMI_CUSTLIST</permkey>
87     <permlevel>FULL</permlevel>
88 </permission>
89 <permission>
90     <permkey>ADMI_CUSTITEMNUMBERFIELD</permkey>
91     <permlevel>FULL</permlevel>
92 </permission>
93 <permission>
94     <permkey>ADMI_CUSTITEMFIELD</permkey>
95     <permlevel>FULL</permlevel>
96 </permission>
97 <permission>
98     <permkey>ADMI_CUSTEVENTFIELD</permkey>
99     <permlevel>FULL</permlevel>
100 </permission>
101 <permission>
102     <permkey>ADMI_CUSTENTRYFORM</permkey>
103     <permlevel>FULL</permlevel>

```

```

104 </permission>
105 <permission>
106   <permkey>ADMI_CUSTENTITYFIELD</permkey>
107   <permlevel>FULL</permlevel>
108 </permission>
109 <permission>
110   <permkey>ADMI_CUSTCOLUMNFIELD</permkey>
111   <permlevel>FULL</permlevel>
112 </permission>
113 <permission>
114   <permkey>ADMI_CUSTCENTER</permkey>
115   <permlevel>FULL</permlevel>
116 </permission>
117 <permission>
118   <permkey>ADMI_CUSTSECTION</permkey>
119   <permlevel>FULL</permlevel>
120 </permission>
121 <permission>
122   <permkey>ADMI_CUSTCATEGORY</permkey>
123   <permlevel>FULL</permlevel>
124 </permission>
125 <permission>
126   <permkey>ADMI_CUSTBODYFIELD</permkey>
127   <permlevel>FULL</permlevel>
128 </permission>
129 <permission>
130   <permkey>ADMI_CRMLIST</permkey>
131   <permlevel>FULL</permlevel>
132 </permission>
133 <permission>
134   <permkey>ADMI_MANAGERROLES</permkey>
135   <permlevel>FULL</permlevel>
136 </permission>
137 <permission>
138   <permkey>ADMI_ADVANCED_TEMPLATES</permkey>
139   <permlevel>FULL</permlevel>
140 </permission>
141 <permission>
142   <permkey>[scriptid=customrecord_sample]</permkey>
143   <permlevel>FULL</permlevel>
144   <restriction>VIEW</restriction>
145 </permission>
146 </permissions>
147 </role>

```

Providing Roles with Access to a Custom Field in SDF

In SuiteCloud Development Framework (SDF), you can assign roles to a custom field using the permission element in the custom field definition of a SuiteCloud project. For more information about custom record type settings, see the help topic [Restricting Access to Custom Fields](#).

Each permission element requires the following information:

Tag name	Possible values	Description
role	Any standard role ID or custom role reference.	The role that can access the custom field.
accesslevel	One of the following integers: 0 (NONE) 1 (VIEW) 2 (EDIT)	The level of access that users of the specific role have to the custom field.
searchlevel	One of the following integers:	The level of access that users of the specific role have to perform searches and reports on the custom field.

	■ 0 (NONE) ■ 1 (RUN) ■ 2 (EDIT)	
--	---	--

The permission element for fields is supported for any custom field type that SDF supports. For more information about which fields are supported, see [Customizations Supported by SuiteCloud Development Framework](#).

For more information, see:

- [role](#)
- [entitycustomfield](#)

Example

The following example gives the bookkeeper role view access to the Demo custom field and prohibits this role from including this field in searches:

```

1 <entitycustomfield scriptid="custentity_demo">
2   <permissions>
3     <permission>
4       <role>BOOKKEEPER</role>
5       <accesslevel>1</accesslevel>
6       <searchlevel>0</searchlevel>
7     </permission>
8   </permissions>
9 </entitycustomfield>

```

Templates

The following topics provide SuiteCloud Development Framework (SDF) information about the SDF custom objects that pertain to templates:

- [Email Templates as XML Definitions](#)
- [Advanced HTML/PDF Templates as XML Definitions](#)

Email Templates as XML Definitions

SuiteCloud Development Framework (SDF) supports email templates that generate personalized email messages to send to business contacts. The emailtemplate SDF custom object can be used to develop custom email templates in a SuiteCloud project. You can create new email templates, or import existing email templates from a NetSuite account into a SuiteCloud project using SuiteCloud IDE plug-in or SuiteCloud CLI.

There are two components to email templates in a SuiteCloud:

- The emailtemplate SDF custom object file, which contains the XML definition of the template record.
- The template definition file, which contains the HTML and FreeMarker code that defines the email template.

You can create email templates in an XML definition by creating both an email template SDF custom object and a template definition file.

The emailtemplate SDF custom object file name must match the scriptID attribute and requires the prefix custemailtmpl, for example custemailtmpl_quoterequest.xml. For information about the available fields for the emailtemplate SDF custom object, see the help topic [emailtemplate](#).

Consider the following information when working with template definition files:

- The template definition file is automatically created when you create an emailtemplate SDF custom object file in SuiteCloud IDE plug-in. The template definition file is also included when you import an emailtemplate SDF custom object into your SuiteCloud project.
- The template definition file can be stored in the objects folder or referenced in the File Cabinet. Only emailtemplate SDF custom objects in account customization projects are able to reference template definition files in the File Cabinet of the target account. SuiteApp projects must contain the template definition file in either the objects folder or the File Cabinet in the project.
- When the template definition file is not referenced in the File Cabinet:
 - The usesmedia field must be set to F in the emailtemplate SDF custom object.
 - The template definition file must be stored in the objects folder with the emailtemplate SDF custom object file.
 - The template definition file name must be of the form scriptID.template.html, where scriptID is the script ID of the emailtemplate object and also the object file name without the file extension. For example, custemailtmpl_quoterequest.template.html is the template definition for the SDF custom object custemailtmpl_quoterequest.xml. The file name provides the means for NetSuite to match the template definition with the template record when the objects are deployed to an account.
- When the template definition file is referenced in the File Cabinet:
 - The usesmedia field must be set to T
 - The mediaitem field of the SDF custom object must contain the path to the template definition file.
 - For account customization projects, referenced files must be stored in the Templates/E-mail Templates or Templates/Marketing Templates directories of the File Cabinet in the SuiteCloud project or target account.
 - For SuiteApp projects, referenced files must be stored in the SuiteApps/appID directory in the SuiteCloud project, where appID is the application ID for the SuiteApp.
 - Supported file types are HTML (.html), FreeMarker Template (.ftl), and Plain Text (.txt).
 - It can be beneficial to reference the template definition file when multiple emailtemplate objects can use the same template but require different attributes, or when the template definition is already stored in the target account.

For information about customizing the template definition, see the following:

- [Customizing Templates for System-Automated Email](#)
- [Scriptable Templates](#)

Email Template XML Definition Examples

The following is an example of an emailtemplate SDF custom object that defines an email template for messages confirming a quote request was received:

```

1 <emailtemplate scriptid="custemailtmpl_quoterequest">
2   <name>Reply to quote request</name>
3   <recordtype>ENTITY</recordtype>
4   <subject>We have received your request</subject>
5   <usesmedia>F</usesmedia>

```



```
6 | </emailtemplate>
```

The emailtemplate SDF custom object example has the file name `custemailtmpl_quoterequest.xml`. The SDF custom object requires a template definition file with the file name `custemailtmpl_quoterequest.template.html`, which contains the HTML and FreeMarker code that defines the email template. The content of `custemailtmpl_quoterequest.template.html` is beyond the scope of this example.

For information about customizing email templates, see the following:

- [Customizing Templates for System-Automated Email](#)
- [Scriptable Templates](#)

The following is an example of the same emailtemplate SDF custom object referencing the template definition file `quoterequest_template.html` in the File Cabinet.

```
1 | <emailtemplate scriptid="custemailtmpl_quoterequest">
2 |   <name>Reply to quote request</name>
3 |   <recordtype>ENTITY</recordtype>
4 |   <subject>We have received your request</subject>
5 |   <usesmedia>T</usesmedia>
6 |   <mediaitem>[/Templates/E-mail Templates/quoterequest_template.html]</mediaitem>
7 | </emailtemplate>
```

Advanced HTML/PDF Templates as XML Definitions

SuiteCloud Development Framework (SDF) supports advanced PDF/HTML templates that can be associated with transaction forms. The `advancedpdftemplate` SDF custom object can be used to develop advanced PDF/HTML templates in a SuiteCloud project. You can create new SDF custom objects, or import existing objects into your SuiteCloud project using SuiteCloud IDE plug-in or SuiteCloud CLI.

There are two components to advanced PDF/HTML templates in a SuiteCloud project:

- The `advancedpdftemplate` SDF custom object file, which contains the XML definition of the template record. The file name requires the prefix `custtmpl`, for example `custtmpl_paymentvoucher.xml`.
- The template definition file, which contains the information that defines the template. There are existing template definitions that you can customize for your business needs. The template definition file is automatically created when you create an `advancedpdftemplate` SDF custom object file in SuiteCloud IDE. The template definition is also included when importing the `advancedpdftemplate` SDF custom object into your SuiteCloud project.

The file name must be of the form `scriptID.template.xml`, where `scriptID` is the script ID of the `advancedpdftemplate` object and also the object file name without the file extension. For example, `custtmpl_paymentvoucher.template.xml` is the template definition for the SDF custom object `custtmpl_paymentvoucher.xml`. The template definition file is located in the same objects folder as the `advancedpdftemplate` SDF custom object file. The file name provides the means for NetSuite to match the template definition with the template record when the objects are deployed to an account.

For information about templates, see the help topics [Reviewing Available Advanced Templates](#) and [Advanced Templates Customization in the Template Editor](#).

To deploy advanced PDF/HTML templates to a target NetSuite account, enable the Advanced PDF/HTML Templates feature. For more information, see the help topic [Enabling the Advanced PDF/HTML Templates Feature](#).

For information about creating or editing advanced PDF/HTML templates, see the following:

- [Advanced PDF/HTML Templates](#)
- [Source Code Editing in the Template Editor](#)

Validating Advanced HTML/PDF Templates in an XML Definition

Consider the following information when validating advancedpdftemplate SDF custom objects:

- The advancedpdftemplate SDF custom object file name requires the prefix custtmpl, for example custtmpl_paymentvoucher.xml.
- The template definition file name must be of the form scriptID.template.xml, where scriptID is the script ID of the advancedpdftemplate object and also the object file name without the file extension. For example, custtmpl_paymentvoucher.template.xml is the template definition for the SDF custom object custtmpl_paymentvoucher.xml. The template definition file is located in the same objects folder as the advancedpdftemplate SDF custom object file.
- The ADVANCEDPRINTING feature must be added to the manifest as a required feature.
- The contents of the template definition file, including form print and email template fields, are not validated. Ensure the correct type of template is specified in those fields. For information about templates, see the help topics [Reviewing Available Advanced Templates](#) and [Advanced Templates Customization in the Template Editor](#).
- The form transaction type must match the advanced PDF template transaction type.
- The savedsearch criteria are not validated.
- Ensure there is only one PDF template within the project that has the preferred field set to T per template type. If there are multiple PDF templates of the same type with the preferred field set to T, they will override each other in sequence because only one per type can be the preferred type.
- If the PDF template is for a custom transaction type, no deployment error is returned when the account is not specified for that transaction type. Under those conditions, an error is displayed when that PDF template is later opened in the NetSuite UI. To avoid the error, specify the account for the custom transaction type in the NetSuite UI prior to opening the PDF template.

Advanced HTML/PDF Template XML Definition Example

The following is an example of an advancedpdftemplate SDF custom object that customizes a payment voucher PDF/HTML template:

```

1 <advancedpdftemplate scriptid="custtmpl_100_1326288_617" standard="STDHTMLPAYMENTVOUCHER">
2   <description>This template is used for payment vouchers.</description>
3   <displaysourcecode>F</displaysourcecode>
4   <isinactive>F</isinactive>
5   <preferred>T</preferred>
6   <title>Custom Payment Voucher PDF/HTML Template</title>
7 </advancedpdftemplate>

```

The advancedpdftemplate SDF custom object example has the file name custtmpl_paymentvoucher.xml. The SDF custom object requires a template definition file with the file name custtmpl_paymentvoucher.template.xml, which contains the HTML code that defines the advanced HTML/PDF template. The content of custtmpl_paymentvoucher.template.xml is beyond the scope of this example, but can be based on the customization of the Standard Payment Voucher PDF/HTML Template, which is a standard advanced PDF/HTML template in NetSuite.

SuiteCommerce Web Site Management

The following topics provide SuiteCloud Development Framework (SDF) information about the SDF custom objects that pertain to SuiteCommerce Web Site Management:

- [CMS Content Types as XML Definitions](#)
- [SSP Applications as XML Definitions](#)

CMS Content Types as XML Definitions

Create and manage CMS content types, sometimes called custom content types (CCT) using SuiteCloud Development Framework (SDF). You can model a `cmscontenttype` definition in a SuiteCloud project and deploy the project to a target NetSuite account. You can either create new CMS content types in your project or import existing ones from a NetSuite account. In the NetSuite application, these records are located under Commerce > Content Management > Content > CMS Content Types. For more information, see the following:

- [Site Management Tools Custom Content Type](#)
- [CMS Content Type Record](#)

The following example shows a CMS content type definition in a SuiteCloud project that uses all available fields:

```

1 <cmscontenttype scriptid="custcontenttype_sample">
2   <customrecordid>[scriptid=customrecord_childdatetime]</customrecordid>
3   <description>This is a brief description of the CMS content type.</description>
4   <iconimagepath>/netsuite.com/sample_image.svg</iconimagepath>
5   <label>My First CMS Content Type</label>
6   <name>Unique Custom Content Type Name</name>
7 </cmscontenttype>

```

A reference to the custom record that is associated with the content type is defined by the `customrecordid` element. This reference must have a one to one relationship to the CMS content type, meaning that the custom record must not be associated with any other CMS content type.

The icon image path, defined by the `iconimagepath` element, is represented by a string value. It must start with a slash (/) character and be in an SVG file format.

CMS content type objects must have values defined for the `name` and `label` elements. The content type name must be unique.

SSP Applications as XML Definitions

Note: SSP application objects in SuiteCloud Development Framework (SDF) only support SuiteScript 1.0. For information about SuiteScript versions, see the help topic [SuiteScript Versioning Guidelines](#).

SSP (SuiteScript Server Page) applications are supported in SuiteCloud Development Framework (SDF). They are used to build custom applications for SuiteCommerce webstores, letting you customize a shopping experience by replacing default web store pages with your own custom assets. For example,

you can use HTML and SuiteScript 1.0 to create these customizations. For more information about SSP applications and SuiteCommerce, see the help topic [SSP Application Overview](#).

You can create and manage a SuiteScript Server Page (SSP) application in a SuiteCloud project and deploy it to a target NetSuite account. You can also import an SSP application from a NetSuite account into a SuiteCloud project. For more information about importing objects, see [Account Component Imports to SuiteCloud Projects](#).

Customizing an SSP application object from your SuiteCloud project is the equivalent of editing or making a new SSP application record from the NetSuite UI. For example, from a NetSuite account, select Customization > Scripting > SSP Application > New from a NetSuite account. An SSP application consists of .ssp files and a library of JavaScript files. The library of files associated with the .ssp files must be stored in the File Cabinet/Web Site Hosting Files folder in the SuiteCloud project.

The following is an image of an SSP application record in Edit mode in the NetSuite UI. Some NetSuite UI and webstore customization differ depending on whether the Site Builder, SuiteCommerce Advanced, or SuiteCommerce Standard module is enabled in the NetSuite account.

Note: Changing precedence, linking to site, or linking to domain are account-specific actions that cannot be specified using SDF. When an SSP application is deployed to an account, it is added to the end of the list and automatically assigned the last precedence number. For example, if it was the third SSP application added to an account, the precedence sequence value is set to 3. For more information, see the help topic [Change SSP Application Precedence](#).

SSP Application ← → List Search

Edit Back Link to Site Link to Domain Actions

Name	Application Folder	Log Level
Reference My Account	Live Hosting Files: /SSP Applications/NetSuite Inc. - My Account/Reference My Account	Debug
ID	URL Root	Owner
webapp8	/MyAccount	Wolfe, J Wolfe, J
Publisher ID	Description	☐ Inactive
App ID	Status	
	Released	
Version		
1.0		

Scripts Supported Touch Points Execution Log History

Function service

Libraries •

Library Script File

Web Site Hosting Files/Live Hosting Files/SSP Applications/NetSuite Inc. - My Account/Reference My Account/ssp_libraries/underscore-1.3.3.js
Web Site Hosting Files/Live Hosting Files/SSP Applications/NetSuite Inc. - My Account/Reference My Account/ssp_libraries/backbone.validation-0.5.2.js
Web Site Hosting Files/Live Hosting Files/SSP Applications/NetSuite Inc. - My Account/Reference My Account/ssp_libraries/commons.js
Web Site Hosting Files/Live Hosting Files/SSP Applications/NetSuite Inc. - My Account/Reference My Account/ssp_libraries/Models.js

Edit Back Link to Site Link to Domain Actions

SSP applications in SDF are represented with an `<sspapplication>` element. To define this object, see the following sections:

- [The SSP Application Element](#)
- [The Libraries \(Scripts\) Element](#)
- [The Entrypoints \(Supported Touch Points\) Element](#)

The SSP Application Element

Note: Some fields may contain or require account-specific values. These values cannot be specified or imported into a SuiteCloud project. These values can only be specified in the SSP application record using the NetSuite UI after the SSP application is deployed to a target account.

Field	Description
sspapplication	Within this element, you can optionally specify the script ID in quotations using a attribute=value pair. <pre> 1 <sspapplication scriptid=""> 2 ... 3 </sspapplication> </pre> For example, the following sets the script ID of an SSP application object to the value webapp_33: <pre> 1 <sspapplication scriptid="webapp_33"> </pre> If you do not enter a value, a default is provided, webapp###. This value cannot be edited after the SSP Application record is saved.
name	This name is like a project name, and is used in NetSuite lists, such as those on the SSP Applications list page. This name is not directly visible to web store users. For example: <pre> 1 <name>Test SSP Application</name> </pre> The character limit is 40.
appfolder	This is the File Cabinet folder where customization files are located, such as a subfolder in the Live Hosting Files folder or WebSite Hosting Files Folder. Specify the application folder as a file reference encased in square brackets. For example: <pre> 1 <appfolder>[/Web Site Hosting Files/Staging Hosting Files/SSP Applications/Sample/Test/]</appfolder> </pre>  Important: The folder path must begin and end with the forward slash (/) character.
rootpath	This is the base URL that is used to link to the SSP application from the web store and may be visible to web store users. It specifies the path that points to the location of SSP application assets. By default, the URL root includes the application publisher and application name. For example: <pre> 1 /Samples/Test/ </pre> The base URL should include at least one path component that distinguishes SSP application assets from other root assets in the domain.  Important: The folder path must begin and end with the forward slash (/) character.
description	(optional)

Field	Description
	This is a description of the SSP application. The description is only displayed internally in NetSuite.
status	Possible values are: ■ TESTING ■ RELEASED Leave as the default of Testing while you are creating and debugging your SSP application. Set to Released after you have completed the SSP application and linked it to your website.
loglevel	(optional) The value in this field determines the type of messages written to the script execution log for the SSP application. Possible values are: ■ DEBUG ■ AUDIT ■ ERROR ■ EMERGENCY The default value is DEBUG.
isinactive	(optional) Possible values are T or F. The default setting is false.

The Libraries (Scripts) Element

The `<sspapplication>` element contains a `<libraries>` element to represent the script subrecord:

Field	Description
libraries	This element represents the script subrecord associated with the SSP application.
library	This element contains the scriptfile reference located in the Web Hosting Files associated with SSP application.
scriptfile	This element takes a file path to reference an external JavaScript file. For example: <pre>1 <scriptfile>[/SuiteScripts/BundleInstallationScript.js]</scriptfile></pre> Files in the SuiteBundles, SuiteScripts, and Web Site Hosting Files folders of your File Cabinet can be selected as library files. After you add a library file to an SSP application, it can be referenced by any of the .ss or .ssp files in the SSP application.

The Entrypoints (Supported Touch Points) Element

The `<sspapplication>` element contains an `<entrypoints>` element to represent the endpoint subrecord:

Field	Description
entrypoints	This element represents the endpoint subrecord, which appears as a supported touch points tab on the SSP application in the NetSuite UI.

Field	Description
	These are the pages where website visitors can interact with the SSP application.
entrypoint	This element contains the elements specifying values for an individual touch point. An entrypoint must include entryitem and entrytype element. The entryparameter is optional.
entrytype	Set the entrytype to the specify the entry page type for the touch point between your SSP application and web store. Possible values are: ■ CART ■ CHECKOUT ■ CUSTCENTER ■ HOMEPAGE ■ LOGIN ■ LOGOUT ■ REGISTER
entryitem	Set the entryitem to reference the SSP file you want to use as a supported touch point. The following file extensions are allowed: ■ .SS ■ .SSP
entryparameter	(optional) Enter any URL parameters to be used

SSP Application Example

The following is an XML definition example of a custom SSP application object, named "Test SSP Application":

```

1  <sspapplication scriptid="webapp_33">
2    <name>Test SSP Application</name>
3    <appfolder>[/Web Site Hosting Files/Staging Hosting Files/SSP Applications/Sample/Test/]</appfolder>
4    <rootpath>/Sample/Test</rootpath>
5    <description>Test SSP Application</description>
6    <status>TESTING</status>
7    <loglevel>DEBUG</loglevel>
8    <isinactive>F</isinactive>
9    <libraries>
10     <library>
11       <scriptfile>[/SuiteScripts/BundleInstallationScript.js]</scriptfile>
12     </library>
13   </libraries>
14   <entrypoints>
15     <entrypoint>
16       <entrytype>LOGIN</entrytype>
17       <entryitem>[/Web Site Hosting Files/Staging Hosting Files/SSP Applications/Sample/Test/test.ssp]</entryitem>
18       <entryparameter>f=g<entryparameter>
19     </entrypoint>
20   </entrypoints>
21 </sspapplication>

```

For more information about:

- Setting up an account to work with SDF and SSP applications, see [Enabling SuiteCloud Development Framework in the Target NetSuite Account \(Administrator Only\)](#) and [Assigning the Developer Role \(Administrator Only\)](#).
- Getting started with SuiteCommerce, see the help topic [Commerce Website Developer Documentation Overview](#).
- Removing SSP application records or associated web hosting content after you have deployed it to an account, see [Differences Between SuiteCloud Projects](#).

For more information about SSP applications in general, see the following topics:

- [SSP Application Overview](#)
- [Deploy to a Custom SSP Application](#)
- [Deploy SCA Customizations to NetSuite](#)
- [SSP Application Governance](#)

SuiteAnalytics Workbooks

The following topics provide SuiteCloud Development Framework (SDF) information about the SDF custom objects that pertain to SuiteAnalytics Workbooks:

- [Workbooks as XML Definitions](#)
- [Custom Workbook Objects](#)
- [Custom Dataset Objects](#)

Workbooks as XML Definitions



Important: Do not modify the contents of the definition structure or its character data. Modifications may damage the workbook data when you deploy the modified object to a target NetSuite account.

You should use NetSuite to modify custom workbooks rather than using SDF to modify workbook objects. After you save your changes in NetSuite and re-import the workbook into your SuiteCloud project, the workbook object gets updated safely and automatically.

Custom workbooks can be created or modified only by using NetSuite. After the workbooks are created, they can be either imported into a SuiteCloud project or deployed to a target NetSuite account. For more information about creating or modifying a custom workbook, see:

- [Custom Workbooks and Datasets](#)
- [SuiteAnalytics Workbook Tutorial](#)

To import a custom workbook into a SuiteCloud project for SuiteCloud Development Framework (SDF), you must import two SDF custom object types that share a common script ID suffix — a workbook and a dataset. Both objects are required to represent a single custom workbook. For more information, see:

- [Custom Workbook Objects](#)
- [Custom Dataset Objects](#)

Usage Requirements and Prerequisites

You must use the Administrator role to import or deploy custom workbooks with SDF. For more information, see:

- [Adding an Account in SuiteCloud Extension for Visual Studio Code](#)
- [SuiteCloud IDE Plug-in for WebStorm Account Setup](#)
- `account:setup`

In addition to using the Administrator role for development, enable the SuiteAnalytics Workbook feature in the target NetSuite account to be able to import and deploy workbooks with SDF. For more information, see the help topic [Enabling SuiteAnalytics Workbook in Your NetSuite Account](#).

Custom Workbook Objects

After importing a custom workbook, the workbook SuiteCloud Development Framework (SDF) custom object contains information about the pivot tables, table views, and charts.

The following is an example of a workbook object that was imported from NetSuite:

```

1 <workbook scriptid="custworkbook_transaction">
2   <name>Transaction Workbook</name>
3   <pivots>
4     <pivot scriptid="custpivot3cbabf23b05847429b9994c169200800"></pivot>
5     <pivot scriptid="custpivot3cbabf23b05847429b8d84a4c1692008"></pivot>
6   </pivots>
7   <charts>
8     <chart scriptid="custchart3224e33738e440089aea8960464b9999"></chart>
9     <chart scriptid="custchart3224e33738e440089aea8960464b9740"></chart>
10  </charts>
11  <tables>
12    <table scriptid="custviewfb73a08a7cb94c7e9edc78cb5c508765"></table>
13    <table scriptid="custviewfb73a08a7cb94c7e9edc78cb5c507663"></table>
14  </tables>
15  <dependencies>
16    <dependency>[scriptid=custdataset_transaction]</dependency>
17  </dependencies>
18  <definition>
19    <![CDATA[
20    ...
21    ]]>
22  </definition>
23 </workbook>

```

The workbook `scriptid` attribute and name element are respectively identical to the script ID and name of the custom workbook in the NetSuite account.

The pivots structure contains the script ID references to the pivot tables in the workbook. For information about creating workbook pivot tables in NetSuite, see the help topic [Workbook Pivot Tables](#).

The charts structure contains the script ID references for the charts in the workbook. For information about creating workbook charts in NetSuite, see the help topic [Workbook Charts](#).

The tables structure contains the script ID references for the tables views in the workbook. For information about creating workbook table views in NetSuite, see the help topic [Workbook Table Views](#).

The dependencies structure of the workbook object only supports a single required dependency element. This element must reference the dataset object that is associated with this workbook object. Dataset objects contain additional information about custom workbooks. For more information, see [Custom Dataset Objects](#).

The definition structure contains character data that is related to the pivots and charts from the workbook.

Warning: Do not modify the contents of the definition structure or its character data. Modifications may damage the workbook data when you deploy the modified object to a target NetSuite account.

You should use NetSuite to modify custom workbooks rather than using SDF to modify workbook objects. After you save your changes in NetSuite and re-import the workbook into your SuiteCloud project, the workbook object gets updated safely and automatically.

For more information about workbook objects, see the help topic [workbook](#).

Custom Dataset Objects

A custom dataset object in SuiteCloud Development Framework (SDF) is derived from a custom workbook. It contains the source data that is associated with a workbook object, and should be imported into your SuiteCloud project along with that workbook object. SDF only supports associating workbooks with custom dataset objects.

The following is an example of a dataset object that was imported from NetSuite:

```

1 <dataset scriptid="custdataset_transaction">
2   <name>Transaction Workbook</name>
3   <dependencies>
4     <dependency>[scriptid=customtransaction_newtype]</dependency>
5   </dependencies>
6   <definition>
7     <![CDATA[
8     ...
9     ]]>
10  </definition>
11 </dataset>

```

The scriptid attribute and name element of the dataset object are derived from its associated workbook object. These two objects share the same script ID suffix. For example, the custdataset_transaction dataset object is associated with a custworkbook_transaction workbook object.

The dependencies structure of the dataset object may contain multiple dependency elements. Each dependency can reference one of several custom objects that are supported by SDF, including custom transaction types, custom segments, custom record types, and various custom fields. Custom dependency references are required if the source data relies on them.

The definition structure contains character data that is related to the source data from the workbook.

Warning: Do not modify the contents of the definition structure or its character data. Modifications may damage the dataset when you deploy the modified object to a target NetSuite account.

You should use NetSuite to modify custom workbooks rather than using SDF to modify dataset objects. After you save your changes in NetSuite and re-import the dataset into your SuiteCloud project, the dataset object gets updated safely and automatically.

For more information about dataset objects, see the help topic [dataset](#).

Artificial Intelligence (AI)

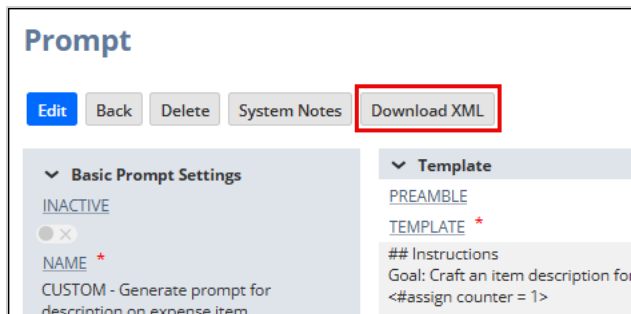
The following topics provide SuiteCloud Development Framework (SDF) information about the SDF custom objects that pertain to artificial intelligence (AI):

- [Prompts as XML Definitions](#)
- [Text Enhance Actions as XML Definitions](#)

Prompts as XML Definitions

A prompt represents a generative artificial intelligence (AI) prompt. Generative AI prompts are the instructions given to a large language model (LLM). The Text Enhance feature uses prompts to provide instructions to the LLM to generate or modify text. For more information, see the help topic [Text Enhance](#). You can also manage prompt records using Prompt Studio and SuiteScript. For more information, see the help topics [Prompt Studio](#) and [N/llm Module](#).

In SDF, the prompt object represents a prompt. To download an XML file that represents a prompt, in Prompt Studio, view a custom prompt, and click **Download XML**.



You can download XML only for custom prompts, not system prompts.

Warning: Do not manually edit the XML in SDF. Modifications to the system-generated XML may result in validation and deployment failures. You should customize prompts using Prompt Studio instead of SDF. For more information, see the help topic [Prompt Studio](#).

Prompt XML Example

The following code sample shows an example of a prompt XML definition.

```

1 <prompt scriptid="custprompttranslate">
2   <action>[scriptid=custteactiontranslate]</action>
3   <description/>
4   <fieldid>all</fieldid>
5   <frequencypenalty>0</frequencypenalty>
6   <language>en</language>
7   <languagevariant/>
8   <maxtokens>0</maxtokens>
9   <modelfamily>COHERE_COMMAND_R</modelfamily>
10  <name>Test Translate</name>
11  <preamble/>
12  <presencepenalty>0</presencepenalty>
13  <prompttype>TEXTENHANCE</prompttype>
14  <recordtype>all</recordtype>
15  <temperature>0.2</temperature>
16  <template>Test</template>
17  <topk>500</topk>
18  <topp>0.7</topp>
19 </prompt>
  
```

Text Enhance Actions as XML Definitions

A Text Enhance action represents an action that can be added to supported text fields as part of the Text Enhance feature. For more information, see the help topic [Text Enhance](#). The following types of text fields support Text Enhance actions:

- Long text
- Text area
- Rich text

In SDF, the `textenhanceaction` object represents a Text Enhance action. To download an XML file that represents a Text Enhance action, in Prompt Studio, view a custom Text Enhance action, and click **Download XML**.

You can download XML only for custom Text Enhance actions, not system Text Enhance actions.



Warning: Do not manually edit the XML in SDF. Modifications to the system-generated XML may result in validation and deployment failures. You should customize Text Enhance actions using Prompt Studio instead of SDF. For more information, see the help topic [Prompt Studio](#).

Text Enhance Action XML Example

The following code sample shows an example of a Text Enhance action XML definition.

```

1 <textenhanceaction scriptid="custteactiontranslate">
2   <ispartial>F</ispartial>
3   <language>en</language>
4   <name>Translate</name>
5 </textenhanceaction>

```

Conversion of a Bundle to an Account Customization Project for SDF

Note: SuiteBundler is still supported, but it will not be updated with any new features.

To take advantage of new features for packaging and distributing customizations, you can use the Copy to Account and SuiteCloud Development (SDF) features instead of SuiteBundler.

Copy to Account is an administrator tool that you can use to copy custom objects between your accounts. The tool can copy one custom object at a time, including dependencies and data. For more information, see the help topic [Copy to Account](#).

SuiteCloud Development Framework is a development framework that you can use to create SuiteApps from an integrated development environment (IDE) on your local computer. For more information, see .

You can retrieve a list of customization bundles from a NetSuite account and import the components into an account customization project type of SuiteCloud project for SuiteCloud Development Framework (SDF).

Before you can convert a bundle, you must enable the **Create Bundles With SuiteBundler** feature in the source NetSuite account. This feature is located under Setup > Company > Setup Tasks > Enable Features.

When converting a bundle, the following types of components cannot be imported:

- Folders that are not supported by SuiteCloud Development Framework (SDF) or that are not in the File Cabinet. For a list of supported folders, see [SuiteCloud Project Structure and File Components](#).
- Files that are larger than 50MB.
- Hidden files.
- Custom record instances.
- Locked records.
- Records that are not supported by SDF.

For more information about records that are supported by SDF, see [Customizations Supported by SuiteCloud Development Framework](#).

If you import a bundle that contains account components that are not supported by SDF, the log indicates which account components were excluded from the import.

To convert a bundle into account customization project components:

- Using SDF CLI, see the help topics [listbundles](#) and [importbundle](#).
- Performing the conversion from the NetSuite account, see [Converting a Bundle into an Account Customization Project from a NetSuite Account](#). Choose this method for converting a large bundle.

 [View the SuiteCloud Development Framework: Converting SuiteBundles to Account Customization Projects video.](#)

Converting a Bundle into an Account Customization Project from a NetSuite Account

Note: SuiteBundler is still supported, but it will not be updated with any new features.

To take advantage of new features for packaging and distributing customizations, you can use the Copy to Account and SuiteCloud Development (SDF) features instead of SuiteBundler.

Copy to Account is an administrator tool that you can use to copy custom objects between your accounts. The tool can copy one custom object at a time, including dependencies and data. For more information, see the help topic [Copy to Account](#).

SuiteCloud Development Framework is a development framework that you can use to create SuiteApps from an integrated development environment (IDE) on your local computer. For more information, see .

You should convert your bundles into account customization projects from a NetSuite account. The converted files are saved as a zip in the File Cabinet. This enables you to use SuiteCloud Development Framework (SDF) for development of your existing bundles.

When you convert the same bundle again, any existing project conversion files are overwritten with the new conversion.

Configuration bundles may not be converted.

Convert a Bundle to an Account Customization Project

1. Go to Customization > SuiteBundler > Create Bundle > List.
2. Select a bundle you own.
3. Note the bundle ID to help you identify the folder where converted files are saved.
4. On the Bundle Details page, select **Convert to SuiteCloud Project**.

Note: If the button is grayed out, it means that a bundle conversion for that bundle is in progress.

5. Click OK.
6. To download the converted files and log, go to your File Cabinet. The file are located at / **SuiteBundles/SDF_Conversions/sdf_conversion_{bundleid}.zip**
7. Unzip and save the files.
8. (Optional) To view the log and check for errors, open the status.xml file.
9. Create a new SDF account project.
10. To import your project components, move the converted file and object folders into your SuiteCloud project.

SuiteCloud Development Framework Error Handling

SuiteCloud Development Framework (SDF) provides extensive validation of SDF custom objects in SuiteCloud projects to catch errors that may occur during the deployment process. Use the messages in the deployment and validation logs to troubleshoot the errors and warnings that you encounter. For more information, see [SuiteCloud Project Validation](#) and [Deployment Logs for SuiteCloud Projects](#).

Most validation messages provide details about the error or warning, probable causes, possible solutions, and preventative measures. However, depending on the customization, some message details can be too generic or there may be multiple solutions to address a given error, depending on your specific business needs.

This section provides additional information about specific error messages that require additional information or further explanation. SDF error handling coverage includes the following error types:

- [entryForm and transactionForm Errors](#)
- [Account Configuration Errors](#)

entryForm and transactionForm Errors

The following errors may occur when working with transaction forms or entry forms:

- [The object value is either invalid or does not exist in the account](#)
- [The field under the group is invalid](#)
- [The property value of the object cannot be modified](#)
- [The sameRowAsPrevious field must be set to false because it is the first entry](#)
- [The object field must not be NOT_YET_SUPPORTED](#)

For more information, see the help topics [entryForm](#) and [transactionForm](#).

The object value is either invalid or does not exist in the account

The <object> value, <ID>, is either invalid or does not exist in the account. The field must be set to a valid system value or a custom value.

Cause: The script ID value contains a typo

■ Solution

Verify the correct spelling of the script ID. Update all ID instances with the correct spelling in the form object, then validate again.

Cause: The script ID value does not exist in the target account

■ Solution

The ID may have been imported from a version or edition of NetSuite, such as a OneWorld account, that supports features that are unavailable in the target NetSuite account. You should remove all references to this ID to make your project compatible with any account type.

The field under the group is invalid

The `<field_name>` field under the `<group_name>` group is invalid. Replace it with any of the following fields: `<field A>`, `<field B>`.

Cause: There is an invalid field name in the SDF custom object

■ Solution

Verify the correct spelling of the field name. Update all field name instances with the correct spelling in the form object, then validate again.

Cause: The field sequence is not in the correct order

Unlike other SDF custom object types, form fields must follow the order as exported from the account.

■ Solution #1

Re-import the form into your SuiteCloud project to determine the correct field order.

■ Solution #2

If you recently changed the field sequence in your recent edits, revert your changes to determine the correct field order.

The property value of the object cannot be modified

The `<property>` value of the `<id>` `<object name>` object cannot be modified.

Cause: The property value is disabled or set to read-only in the target account

■ Solution

Set the property to the default value that is used in the account that you are deploying to.



Note: A read-only property may have a default value that is different across multiple account types.

The sameRowAsPrevious field must be set to false because it is the first entry

The field with the `<field>` ID contains an invalid value. The `sameRowAsPrevious` field must be set to false because it is the first entry.

Cause: SameRowAsPrevious was enabled for TOP and BOTTOM fields or for the first field with a field group

■ Solution

For the top and bottom position fields, set `<SameRowAsPrevious>F</SameRowAsPrevious>`.

For the first field within a field group, set `<SameRowAsPrevious>F</SameRowAsPrevious>`.

The object field must not be NOT_YET_SUPPORTED

The object field <field> must not be NOT_YET_SUPPORTED.

Cause: You selected a custom template for the printing type but only standard templates are supported

■ **Solution**

Change the custom template to a standard template.

Account Configuration Errors

The following errors may occur when working with account configuration errors:

- Resolve the following conflicts to disable the feature
- Disable the dependent feature before disabling the feature
- The feature must be enabled to deploy this project
- The feature is duplicated
- Enable the dependent feature before enabling the feature
- The feature is missing its field
- The feature's field requires a value
- The feature is not available on this account
- The configuration field from the feature is missing
- Feature is not modifiable
- Feature(s) definition must start with tag but found tag
- An invalid feature ID
- Is an invalid feature state
- You cannot use SuiteCloud Development Framework to enable the feature because it requires a Terms of Service user agreement

Resolve the following conflicts to disable the feature

Resolve the following conflicts to disable the <feature_name> feature: <conflict(s)>

Cause: The feature is dependent on the state of more than one feature with a conflicting state

■ **Solution**

Adjust the states of the features referenced in the error message.

Disable the dependent feature before disabling the feature

Disable the <feature_name> feature before disabling the <other_feature_name> feature.

Cause: The feature is dependent on the state of another feature that is either enabled in the target account, or set to enabled in the account configuration

■ **Solution #1**

Disable the feature in the target account.

■ **Solution #2**

Set the feature to disabled in the AccountConfiguration/features.xml file in the project.

The feature must be enabled to deploy this project

The <feature_name> feature must be enabled to deploy this project.

Cause: The feature is set to 'required' in the manifest but also set to disabled in the account configuration

■ **Solution #1**

Change the required state of the feature in the manifest file in the project.

■ **Solution #2**

Set the feature to enabled in the AccountConfiguration/features.xml file in the project.

The feature is duplicated

The <feature_ID> feature is duplicated.

Cause: There are multiple occurrences of the referenced feature ID

■ **Solution**

Remove all the duplicates.

Enable the dependent feature before enabling the feature

Enable the <feature_name> feature before enabling the <other_feature_name> feature.

Cause: The feature is dependent on the state of another feature that is either disabled in the target account, or set to disabled in the account configuration

■ **Solution #1**

Enable the feature in the target account.

■ **Solution #2**

Set the feature to enabled in the AccountConfiguration/features.xml file in the project.

The feature is missing its field

The feature is missing its <field_name> field.

Cause: A feature entry in the account configuration is missing its ID field

■ Solution

Add the missing field ID to the feature definition in the AccountConfiguration/features.xml file in the project.

The feature's field requires a value

The feature's <field_name> field requires a value.

Cause: A feature entry in the account configuration is missing its field value

■ Solution

Add the missing field value to the feature definition in the AccountConfiguration/features.xml file in the project.

The feature is not available on this account

The <feature_name> feature is not available on this account.

Cause: The feature is valid but does not exist in the target account

For example, you may have imported some features from a OneWorld account that are not available in other account types. This error occurs when you try to deploy a OneWorld account configuration to other account types.

■ Solution #1

Remove the referenced feature definition from the AccountConfiguration/features.xml file in the project so that you can deploy to the target account.

■ Solution #2

Only deploy the project to account types that support the referenced feature.

The configuration field from the feature is missing

The <field_name> configuration field from the <feature_name> feature is missing.

Cause: A feature entry in the account configuration is missing a required field (that is the status field)

■ Solution

Add the missing field to the feature definition in the AccountConfiguration/features.xml file in the project.

Feature is not modifiable

<feature_name> is not modifiable.

Cause: There was an attempt to modify a feature (DOCUMENTS or SUITEAPPDEVELOPMENTFRAMEWORK) that SDF requires to function

■ Solution

Remove the reference to the feature definition in the AccountConfiguration/features.xml file in the project.

Feature(s) definition must start with tag but found tag

Feature definition must start with the <tag_name> tag but found the <tag_name> tag.

Features definition must start with the <tag_name> tag but found the <tag_name> tag.

Cause: There was a typo or misspelling in the AccountConfiguration/features.xml file

■ Solution

Change the referenced tag to <feature> or <features>, depending on where the error occurred.

An invalid feature ID

<ID> is an invalid feature ID.

Cause: There was a typo or misspelling in the AccountConfiguration/features.xml file

■ Solution

Remove the reference to the invalid ID in the AccountConfiguration/features.xml file in the project.

Is an invalid feature state

<state_value> is an invalid feature state. Set the state to ENABLED or DISABLED.

Cause: There was a typo or misspelling in the AccountConfiguration/features.xml file

■ **Solution**

Change the referenced feature state to ENABLED or DISABLED.

You cannot use SuiteCloud Development Framework to enable the feature because it requires a Terms of Service user agreement

You cannot use SuiteCloud Development Framework (SDF) to enable the <feature_name> feature because it requires a Terms of Service user agreement.

Cause: There was an attempt to change the state of a feature that requires a Terms of Service user agreement

■ **Solution**

Remove the reference to the feature definition in the AccountConfiguration/features.xml file in the project.

Circular Dependency Warnings

When circular dependencies exist in a SuiteCloud project for SuiteCloud Development Framework (SDF) or an SDF SuiteApp but there are no errors, there are no warnings for the circular dependencies outputted to the SuiteCloud project deployment log and SuiteApp installation log. Circular dependency warnings are only logged when there are errors that cause deployment or installation to fail.

Circular dependencies detected

Circular dependencies are detected when SDF custom objects reference each other in a way that creates direct dependencies. Circular dependencies that are detected on SuiteCloud project validation and deployment (or SuiteApp installation) log the warning “Circular dependencies detected” for each circular dependency when SuiteCloud project deployment (or SDF SuiteApp installation) fails due to any error.

Cause: SDF custom objects reference each other in a way that creates direct dependencies

■ **Solution**

You can resolve the circular dependency warnings by ensuring that each SDF custom object does not reference any SDF custom object that references it.